
声明

责任。工程和软件开发领域的知识和最佳实践不断变化。从业者和研究人员在评估和使用本文描述的任何信息、方法、化合物或实验时，必须始终依赖自己的经验和知识。在使用此类信息或方法时，他们应注意自身和他人的安全，包括他们对其负有专业责任的各方。

在法律允许的最大范围内，作者、贡献者或编辑均不对因产品责任、疏忽或其他原因，或因使用或操作本文所含材料中的任何方法、产品、说明或想法而造成的任何人员伤亡和/或财产损失承担任何责任。

商标。公司用来区分其产品的名称通常被声明为商标或注册商标。Intel、Intel Core、Intel Xeon、Intel Pentium、Intel VTune 和 Intel Advisor 是 Intel Corporation 在美国和/或其他国家的商标。AMD 是 Advanced Micro Devices Corporation 在美国和/或其他国家的商标。Arm 是 Arm Limited（或其子公司）在美国和/或其他地方的商标。但是，读者应联系相关公司以获取有关商标和注册的完整信息。

关联。在撰写本文时，本书作者（Denis Bakhvalov）是 Intel Corporation 的员工。书中提供的信息不是上述公司的官方立场，而是作者的个人知识和观点。主要作者没有从 Intel Corporation 获得任何写作本书的财务赞助。

广告。本书不宣传任何软件、硬件或任何其他产品。

版权

版权所有 © 2024 Denis Bakhvalov。

前言

关于作者

Denis Bakhvalov 是 Intel 的性能工程师，负责优化生产应用程序和基准测试。此前，他是 Intel 编译器团队的一员，该团队为各种不同架构开发 C++ 编译器。Denis 于 2008 年开始职业生涯，担任软件开发人员，参与大型 C++ 企业金融应用程序的开发。在加入 Intel 之前，他在 Nokia 工作了三年，编写嵌入式软件。

性能工程和编译器一直是他的主要兴趣之一。2016 年，Denis 开始撰写 `easyperf.net` 博客，分享关于低级性能优化、C/C++ 编译器和 CPU 微架构的知识。工作之余，Denis 喜欢足球、国际象棋和旅行。

联系方式：

- 邮箱：dendibakh@gmail.com
- X (原 Twitter)：@dendibakh
- LinkedIn：@dendibakh

致读者

我开始写这本书的目标很简单：帮助软件开发者更好地理解其应用程序的性能。我知道低级性能工程这个主题对初学者甚至有经验的开发者来说可能有多难。我记得刚开始学习性能分析的日子。我看着陌生的指标，试图匹配不匹配的数据，感到困惑。花了多年时间才终于“顿悟”，所有拼图碎片都拼在一起了……尽管有时我仍然在同样的问题上挣扎。

当我刚开始学习性能工程时，关于这个主题的唯一优质信息来源是软件开发者手册，这不是主流开发者喜欢阅读的。坦白说，我希望在我尝试学习低级性能分析时就有这本书。2016 年，我开始在我的博客上分享我学到的东西，并收到了读者的一些积极反馈。其中一些人建议我将这些信息汇集成一本书。这本书是他们的功劳。

很多人问我为什么决定自出版。事实上，我最初尝试向几家知名出版社推销，但他们没有看到制作这样一本书的经济利益。然而，我真的想写它，所以我决定无论如何都要做。最终，它效果相当好，所以我决定自出版第二版。

第一版于 2020 年 11 月发布。它受到了社区的欢迎，但读者给了我很多建设性的批评。最普遍的反馈是增加实验练习。一些读者抱怨它过于关注 Intel CPU，没有涵盖 AMD 和 ARM 等其他架构。其他读者建议我应该涵盖系统性能，而不仅仅是 CPU 性能。第二版在所有这些方向和许多其他方向上都有扩展。它的篇幅是第一本书的两倍。

具体来说，我想强调我在第二版中添加的练习。我创建了一个补充在线课程“Performance Ninja”，包含超过二十个实验室作业。我建议你使用这些小谜题来练习优化技术并检查你对材料的理解。我认为这是本书与其他书不同的最佳部分。我希望它能使你的学习过程更加有趣。有关在线课程的更多详细信息，请参见 Section 1.7。

我亲身经历知道低级性能优化并不容易。我试图尽可能清晰地解释所有内容，但这个主题非常复杂。需要实验和练习才能完全理解材料。我鼓励你花时间通读各章，并尝试在线课程中提供的示例。

在我的职业生涯中，我从不回避软件优化任务。每当设法使我的程序运行得更快时，我总会获得多巴胺的刺激。发现某事物的兴奋感和自豪感让我更加好奇并渴望更多。我最初的性能工作非常无结构。现在它是我的职业，但当让软件运行得更快时，我仍然感到非常快乐。我希望你也能体验到发现性能问题的乐趣，以及修复它们的满足感。

我衷心希望这本书能帮助你学习低级性能分析。如果你因此使应用程序更快，我将认为我的使命完成了。

你会发现我在书中某些地方使用“我们”而不是“我”。这是因为得到了其他人的很多帮助。完整的贡献者名单可以在书末的“致谢”部分找到。

本书的 PDF 版本和“Performance Ninja”在线课程免费提供。这是我回馈社区的方式。

目标受众

如果你正在从事性能关键型应用程序的工作，这本书适合你。它主要面向高性能计算（HPC）、AI、游戏开发、数据中心应用程序（如 Meta、Google 等）、高频交易（HFT）以及其他性能优化价值广为人知和赞赏的行业的软件开发者。

本书也将对任何想要更好地理解其应用程序性能并了解如何改进它的开发者有用。你可能只是对性能工程充满热情，想了解更多。或者你可能想成为房间里最聪明的工程师；那也没问题。我希望本书介绍的材料能帮助你发展新技能，应用于日常工作，并 potentially 推动你的职业生涯发展。

理解本书示例需要 C 和 C++ 编程语言的最低背景。阅读基本 x86/ARM 汇编的能力是可取的，但不是严格要求。我还期望熟悉计算机架构和操作系统的基本概念，如“CPU”、“内存”、“进程”、“线程”、“虚拟”和“物理内存”、“上下文切换”等。如果这些术语对你来说是新的，我建议先学习这些先决条件。

我建议你从头开始逐章阅读本书。如果你认为自己是性能分析的初学者，我不建议跳过章节。阅读完后，当你遇到性能问题且不确定如何修复时，可以使用本书作为想法来源。你可以浏览本书的第二部分，看看哪些优化技术可以应用于你的代码。

我将在我的博客上发布勘误表和有关本书的其他信息，网址为：<https://easyperf.net/blog/2024/11/11/Book-Updates-Errata>。

Table Of Contents

Table Of Contents	4
1 引言	9
1.1 为什么软件很慢?	10
1.2 为什么关心性能?	11
1.3 什么是性能分析?	12
1.4 什么是性能调优?	13
1.5 本书讨论什么?	13
1.6 本书不讨论什么?	15
1.7 练习	15
第一部分: 现代 CPU 上的性能分析	17
2 度量性能	17
2.1 现代系统中的噪声	17
2.2 在生产环境中度量性能	19
2.3 持续基准测试	19
2.4 手动性能测试	21
2.5 软件和硬件计时器	24
2.6 微基准测试	25
2.7 主动基准测试	26
3 CPU 微架构	28
3.1 指令集架构	28
3.2 流水线	28
3.3 利用指令级并行 (ILP)	30
3.3.1 乱序 (OOO) 执行	31
3.3.2 超标量引擎	32
3.3.3 推测执行	33
3.4 SIMD 多处理器	33
3.5 利用线程级并行	35
3.5.1 多核系统	35
3.5.2 同时多线程	35
3.5.3 混合架构	37
3.6 内存层次结构	37
3.6.1 缓存层次结构	38
3.7 虚拟内存	39

3.7.1	转换后备缓冲区 (TLB)	41
3.7.2	大页	41
3.8	现代 CPU 设计	42
3.8.1	CPU 前端	42
3.8.2	CPU 后端	44
3.8.3	加载-存储单元	46
3.8.4	TLB 层次结构	47
3.9	性能监控单元	48
3.9.1	性能监控计数器	49
4	性能分析中的术语和指标	53
4.1	退休指令与执行指令	53
4.2	CPU 利用率	53
4.3	CPI 和 IPC	54
4.4	微操作	55
4.5	流水线槽	56
4.6	核心周期与参考周期	57
4.7	缓存未命中	57
4.8	分支预测错误	58
4.9	性能指标	59
4.10	内存延迟和带宽	59
4.11	案例研究：分析四个基准测试的性能指标	61
5	性能分析方法	64
5.1	代码检测	64
5.2	跟踪	68
5.3	收集性能监控事件	69
5.3.1	使用标记 API	70
5.4	采样	71
5.4.1	用户模式和基于硬件事件的采样	71
5.4.2	查找热点	72
5.5	Roofline 性能模型	73
5.6	静态性能分析	75
5.6.1	案例研究：使用 UICA 优化 FMA 吞吐量	76
5.7	编译器优化报告	76
6	用于性能分析的 CPU 特性	79
6.1	Top-down 微架构分析	80
6.1.1	Intel 平台上的 TMA	80
6.1.2	AMD 平台上的 TMA	82
6.1.3	Arm 平台上的 TMA	84
6.1.4	TMA 总结	85
6.2	分支记录机制	86
6.3	基于硬件的采样功能	87
6.3.1	Intel 平台上的 PEBS	88
6.3.2	AMD 平台上的 IBS	89

7	性能分析工具概述	91
7.1	Intel VTune Profiler	91
7.2	AMD uProf	92
7.3	Apple Xcode Instruments	93
7.4	Linux Perf	95
7.5	火焰图	97
7.6	Windows 事件跟踪	98
7.7	专用和混合分析器	99
7.8	内存分析	100
7.8.1	内存使用	100
7.8.2	案例研究：分析 Stockfish 的堆分配	101
7.9	持续分析	101
	第二部分：源代码调优	105
8	优化内存访问	107
8.1	缓存友好数据结构	107
8.1.1	顺序访问数据	108
8.1.2	使用适当的容器。	109
8.1.3	数据打包	109
8.2	动态内存分配	109
8.3	解决内存带宽限制	110
8.4	减少 DTLB 未命中	111
8.4.1	显式大页	111
8.4.2	透明大页	112
8.5	显式内存预取	112
9	优化计算	115
9.1	数据依赖	115
9.2	内联函数	116
9.3	循环优化	117
9.3.1	低级优化。	117
9.4	向量化	119
9.4.1	编译器自动向量化	119
9.4.2	发现向量化机会。	120
9.5	编译器内置函数	120
10	优化分支预测	123
10.1	用查找替换分支	124
10.2	用算术替换分支	125
10.3	用选择替换分支	125
10.4	多个测试单个分支	127
11	机器代码布局优化	129
11.1	机器代码布局	129
11.2	基本块	130

11.3	基本块放置	130
11.4	基本块对齐	132
11.5	函数分割	133
11.6	函数重排序	135
11.7	配置文件引导优化	136
11.8	减少 ITLB 未命中	138
11.9	案例研究：度量代码占用	139
12	其他调优领域	142
12.1	CPU 特定优化	142
12.1.1	ISA 扩展	143
12.2	微架构特定性能问题	143
12.2.1	内存顺序违规	143
12.3	低延迟调优技术	145
12.3.1	避免小页面错误	145
12.4	系统调优	146
12.5	案例研究：对最后一级缓存大小的敏感性	147
13	优化多线程应用程序	149
13.1	并行效率指标	149
13.2	多线程程序中的性能扩展	150
13.2.1	线程数扩展案例研究	151
13.3	任务调度	153
13.4	缓存一致性	155
13.4.1	缓存一致性协议	155
13.4.2	真共享	156
13.4.3	假共享	156
13.5	高级分析工具	156
13.5.1	Coz	156
13.5.2	eBPF 和 GAPP	157
	后记	159
	致谢	161
	术语表	163
	主要 CPU 微架构列表	164
	附录 A. 减少测量噪声	165
	附录 B. 启用大页	166
	Windows	166
	Linux	166
	附录 C. Intel 处理器跟踪	167
	13.6 收集和解码跟踪	168

附录 D. Windows 事件跟踪分析	169
References	171

1 引言

性能为王：这在十年前是正确的，现在更是如此。根据 [domo.com, 2017] 的数据，2017 年全球每天产生 250 亿亿¹ 字节的数据。[statista.com, 2024] 预测到 2024 年这个数字将达到每天 4000 亿亿字节。在这个日益数据驱动的世界中，信息交换的增长既需要更快的软件，也需要更快的硬件。

软件程序员几十年来一直享有“轻松时光”，这要归功于摩尔定律。软件供应商可以依赖新一代硬件来加速他们的软件产品，即使他们不投入人力来改进代码。这种策略不再有效。从图 fig. 1.1 可以看出，单线程² 性能增长正在放缓。从 1990 年到 2000 年，SPECint 基准测试中的单线程性能增长了大约 25 到 30 倍，主要由更高的 CPU 频率和改进的微架构驱动。

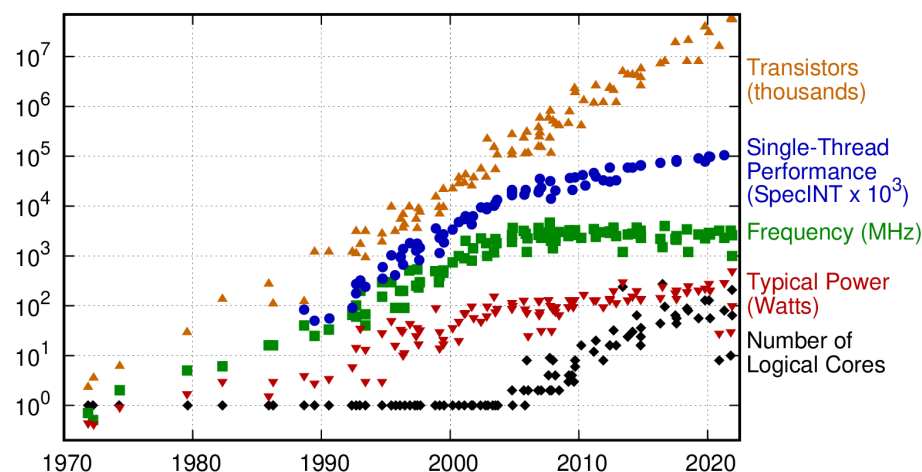


Figure 1.1: 50 年微处理器趋势数据。© 图片由 K. Rupp 通过 karlrupp.net 提供。2010 年之前的原始数据由 M. Horowitz、F. Labonte、O. Shacham、K. Olukotun、L. Hammond 和 C. Batten 收集和绘制。2010-2021 年的新数据由 K. Rupp 收集。

2000 年到 2010 年间，单线程 CPU 性能增长更为温和（大约 4 到 5 倍）。当时，由于功耗、散热挑战、电压缩放限制（登纳德缩放³）和其他基本问题，时钟频率在 4GHz 左右达到顶峰。尽管时钟频率停滞不前，架构改进仍在继续：更好的分支预测、更深的流水线、更大的缓存和更高效的执行单元。

从 2010 年到 2020 年，单线程性能仅增长了 2 到 3 倍。在此期间，CPU 制造商开始更多地关注多核处理器和并行性，而不仅仅是提高单线程性能。

现代处理器中的晶体管数量继续增加。例如，苹果芯片中的晶体管数量在大约四年的时间里从 M1 的 160 亿增长到 M2 的 200 亿，再到 M3 的 250 亿，最后到 M4 的 280 亿。晶体管数量的增长使制造商能够在处理器中添加更多核心。截至 2024 年，你可以购买一个在单个 CPU 插槽

¹ 亿亿是 10 的 18 次方 (10¹⁸)。

² 单线程性能是 CPU 核心中单个硬件线程在隔离测量时的性能。

³ 登纳德缩放 - https://en.wikipedia.org/wiki/Dennard_scaling

上拥有超过 100 个逻辑核心的高端服务器处理器。这非常令人印象深刻。不幸的是，这并不总是转化为更好的性能。通常，应用程序性能不会随着额外的 CPU 核心而扩展。

由于硬件每一代不再提供显著的性能提升，我们必须开始更加关注代码运行的速度。在寻求提高性能的方法时，开发者不应该依赖硬件。相反，他们应该开始优化应用程序的代码。

“今天的软件效率极低；软件程序员重新进入优化的最佳时机。” - Marc Andreessen, 美国企业家和投资者 (a16z 播客)

1.1 为什么软件很慢？

如果世界上所有的软件都高效地利用了所有可用的硬件资源，那么这本书就不会存在。我们不需要对软件做任何改变，而是依赖现有处理器提供的能力。但你已经知道现实并非如此，对吧？现实是现代软件极其低效。公共云中的典型服务器系统通常运行着优化不佳的代码，消耗的电力比它本可以消耗的更多（增加碳排放并加剧其他环境问题）。如果我们能让所有软件运行速度提高两倍，我们就有可能将计算的碳足迹减少一半。

论文 [Leiserson et al., 2020] 的作者提供了一个很好的例子，说明了”默认”软件 and 高度优化软件之间的性能差距。表 tbl. ?? 总结了对两个 4096×4096 矩阵相乘程序进行性能工程后的加速比。应用多项优化后的最终结果是一个运行速度快 60,000 多倍的程序。提供这个例子的目的不是批评 Python 或 Java（它们都是很好的语言），而是打破软件默认性能”足够好”的信念。大多数程序都在第 1-5 行之间。源代码级别的改进潜力是巨大的。

版本实现	绝对加	速比相对加	速比
1	Python	1	—
2	Java	11	10.8
3	C	47	4.4
4	并行循环	366	7.8
5	并行分治	6,727	18.4
6	加向量化 2	3,224	3.5
7	加 AVX 内置函数 62	,806	2.7

表：对两个 4096×4096 矩阵相乘程序进行性能工程后的加速比，运行在配备总共 60 GB 内存的双路 Intel Xeon E5-2666 v3 系统上。来自 [Leiserson et al., 2020]。{#tbl:PlentyOfRoom}

那么，让我们谈谈是什么阻止了系统默认达到最佳性能。以下是一些最重要的因素：

- 1. **CPU 限制：**人们很容易问：“为什么硬件不能解决我们所有的问题？”现代 CPU 执行指令的速度令人难以置信，并且每一代都在变得更好。但是，如果用于执行任务的指令不是最优的甚至是冗余的，它们仍然做不了太多。处理器不能神奇地将次优代码转化为性能更好的代码。例如，如果我们实现冒泡排序，CPU 不会尝试识别它并使用更好的替代方案（如快速排序）。它会盲目地执行被告知要做的任何事情。
- 2. **编译器限制：**“但这不正是编译器应该做的吗？为什么编译器不能解决我们所有的问题？”的确，如今的编译器非常智能，但仍然可能生成次优代码。编译器在消除冗余工作方面很出色，但在做出更复杂的决策（如向量化）时，它们可能无法生成最佳代码。性能专家通常能想出比编译器能力更聪明的向量化循环方法。当编译器必须决定是否执行代码转换时，它们依赖

于复杂的成本模型和启发式方法，这些可能不适用于所有可能的情况。例如，对于编译器是否应该总是将函数内联到调用位置这个问题，没有二元的“是”或“否”答案。这通常取决于编译器应该考虑的许多因素。此外，除非编译器绝对确定安全，否则它们不能执行优化。编译器可能很难证明优化在所有可能的情况下都是正确的，从而禁止某些转换。最后，编译器通常不会尝试“英雄式”优化，比如转换程序使用的数据结构。

3. **算法复杂度分析限制：**一些开发者过度迷恋算法复杂度分析，这导致他们选择具有最优算法复杂度的流行算法，即使它可能不是给定问题最有效的算法。考虑两种排序算法，插入排序和快速排序，后者在平均情况下的大 O 表示法中明显胜出：插入排序是 $O(N^2)$ ，而快速排序只有 $O(N \log N)$ 。然而，对于相对较小的 N （最多 50 个元素），插入排序的性能优于快速排序。复杂度分析无法考虑各种算法的所有低级性能影响，因此人们只是将它们封装在隐式常数 C 中，这有时会对性能产生很大影响。仅计算用于排序的比较和交换次数，忽略了缓存未命中和分支预测错误，而这些在今天实际上是非常昂贵的。盲目信任大 O 表示法而不对目标工作负载进行测试可能会引导开发者走上错误的道路。因此，对于某个问题的最新算法不一定在实践中对每个可能的输入都是性能最高的。

除了上述限制之外，编程范式也会产生开销。优先考虑代码清晰度、可读性和可维护性的编码实践可能会降低性能。高度通用和可重用的代码可能会引入不必要的复制、运行时检查、函数调用、内存分配等。例如，面向对象编程中的多态通常使用虚函数实现，这会引入性能开销。⁴

上述所有因素都对软件征收了“性能税”。通常有很大的机会来调整我们软件的性能，以发挥其全部潜力。

1.2 为什么关心性能?

除了硬件单线程性能增长放缓之外，还有一些其他业务原因需要关心性能。在 PC 时代，^[12] 慢速软件的成本由用户承担，因为低效软件运行在用户计算机上。软件供应商没有直接的动力去优化其应用程序的代码。随着 SaaS（软件即服务）和云计算的出现，慢速软件的成本又回到了软件提供商身上，而不是他们的用户。如果你是像 Meta 或 Netflix 这样的 SaaS 公司，^[4] 无论你在本地硬件上运行服务还是使用公共云，你都要为服务器消耗的电力付费。低效软件直接侵蚀你的利润率和市场估值。根据 Synergy Research Group，^[5] 2020 年全球云服务支出超过 1000 亿美元，根据 Gartner，^[6] 到 2024 年将超过 6750 亿美元。

多年来，性能工程一直是一个小众领域，但现在它正在成为主流。许多公司已经意识到性能工程的重要性，并愿意为此支付高昂的报酬。

达到表 tbl. ?? 中的性能级别 4 相当容易。事实上，你不需要这本书就能达到。用原生编程语言之一编写程序，将工作分配给多个线程，选择一个好的优化编译器，你就能达到。不幸的是，你的程序的性能将比最优目标慢大约 200 倍。

本书中的方法侧重于从应用程序中榨取最后一丝性能。这种改进可以归因于表 tbl. ?? 中的第 6 和第 7 行。将要讨论的改进类型通常不大，通常不超过 10%。但是，不要低估 10% 加速的重要性。SQLite 如今无处不在，不是因为它的开发者有一天让它快了 50%，而是因为他们多年来一丝不苟地做了数百个 0.1% 的改进。这些小改进的累积效应才是造成差异的原因。

小改进的影响对于在云中运行的大型分布式应用程序非常相关。根据 [Hennessy, 2018]，在 2018 年，Google 在运行云的实际计算服务器上花费的金额与其在电力和冷却基础设施上花费的金额大致相同。能源效率是一个非常重要的问题，可以通过优化软件来改善。

⁴ 我不否定设计模式和整洁代码原则，但我鼓励一种更细致的方法，其中性能也是开发过程中的关键考虑因素。

“在这样的 [Google] 规模下，理解性能特性变得至关重要——即使是性能或利用率的微小改进也能转化为巨大的成本节约。”[Kanev et al., 2015]

除了云成本之外，还有另一个因素在起作用：人们对慢速软件的看法。Google 报告说，搜索延迟 500 毫秒会导致流量减少 20%。[3] 对于 Yahoo!，页面加载快 400 毫秒会导致流量增加 5-9%。[8] 在大数字的游戏中，小改进可以产生重大影响。这些例子证明，服务运行得越慢，使用它的人就越少。

在云服务之外，还有许多其他性能关键行业，性能工程不需要证明其合理性，例如人工智能（AI）、高性能计算（HPC）、高频交易（HFT）、游戏开发等。此外，性能不仅在高度专业化的领域需要，它也与通用应用程序和服务相关。我们每天使用的许多工具如果无法满足其性能要求，根本就不存在。例如，集成到 Microsoft Visual Studio IDE 中的 Visual C++ IntelliSense[2] 功能具有非常严格的性能约束。为了让 IntelliSense 自动补全功能工作，它必须在毫秒内解析整个源代码库。[9] 如果源代码编辑器需要几秒钟才能建议自动补全选项，没有人会使用它。这样的功能必须非常响应，并在用户输入新代码时提供有效的延续。

“不是所有快速的软件都是世界级的，但所有世界级的软件都是快速的。性能是 __ 杀手级 __ 特性。” —Shopify CEO Tobi Lutke。

我希望不言而喻的是，人们讨厌使用慢速软件，尤其是当他们的生产力因此下降时。表 1.2 显示，大多数人认为 2 秒或更长的延迟是”长时间等待”，并在等待 10 秒后切换到其他东西（我认为会更快）。如果你想保持用户的注意力，你的应用程序必须快速响应。

交互类别	人类感知响	应时间
快	最小可察觉延迟 1	00ms-200ms
交互式	快，但太慢无法描述为快	300ms-500ms
停顿	不快但仍然感觉响应	500ms-1 秒
等待	由于场景工作量大而不快	1 秒-3 秒
长时间等待	不再感觉响应	2 秒-5 秒
强制等待	保留给不可避免的长/复杂场景	5 秒-10 秒
长时间运行	用户可能会在操作期间切换	10 秒-30 秒

表：人-软件交互类别。来源：Microsoft Windows 博客。[11] {#tbl:WindowsResponsiveness}

应用程序性能可能会驱使你的客户转向竞争对手的产品。通过强调性能，你可以为你的产品提供竞争优势。

1.3 什么是性能分析？

你是否曾经发现自己与同事争论某段代码的性能？那么你可能知道预测哪段代码效果最好有多难。现代处理器中有这么多活动部件，即使对代码的微小调整也可能触发明显的性能变化。在优化应用程序时依赖直觉通常会导致随机的”修复”，而没有真正的性能影响。

经验不足的开发者有时会更改代码，并声称它应该运行得更快。一个这样的例子是在整个代码库中将 i++（后递增）替换为 ++i（前递增）（假设 i 的前值未被使用）。在一般情况下，这种更改不会对生成的代码产生任何影响：每个优秀的优化编译器都会识别出 i 的前值未被使用，并无论如何都会消除冗余副本。本书的第一个建议是：不要仅仅依赖你的直觉。总是测量。

许多流传于世的微优化技巧在过去是有效的，但当前的编译器已经学会了它们。此外，有些人倾向于过度使用遗留的位操作技巧。其中一个例子是 XOR 交换惯用法。⁵ 实际上，简单的 `std::swap` 会产生等效或更快的代码。这种意外的更改不太可能提高应用程序的性能。找到正确的调优位置应该是仔细性能分析的结果，而不是直觉或猜测。

性能分析是收集有关程序执行方式的信息并解释它以找到优化机会的过程。最终在程序源代码中做出的任何更改都应该由分析和解释收集的数据驱动。我们将向你展示如何使用性能分析技术，即使在大型且不熟悉的代码库中也能发现优化机会。有许多性能分析方法。根据问题的不同，其中一些会比其他方法更有效。随着经验的积累，你将发展出自己的策略，了解何时使用每种方法。

1.4 什么是性能调优？

定位性能瓶颈只是工程师工作的一半。另一半是正确地修复它。有时更改程序源代码中的一行可以产生显著的性能提升。错过这样的机会可能非常浪费。性能分析和调优都是关于找到并修复这一行。

为了利用现代 CPU 的所有计算能力，你需要了解它们的工作原理。或者正如性能工程师喜欢说的，你需要有“机械同理心”。这个术语借自赛车世界。它意味着一个对汽车工作原理有良好理解的赛车手比没有理解的竞争对手更有优势。同样的道理也适用于性能工程。不可能了解现代 CPU 运作的细节，但你需要对它有一个良好的心智模型，以榨取最后一丝性能。

这就是我所说的低级优化。这是一种考虑底层硬件能力细节的优化。它不同于高级优化，后者更多关于应用程序级逻辑、算法和数据结构。正如你将在书中看到的，大多数低级优化可以应用于各种现代处理器。要成功实施低级优化，你需要对底层硬件有良好的理解。

“在后摩尔时代，使代码快速运行变得越来越重要，特别是根据运行它的硬件进行定制。”[Leiserson et al., 2020]

过去，软件开发者有更多的机械同理心，因为他们经常必须处理硬件实现的细微差别。在 PC 时代，开发者通常直接在操作系统之上编程，中间可能有一些库。随着世界进入云时代，软件栈变得更深、更宽、更复杂。栈的顶层（大多数开发者工作的地方）已经离硬件更远。这种演进的负面影响是，现代应用程序的开发者对其软件运行的实际硬件亲和力较低。这本书将帮助你与现代处理器建立强大的联系。

Donald Knuth 有一句名言：“过早优化是万恶之源”。[Knuth, 1974] 但相反的情况也经常是真的。延迟的性能工程可能为时已晚，造成与过早优化同样多的恶果。对于从事性能关键项目的开发者来说，了解底层硬件的工作原理至关重要。在这种角色中，不以硬件为重点的程序开发从一开始就是失败的。⁶ 软件的性能特征必须与正确性和安全性一样，从第一天起就成为主要目标。糟糕的性能可能像安全漏洞一样容易杀死一个产品。

性能工程是一项重要且有回报的工作，但它可能非常耗时。事实上，性能优化是一场没有终点的游戏。总有一些东西需要优化。不可避免地，开发者会达到收益递减的点，此时进一步的改进不被预期的工程成本所证明。知道何时停止优化是性能工作的关键方面。

1.5 本书讨论什么？

本书旨在帮助开发者更好地理解其应用程序的性能，学会找到低效之处并消除它们。

- 为什么我的更改导致 2 倍的性能下降？

⁵ 基于 XOR 的交换惯用法 - https://en.wikipedia.org/wiki/XOR_swap_algorithm

⁶ ClickHouse DB 是一个成功的软件产品的例子，它围绕一个虽小但非常高效的核心构建。

- 我们的客户抱怨我们的应用程序太慢。我应该如何调查？
- 为什么我手写的压缩算法比传统算法慢？
- 我是否将程序优化到了其全部潜力？
- 我的平台上有哪些性能分析工具？
- 有哪些技术可以减少缓存未命中和分支预测错误？

我希望在读完本书后，你将能够回答这些问题。

本书分为两部分。第一部分（第 2-7 章）教你如何找到性能问题，第二部分（第 8-13 章）教你如何修复它们。

- 第 2 章讨论公平的性能实验及其分析。它介绍了性能测试和比较结果的最佳实践。
- 第 3 章介绍 CPU 微架构，重点介绍 Intel 的 Golden Cove 微架构。
- 第 4 章涵盖性能分析中使用的术语和指标。在本章末尾，我们介绍一个案例研究，展示在四个真实应用程序上收集的各种性能指标。
- 第 5 章探讨最流行的性能分析方法。我们描述分析工具如何工作以及它们可以收集什么样的数据。
- 第 6 章研究现代 Intel、AMD 和基于 ARM 的 CPU 提供的用于支持和增强性能分析的功能。它展示它们如何工作以及它们帮助解决什么问题。
- 第 7 章概述 Linux、Windows 和 MacOS 上可用的最流行的性能分析工具。
- 第 8 章关于优化内存访问、缓存友好代码、数据结构重组和其他技术。
- 第 9 章关于优化计算；它探讨数据依赖、函数内联、循环优化和向量化。
- 第 10 章关于无分支编程，用于避免分支预测错误。
- 第 11 章关于机器代码布局优化，如基本块放置、函数分割和配置文件引导优化。
- 第 12 章包含前四章未考虑的优化主题，但仍然重要到足以在本书中找到它们的位置。在本章中，我们讨论 CPU 特定的优化，检查几个与微架构相关的性能问题，探讨用于优化低延迟应用程序的技术，并就如何调整系统以获得最佳性能提供建议。
- 第 13 章讨论分析多线程应用程序的技术。它深入探讨了优化多线程应用程序的一些最重要挑战。我们提供五个真实多线程应用程序的案例研究，解释为什么它们的性能不随 CPU 线程数扩展。我们还讨论缓存一致性问题（如“伪共享”）以及一些旨在分析多线程应用程序的工具。

在本书末尾，有一个术语表和主要 CPU 供应商的微架构列表。每当你看到不熟悉的缩写或需要刷新对最近 Intel、AMD 和 ARM 芯片系列的记忆时，请参考这些资源。

本书中提供的示例主要基于开源软件：Linux 作为操作系统，基于 LLVM 的 Clang 编译器用于 C 和 C++ 语言，以及各种开源应用程序和基准测试⁷，你可以构建和运行它们。原因不仅是这些项目的流行，还因为它们的源代码是开放的，这使我们能够更好地理解它们工作原理的底层机制。这对于学习本书中介绍的概念特别有用。这并不意味着我们永远不会展示专有工具。例如，我们广泛使用 Intel® VTune™ Profiler。

有时可以通过各种提示强制编译器生成所需的机器代码来获得有吸引力的加速。你会在本书中找到许多这样的例子。虽然以前的编译器经验对性能工作有很大帮助，但大多数时候你不需要成为编译器专家来推动应用程序的性能改进。大多数优化可以在源代码级别完成，而无需深入研究编译器源代码。

⁷ 有些人不喜欢他们的应用程序被称为“基准测试”。他们认为基准测试是合成和人为的，不能很好地代表现实场景。在本书中，我们互换使用“基准测试”、“工作负载”和“应用程序”这些术语，无意冒犯任何人。

1.6 本书不讨论什么？

系统性能取决于不同的组件：CPU、DRAM、I/O 和网络设备等。应用程序可能受益于调整系统的各种组件，具体取决于瓶颈在哪里。通常，工程师应该分析整个系统的性能。然而，系统性能的最大因素是其核心，CPU。这就是为什么本书主要从 CPU 的角度关注性能分析。我们还相当广泛地讨论了内存子系统，但我们不探讨 I/O 和网络性能。

同样，软件栈包括许多层，例如固件、BIOS、操作系统、库和应用程序的源代码。然而，由于大多数底层不在我们的直接控制之下，主要关注点将是源代码级别。

本书的范围不超过单个 CPU 插槽，因此我们不会讨论分布式、NUMA 和异构系统的优化技术。本书不讨论使用 OpenCL 和 openMP 等解决方案将计算卸载到加速器（GPU、FPGA 等）。

我试图使本书适用于大多数现代 CPU，包括 Intel、AMD、Apple 和其他基于 ARM 的处理器。如果它没有涵盖你喜欢的架构，我很抱歉。尽管如此，本书中讨论的许多原则也很好地适用于其他处理器。同样，本书中的大多数示例都在 Linux 上运行，但大多数时候这并不重要，因为相同的技术也适用于在 Windows 和 macOS 操作系统上运行的应用程序。

本书中的代码片段用 C 或 C++ 编写，但在很大程度上，本书中的想法可以应用于其他编译为本机代码的语言，如 Rust、Go，甚至 Fortran。由于本书针对接近硬件运行的用户模式应用程序，我们不会讨论托管环境，例如 Java。

最后，我假设读者对他们开发的软件拥有完全控制权，包括他们使用的库和编译器的选择。因此，本书不是关于调整购买的商业软件包，例如调整 SQL 数据库查询。

1.7 练习

作为本书的补充材料，我开发了“Performance Ninja”，一个免费的在线课程，你可以在其中练习低级性能分析和调优。它可在以下 URL 获取：<https://github.com/dendibakh/perf-ninja>。它包含一系列专注于特定性能问题的实验室作业。每个实验室作业可能需要 30 分钟到 4 小时，具体取决于你的背景和实验室作业本身的复杂性。

按照 GitHub 仓库的名称，我们将使用 perf-ninja 来指代在线课程。在每章末尾的“问题和练习”部分，你可能会找到来自 perf-ninja 的作业。例如，当你看到 perf-ninja::warmup 时，它对应于 GitHub 仓库中名为“Warmup”的实验室作业。我们鼓励你解决这些谜题以巩固你的知识。

你可以在本地机器上解决作业，也可以将代码更改提交到 GitHub 进行自动验证和基准测试。如果你选择后者，请按照仓库的“Get Started”页面上的说明操作。我们还在本书中使用了 perf-ninja 的示例。这使你可以在自己的机器上重现特定的性能问题并进行实验。

章节总结

- 单线程 CPU 性能的增长速度不如几十年前那样快。当硬件每一代不再提供显著的性能提升时，开发者应该开始优化其软件的代码。
- 现代软件极其低效。公共云中的典型服务器系统通常运行着优化不佳的代码，消耗的电力比它本可以消耗的更多，这会增加碳排放并加剧其他环境问题。
- 存在某些限制阻止应用程序达到其全部性能潜力。CPU 不能神奇地加速慢速算法。编译器远非为每个程序生成最优代码。大 O 表示法并不总是性能的良好指标，因为它没有考虑硬件特性。

- 多年来，性能工程一直是一个小众领域。但现在随着软件供应商意识到其优化不佳的软件对其利润的影响，它正在成为主流。
- 人们绝对讨厌使用慢速软件，尤其是当他们的生产力因此下降时。不是所有快速的软件都是世界级的，但所有世界级的软件都是快速的。性能是 __ 杀手级 __ 特性。
- 软件调优比过去 40 年更重要，它将是未来性能提升的关键驱动力之一。低级性能调优的重要性不应被低估，即使只是 1% 的改进。这些小改进的累积效应才是造成差异的原因。
- 要榨取最后一丝性能，你需要对现代 CPU 的工作原理有良好的心智模型。
- 预测某段代码的性能几乎是不可能的，因为有太多因素影响现代平台的性能。在实施软件优化时，开发者不应该依赖直觉，而应该使用仔细的性能分析。

第一部分：现代 CPU 上的性能分析

2 度量性能

理解应用程序性能的第一步是度量它。任何曾经关心过性能评估的人可能都知道，有时进行公平的性能测量并从中得出准确的结论有多难。性能测量可能非常出人意料和违反直觉。更改源代码中看似无关的部分可能会让我们惊讶地发现对程序性能的重大影响。由于各种原因，测量可能会持续高估或低估真实性能，导致扭曲的结果不能准确反映现实。这种现象称为测量偏差。

性能问题通常比大多数功能问题更难重现和找到根本原因。程序的每次运行通常在功能上是相同的，但从性能角度来看有所不同。例如，解压缩 zip 文件时，我们一次又一次地获得相同的结果，这意味着此操作是可重现的。但是，无法重现此操作的相同 CPU 逐周期性能概况。

进行公平的性能实验是获得准确有意义结果的重要一步。你需要确保你正在查看正确的问题，而不是在调试一些无关的问题。设计性能测试和配置环境都是评估性能过程中的重要组成部分。

由于测量偏差，性能评估通常涉及统计方法，这些方法本身值得写一整本书。这个领域有许多边缘情况和大量研究。我们不会深入探讨用于评估性能测量的统计方法。相反，我们只讨论高级概念并给出基本方向。我们鼓励你自己深入研究。

在本章中，我们：

- 简要介绍为什么现代系统会产生嘈杂的性能测量以及你可以做什么。
- 解释为什么在生产部署中度量性能很重要。
- 提供关于如何正确收集和分析性能测量的通用指导。
- 探索如何在代码库中实施更改时自动检测性能回归。
- 描述开发人员可以在基于时间的测量中使用的软件和硬件计时器。
- 讨论如何编写好的微基准测试以及你在执行过程中可能遇到的一些常见陷阱。

2.1 现代系统中的噪声

硬件和软件中有许多旨在提高性能的功能，但并非所有功能都具有确定性行为。考虑动态频率缩放 (DFS)，这是一个允许 CPU 将其频率提高到远高于基本频率的功能，使其运行速度显著加快。DFS 通常也称为 *turbo* 模式。不幸的是，CPU 不能长时间保持在 *turbo* 模式，否则可能面临过热的风险。因此一段时间后，它会降低频率以保持在其热限制内。DFS 通常很大程度上取决于当前系统负载和外部因素，如核心温度，这使得预测其对性能测量的影响变得困难。

图 fig. 2.1 显示了 DFS 可能导致性能变化的典型示例。在我们的场景中，我们在“冷”处理器上连续运行了两次基准测试。⁸ 在第一秒内，基准测试的第一次迭代以 4.4 GHz 的最大 *turbo* 频率运行，但后来 CPU 不得不将其频率降低到 4 GHz 以下。第二次运行没有提升 CPU 频率的优势，也没有进入 *turbo* 模式。即使我们运行了完全相同版本的基准测试两次，它们运行的环境也不相同。如你所见，第一次运行比第二次运行快 200 毫秒，这是因为开始时它以更高的 CPU

⁸ 冷处理器是指处于空闲模式一段时间的 CPU，允许其冷却温度。

频率运行。当你在笔记本电脑上对软件进行基准测试时，这种情况经常发生，因为笔记本电脑的散热有限。⁹

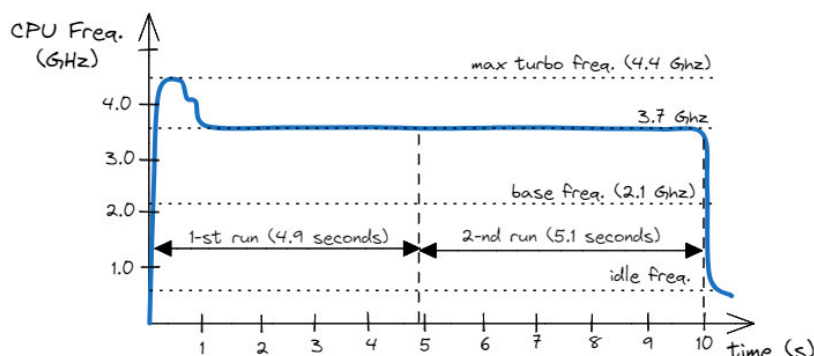


Figure 2.1: 由动态频率缩放引起的性能变化：第一次运行比第二次快 200 毫秒。

频率缩放是硬件功能可能导致测量变化的一个例子，然而，变化也可能来自软件。考虑对 `git status` 命令进行基准测试，该命令访问磁盘上的许多文件。在这种场景中，文件系统在性能中起着重要作用；特别是文件系统缓存。在第一次运行时，文件系统缓存中缺少所需的条目。文件系统缓存无效，我们的 `git status` 命令运行非常慢。然而，第二次文件系统缓存会预热，使其比第一次运行快得多。

你可能正在考虑在测量之前包含一次预运行。这确实有帮助，不幸的是，测量偏差也可能在运行中持续存在。[Mytkowicz et al., 2009] 论文证明，UNIX 环境大小（即存储环境变量所需的总字节数）或链接顺序（传递给链接器的目标文件顺序）可能以不可预测的方式影响性能。还有许多其他方式内存布局可能影响性能测量。¹⁰

具有一致性能需要以相同的条件运行基准测试的所有迭代。在基准测试的每次运行中实现 100% 一致的结果是不可能的，但也许你可以通过仔细控制环境来接近。消除系统中的不确定性有助于明确定义的、稳定的性能测试，例如微基准测试。

考虑当你实施了代码更改并希望通过基准测试程序的“之前”和“之后”版本来了解相对加速比的情况。在这种场景中，你可以控制系统中的大部分可变性，包括硬件配置、操作系统设置、后台进程等。禁用具有不确定性能影响的功能将帮助你获得更一致和准确的比较。你可以在附录 A 中找到此类功能的示例以及如何禁用它们。此外，还有一些工具可以设置环境以确保基准测试结果具有低方差；其中一个工具是 `temci`¹¹。

但是，不可能复制完全相同的环境并完全消除偏差：可能存在不同的温度条件、功率传输峰值、意外的系统中断等。追踪系统中所有潜在的噪声和变化源可能是一个永无止境的故事。有时无法实现，例如，当你对大型分布式云服务进行基准测试时。

⁹ 请记住，即使运行 Windows 任务管理器或 Linux `top` 程序也会影响测量，因为将激活额外的 CPU 核心并将其分配给它。这可能会影响运行实际基准测试的核心的频率。

¹⁰ [Curtisinger & Berger, 2013] 中介绍了一种启用统计上合理的性能分析的方法。这项工作表明，可以通过在运行时反复随机化代码、堆栈和堆对象的位置来消除来自内存布局的测量偏差。遗憾的是，这些想法没有进一步发展，目前这个项目几乎被放弃了。

¹¹ `Temci` - <https://github.com/partimenerd/temci>。

当你想要度量更改的真实世界性能影响时，你不应该消除系统不确定性行为。是的，这些功能可能导致性能不稳定，但它们旨在提高系统的整体性能。你的应用程序的用户很可能启用了它们以提供更好的性能。因此，当你分析生产应用程序的性能时，你应该尝试复制你正在优化的目标系统配置。向系统引入任何人工调优都会改变你的服务用户在实践中看到的结果。¹²

2.2 在生产环境中度量性能

当应用程序在共享基础设施中运行时，例如在公共云中，通常会有来自其他客户的工作负载在同一服务器上运行。随着虚拟化和容器等技术变得越来越流行，公共云提供商试图充分利用其服务器的容量。不幸的是，这为在此类环境中度量性能制造了额外的障碍。当你的应用程序与邻居进程共享资源时，其性能可能变得非常不可预测。

通过在实验室中重现特定场景来分析生产工作负载可能非常棘手。有时无法重现“内部”性能测试的确切行为。这就是为什么云提供商和超大规模提供商提供工具来直接在生产系统上监控性能。在实验室环境中表现良好的代码更改不一定在生产中总是表现良好。请咨询你的云服务提供商，了解如何启用生产实例的性能监控。我们在 Section 7.9 中概述了持续分析器。

大型服务提供商实施监控用户设备性能的遥测系统正在成为一种趋势。其中一个例子是 Netflix Icarus¹³ 遥测服务，它运行在遍布全球的数千种不同设备上。这样的遥测系统帮助 Netflix 了解用户如何感知 Netflix 应用程序的性能。它使 Netflix 工程师能够分析从许多设备收集的数据，并找到否则不可能找到的问题。这种数据能够就优化工作的重点做出更明智的决策。

监控生产部署的一个重要注意事项是测量开销。因为任何类型的监控都会影响运行服务的性能，我们建议使用轻量级分析方法。根据 [Ren et al., 2010]: “要在服务真实流量的数据中心机器上进行持续分析，极低的开销至关重要”。通常，可接受的聚合开销被认为低于 1%。通过限制分析的机器集以及更不频繁地捕获数据样本，可以减少性能监控开销。

2.3 持续基准测试

我们刚刚讨论了为什么你应该在生产环境中监控性能。另一方面，设置持续的“内部”测试仍然有益，可以在早期发现性能问题，即使并非每个性能回归都能在实验室中被捕获。

软件供应商不断寻求加速产品交付市场步伐的方法。许多公司每隔几个月或几周就部署新编写的代码。不幸的是，软件产品并不随着每个新版本而获得更好的性能。性能缺陷倾向于以惊人的速度泄露到生产软件中 [Jin et al., 2012]。大量的代码更改对其性能影响的彻底分析提出了挑战。

性能回归是使软件比前一版本运行更慢的缺陷。捕获性能回归（或改进）需要检测更改了程序性能的提交。从数据库系统到搜索引擎再到编译器，性能回归几乎被所有大型软件系统在其持续演进和部署生命周期中普遍经历。在软件开发过程中可能无法完全避免性能回归，但通过适当的测试和诊断工具，可以显著降低此类缺陷悄悄泄露到生产代码中的可能性。

使用图表跟踪应用程序的性能很有用，如图 fig. 2.2 所示。使用这样的图表，你可以看到历史趋势并找到性能改善或恶化的时刻。通常，你跟踪的每个性能测试都会有一条单独的线。不要在单个图表上包含太多基准测试，否则会显得非常嘈杂。

¹² 禁用不确定性性能功能的另一个缺点是它使基准测试运行时间更长。这在持续集成和持续交付（CI/CD）性能测试中尤为重要，因为整个基准测试套件的运行时间有限制。

¹³ 在 CMG 2019 上展示，https://www.youtube.com/watch?v=4RG2DUK03_0。

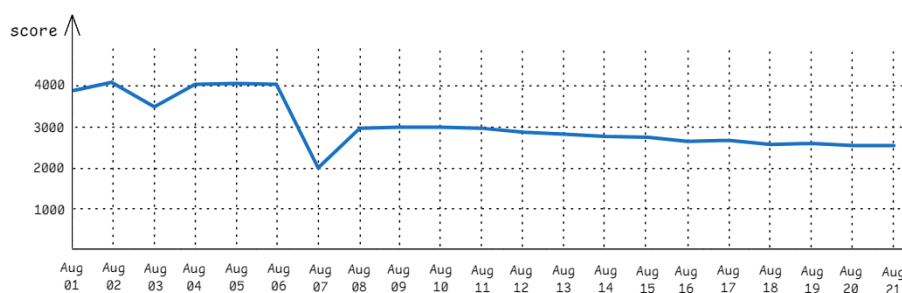


Figure 2.2: 应用程序的性能图（越高越好），显示 8 月 7 日性能大幅下降，之后下降较小。

让我们考虑一些检测性能回归的潜在解决方案。想到的第一个选项是：让人类查看图表。对于图 fig. 2.2 中的图表，人类可能会捕获 8 月 7 日发生的性能回归，但他们是否会检测到后来较小的回归并不明显。人们往往会很快失去注意力，可能会错过回归，尤其是在繁忙的图表上。除此之外，这是一项耗时且无聊的工作，必须每天执行。

8 月 3 日还有另一个有趣的性能下降。开发者也可能捕获它，但我们大多数人都会倾向于忽略它，因为性能在第二天恢复了。但我们确定那只是测量中的小故障吗？如果这是一个真实的回归，被 8 月 4 日的优化所补偿了呢？如果我们能修复回归并保留优化，我们的性能分数将达到 4500 左右。不要忽视这种情况。一种处理方法是重复 8 月 2 日至 8 月 4 日的测量，并检查该期间的代码更改。

第二个选项是设置一个阈值，比如 2%。每个性能在该阈值内的代码修改都被视为噪声，超过阈值的都被视为回归。这比第一个选项好一些，但仍有其缺点。性能测试中的波动是不可避免的：有时，即使无害的代码更改也可能触发基准测试中的性能变化。¹⁴ 为阈值选择正确的值极其困难，也不能保证低的误报和漏报率。将阈值设置得太低可能会导致分析一堆不是由源代码更改引起的小回归，而是由一些随机噪声引起的。将阈值设置得太高可能会导致过滤掉真正的性能回归。

小回归可以慢慢积累成更大的回归，这可能被忽视。回到图 fig. 2.2，注意从 8 月 11 日到 8 月 21 日持续的下降趋势。该期间以 3000 分开始，以 2600 分结束。这大约是 10 天内 15% 的回归，或平均每天 1.5%。如果我们设置 2% 的阈值，所有回归都会被过滤掉。然而，正如我们所看到的，累积回归远大于阈值。

尽管如此，这个选项对许多项目来说效果相当好，特别是如果基准测试中的噪声水平非常低。此外，你可以为每个测试调整阈值。一个持续集成 (CI) 系统的例子是 LUCI，¹⁵ 它是 Chromium 项目的一部分，每个测试都需要设置显式阈值来警报回归。

值得一提的是，随时间跟踪性能结果要求你维护用于运行基准测试的机器的相同配置。配置的更改可能会使所有先前的性能结果无效。你可能决定用新配置重新收集所有历史测量，但这非常昂贵。

另一个最近变得流行的选项使用统计方法来识别性能回归。它利用一种称为“变化点检测” (CPD, 参见 [Matteson & James, 2014]) 的算法，该算法利用历史数据并识别性能发生变化的时间点。许多性能监控系统采用了 CPD 算法，包括几个开源项目。你可以在网上搜索找到最适合你需求的一个。

¹⁴ 以下文章显示，更改函数顺序或删除死函数可能导致性能变化：https://easypref.net/blog/2018/01/18/Code_alignment_issues

¹⁵ LUCI - https://chromium.googlesource.com/chromium/src.git/+/_master/docs/tour_of_luci_ui.md

CPD 的显著优势是它不需要设置阈值。该算法评估大量近期结果窗口，使其能够将异常值视为噪声并产生更少的误报。CPD 的缺点是缺乏即时反馈。例如，考虑一个具有以下运行时间历史测量的性能测试：5 秒、6 秒、5 秒、5 秒、7 秒。如果下一个基准测试结果是 11 秒，那么可能会超过阈值并立即生成警报。但是，在使用 CPD 算法的情况下，此时不会做任何事情。如果在下一次运行中，性能恢复到 5 秒，那么它可能会将其视为误报而不生成警报。相反，如果下一次或两次运行分别产生 10 秒和 12 秒，那么 CPD 算法才会触发警报。

对于哪种方法更好没有明确的答案。如果你的开发流程需要即时反馈，例如在合并拉取请求之前评估它，那么使用阈值是更好的选择。此外，如果你可以从系统中消除大量噪声并获得稳定的性能结果，那么使用阈值更合适。在非常安静的系统中，前面提到的 11 秒测量可能表示真正的性能回归，因此我们需要尽早标记它。相反，如果你的系统中有很多噪声，例如你运行分布式宏基准测试，那么 11 秒的结果可能只是误报。在这种情况下，你最好使用变化点检测。

典型的 CI 性能跟踪系统应自动化以下操作：

1. 设置被测系统。
2. 运行基准测试套件。
3. 报告结果。
4. 确定性能是否发生变化。
5. 对意外的性能变化发出警报。
6. 可视化结果供人类分析。

CI 性能跟踪系统的另一个理想功能是允许开发者在将补丁提交到代码库之前为其提交性能评估作业。这大大简化了开发者的工作并促进了更快的实验周转。代码更改的性能影响通常包含在签入标准列表中。

如果由于某种原因性能回归已经滑入代码库，及时检测它非常重要。首先，因为自发生以来合并的更改较少。这让我们能够在负责回归的人转向另一个任务之前，让他们调查问题。此外，开发者接近回归要容易得多，因为所有细节在他们脑海中仍然清晰，而几周后就不是这样了。

最后，CI 系统应该不仅对软件性能回归发出警报，还应应对意外的性能改进发出警报。例如，有人可能签入一个看似无害的提交，该提交在自动跟踪工具中提高了 10% 的性能。你的第一反应可能是庆祝这个意外的性能提升并继续你的一天。但是，虽然此提交可能通过了 CI 管道中的所有功能测试，但这种意外改进可能揭示了功能测试中的一个缺口，该缺口仅在性能回归结果中表现出来。例如，该更改导致应用程序跳过了某些工作部分，而这些部分没有被功能测试覆盖。这种情况经常发生，值得明确提及：将自动性能回归工具视为整体软件测试框架的一部分。

总之，我们强烈建议设置自动化的统计性能跟踪系统。尝试使用不同的算法，看看哪种最适合你的应用程序。这肯定需要时间，但它将是项目未来性能健康的可靠投资。

2.4 手动性能测试

在上一节中，我们讨论了 CI 系统如何帮助评估代码更改的性能影响。然而，由于硬件不可用、设置对测试基础设施来说太复杂、需要收集额外指标等原因，可能并不总是能够利用这样的系统。在本节中，我们提供本地性能评估的基本建议。

我们通常通过以下方式度量代码更改的性能影响：1) 度量基线性能，2) 度量修改后程序的性能，3) 将它们相互比较。例如，我们有一个递归计算斐波那契数的程序（基线），我们决定用循环重写它（修改后）。两个版本在功能上都是正确的，并产生相同的斐波那契数。现在我们需要比较两个版本程序的性能。

强烈建议不仅获取单次测量，而是多次运行基准测试。如果你仅基于单次测量进行比较，你就会增加数字被我们在 Section 2.1 中讨论的测量偏差扭曲的风险。因此，我们为基线收集了 N 次性能测量，为修改后的程序版本收集了 N 次测量。我们称一组性能测量为性能分布。现在我们需要聚合和比较这两个分布，以决定哪个版本的程序更快。

比较两个性能分布最直接的方法是取两个分布中 N 次测量的平均值并计算比率。对于我们在本书中讨论的代码改进类型，这种简单方法在大多数情况下效果良好。然而，比较性能分布相当微妙，有许多方式可能被测量愚弄并可能得出错误的结论。我们不会深入探讨统计分析的细节，而是建议你阅读一本关于该主题的教科书。特别推荐给性能工程师的参考书是 Dror G. Feitelson 的《Workload Modeling for Computer Systems Performance Evaluation》，¹⁶ 其中有关于模态分布、偏斜度和其他相关主题的更多信息。

数据科学家通常通过绘图来呈现测量结果。这消除了有偏差的结论，并允许读者自己解释数据。绘制分布的流行方式之一是使用箱线图（也称为箱须图）。在图 fig. 2.3 中，我们可视化了同一功能程序两个版本（“之前”和“之后”）的性能分布。每个分布中有 70 个性能数据点。

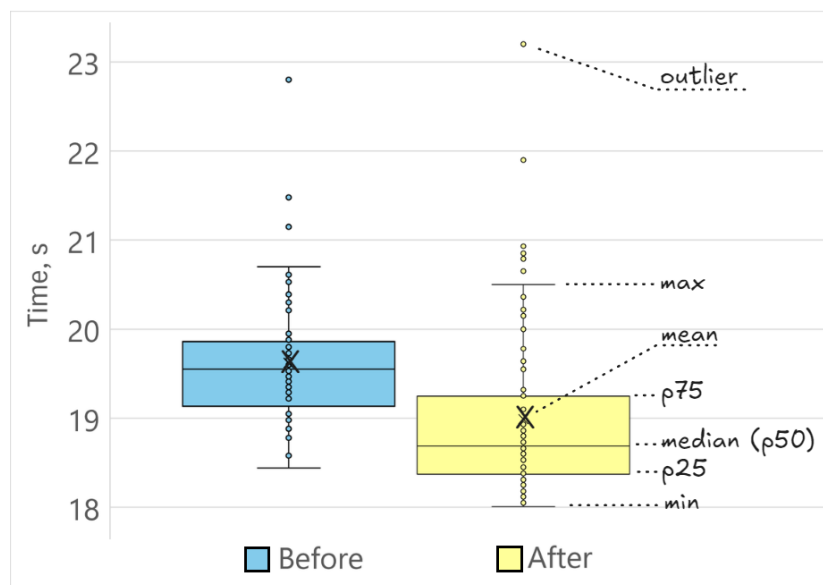


Figure 2.3: 以箱线图呈现的程序“之前”和“之后”版本的性能测量（越低越好）。

让我们描述图像上指示的术语：

- 平均值（通常称为平均数）是数据集中所有值的总和除以值的数量。用 X 表示。
- 中位数是数据集排序后的中间值。与第 50 百分位数 (p50) 相同。
- 第 25 百分位数 (p25) 将最低 25% 的数据与最高 75% 分开。
- 第 75 百分位数 (p75) 将最低 75% 的数据与最高 25% 分开。
- 异常值是与其他数据集中其他样本显著不同的数据点。异常值可能由数据中的可变性或实验误差引起。

¹⁶ 书籍《Workload Modeling for Computer Systems Performance Evaluation》 - <https://www.cs.huji.ac.il/~feit/wlmod/>

- 最小值和最大值（须线）代表不被视为异常值的最极端数据点。

通过查看图 fig. 2.3 中的箱线图，我们可以感觉到我们的代码更改对性能有积极影响，因为”之后”样本通常比”之前”快。但是，有一些”之前”测量比”之后”快。箱线图允许在同一个图表上比较多个分布。使用箱线图可视化性能分布的好处在 Stefan Marr 的博客文章中进行了描述。¹⁷

性能加速可以通过取两个平均值的比率来计算。在某些情况下，你可以使用其他指标来计算加速，包括中位数、最小值和第 95 百分位数，具体取决于哪个更能代表你的分布。

标准差量化数据集中的值平均偏离平均值的程度。低标准差表示数据点接近平均值，而高标准差表示它们分布在更广泛的范围内。除非分布具有低标准差，否则不要计算加速。如果测量中的标准差与平均值处于同一数量级，则平均值不是代表性指标。考虑采取措施减少测量中的噪声。如果不可能，请将结果显示为关键指标的组合，如平均值、中位数、标准差、百分位数、最小值、最大值等。

性能提升通常以两种方式表示：作为加速因子或百分比改进。如果一个程序最初需要 10 秒运行，你将其优化到 1 秒，那就是 10 倍加速。我们从原始程序中减少了 9 秒运行时间，那就是 90% 的时间减少。计算百分比改进的公式如下所示。在本书中，我们将使用两种方式来表示加速。

$$\text{百分比加速} = \left(1 - \frac{\text{新时间}}{\text{旧时间}}\right) \times 100\%$$

计算准确加速比最重要的因素之一是收集丰富的样本，即运行基准测试大量次数。这听起来可能很明显，但并不总是可以实现。例如，一些 SPEC CPU 2017 基准测试¹⁸ 在现代机器上运行超过 10 分钟。这意味着仅产生三个样本就需要 1 小时：每个版本的程序 30 分钟。想象一下，你的套件中不仅有一个基准测试，而是数百个。即使你将工作分配到多台机器上，收集统计上足够的数据也会变得非常昂贵。

如果获取新测量很昂贵，不要急于收集大量样本。通常你可以从仅仅三次运行中学到很多。如果你看到这三个样本中的标准差非常低，你可能从收集更多测量中学不到新东西。这对于具有底层一致性的程序（例如静态基准测试）来说非常典型。但是，如果你看到异常高的标准差，我不建议启动新运行并期望获得”更好的统计”。你应该弄清楚是什么导致了性能变化以及如何减少它。

在自动化设置中，你可以通过根据标准差动态限制基准测试迭代次数来实现自适应策略，即你收集样本直到获得落在特定范围内的标准差。分布中的标准差越低，你需要的样本数量越少。一旦你获得低于阈值的标准差，你就可以停止收集测量。这种策略在 [Akinshin, 2019, Chapter 4] 中有更详细的解释。

需要注意的一件事是异常值的存在。丢弃一些样本（例如冷运行）作为异常值是可以的，但不要故意从测量集中丢弃不需要的样本。异常值可能是某些类型基准测试最重要的指标之一。例如，当对具有实时约束的软件进行基准测试时，第 99 百分位数可能非常有趣。

我建议使用基准测试工具，因为它们自动化性能测量。例如，Hyperfine¹⁹ 是一个流行的跨平台命令行基准测试工具，可以自动确定运行次数，并将结果可视化为包含平均值、最小值、最大值的表格或箱线图。

¹⁷ Stefan Marr 关于箱线图的博客文章 - <https://stefan-marr.de/2024/06/5-reasons-for-box-plots-as-default/>

¹⁸ SPEC CPU 2017 基准测试 - <http://spec.org/cpu2017/Docs/overview.html#benchmarks>

¹⁹ hyperfine - <https://github.com/sharkdp/hyperfine>

在接下来的两节中，我们将讨论如何度量挂钟时间（延迟），这是最常见的情况。但是，有时我们也可能想要度量其他内容，如每秒请求数（吞吐量）、堆分配、上下文切换等。

2.5 软件和硬件计时器

为了对执行时间进行基准测试，工程师通常使用两种不同的计时器，所有现代平台都提供：

- **系统级高分辨率计时器**：这是一个系统计时器，通常实现为自称为纪元²⁰的任意起始日期以来经过的刻度的简单计数。这个时钟是单调的；即它总是向上走。系统时间可以通过系统调用从操作系统获取。在 Linux 系统上访问系统计时器可以通过 `clock_gettime` 系统调用。系统计时器具有纳秒分辨率，在所有 CPU 之间一致，并且独立于 CPU 频率。尽管系统计时器可以返回纳秒精度的时间戳，但它不适用于度量短时间运行的事件，因为通过 `clock_gettime` 系统调用获取时间戳需要很长时间。但对于持续时间超过一微秒的事件是可以的。在 C++ 中访问系统计时器的标准方法是使用 `std::chrono`，如 Listing ?? 所示。

Listing: 使用 C++ `std::chrono` 访问系统计时器

```
#include <cstdint>
#include <chrono>

// 返回以纳秒为单位的经过时间
uint64_t timeWithChrono() {
    using namespace std::chrono;
    auto start = steady_clock::now();
    // 运行某些内容
    auto end = steady_clock::now();
    uint64_t delta = duration_cast<nanoseconds>(end - start).count();
    return delta;
}
```

- **时间戳计数器 (TSC)**：这是一个实现为硬件寄存器的硬件计时器。TSC 是单调的并且具有恒定速率，即它不考虑频率变化。每个 CPU 都有自己的 TSC，它只是经过的参考周期数（参见 Section 4.6）。它适用于度量持续时间从纳秒到一分钟的短事件。在 x86 平台上，TSC 的值可以通过使用编译器的内置函数 `__rdtsc` 来获取，如 Listing ?? 所示，它在底层使用 RDTSC 汇编指令。有关使用 RDTSC 汇编指令对代码进行基准测试的更多低级详细信息，请参见白皮书 [Paoloni, 2010]。在 ARM 平台上，你可以读取 CNTVCT_EL0，计数器-定时器虚拟计数寄存器。

Listing: 使用 `__rdtsc` 编译器内置函数访问 TSC

```
#include <x86intrin.h>
#include <cstdint>

// 返回经过的参考时钟数
uint64_t timeWithTSC() {
    uint64_t start = __rdtsc();
```

²⁰ Unix 纪元从 1970 年 1 月 1 日 00:00:00 UT 开始: https://en.wikipedia.org/wiki/Unix_epoch.

```
// 运行某些内容
return __rdtsc() - start;
}
```

选择使用哪个计时器非常简单，取决于你要度量的东西的持续时间。如果你度量非常短时间内的内容，TSC 将给你更好的精度。相反，使用 TSC 度量运行数小时的程序是没有意义的。除非你需要周期精度，否则系统计时器对于大多数情况应该足够了。重要的是要记住，访问系统计时器通常比访问 TSC 具有更高的延迟。进行 `clock_gettime` 系统调用可能比执行 `RDTSC` 指令慢得多。后者大约需要 5 纳秒 (20 个 CPU 周期)，而前者大约需要 500 纳秒。这对于最小化测量开销可能很重要，特别是在生产环境中。CppPerformanceBenchmarks 仓库的 wiki 页面上提供了不同平台上访问计时器的不同 API 的性能比较。²¹

2.6 微基准测试

微基准测试是人们编写的小型自包含程序，用于快速测试假设。通常，微基准测试用于选择某个相对较小的算法或功能的最佳实现。几乎所有现代语言都有基准测试框架。在 C++ 中，你可以使用 Google benchmark²² 库，C# 有 BenchmarkDotNet²³ 库，Julia 有 BenchmarkTools²⁴ 包，Java 有 JMH²⁵ (Java 微基准测试工具)，Rust 有 Criterion²⁶ 包等。

编写微基准测试时，确保你想测试的场景在运行时确实由你的微基准测试执行非常重要。优化编译器可能会消除重要的代码，使实验无用，甚至更糟，引导你得出错误的结论。在下面的示例中，现代编译器可能会消除整个循环：

```
// foo 不对字符串创建进行基准测试
void foo() {
    for (int i = 0; i < 1000; i++)
        std::string s("hi");
}
```

像这样的错误在论文“Always Measure One Level Deeper” [Ousterhout, 2018] 中得到了很好的捕捉，作者倡导更科学的方法，并从不同角度度量性能。遵循论文的建议，我们应该检查基准测试的性能概况，并确保预期的代码作为热点突出显示。有时可以立即发现异常计时，因此在分析和比较基准测试运行时使用常识。

防止编译器优化掉重要代码的流行方法之一是使用 `DoNotOptimize-like`²⁷ 辅助函数，它们在底层执行必要的内联汇编魔法：

```
// foo 对字符串创建进行基准测试
void foo() {
    for (int i = 0; i < 1000; i++) {
        std::string s("hi");
    }
}
```

²¹ CppPerformanceBenchmarks wiki - <https://gitlab.com/chriscox/CppPerformanceBenchmarks/-/wikis/ClockTimeAnalysis>

²² Google benchmark 库 - <https://github.com/google/benchmark>

²³ BenchmarkDotNet - <https://github.com/dotnet/BenchmarkDotNet>

²⁴ Julia BenchmarkTools - <https://github.com/JuliaCI/BenchmarkTools.jl>

²⁵ Java Microbenchmark Harness - <http://openjdk.java.net/projects/code-tools/jmh/etc>

²⁶ Criterion.rs - <https://github.com/bheisler/criterion.rs>

²⁷ 对于 JMH，这被称为 `Blackhole.consume()`。

```

    DoNotOptimize(s);
  }
}

```

如果编写得当，微基准测试可以成为性能数据的良好来源。它们通常用于比较关键函数的不同实现的性能。好的基准测试在现实条件下测试性能。相反，如果基准测试使用与实践不同的合成输入，那么基准测试可能会误导你并引导你得出错误的结论。除此之外，当基准测试在没有其他高需求进程的系统上运行时，它拥有所有可用资源，包括 DRAM 和缓存空间。这样的基准测试可能会拥护更快版本的函数，即使它比另一个版本消耗更多内存。但是，如果有邻居进程消耗了 DRAM 的很大一部分，导致属于基准测试进程的内存区域被交换到磁盘，结果可能会相反。

出于同样的原因，在得出从函数单元测试获得的结果时要小心。现代单元测试框架（如 GoogleTest）提供每个测试的持续时间。但是，这些信息不能替代精心编写的基准测试，该基准测试在实际条件下使用现实输入测试函数（更多内容请参见 [Fog, 2023, chapter 16.2]）。并不总是能够复制与实践完全相同的输入和环境，但这是开发者在编写好的基准测试时应该考虑的。

2.7 主动基准测试

正如你在前面的章节中看到的，度量性能是一项复杂的任务，沿途有许多陷阱。作为人类，我们倾向于欢迎有利的结果并忽略不利的结果。这经常导致以“运行即忘”的风格进行基准测试，没有额外的分析，并忽略了任何潜在的问题。以这种方式进行的测量可能不完整、具有误导性甚至错误。考虑以下两种场景：

- 开发者 A 在团队会议上：“如果我们在整个 C++ 代码库中的类声明中添加 `final` 关键字，它将使我们的代码快 5%，某些测试显示高达 30% 的加速。”
- 开发者 B 在下一次团队会议上：“我仔细研究了在类声明中添加 `final` 关键字的性能影响。首先，我执行了更长的测试，没有测量到大于 5% 的加速。最初观察到的 30% 加速是由测试不稳定性引起的异常值。我还注意到用于测量的两台机器配置不同：虽然 CPU 相同，但其中一台机器具有更快的内存模块。我在同一台机器上重新运行了测试，观察到 1% 以内的性能差异。我比较了更改前后的生成机器代码，没有发现显著差异。此外，我还比较了执行的指令数、缓存未命中、页面错误、上下文切换等，没有发现任何异常。此时，我得出结论，与我们可以进行的其他优化相比，`final` 关键字的性能影响可以忽略不计。”

开发者 A 进行的基准测试是被动的。结果在没有任何技术解释的情况下呈现，并且性能影响被夸大了。相比之下，开发者 B 执行了主动基准测试。²⁸ 她确保了适当的机器配置，进行了广泛的测试，深入研究了一层，并收集了尽可能多的指标来支持她的结论。她的分析解释了她观察到的性能结果的底层技术原因。

你应该有良好的直觉来发现可疑的基准测试结果。每当你看到呈现看起来好得令人难以置信且没有任何技术解释的基准测试结果的出版物时，你应该持怀疑态度。呈现测量结果没有错，但正如 John Ousterhout 所说，“性能测量在被证明无罪之前应被视为有罪。”[Ousterhout, 2018] 验证结果的最好方法是通过主动基准测试。主动基准测试比被动基准测试需要更多的努力，但它是获得可靠结果的唯一方法。

思考与练习

1. 用一系列测量值的平均值来确定程序运行时间，这样做总是安全的吗？有哪些潜在的陷阱？

²⁸ Brendan Gregg 创造术语 - <https://www.brendangregg.com/activebenchmarking.html>.

2. 假设你已经定位了一个性能缺陷，现在正在开发环境中修复它。你会如何降低系统噪声，以获得更明显的基准测试结果？
3. 用函数级单元测试来追踪程序的整体性能，这样做合适吗？
4. 你的组织是否已经部署了性能回归追踪系统？如果有，它能否进一步改进？如果没有，请思考如何构建一套这样的系统。需要考虑的因素包括：哪些部分会变化、哪些不会（源代码、编译器、硬件配置等），变化发生的频率，测量方差有多大，基准测试套件的运行时间有多长，以及你能运行多少次迭代。

本章小结

- 现代系统的性能具有不确定性。消除系统中的不确定性有助于获得定义明确、稳定的性能测试结果，例如微基准测试 (Microbenchmark)。
- 在生产环境中测量性能是评估用户对服务响应速度感知的必要手段。然而，这需要应对嘈杂的环境，并使用统计方法来分析结果。
- 部署自动化的性能追踪系统可以有效防止性能回归泄漏到生产软件中。这类 CI 系统应能运行自动化性能测试、可视化结果，并在发现性能异常时发出告警。
- 可视化性能分布有助于比较性能结果。这是向广大受众展示性能结果的安全方式。
- 为了测量执行时间，工程师可以使用两种不同的计时器：系统级高精度计时器和时间戳计时器 (Time Stamp Counter)。前者适合测量持续时间超过一微秒的事件，后者可以高精度地测量短时间事件。
- 微基准测试适合快速实验，但你始终应该在实际应用和真实条件下验证你的想法。通过检查性能分析结果，确保你测量的是正确的代码。
- 始终深入一层测量，收集尽可能多的指标来支撑你的结论，并准备好为你观察到的性能结果解释底层技术原因。

3 CPU 微架构

本章简要总结了对软件性能有直接影响的关键 CPU 微架构特性。本章的目标不是涵盖 CPU 架构的所有细节和权衡，这些在文献 [Hennessy & Patterson, 2017]、[Shen & Lipasti, 2013] 中已有广泛介绍。我提供了现代处理器中特性的回顾，为读者准备本书接下来的内容。

3.1 指令集架构

指令集架构 (ISA) 是软件和硬件之间的契约，定义了通信规则。Intel x86-64、²⁹ Armv8-A 和 RISC-V 是当前广泛部署的 ISA 的例子。所有这些是 64 位架构，即所有地址计算使用 64 位。ISA 开发者和 CPU 架构师通常确保符合规范的软件或固件将在使用该规范构建的任何处理器上执行。广泛部署的 ISA 通常还确保向后兼容性，以便为处理器 GenX 版本编写的代码将继续在 GenX+i 上执行。

大多数现代架构可以归类为通用基于寄存器的加载-存储架构，如 RISC-V 和 ARM，其中操作数被显式指定，并且仅使用加载和存储指令访问内存。X86 ISA 是寄存器-内存架构，其中操作可以在寄存器以及内存操作数上执行。除了在 ISA 中提供基本功能（如使用整数和浮点数的加载、存储、控制和标量算术运算）之外，广泛部署的架构继续增强其 ISA 以支持新的计算范式。这些包括增强的向量处理指令（例如 Intel AVX2、AVX512、ARM SVE、RISC-V “V” 向量扩展）和矩阵/张量指令 (Intel AMX、ARM SME)。广泛使用这些高级指令的应用程序通常会看到显著的性能提升。

现代 CPU 支持浮点和整数算术运算的 32 位和 64 位精度。随着机器学习和 AI 领域的快速发展，业界对替代数字格式重新产生了兴趣，以推动显著的性能改进。研究表明，机器学习模型使用更少的位数来表示变量时表现同样好，节省了计算和内存带宽。因此，大多数主流 ISA 最近添加了对较低精度数据类型的支持，如 8 位和 16 位整数和浮点类型 (int8、fp8、fp16、bf16)，以及传统的 32 位和 64 位算术运算格式。

3.2 流水线

流水线是用于使 CPU 变快的基础技术，其中多条指令在执行期间重叠。CPU 中的流水线从汽车装配线中汲取灵感。指令的处理被分为阶段。这些阶段并行运行，处理不同指令的不同部分。DLX 是 John L. Hennessy 和 David A. Patterson 于 1994 年设计的相对简单的架构。如 [Hennessy & Patterson, 2017] 中定义的，它有一个 5 级流水线，包括：

1. 指令取指 (IF)
2. 指令解码 (ID)
3. 执行 (EXE)
4. 内存访问 (MEM)
5. 写回 (WB)

图 fig. 3.1 显示了 5 级流水线 CPU 的理想流水线视图。在周期 1 中，指令 x 进入流水线的 IF 阶段。在下一个周期中，当指令 x 移动到 ID 阶段时，程序中的下一条指令进入 IF 阶段，依

²⁹ 在本书中，我经常简称为 x86，但我假设是 x86-64，这是 x86 指令集的 64 位版本，于 1999 年首次发布。此外，我使用 ARM 指代 ISA，使用 Arm 指代公司。

Instruction	Clock cycle								
	1	2	3	4	5	6	7	8	9
Instruction x	IF	ID	EXE	MEM	WB				
Instruction x+1		IF	ID	EXE	MEM	WB			
Instruction x+2			IF	ID	EXE	MEM	WB		
Instruction x+3				IF	ID	EXE	MEM	WB	
Instruction x+4					IF	ID	EXE	MEM	WB

Figure 3.1: 简单的 5 级流水线图。

此类推。一旦流水线填满，如上面的周期 5 所示，CPU 的所有流水线阶段都忙于处理不同的指令。没有流水线，指令 $x+1$ 在指令 x 完成其工作之前无法开始执行。

现代高性能 CPU 有多个流水线阶段，通常范围从 10 到 20（有时更多），具体取决于架构和设计目标。这涉及到比前面介绍的简单 5 级流水线复杂得多的设计。例如，解码阶段可能被分成几个新阶段。我们还可以在执行阶段之前添加新阶段来缓冲解码后的指令，等等。

流水线 CPU 的吞吐量定义为每单位时间完成并退出流水线的指令数。任何给定指令的延迟是通过流水线所有阶段的总时间。由于流水线的阶段链接在一起，每个阶段必须准备好以锁定步调移动到下一条指令。将指令从一个阶段移动到下一个阶段所需的时间定义了 CPU 的基本机器周期或时钟。为给定时钟选择的值由流水线中最慢的阶段定义。CPU 硬件设计者努力平衡阶段中可以完成的工作量，因为这直接影响 CPU 的操作频率。

在实际实现中，流水线引入了几个限制，限制了图 fig. 3.1 中所示的流畅执行。流水线冒险阻止了理想的流水线行为，导致停顿。冒险的三类是结构冒险、数据冒险和控制冒险。幸运的是，对于程序员来说，在现代 CPU 中，所有类型的冒险都由硬件处理。

- **结构冒险**：由资源冲突引起，即当两条指令竞争同一资源时。这种冒险的一个例子是当两条 32 位加法指令准备好在同一周期中执行，但该周期中只有一个执行单元可用。在这种情况下，我们需要选择执行两条指令中的哪一条，哪一条将在下一个周期执行。在很大程度上，可以通过复制硬件资源来消除它们，例如使用多个执行单元、指令解码器、多端口寄存器文件等。然而，这在硅面积和功耗方面可能变得相当昂贵。
- **数据冒险**：由程序中的数据依赖引起，分为三种类型：

写后读（RAW）冒险要求依赖读在写之后执行。当指令 $x+1$ 在先前指令 x 写入源之前读取源时发生，导致读取错误的值。CPU 实现从流水线的较后阶段到较早阶段的数据转发（称为“旁路”），以减轻与 RAW 冒险相关的惩罚。其思想是指令 x 的结果可以在指令 x 完全完成之前转发给指令 $x+1$ 。如果我们看一下示例：

```
R1 = R0 ADD 1
R2 = R1 ADD 2
```

寄存器 R1 存在 RAW 依赖。如果我们直接在加法 `R0 ADD 1` 完成后取值（从 EXE 流水线阶段），我们不需要等待 WB 阶段完成（当值将被写入寄存器文件时）。旁路有助于节省几个周期。流水线越长，旁路越有效。

写后读 (WAR) 冒险要求依赖写在读之后执行。当指令在较早指令读取源之前写入寄存器时发生, 导致读取错误的新值。WAR 冒险不是真正的依赖, 可以通过称为寄存器重命名的技术消除。这是一种将逻辑寄存器与物理寄存器抽象分离的技术。CPU 通过保持大量物理寄存器来支持寄存器重命名。逻辑 (架构) 寄存器 (由 ISA 定义的) 只是更宽寄存器文件上的别名。通过这种架构状态的解耦, 解决 WAR 冒险很简单: 我们只需要为写操作使用不同的物理寄存器。例如:

<pre> ; 机器代码, WAR 冒险 ; (架构寄存器) R1 = R0 ADD 1 1 R0 = R2 ADD 2 2 </pre>	=>	<pre> ; 寄存器重命名后 ; (物理寄存器) R101 = R100 ADD R103 = R102 ADD </pre>
---	----	--

在原始汇编代码中, 寄存器 R0 存在 WAR 依赖。对于左边的代码, 我们不能重新排序指令的执行, 因为它可能会在 R1 中留下错误的值。但是, 我们可以利用我们大量的物理寄存器来克服这个限制。为此, 我们需要从写操作 (R0 = R2 ADD 2) 开始重命名 R0 寄存器的所有出现, 并使用空闲寄存器。重命名后, 我们给这些寄存器分配与物理寄存器对应的新名称, 比如 R103。通过重命名寄存器, 我们消除了初始代码中的 WAR 冒险, 我们可以安全地以任何顺序执行两个操作。

写后写 (WAW) 冒险要求依赖写在写之后执行。当指令在较早指令写入同一寄存器之前写入该寄存器时发生, 导致存储错误的值。WAW 冒险也通过寄存器重命名消除, 允许两个写操作以任何顺序执行, 同时保留正确的最终结果。下面是消除 WAW 冒险的示例。

<pre> ; 机器代码, WAW 冒险 ; (架构寄存器) R1 = R0 ADD 1 1 R2 = R1 SUB R3 ; RAW R103 ; RAW R1 = R0 MUL 3 ; WAW 和 WAR </pre>	=>	<pre> ; 寄存器重命名后 ; (物理寄存器) R101 = R100 ADD R102 = R101 SUB R104 = R100 MUL 3 </pre>
---	----	--

你会在许多生产程序中看到类似的代码。在我们的示例中, R1 保存 ADD 操作的临时结果。一旦 SUB 指令完成, R1 立即被重用以存储 MUL 操作的结果。左边的原始代码具有所有三种类型的数据依赖。ADD 和 SUB 之间存在 R1 上的 RAW 依赖, 并且它必须在寄存器重命名后保留。此外, MUL 操作的同一 R1 寄存器上存在 WAW 和 WAR 冒险。同样, 我们需要重命名寄存器来消除这两个冒险。注意, 寄存器重命名后, MUL 操作有一个新的目标寄存器 (R104)。现在我们可以安全地将 MUL 与其他两个操作重新排序。

- **控制冒险:** 由程序流的变化引起。它们来自对改变程序流的分支和其他指令的流水线化。确定分支方向 (已采取 vs. 未采取) 的分支条件在执行流水线阶段中解析。因此, 除非控制冒险被消除, 否则下一条指令的取指不能流水线化。下一节中描述的动态分支预测和推测执行等技术用于减轻控制冒险。

3.3 利用指令级并行 (ILP)

程序中的大多数指令都适合流水线化和并行执行, 因为它们是独立的。现代 CPU 实现了许多额外的硬件功能来利用这种指令级并行 (ILP), 即单个指令流内的并行性。与先进的编译器技术协同工作, 这些硬件功能提供了显著的性能改进。

3.3.1 乱序 (OOO) 执行

图 fig. 3.1 中的流水线示例显示所有指令按顺序通过流水线的不同阶段，即按它们在程序中出现的顺序，也称为程序顺序。大多数现代 CPU 支持乱序 (OOO) 执行，其中顺序指令可以以任意顺序进入执行阶段，仅受其依赖性和资源可用性的限制。具有 OOO 执行的 CPU 必须仍然提供与所有指令按程序顺序执行相同的结果。

指令在最终执行后被称为退休，其结果正确且在架构状态中可见。为确保正确性，CPU 必须按程序顺序退休所有指令。OOO 执行主要用于避免因依赖性引起的停顿而导致的 CPU 资源利用不足，特别是在超标量引擎中，我们稍后将讨论。

Instruction	Clock cycle									
	1	2	3	4	5	6	7	8	9	10
Instruction x	IF	ID	EXE	MEM	WB					
Instruction x+1		IF	ID			EXE	MEM	WB		
Instruction x+2			IF	ID	EXE	MEM			WB	
Instruction x+3				IF	ID		EXE	MEM		WB

Figure 3.2: 乱序执行的概念。

图 fig. 3.2 说明了乱序执行的概念。假设指令 x+1 由于某些冲突在周期 4 和 5 期间无法执行。顺序 CPU 将阻止所有后续指令进入 EXE 流水线阶段，因此指令 x+2 只能在周期 7 开始执行。在具有 OOO 执行的 CPU 中，只要指令没有任何冲突（例如其输入可用、执行单元未被占用等），它就可以开始执行。如图所示，指令 x+2 在指令 x+1 之前开始执行。指令 x+3 不能在周期 6 进入 EXE 阶段，因为它已被指令 x+1 占用。所有指令仍然按顺序退休，即指令按程序顺序完成 WB 阶段。

OOO 执行通常比顺序执行带来显著的性能改进。但是，它也引入了额外的复杂性和功耗。

重新排序指令的过程通常称为指令调度。调度的目标是最小化流水线冒险并最大化 CPU 资源利用率的方式发出指令。指令调度可以在编译时完成（静态调度），也可以在运行时完成（动态调度）。让我们详细讨论这两种选择。

3.3.1.1 静态调度 Intel Itanium 是静态调度的一个例子。通过超标量、多执行单元机器的静态调度，调度从硬件转移到编译器，使用一种称为 VLIW（超长指令字）的技术。其基本原理是通过要求编译器选择正确的指令组合来保持机器完全利用，从而简化硬件。编译器可以使用软件流水线和循环展开等技术，比硬件结构合理支持的更远地前瞻，以找到正确的 ILP。

Intel Itanium 从未取得成功，原因有几个。其中之一是与 x86 代码缺乏兼容性。另一个事实是，由于可变的加载延迟，编译器难以保持 CPU 忙碌的方式调度指令。x86 ISA 的 64 位扩展 (x86-64) 由 AMD 在同一时间窗口引入，与 IA-32 (x86 ISA 的 32 位版本) 兼容，并最终成为其真正的后继者。最后的 Intel Itanium 处理器于 2021 年出货。

3.3.1.2 动态调度 为了克服静态调度的问题，现代处理器使用动态调度。动态调度的两个最重要的算法是 Scoreboarding^[4] 和 Tomasulo 算法。^[5]

Scoreboarding 最早在 1960 年代的 CDC6600 处理器中实现。它的主要缺点是它不仅保留真正的依赖 (RAW)，还保留虚假依赖 (WAW 和 WAR)，因此它提供了次优的 ILP。虚假依赖是由少量的架构寄存器引起的，在现代 ISA 中通常在 16 到 32 之间。这就是为什么所有现代处理器都采用 Tomasulo 算法进行动态调度。Tomasulo 算法由 Robert Tomasulo 在 1960 年代发明，首先在 IBM360 Model 91 中实现。

为了消除虚假依赖，Tomasulo 算法利用了我们在上一节讨论的寄存器重命名。因此，与 Scoreboarding 相比，性能得到了极大提高。但是，携带 RAW 依赖的指令序列（也称为依赖链）对于 OOO 执行仍然存在问题，因为在寄存器重命名后 ILP 没有增加，因为所有 RAW 依赖都被保留。依赖链通常在循环中找到（循环携带依赖），其中当前循环迭代依赖于前一次迭代产生的结果。

实现动态调度的另外两个关键组件是重排序缓冲区 (ROB) 和保留站 (RS)。ROB 是一个循环缓冲区，跟踪每条指令的状态，在现代处理器中，它有几百个条目。通常，ROB 的大小决定了硬件可以独立调度指令的距离。指令按程序顺序插入 ROB，可以乱序执行，并按程序顺序退休。当指令放入 ROB 时进行寄存器重命名。

从 ROB 中，指令被插入到条目少得多的 RS 中。一旦指令进入 RS，它们就等待其输入操作数变为可用。当输入可用时，指令可以被发出到相应的执行单元。因此，一旦操作数变得可用，指令就可以按任何顺序执行，不再绑定到程序顺序。现代处理器变得更宽（可以在一个周期中执行更多指令）和更深（更大的 ROB、RS 和其他缓冲区），这表明在生产应用程序中有很多潜力来发现更多 ILP。

3.3.2 超标量引擎

大多数现代 CPU 是超标的，即它们可以在任何给定周期中发出多条指令。发射宽度是在同一周期内可以发出的最大指令数（参见 Section 4.5）。2024 年主流 CPU 的典型发射宽度范围从 6 到 9。为了确保正确的平衡，这种超标量引擎也有多个执行单元和/或流水线执行单元。CPU 还将超标量能力与深流水线和乱序执行相结合，以从给定的软件中提取最大的 ILP。

Instruction	Clock cycle					
	1	2	3	4	5	6
Instruction x	IF	ID	EXE	MEM	WB	
Instruction x+1	IF	ID	EXE	MEM	WB	
Instruction x+2		IF	ID	EXE	MEM	WB
Instruction x+3		IF	ID	EXE	MEM	WB

Figure 3.3: 在 2 路超标量 CPU 中执行的代码的流水线图。

图 fig. 3.3 显示了支持 2 宽发射的 CPU 的流水线图。注意，每个周期中可以在流水线的每个阶段处理两条指令。例如，指令 x 和 x+1 都在周期 3 期间开始执行。这可以是两条相同类型的指令（例如两条加法）或两条不同的指令（例如一条加法和一条分支）。超标量处理器复制执行资源，以保持流水线中的指令在没有结构冲突的情况下流动。例如，为了支持同时解码两条指令，我们需要 2 个独立的解码器。

3.3.3 推测执行

如上一节所述，如果指令在分支条件解决之前被停顿，控制冒险可能导致流水线中的显著性能损失。避免这种性能损失的一种技术是硬件分支预测。使用这种技术，CPU 预测分支的可能目标，并开始从预测路径执行指令（称为推测执行）。

让我们考虑 Listing ?? 中的一个示例。为了使处理器理解接下来应该执行哪个函数，它应该知道条件 $a < b$ 是假还是真。在不知道的情况下，CPU 等待直到分支指令的结果确定，如图 fig. ?? 所示。

Listing: 推测执行

```
if (a < b)
    foo();
else
    bar();
```

3.4 SIMD 多处理器

另一种促进并行处理的技术称为单指令多数据 (SIMD)，它在几乎所有高性能处理器中都有使用。顾名思义，在 SIMD 处理器中，单条指令在单个周期内使用多个独立的功能单元对多个数据元素进行操作。向量和矩阵上的操作非常适合 SIMD 架构，因为向量或矩阵的每个元素都可以使用相同的指令进行处理。SIMD 架构能够更高效地处理大量数据，最适合涉及向量操作的数据并行应用程序。

图 fig. 3.4 显示了 Listing ?? 中代码的标量和 SIMD 执行模式。在传统的单指令单数据 (SISD) 模式（也称为标量模式）中，加法操作分别应用于数组 a 和 b 的每个元素。然而，在 SIMD 模式下，加法同时应用于多个元素。如果我们针对具有能够对 256 位向量执行操作的执行单元的 CPU 架构，我们可以用单条指令处理四个双精度元素。这导致发出 4 倍少的指令，并可能获得比四个标量计算 4 倍的加速。

Listing: SIMD 执行

```
double *a, *b, *c;
for (int i = 0; i < N; ++i) {
    c[i] = a[i] + b[i];
}
```

对于常规整数 SISD 指令，处理器使用通用寄存器。类似地，对于 SIMD 指令，CPU 有一组 SIMD 寄存器来保存从内存加载的数据和存储计算的中间结果。在我们的示例中，对应于数组 a 和 b 的两个 256 位连续数据区域将从内存加载并存储在两个单独的向量寄存器中。接下来，将进行逐元素加法，结果将存储在一个新的 256 位向量寄存器中。最后，结果将从向量寄存器写入对应于数组 c 的 256 位内存区域。注意，数据元素可以是整数或浮点数。

向量执行单元在逻辑上被分为通道。在 SIMD 的上下文中，通道是指 SIMD 执行单元内的不同数据通路，并处理向量的一个元素。在我们的示例中，每个通道处理 64 位元素（双精度），因此 256 位寄存器中将有 4 个通道。

大多数流行的 CPU 架构都具有向量指令，包括 x86、PowerPC、ARM 和 RISC-V。1996 年 Intel 发布了 MMX，这是一种为多媒体应用程序设计的 SIMD 指令集。继 MMX 之后，Intel 引入了具有附加功能和增加向量大小的新指令集：SSE、AVX、AVX2 和 AVX-512。

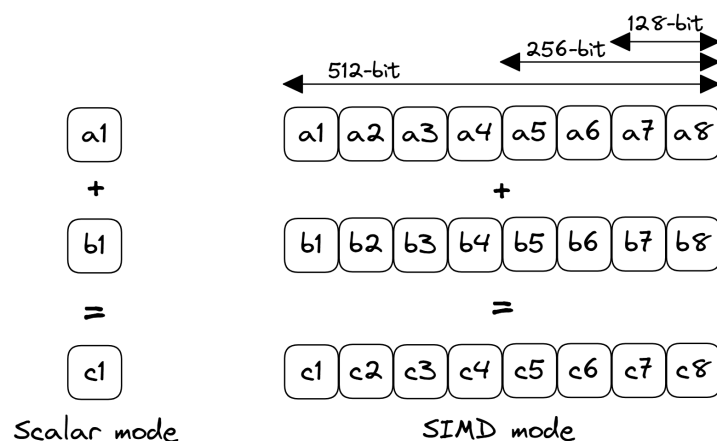


Figure 3.4: 标量和 SIMD 操作示例。

ARM 在其架构的各种版本中可选地支持 128 位 NEON 指令集。在版本 8 (aarch64) 中, 此支持变为强制性的, 并添加了新指令。

随着新指令集的可用, 开始使它们对软件工程师可用。利用 SIMD 指令所需的软件更改称为代码向量化。最初, SIMD 指令是用汇编语言编程的。后来, 引入了特殊的编译器内置函数, 这些是提供与 SIMD 指令一对一映射的小函数。今天, 所有主要编译器都支持流行处理器的自动向量化, 即它们可以直接从用 C/C++、Java、Rust 和其他语言编写的高级代码生成 SIMD 指令。

为了使代码能够在支持不同向量长度的系统上运行, Arm 引入了 SVE 指令集。它的定义特征是可变伸缩向量的概念: 它们的长度在编译时未知。使用 SVE, 无需将软件移植到每个可能的向量长度。当较新的 CPU 代中提供更宽的向量时, 用户不必重新编译其应用程序的源代码来利用它们。可变伸缩向量的另一个示例是 RISC-V V 扩展 (RVV), 于 2021 年底获得批准。一些实现支持相当宽 (2048 位) 的向量, 最多可以将八个组合在一起以产生 16384 位向量, 这大大减少了执行的指令数。在每次循环迭代中, SVE 代码通常执行 `ptr += number_of_lanes`, 其中 `number_of_lanes` 在编译时未知。ARM SVE 提供特殊指令用于此类长度相关的操作, 而 RVV 使程序员能够查询/设置 `number_of_lanes`。

回到 Listing ?? 中的示例, 如果 `N` 等于 5, 我们有一个 256 位向量, 我们无法在单次迭代中处理所有元素。我们可以使用单条 SIMD 指令处理前四个元素, 但第 5 个元素需要单独处理。这称为循环余数。循环余数是必须处理少于向量宽度的元素的循环部分, 需要额外的标量代码来处理剩余元素。可变伸缩向量 ISA 扩展没有这个问题, 因为它们可以在单条指令中处理任意数量的元素。循环余数问题的另一个解决方案是使用掩码, 它允许根据条件选择性地启用或禁用 SIMD 通道。

此外, CPU 越来越多地加速机器学习中经常使用的矩阵乘法。自 2023 年以来在服务器处理器中支持的 Intel AMX 扩展, 将 16×64 和 64×16 形状的 8 位矩阵相乘, 累加到 32 位 16×16 矩阵中。相比之下, Apple CPU 中不相关但同名的 AMX 扩展以及 ARM 的 SME 扩展分别计算行和列的外积, 分别存储在特殊的 512 位寄存器或可变伸缩向量中。

最初, SIMD 由多媒体应用程序和科学计算驱动, 但后来在许多其他领域找到了用途。随着时间的推移, SIMD 指令集中支持的操作集稳步增加。除了图 fig. 3.4 所示的直接算术之外, SIMD 的

新用例包括：

- 字符串处理：查找字符、验证 UTF-8、³⁰ 解析 JSON³¹ 和 CSV；³²
- 哈希、³³ 随机生成、³⁴ 密码学 (AES)；
- 列式数据库 (位打包、过滤、连接)；
- 内置类型排序 (VQSort、³⁵ QuickSelect)；
- 机器学习和人工智能 (加速 PyTorch、TensorFlow)。

3.5 利用线程级并行

前面描述的技术依赖于程序中可用的并行性来加速执行。除此之外，CPU 还支持利用在 CPU 上执行的进程和/或线程之间的并行性的技术。接下来，我们将讨论三种利用线程级并行 (TLP) 的技术：多核系统、同时多线程和混合架构。这些技术使得从可用硬件资源中榨取最大效率并提高系统吞吐量成为可能。

3.5.1 多核系统

随着处理器架构师开始达到半导体设计和制造的实际限制，GHz 竞赛放缓，设计者不得不专注于其他创新来提高 CPU 性能。关键方向之一是多核设计，试图增加每一代的核心数。想法是在单个芯片上复制多个处理器核心，让它们同时服务于不同的程序。例如，一个核心可以运行网络浏览器，另一个核心可以渲染视频，还有一个可以播放音乐，所有这些同时进行。对于服务器机器，来自不同客户端的请求可以在单独的核心上处理，这可以大大提高此类系统的吞吐量。

第一个面向消费者的双核处理器是 Intel Core 2 Duo，于 2005 年发布，随后是 AMD Athlon X2 架构，于同年晚些时候发布。多核系统导致许多软件组件被重新设计，并影响了我们编写代码的方式。如今，面向消费者的设备中几乎所有处理器都是多核 CPU。在撰写本书时，高端笔记本电脑包含超过十个物理核心，服务器处理器在单个插槽上包含超过 100 个核心。

这听起来非常令人印象深刻，但我们不能无限地添加核心。首先，每个核心在工作时都会产生热量，通过处理器封装安全地散发这些热量仍然是一个挑战。这意味着当更多核心运行时，热量可能迅速超过冷却能力。在这种情况下，多核处理器会降低时钟频率。这就是为什么你可以看到具有大量核心的服务器芯片的频率远低于进入笔记本电脑和台式机的处理器的原因之一。

多核系统中的核心相互连接，并连接到共享资源，如最后一级缓存和内存控制器。这种通信通道称为互连，通常具有环形或网状拓扑结构。CPU 设计者的另一个挑战是随着核心数量的增长保持机器平衡。当你复制核心时，某些资源保持共享，例如内存总线和最后一级缓存。这导致随着核心的添加，性能收益递减，除非你也解决其他共享资源的吞吐量，例如互连带宽、最后一级缓存大小和带宽以及内存带宽。共享资源经常成为多核系统中性能问题的来源。

3.5.2 同时多线程

提高多线程性能的更复杂方法是同时多线程 (SMT)。人们经常使用超线程一词来描述同一件事。该技术的目标是充分利用 CPU 流水线的可用宽度。SMT 允许多个软件线程在同一个物理核心

³⁰ UTF-8 验证 - <https://github.com/rusticstuff/simdutf8>

³¹ 解析 JSON - <https://github.com/simdjson/simdjson>

³² 解析 CSV - <https://github.com/geofflangdale/simdcsv>

³³ SIMD 哈希 - <https://github.com/google/highwayhash>

³⁴ 随机生成 - abseil 库

³⁵ 排序 - VQSort

上使用共享资源同时运行。更准确地说，来自多个软件线程的指令在同一个周期内并发执行。这些不必是来自同一进程的线程；它们可以是恰好调度在同一物理核心上的完全不同的程序。

图 fig. 3.5 显示了在非 SMT 和 2 路 SMT (SMT2) 处理器上执行的示例。在两种情况下，处理器流水线的宽度都是四，每个槽位代表发出新指令的机会。100% 机器利用率是没有未使用槽位的情况，这在实际工作负载中永远不会发生。很容易看出，对于非 SMT 情况，有许多未使用的槽位，因此可用资源没有得到很好的利用。这可能由于多种原因发生。例如，在周期 3 中，线程 1 无法向前推进，因为所有指令都在等待其输入变得可用。非 SMT 处理器只会停顿，而启用 SMT 的处理器则利用这个机会从另一个线程调度有用的工作。这里的目标是通过另一个线程占用未使用的槽位，以提高硬件利用率和多线程性能。

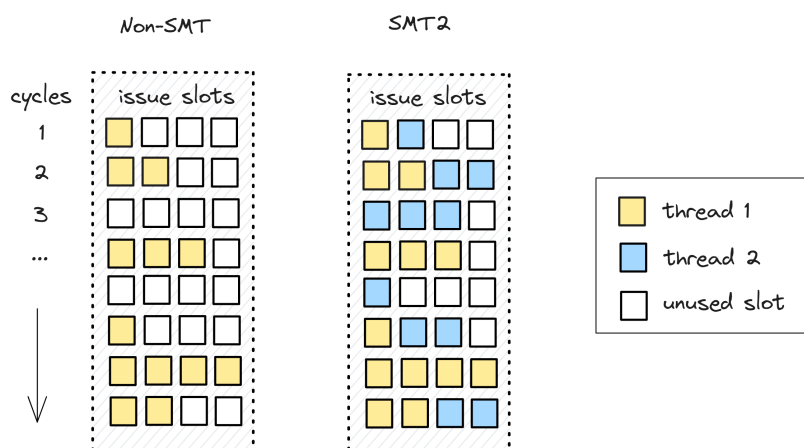


Figure 3.5: 在 4 宽非 SMT 和 4 宽 SMT2 处理器上的执行。

使用 SMT2 实现，每个物理核心由两个逻辑核心表示，操作系统将其视为两个可用于接受工作的独立处理器。考虑我们有 16 个准备运行的软件线程，但只有 8 个物理核心的情况。在非 SMT 系统中，只有 8 个线程将同时运行，而使用 SMT2，我们可以同时执行所有 16 个线程。在另一个假设情况下，如果两个程序在启用 SMT 的核心上运行，并且每个程序始终只使用四个可用槽位中的两个，那么它们很可能会像在该物理核心上单独运行时一样快。

尽管两个程序在同一个处理器核心上运行，但它们彼此完全分离。在启用 SMT 的处理器中，即使指令是混合的，它们也有不同的上下文，这有助于保持执行的正确性。为了支持 SMT，CPU 必须复制架构状态（程序计数器、寄存器）以维护线程上下文。其他 CPU 资源可以共享。在典型实现中，缓存在硬件线程之间动态共享。跟踪 OOO 和推测执行的资源可以复制或分区。

在 SMT2 核心中，两个逻辑核心确实在同一时间运行。在 CPU 前端，它们以交替顺序获取指令（每个周期或几个周期一次）。在后端，每个周期处理器从所有线程中选择指令执行。指令执行是混合的，因为处理器在两个线程之间动态调度执行单元。

因此，SMT 是一种非常灵活的设置，可以恢复未使用的 CPU 发射槽位。SMT 提供与单线程相同的性能，此外还为多线程提供好处。选择支持 SMT 的现代 CPU 通常实现双向 (SMT2)，有时是四向 (SMT4) SMT。

SMT 也有其自身的缺点。由于某些资源在逻辑核心之间共享，它们最终可能竞争使用这些资源。

最可能的 SMT 惩罚是由 L1 和 L2 缓存的竞争引起的。由于它们在两个逻辑核心之间共享，它们可能缺乏缓存空间并强制驱逐将来会被另一个线程使用的数据。

SMT 给软件开发者带来了相当大的负担，因为它使预测和度量在 SMT 核心上运行的应用程序的性能变得更加困难。想象一下，你正在 SMT 核心上运行性能关键代码，突然操作系统将另一个高需求作业放在兄弟逻辑核心上。你的代码几乎占满了机器的资源，现在你需要与其他人共享它们。这个问题在云环境中尤其突出，因为你无法预测你的应用程序是否会有嘈杂的邻居。

某些同时多线程实现也存在安全问题。研究人员表明，一些早期实现存在漏洞，通过该漏洞，一个应用程序可以通过监控其缓存使用情况，从在同一处理器的兄弟逻辑核心上运行的另一个应用程序中窃取关键信息（如加密密钥）。我们不会深入探讨这个主题，因为硬件安全超出了本书的范围。

3.5.3 混合架构

计算机架构师还开发了混合 CPU 设计，其中两种（或更多）类型的核心放在同一个处理器中。通常，更强大的核心与相对较慢的核心耦合以解决不同的目标。在这样的系统中，大核心用于延迟敏感的任务，小核心提供降低的功耗。但是，两种类型的核心也可以同时使用以提高多线程性能。所有核心都可以访问相同的内存，因此工作负载可以在大核心和小核心之间动态迁移。目的是创建一个可以更好地适应动态计算需求并使用更少功耗的多核处理器。例如，视频游戏具有单线程突发执行的部分，也有可以扩展到多个核心的工作部分。

第一个主流混合架构是 Arm 的 big.LITTLE，于 2011 年 10 月推出。其他供应商也采用了这种方法。Apple 于 2020 年推出了其 M1 芯片，具有四个高性能“Firestorm”和四个节能“Icestorm”核心。Intel 于 2021 年推出了其 Alder Lake 混合架构，顶级配置中有八个 P 核和八个 E 核。

混合架构结合了两种核心类型的优势，但它们也带来了一系列挑战。首先，它要求核心完全兼容 ISA，即它们应该能够执行相同的指令集。否则，调度将受到限制。例如，如果大核心具有一些小核心上没有的特殊指令，那么你可能只能将大核心分配给使用此类指令的工作负载。这就是为什么供应商在为混合处理器选择 ISA 时通常使用“最大公分母”方法。

即使使用兼容 ISA 的核心，调度也变得具有挑战性。不同类型的工作负载需要特定的调度方案，例如突发执行 vs. 稳定执行、低 IPC vs. 高 IPC、³⁶ 低重要性 vs. 高等重要性等。这很快变得不简单。以下是最佳调度的一些考虑因素：

- 利用小核心节省功耗。不要为后台工作唤醒大核心。
- 识别可以卸载到较小核心的候选者（低重要性、低 IPC）。类似地，将高重要性、高 IPC 任务提升到大核心。
- 分配新任务时，首先使用空闲的大核心。在 SMT 的情况下，使用两个逻辑线程都空闲的大核心。然后，使用空闲的小核心。然后，使用大核心的兄弟逻辑线程。

从程序员的角度来看，不需要代码更改来利用混合系统。这种方法在面向客户的设备中非常流行，特别是在智能手机中。

3.6 内存层次结构

为了有效利用 CPU 中配置的所有硬件资源，机器需要在正确的时间获得正确的数据。未能做到这一点需要从主内存获取变量，这大约需要 100 纳秒。从 CPU 的角度来看，这是很长的时间。理

³⁶ IPC - 每周期指令数，参见 Section 4.3。

解内存层次结构对于提供 CPU 的性能能力至关重要。大多数程序都表现出局部性：它们不均匀地访问所有代码或数据。CPU 内存层次结构建立在两个基本属性之上：

- **时间局部性**：当访问给定内存位置时，同一位置可能很快会被再次访问。理想情况下，我们希望下次需要此信息时它在缓存中。
- **空间局部性**：当访问给定内存位置时，附近的位置可能很快会被访问。这指的是将相关数据放在一起。当程序从内存读取单个字节时，通常会获取更大的内存块（缓存行），因为程序很可能很快需要该数据。

本节概述了现代 CPU 支持的内存层次结构系统的关键属性。

3.6.1 缓存层次结构

缓存是 CPU 流水线发出的任何请求（代码或数据）的内存层次结构的第一级。理想情况下，流水线在具有最小访问延迟的无限缓存中表现最佳。实际上，任何缓存的访问时间都会随着大小的增加而增加。因此，缓存被组织为层次结构，由最接近执行单元的小而快速的存储块组成，由更大、更慢的块备份。缓存层次结构的特定级别可以专门用于代码（指令缓存，I-cache）或数据（数据缓存，D-cache），或在代码和数据之间共享（统一缓存）。此外，层次结构的某些级别可以是特定核心私有的，而其他级别可以在核心之间共享。

缓存被组织为具有定义大小的块，也称为缓存行。现代 CPU 中典型的缓存行大小为 64 字节。然而，这里值得注意的例外是 Apple 处理器（如 M1、M2 及更高版本）中的 L2 缓存，它在 128B 缓存行上运行。最接近执行流水线的缓存大小通常范围从 32 KB 到 128 KB。中级缓存倾向于具有 1MB 及以上。现代 CPU 中的最后一级缓存可以是数十甚至数百兆字节。

3.6.1.1 数据在缓存中的放置 请求的地址用于访问缓存。在直接映射缓存中，给定的块地址只能出现在缓存中的一个位置，由下面显示的映射函数定义。直接映射缓存相对容易构建且访问速度快，但它们具有高未命中率。

$$\text{缓存中的块数} = \frac{\text{缓存大小}}{\text{缓存块大小}}$$

$$\text{直接映射位置} = (\text{块地址}) \bmod (\text{缓存中的块数})$$

在全相联缓存中，给定的块可以放在缓存中的任何位置。这种方法涉及高硬件复杂性和慢访问时间，因此被认为不适用于大多数用例。

直接映射和全相联映射之间的中间选项是组相联映射。在这种缓存中，块被组织为组，通常每个组包含 2、4、8 或 16 个块。给定地址首先映射到一个组。在组内，地址可以放在该组中的任何块中。每组有 m 个块的缓存被描述为 m 路组相联缓存。组相联缓存的公式为：

$$\text{缓存中的组数} = \frac{\text{缓存中的块数}}{\text{每组块数（相联度）}}$$

$$\text{组（} m \text{ 路）相联位置} = (\text{块地址}) \bmod (\text{缓存中的组数})$$

考虑一个 L1 缓存的示例，其大小为 32 KB，具有 64 字节缓存行、64 组和 8 路。这种缓存中的总缓存行数为 $32 \text{ KB} / 64 \text{ 字节} = 512$ 行。新行只能插入到其适当的组中（64 个组之一）。一旦确定了组，新行可以放在该组中的 8 路之一中。类似地，当你稍后搜索此缓存行时，你首先确定组，然后只需要检查该组中最多 8 路。

这是 Apple M1 处理器缓存组织的另一个示例。每个性能核心内的 L1 数据缓存可以存储 128 KB，具有 256 组，每组 8 路，并在 64 字节行上运行。性能核心形成一个集群并共享 L2 缓存，该缓存可以保持 12 MB，是 12 路组相联的，并在 128 字节行上运行。[Apple, 2024]

3.6.1.2 在缓存中查找数据 m 路组相联缓存中的每个块都有一个与之关联的地址标签。此外，标签还包含状态位，例如指示数据是否有效的位。标签还可以包含额外的位来指示访问信息、共享信息等。

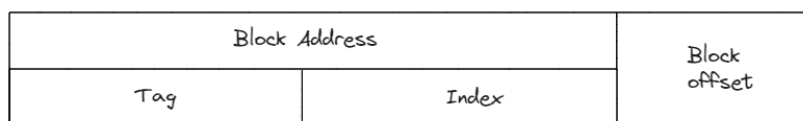


Figure 3.6: 缓存查找的地址组织。

图 fig. 3.6 显示了从流水线生成的地址如何用于检查缓存。最低有效地址位定义给定块内的偏移量；块偏移位（32 字节缓存行为 5 位，64 字节缓存行为 6 位）。使用基于上述公式的索引位选择组。一旦选择了组，标签位用于与该组中的所有标签进行比较。如果其中一个标签与传入请求的标签匹配且有效位被设置，则产生缓存命中。与该块条目关联的数据（与标签查找并行从缓存的数据数组中读出）提供给执行流水线。当标签不匹配时，会发生缓存未命中。

3.6.1.3 管理未命中 当发生缓存未命中时，缓存控制器必须选择缓存中的一个块进行替换，以分配导致未命中的地址。对于直接映射缓存，由于新地址只能分配在单个位置，因此取消分配映射到该位置的先前条目，并将新条目安装在其位置。在组相联缓存中，由于新缓存块可以放在组的任何块中，因此需要替换算法。使用的典型替换算法是 LRU（最近最少使用）策略，其中最近访问最少的块被驱逐以腾出空间给新数据。另一个替代方案是随机选择一个块作为受害者块。

3.6.1.4 管理写入 对缓存的写访问比数据读取频率低。处理缓存中的写入更困难，CPU 实现使用各种技术来处理这种复杂性。软件开发者应特别注意硬件支持的各种写入缓存流程，以确保其代码的最佳性能。

CPU 设计使用两种基本机制来处理在缓存中命中的写入：

- 在写直达缓存中，命中数据同时写入缓存中的块和层次结构的下一个较低级别。
- 在写回缓存中，命中数据仅写入缓存。随后，层次结构的较低级别包含过时数据。修改行的状态通过标签中的脏位跟踪。当修改的缓存行最终从缓存中驱逐时，写回操作强制数据被写回下一个较低级别。

3.7 虚拟内存

虚拟内存（Virtual Memory）是一种将 CPU 附带的物理内存共享给其上运行的所有进程的机制。虚拟内存提供了保护机制，防止一个进程访问分配给其他进程的内存。虚拟内存还提供了重定位能力，即可以将程序加载到物理内存的任意位置，而无需修改程序中的地址。

在支持虚拟内存的 CPU 中，程序使用虚拟地址进行访问。但用户代码操作的是虚拟地址，而从内存中检索数据需要物理地址。此外，为了有效管理有限的物理内存，内存被划分为页面。因此，应用程序操作的是操作系统提供的一组页面。

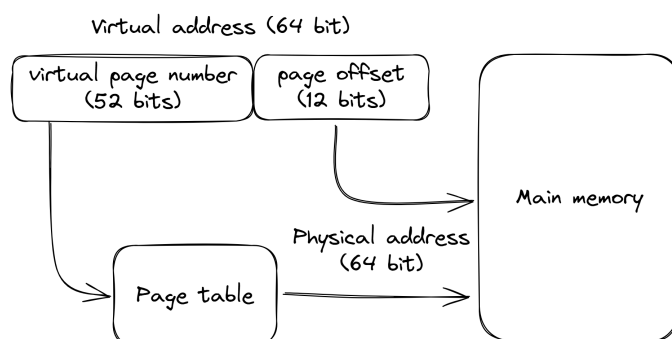


Figure 3.7: 4KB 页面的虚拟地址到物理地址转换。

访问数据和代码（指令）都需要进行虚拟地址到物理地址的转换。图 fig. 3.7 展示了页面大小为 4KB 的系统的地址转换机制。虚拟地址被分为两部分。虚拟页号（最高的 52 位）用于索引页表，生成虚拟页号与对应物理页之间的映射。最低的 12 位用于 4KB 页面内的偏移。这些位不需要转换，直接“按原样”用于访问物理内存位置。

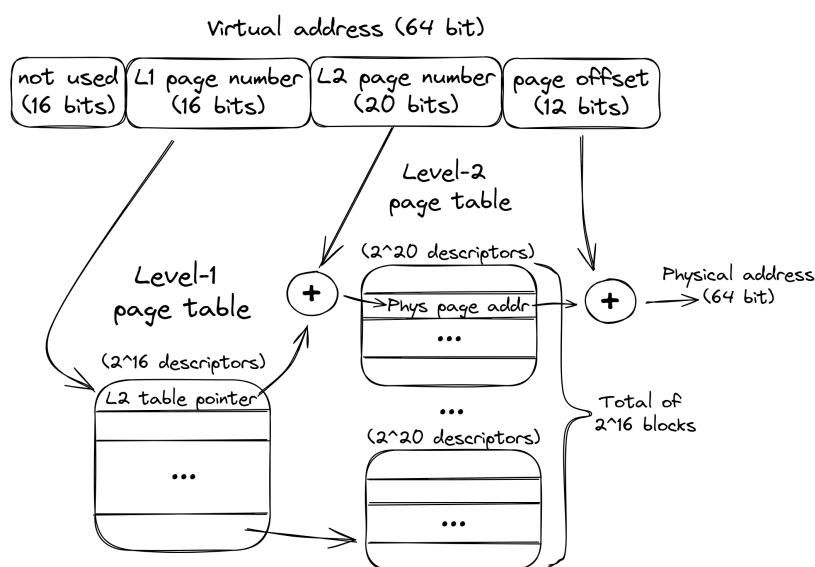


Figure 3.8: 两级页表示例。

页表可以是单级的，也可以是多级嵌套的。图 fig. 3.8 展示了一个两级页表的示例。注意地址被拆分成了更多部分。首先，最高的 16 位未被使用。这看起来像是浪费了比特位，但即使只用剩余的 48 位，我们也能寻址 256 TB 的总内存 (2^{48})。有些应用会利用这些未使用的位来存储元数据，这种技术称为指针标记 (Pointer Tagging)。

多级页表是一个基数树 (Radix Tree)，其中存储了物理页地址以及一些元数据。要在两级页表中

查找地址转换，我们首先使用第 32..47 位作为索引访问一级页表，也称为页表目录 (Page Table Directory)。目录中的每个描述符指向一个二级表块 (共 2^{16} 个)。找到对应的二级表块后，再使用第 12..31 位找到物理页地址。将其与页面偏移 (第 0..11 位) 拼接，就得到了物理地址，可用于从 DRAM 中检索数据。

页表的具体格式由 CPU 决定，原因我们稍后会讨论。因此页表组织方式的变化受限于 CPU 的支持范围。现代 CPU 同时支持 48 位指针的四级页表 (256 TB 总内存) 和 57 位指针的五级页表 (128 PB 总内存)。

将页表拆分为多级并不会改变总的可寻址内存范围。但嵌套方式不需要将整个页表存储为连续数组，也不会分配没有描述符的块。这节省了内存空间，但在遍历页表时增加了开销。

未能提供物理地址映射的情况称为缺页 (Page Fault)。当请求的页面无效或当前不在主内存中时，就会发生缺页。最常见的两个原因是：1) 操作系统承诺分配一个页面，但尚未用物理页面支撑它；2) 被访问的页面已被换出到磁盘，当前不在 RAM 中。

3.7.1 转换后备缓冲区 (TLB)

在多级页表中进行搜索可能开销很大，需要遍历层级结构并可能产生多次间接访问。这种遍历称为页表遍历 (Page Walk)。为了缩短地址转换时间，CPU 提供了一种称为转换后备缓冲区 (Translation Lookaside Buffer, TLB) 的硬件结构，用于缓存最近使用的地址转换。与常规缓存类似，TLB 通常被设计为层次结构：L1 ITLB (指令) 和 L1 DTLB (数据)，后接共享的 (指令和数据) L2 STLB。

为了降低内存访问延迟，L1 缓存查找可以与 DTLB 查找部分重叠——这得益于缓存关联度和大小上的一个约束，使得 L1 组选择可以在不需要物理地址的情况下完成。³⁷ 然而，更高层级的缓存 (L2 和 L3) ——它们通常也是物理索引、物理标记 (PIPT) 的——无法利用这种优化，因此需要在缓存查找之前完成地址转换。

TLB 层次结构缓存了相对较大内存空间的地址转换。但 TLB 未命中的代价仍然很高。为了加速 TLB 未命中的处理，CPU 提供了一种称为硬件页表遍历器 (Hardware Page Walker) 的机制。该单元可以直接在硬件中执行页表遍历，通过发出必要的指令来遍历页表，而无需中断内核。这就是页表格式由 CPU 决定的原因——操作系统必须遵循这一格式。高端处理器有多个硬件页表遍历器，可以同时处理多个 TLB 未命中。然而，即使有了现代 CPU 提供的所有加速手段，TLB 未命中仍然是许多应用的性能瓶颈。

3.7.2 大页

较小的页面大小可以更高效地管理可用内存并减少碎片。但缺点是需要更多的页表条目来覆盖相同的内存区域。考虑两种页面大小：x86 上默认的 4KB 和 2MB 的大页 (Huge Page)。对于操作 10MB 数据的应用，第一种情况需要 2560 个条目，而如果将地址空间映射到大页上，只需要 5 个条目。Linux 上称它们为 *Huge Pages*，FreeBSD 上称为 *Super Pages*，Windows 上称为 *Large Pages*，但含义相同。在本书后续内容中，我们统一称为大页。

图 fig. 3.9 展示了一个指向大页内数据的地址示例。与默认页面大小一样，使用大页时的具体地址格式由硬件决定，但幸运的是，作为程序员我们通常无需关心这一点。

³⁷ PIPT L1 缓存要同时满足 VIPT 特性所需的最低相联度——无需将索引转换为物理地址即可访问组——参见 <https://stackoverflow.com/a/59288185>。

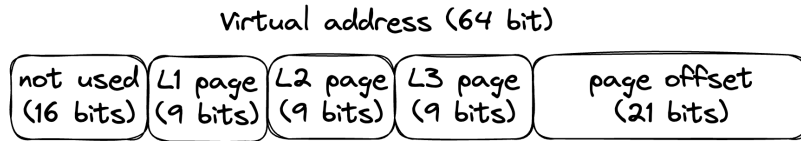


Figure 3.9: 指向 2MB 页面内数据的虚拟地址。

使用大页大幅减轻了 TLB 层次结构的压力，因为所需的 TLB 条目更少，TLB 命中的概率大大提高。我们将在 Section 8.4 和 Section 11.8 中讨论如何使用大页来减少 TLB 未命中的频率。使用大页的缺点是内存碎片化，以及在某些情况下页面分配延迟的不确定性——因为操作系统更难管理大块内存并确保有效利用可用内存。为了在运行时满足 2MB 大页的分配请求，操作系统需要找到一块连续的 2MB 内存区域。如果找不到，操作系统就需要重新组织页面，导致分配延迟增加。

3.8 现代 CPU 设计

为了解本章讨论的所有概念在实践中如何应用，让我们来看看 Intel 第 12 代核心 Golden Cove 的实现，它于 2021 年发布。该核心被用作 Alder Lake 和 Sapphire Rapids 平台中的 P-core（性能核心）。图 fig. 3.10 展示了 Golden Cove 核心的框图。注意，本节只描述单个核心，而非整个处理器。因此，我们将跳过关于频率、核心数量、L3 缓存、核心互联、内存延迟和带宽的讨论。

核心分为负责取指和将 x86 指令解码为 μop ³⁸ 的有序前端，以及 6 宽度的乱序后端。Golden Cove 核心支持 2 路 SMT。它拥有 32KB 的一级指令缓存（L1 I-cache）和 48KB 的一级数据缓存（L1 D-cache）。L1 缓存由统一的 1.25MB（服务器芯片为 2MB）L2 缓存支撑。L1 和 L2 缓存是每个核心私有的。在本节末尾，我们还会介绍 TLB 层次结构。

3.8.1 CPU 前端

CPU 前端由若干功能单元组成，负责从内存中取指和解码指令。其主要目的是将准备好的指令输送给 CPU 后端，后者负责指令的实际执行。

从技术上讲，取指是执行指令的第一阶段。但当程序进入稳态后，分支预测单元（BPU）主导着 CPU 前端的工作。这一点在图中由从 BPU 指向指令缓存的箭头表示。BPU 预测所有分支指令的目标，并基于预测结果引导下一次取指。

BPU 的核心是一个包含 12K 条目的分支目标缓冲区（BTB），其中存储了分支及其目标的信息。预测算法利用这些信息工作。每个时钟周期，BPU 生成下一个取指地址并传递给 CPU 前端。

CPU 前端每个周期从 L1 I-cache 取指 32 字节的 x86 指令。如果启用了 SMT，这 32 字节由两个线程共享，因此每个线程每隔一个周期获得 32 字节。这些是复杂的变长 x86 指令。首先，预解码阶段通过检查指令块来确定并标记变长指令的边界。在 x86 中，指令长度可从 1 到 15 字节不等。此阶段还会识别分支指令。预解码阶段将最多 6 条指令（也称为宏指令）移入指令队列，指令队列在两个线程之间分配。指令队列还支持宏操作融合单元，用于检测两条宏指令是否可以融合为一条微操作（ μop ）。这种优化节省了流水线后续阶段的带宽。

³⁸ 复杂的 CISC 指令被转换为简单的 RISC 微操作，参见 Section 4.4。

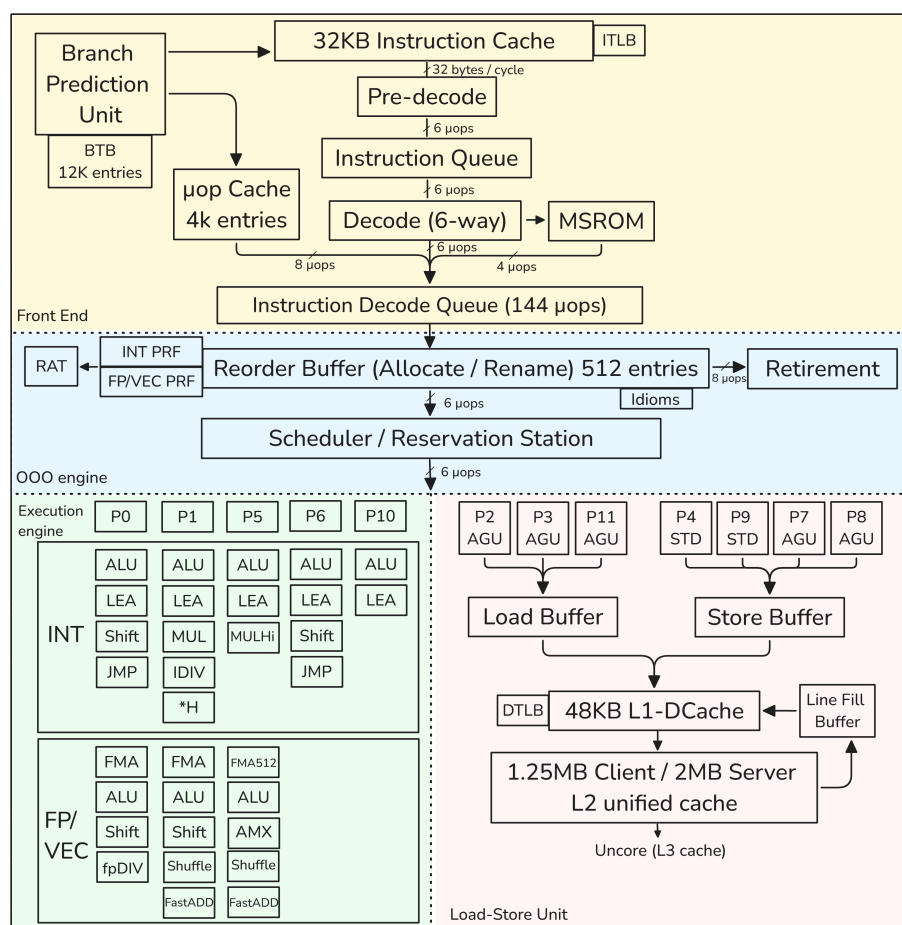


Figure 3.10: Intel Golden Cove 微架构的 CPU 核心框图。

随后，每个周期最多 6 条预解码指令从指令队列发送到解码单元。两个 SMT 线程交替访问该接口。6 路解码器将复杂的宏操作转换为固定长度的 μop 。解码后的 μop 被排入指令解码队列 (IDQ)，在图中标记为“ μop Queue”。

前端的一个重要性能提升特性是 μop 缓存 (μop Cache)，你也可能经常看到人们称其为解码流缓冲区 (Decoded Stream Buffer, DSB)。其动机是将宏操作到 μop 的转换缓存在一个独立的结构中，与 L1 I-cache 并行工作。当 BPU 生成新的取指地址时， μop 缓存也会被检查，看 μop 转换结果是否已经可用。频繁出现的宏操作会在 μop 缓存中命中，流水线将避免对 32 字节的指令块重复执行开销高昂的预解码和解码操作。 μop 缓存每个周期可提供 8 个 μop ，最多可容纳 4K 个条目。

一些非常复杂的指令可能需要的 μop 数量超过解码器的处理能力。这类指令的 μop 由微码序列器 (Microcode Sequencer, MSROM) 提供。这类指令的例子包括字符串操作、加密、同步等的硬件操作支持。此外，MSROM 还保存着用于处理异常情况的微码操作，例如分支误预测

(需要流水线刷新)、浮点辅助 (例如, 当指令操作非规格化浮点值时) 等。MSROM 每周期可向 IDQ 推送最多 4 个 μop 。

指令解码队列 (IDQ) 提供了有序前端与乱序后端之间的接口。IDQ 按序排列 μop , 在单线程模式下每个逻辑处理器可容纳 144 个 μop , SMT 激活时每个线程 72 个 μop 。有序 CPU 前端到此结束, 乱序 CPU 后端从这里开始。

3.8.2 CPU 后端

CPU 后端采用乱序执行引擎来执行指令并存储结果。我在图 fig. 3.11 中复制了描绘 Golden Cove 乱序引擎的部分框图。

乱序引擎的核心是 512 条目的重排序缓冲区 (ReOrder Buffer, ROB)。它有以下几个用途。首先, 它提供寄存器重命名 (Register Renaming)³⁹。虽然只有 16 个通用整数寄存器和 32 个浮点/SIMD 架构寄存器, 但物理寄存器的数量要多得多。⁴⁰ 物理寄存器位于一个称为物理寄存器文件 (Physical Register File, PRF) 的结构中。整数和浮点/SIMD 寄存器各有独立的 PRF。架构可见寄存器到物理寄存器的映射保存在寄存器别名表 (Register Alias Table, RAT) 中。

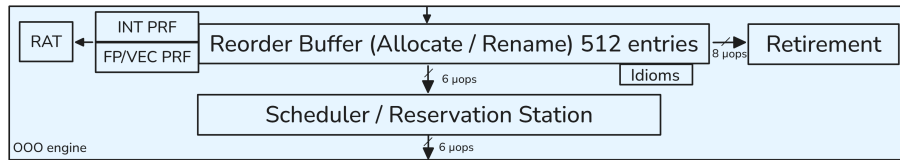


Figure 3.11: Intel Golden Cove 微架构的 CPU 后端框图。

其次, ROB 分配执行资源。当一条指令进入 ROB 时, 会分配一个新条目并为其分配资源, 主要包括一个执行单元和目标物理寄存器。ROB 每周期最多可分配 6 个 μop 。

第三, ROB 追踪推测执行。当一条指令完成执行后, 其状态会被更新, 并保留在那里直到之前的指令全部完成。这样做是因为指令必须按程序顺序退休 (Retire)。一旦指令退休, 其 ROB 条目被释放, 指令的结果变得可见。退休阶段比分配阶段更宽: ROB 每周期可退休 8 条指令。

某些操作处理器会以特殊方式处理, 通常称为惯用法 (Idiom), 它们不需要执行或执行开销更低。处理器会识别这些情况并允许它们比常规指令更快完成。以下是一些例子:

- **置零操作:** 为了将寄存器置零, 编译器通常使用 XOR / PXOR / XORPS / XORPD 指令, 例如 XOR EAX, EAX, 编译器更倾向于使用它而不是等效的 MOV EAX, 0x0, 因为 XOR 编码使用更少的字节数。这种置零惯用法不会像其他常规指令那样被执行, 而是在 CPU 前端中解析完成, 从而节省了执行资源。指令随后照常退休。
- **移动消除 (Move Elimination):** 与前一种类似, 寄存器到寄存器的 mov 操作, 例如 MOV EAX, EBX, 以零周期延迟执行。
- **NOP 指令:** NOP 通常用于填充或对齐目的。它直接被标记为完成, 无需分配到保留站。

³⁹ 重命名必须按程序顺序进行。

⁴⁰ 约有 300 个物理通用寄存器 (GPR), 向量寄存器的数量类似。实际寄存器数量未公开。

- **其他旁路优化：**CPU 架构师还优化了某些算术操作。例如，任何数乘以 1 总是得到该数本身。任何数除以 1 也是如此。任何数乘以 0 总是得到 0，等等。某些 CPU 可以在运行时识别这些情况，并以比常规乘法或除法更短的延迟执行它们。

“调度器/保留站” (Scheduler / Reservation Station, RS) 是追踪给定 μop 所有资源可用性并在其就绪时将 μop 分派到执行端口的结构。执行端口是连接调度器与执行单元的通路。每个执行端口可连接多个执行单元。当一条指令进入 RS 时，调度器开始追踪其数据依赖关系。一旦所有源操作数可用，RS 就尝试将 μop 分派到空闲的执行端口。RS 的条目数比 ROB 少。⁴¹ 它每周期最多可分派 6 个 μop 。

我在图 fig. 3.12 中复制了描绘 Golden Cove 执行引擎和加载-存储单元的部分框图。共有 12 个执行端口：

- 端口 0、1、5、6 和 10 提供整数 (INT) 操作，其中部分端口还处理浮点和向量 (FP/VEC) 操作。
- 端口 2、3 和 11 用于地址生成 (AGU) 和加载操作。
- 端口 4 和 9 用于存储操作 (STD)。
- 端口 7 和 8 用于地址生成。

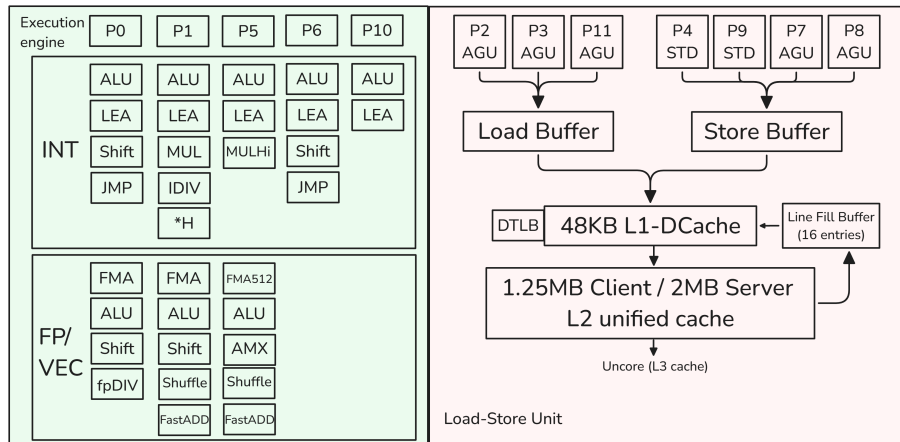


Figure 3.12: Intel Golden Cove 微架构的执行引擎和加载-存储单元框图。

需要内存操作的指令由加载-存储单元 (端口 2、3、11、4、9、7 和 8) 处理，我们将在下一节讨论。如果操作不涉及加载或存储数据，则会被分派到执行引擎 (端口 0、1、5、6 和 10)。某些指令可能需要两个 μop ，且必须在不同的执行端口上执行，例如加载加法。

例如，Integer Shift 操作只能分派到端口 0 或 6，而 Floating-Point Divide 操作只能分派到端口 0。当调度器需要分派两个要求同一执行端口的操作时，其中一个将不得不延迟。

浮点/向量 (FP/VEC) 栈执行浮点标量和所有打包 (SIMD) 操作。例如，端口 0、1 和 5 可以处理以下类型的 ALU 操作：打包整数、打包浮点和标量浮点。整数和向量/浮点寄存器文件是分开放置的。在 INT 栈和 FP/VEC 栈之间移动值的操作 (例如转换、提取或插入) 会产生额外的开销。

⁴¹ 人们测量到 RS 中约有 200 个条目，但实际条目数未公开。

3.8.3 加载-存储单元

加载-存储单元 (Load-Store Unit, LSU) 负责内存操作。Golden Cove 核心通过端口 2、3 和 11 最多可发出三个加载操作 (三个 256 位或两个 512 位)。AGU 是地址生成单元 (Address Generation Unit)，用于访问内存位置所需。它还可以通过端口 4、9、7 和 8 每周期最多发出两个存储操作 (两个 256 位或一个 512 位)。STD 是存储数据 (Store Data)。

注意，加载和存储操作都需要 AGU 来执行动态地址计算。例如，在指令 `vmovss DWORD PTR [rsi+0x4],xmm0` 中，AGU 负责计算 `rsi+0x4`，该地址将用于存储 `xmm0` 中的数据。

一旦加载或存储操作离开调度器，LSU 就负责访问数据。加载操作将取到的值保存到寄存器中。存储操作将寄存器中的值传输到内存位置。LSU 有一个加载缓冲区 (Load Buffer，也称加载队列) 和一个存储缓冲区 (Store Buffer，也称存储队列)；它们的大小未公开。⁴² 加载缓冲区和存储缓冲区在调度器分派时接收操作。

当内存加载请求到来时，LSU 使用虚拟地址查询 L1 缓存，同时在 TLB 中查找物理地址转换。这两个操作同时发起。L1 D-cache 的大小为 48KB。如果两个操作都命中，加载操作将数据传送到整数或浮点寄存器，并离开加载缓冲区。类似地，存储操作会将数据写入数据缓存并退出存储缓冲区。

如果 L1 未命中，硬件会发起对 (私有) L2 缓存标签的查询。在查询 L2 缓存的同时，会分配一个 64 字节宽的填充缓冲区 (Fill Buffer, FB) 条目，用于在缓存行到达时保存它。Golden Cove 核心有 16 个填充缓冲区。为了降低延迟，还会在 L2 缓存查找的同时向 L3 缓存发送推测性查询。此外，如果两个加载访问同一个缓存行，它们会命中同一个 FB。这两个加载会被“粘合”在一起，只发起一次内存请求。

如果确认 L2 未命中，加载操作继续等待 L3 缓存的结果，这会带来更高的延迟。从这一刻起，请求离开核心，进入非核心 (Uncore) —— 你有时可能在性能分析工具中看到这个术语。核心的未完成未命中由超级队列 (Super Queue, SQ，图中未显示) 追踪，它可以追踪最多 48 个非核心请求。在 L3 未命中的场景下，处理器开始准备内存访问。进一步的细节超出了本章的范围。

当存储操作修改一个内存位置时，处理器需要先加载整条缓存行，修改它，然后写回内存。如果要写的地址不在缓存中，它会经历与加载非常类似的机制来将数据取入。存储操作在数据写入缓存层次结构之前无法完成。

当然，存储操作也有一些优化。首先，如果我们处理的是修改整条缓存行的存储或多个相邻存储 (也称为流式存储)，就不需要先读取数据，因为所有字节都会被覆盖。因此，处理器会尝试合并写操作以填满整条缓存行。如果成功，就不需要内存读取操作。

其次，写合并 (Write Combining) 使得多个存储可以被组装在一起，作为一个单元写入更远的缓存层次结构。因此，如果多个存储修改同一条缓存行，只会向内存子系统发出一次内存写入。所有这些优化都在存储缓冲区内部完成。存储指令将要写入的数据从寄存器复制到存储缓冲区。从那里，数据可能被写入 L1 缓存，也可能与对同一缓存行的其他存储合并。存储缓冲区容量有限，因此它只能在一段时间内保存对缓存行的部分写入请求。然而，当数据在存储缓冲区中等待写入时，其他加载指令可以直接从存储缓冲区读取数据 (存储到加载转发, Store-to-Load Forwarding)。此外，LSU 支持在存在较旧的存储包含加载所需全部字节且数据已产生并可用在存储队列中时进行存储到加载转发。

⁴² 加载缓冲区和存储缓冲区的大小未公开，但人们测量到分别有 192 和 114 个条目。

最后，在某些情况下，我们可以通过使用所谓的非时间（Non-temporal）内存访问来提高缓存利用率。如果我们执行部分存储（例如，覆盖缓存行中的 8 字节），就需要先读取整条缓存行。这条新缓存行会替换缓存中的另一条行。但如果我们知道不需要再次使用这些数据，那么最好不要在缓存中为该行分配空间。非时间内存访问是特殊的 CPU 指令，它们不会将取到的行保留在缓存中，使用后立即丢弃。

在典型的程序执行过程中，可能有数十个内存访问同时在进行中。在大多数高性能处理器中，加载和存储操作的顺序不一定与程序顺序相同，这称为弱序内存模型（Weakly Ordered Memory Model）。出于优化目的，处理器可以重新排序内存读写操作。考虑一种情况：当加载遇到缓存未命中，必须等待数据从内存返回时，处理器允许后续的加载在等待数据的加载之前继续执行。这使得后面的加载可以先于前面的加载完成，不会不必要地阻塞执行。这种加载/存储重排序使内存单元能够并行处理多个内存访问，直接转化为更高的性能。

LSU 动态重排序操作，支持加载旁路较旧的加载，以及加载旁路较旧的非冲突存储。但有几个例外。与常规算术指令之间的依赖关系一样，加载和存储之间也存在内存依赖关系。换句话说，加载可能依赖于较早的存储，反之亦然。首先，存储不能与较旧的加载重排序：

```
Load R1, MEM_LOC_X
Store MEM_LOC_X, 0
```

如果我们允许存储在加载之前执行，那么 R1 寄存器可能会从内存位置 MEM_LOC_X 读取到错误的值。

另一种有趣的情况是加载消费了较早存储的数据：

```
Store MEM_LOC, 0
Load R1, MEM_LOC
```

如果加载消费了一个尚未完成的存储的数据，我们不应允许该加载继续执行。但如果我们还不知道存储的地址呢？在这种情况下，处理器预测加载和存储之间是否存在潜在的数据转发，以及重排序是否安全。这就是内存消歧（Memory Disambiguation）。当加载开始执行时，它必须与所有较旧的存储进行检查，看是否存在潜在的存储转发。有四种可能的场景：

- 预测：不依赖；结果：不依赖。这是内存消歧成功的情况，产生最优性能。
- 预测：依赖；结果：不依赖。处理器过于保守，没有让加载在存储之前执行。这是性能优化的遗漏机会。
- 预测：不依赖；结果：依赖。这是内存顺序违例（Memory Order Violation）。与分支误预测类似，处理器必须刷新流水线、回滚执行并重新开始。代价非常高。
- 预测：依赖；结果：依赖。加载和存储之间存在内存依赖，处理器正确预测了它。没有遗漏机会。

值得一提的是，从存储到加载的转发在实际代码中相当常见。特别是任何对其数据结构使用读-修改-写访问的代码，都可能触发这类问题。由于乱序窗口很大，CPU 可以轻松地同时处理多个读-修改-写序列，因此一个序列的读操作可能在前一个序列的写操作完成之前就发生。这类示例见 Section 12.2。

3.8.4 TLB 层次结构

回顾 Section 3.7.1 中的内容，虚拟地址到物理地址的转换被缓存在 TLB 中。Golden Cove 的 TLB 层次结构如图 fig. 3.13 所示。与常规数据缓存类似，它有两级，其中一级为指令（ITLB）

和数据 (DTLB) 分别设有独立实例。L1 ITLB 有 256 个条目用于常规 4K 页面, 覆盖 1MB 内存; L1 DTLB 有 96 个条目, 覆盖 384 KB。

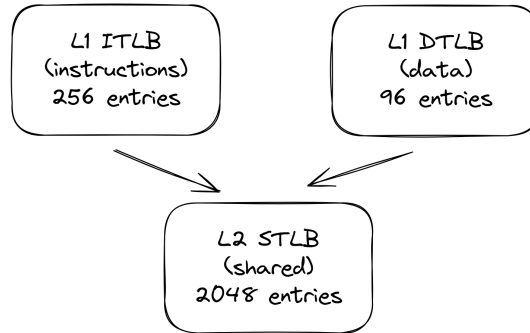


Figure 3.13: Intel Golden Cove 微架构的 TLB 层次结构。

层次结构的第二级 (STLB) 同时缓存指令和数据的地址转换。它是一个更大的存储, 用于服务在 L1 TLB 中未命中的请求。L2 STLB 可容纳 2048 个最近的数据和指令页面地址转换, 总共覆盖 8MB 的内存空间。2MB 大页可用的条目更少: L1 ITLB 有 32 个条目, L1 DTLB 有 32 个条目, L2 STLB 只能使用 1024 个条目, 且与常规 4KB 页面共享。

如果在 TLB 层次结构中未找到地址转换, 就必须通过“遍历”内核页表从 DRAM 中检索。回顾一下, 页表是作为子表的基数树构建的, 子表的每个条目保存着指向树下一级的指针。

加速页表遍历过程的关键要素是一组页结构缓存 (Paging-Structure Caches)⁴³, 它们缓存了页表结构中的热点条目。对于四级页表, 最低 12 位 (11:0) 用于页面偏移 (不进行转换), 第 47:12 位用于页号。TLB 中的每个条目都是一个完整的转换, 而页结构缓存只覆盖上面三级 (第 47:21 位)。其思想是减少 TLB 未命中时需要执行的加载次数。例如, 如果没有这类缓存, 我们需要执行 4 次加载, 这会增加指令完成的延迟。但借助页结构缓存, 如果我们找到了地址一级和二级的转换 (第 47:30 位), 就只需要执行剩余的 2 次加载。

Golden Cove 微架构有四个专用的页表遍历器, 可以同时处理 4 个页表遍历。在 TLB 未命中时, 这些硬件单元会向内存子系统发出所需的加载操作, 并用新条目填充 TLB 层次结构。页表遍历器产生的页表加载可以命中 L1、L2 或 L3 缓存 (细节未公开)。最后, 页表遍历器可以预判未来的 TLB 未命中, 并在未命中实际发生之前推测性地执行页表遍历来更新 TLB 条目。

Golden Cove 的规格说明未公开两个 SMT 线程之间如何共享资源。但通常来说, 缓存、TLB 和执行单元是完全共享的, 以提高这些资源的动态利用率。另一方面, 用于在主要流水线阶段之间暂存指令的缓冲区要么是复制的, 要么是分区的。这些缓冲区包括 IDQ、ROB、RAT、RS、加载缓冲区和存储缓冲区。PRF 也是复制的。

3.9 性能监控单元

每款现代 CPU 都提供了用于监控性能的设施, 这些设施被整合到性能监控单元 (Performance Monitoring Unit, PMU) 中。该单元集成了帮助开发者分析应用性能的各种功能。图 fig. 3.14 展示了现代 Intel CPU 中 PMU 的示例。大多数现代 PMU 都有一组性能监控计数器

⁴³ AMD 的等价物称为页表遍历缓存 (Page Walk Caches)。

(Performance Monitoring Counter, PMC), 可用于收集程序执行过程中发生的各种性能事件。我们将在 Section ?? 中讨论如何使用 PMC 进行性能分析。此外, PMU 还有其他增强性能分析的功能, 如 LBR、PEBS 和 PT, Section 6 专门讨论这些主题。

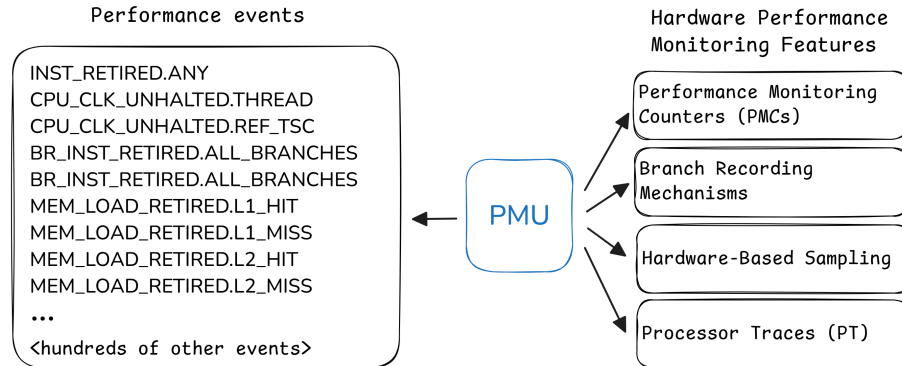


Figure 3.14: 现代 Intel CPU 的性能监控单元。

随着每一代新 CPU 的设计演进, 其 PMU 也在不断发展。在 Linux 上, 可以使用 `cpuid` 命令确定 CPU 中 PMU 的版本, 如 Listing ?? 所示。类似的信息也可以通过检查 `dmesg` 命令输出的内核消息缓冲区来获取。每个 Intel PMU 版本的特性以及与前一版本的变更, 可以在 [Intel, 2023, Volume 3B, Chapter 20] 中找到。

Listing: 查询你的 PMU

```
$ cpuid
...
Architecture Performance Monitoring Features (0xa/eax):
    version ID                      = 0x4 (4)
    number of counters per logical processor = 0x4 (4)
    bit width of counter              = 0x30
    (48)
...
Architecture Performance Monitoring Features (0xa/edx):
    number of fixed counters        = 0x3 (3)
    bit width of fixed counters     = 0x30 (48)
...
```

3.9.1 性能监控计数器

如果我们想象一个简化的处理器视图, 它可能如图 fig. 3.15 所示。正如我们在本章前面讨论的, 现代 CPU 有缓存、分支预测器、执行流水线和其他单元。当 PMC 连接到多个单元时, 可以从收集有用的统计信息。例如, 它可以统计经过了多少个时钟周期、执行了多少条指令、发生了多少次缓存未命中或分支误预测等性能事件。

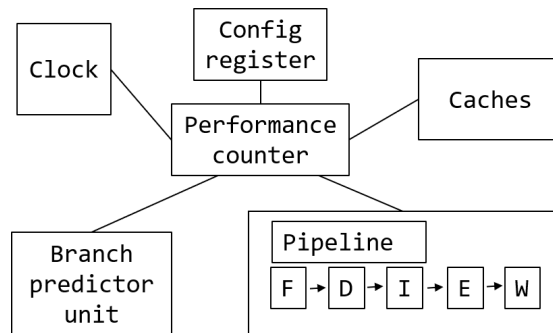


Figure 3.15: 带有性能监控计数器的 CPU 简化视图。

PMC 通常为 48 位宽，这使得分析工具可以长时间运行而无需中断程序执行。⁴⁴ 性能计数器是一种作为型号特定寄存器 (Model-Specific Register, MSR) 实现的硬件寄存器。这意味着计数器的数量和宽度可能因型号而异，你不能依赖 CPU 中计数器的数量始终相同。你应该始终先查询，例如使用 `cpuid` 等工具。PMC 通过 `RDMSR` 和 `WRMSR` 指令访问，这些指令只能在内核空间中执行。幸运的是，只有当你在开发性能分析工具（如 Linux `perf` 或 Intel VTune Profiler）时才需要关心这一点。这些工具处理了 PMC 编程的所有复杂性。

工程师在分析应用时，收集已执行指令数和经过的周期数是非常常见的。这就是为什么某些 PMU 有专门的 PMC 来收集这类事件。固定计数器 (Fixed Counter) 始终测量 CPU 核心内的同一指标。可编程计数器 (Programmable Counter) 则由用户选择要测量的内容。

例如，在 Intel Skylake 架构 (PMU 版本 4，参见 Listing ??) 中，每个物理核心有三个固定计数器和八个可编程计数器。三个固定计数器分别设置为核心时钟、参考时钟和退休指令数（详见 Section 4 中关于这些指标的说明）。AMD Zen4 和 Arm Neoverse V1 核心每个处理器核心支持 6 个可编程性能监控计数器，没有固定计数器。

PMU 提供超过一百个可监控事件的情况并不罕见。图 fig. 3.14 只展示了现代 Intel CPU 上可用于监控的性能监控事件的一小部分。不难注意到，可用 PMC 的数量远少于性能事件的数量。不可能同时统计所有事件，但分析工具通过在程序执行期间在性能事件组之间进行多路复用来解决这个问题（参见 Section ??）。

- 对于 Intel CPU，完整的性能事件列表可以在 [Intel, 2023, Volume 3B, Chapter 20] 或 perfmon-events.intel.com 上找到。
- AMD 没有为每款 AMD 处理器发布性能监控事件列表。好奇的读者可以在 Linux `perf` 源代码⁴⁵中找到一些信息。此外，你可以使用 AMD `uProf` 命令行工具列出可用于监控的性能事件。关于 AMD 性能计数器的一般信息可以在 [AMD, 2023, 13.2 Performance Monitoring Counters] 中找到。
- 对于 ARM 芯片，性能事件的定义不那么统一。各厂商按照 ARM 架构实现核心，但性能事件在含义和支持的事件上都有所不同。对于 Arm 自行设计的 Neoverse V1 核心，性

⁴⁴ 当 PMC 的值溢出时，必须中断程序的执行。性能分析工具随后应保存溢出的事实。我们将在 Section 5 中更详细地讨论这一点。

⁴⁵ AMD 核心的 Linux 源代码——<https://github.com/torvalds/linux/blob/master/arch/x86/events/amd/core.c>

能事件列表可以在 [Arm, 2022] 中找到。对于 Arm Neoverse V2 和 V3 微架构，性能事件列表可在 Arm 官网找到。⁴⁶

思考与练习

1. 描述流水线、乱序执行和推测执行。
2. 寄存器重命名如何帮助加速执行？
3. 描述空间局部性和时间局部性。
4. 大多数现代处理器的缓存行大小是多少？
5. 列出构成 CPU 前端和后端的组件。
6. 四级页表的组织结构是什么？什么是缺页？
7. x86 和 ARM 架构的默认页面大小分别是多少？
8. TLB（转换后备缓冲区）起什么作用？

本章小结

- 指令集架构 (ISA) 是软件与硬件之间的基本契约。ISA 是计算机的抽象模型，定义了可用的操作和数据类型集合、寄存器集合、内存寻址方式等。你可以用多种不同方式实现特定的 ISA。例如，你可以设计一个优先考虑能效的“小”核心，或者一个面向高性能的“大”核心。
- 实现的细节被封装在 CPU“微架构”这个术语中。这个主题已经被数千名计算机科学家长期研究。多年来，许多巧妙的创意被发明并应用到量产 CPU 中。其中最引人注目的包括流水线、乱序执行、超标量引擎、推测执行和 SIMD 处理器。所有这些技术都有助于开发指令级并行性 (ILP) 并提升单线程性能。
- 与单线程性能并行地，硬件设计师开始推动多线程性能。绝大多数面向消费者的现代设备都配备了包含多个核心的处理器。有些处理器借助同步多线程 (Simultaneous Multithreading, SMT) 技术将可观察的 CPU 核心数量翻倍。SMT 使多个软件线程能够使用共享资源在同一物理核心上同时运行。这一方向上更新的技术称为“混合” (Hybrid) 处理器，它将不同类型的核心组合在一个封装中，以更好地支持多样化的工作负载。
- 现代计算机的内存层次结构包含多级缓存，反映了访问速度与容量之间的不同权衡。L1 缓存最接近核心，速度快但容量小。L3/LLC 缓存较慢但容量更大。DDR 是大多数平台使用的主流 DRAM 技术。DRAM 模块在 rank 数量和内存宽度上有所不同，这可能会对系统性能有轻微影响。处理器可以有多个内存通道，以同时访问多个 DRAM 模块。
- 虚拟内存是与 CPU 上运行的所有进程共享物理内存的机制。程序在访问时使用虚拟地址，虚拟地址会被转换为物理地址。内存空间被划分为页面。x86 的默认页面大小为 4KB，ARM 为 16KB。只有页地址会进行转换，页面内的偏移量按原样使用。操作系统在页表中保存转换映射，页表以基数树的形式实现。有硬件特性可以提升地址转换的性能：主要是转换后备缓冲区 (TLB) 和硬件页表遍历器。此外，开发者可以利用大页来缓解某些情况下地址转换的开销 (参见 Section 8.4)。
- 我们分析了 Intel 近期 Golden Cove 微架构的设计。从逻辑上看，核心分为前端和后端。前端由分支预测单元 (BPU)、L1 I-cache、取指和解码逻辑以及 IDQ 组成，IDQ 将指令输送给 CPU 后端。后端由乱序执行引擎、执行单元、加载-存储单元、L1 D-cache 和 TLB 层次结构组成。

⁴⁶ Arm 遥测——<https://developer.arm.com/telemetry>

- 现代处理器具有封装在性能监控单元（PMU）中的性能监控功能。该单元围绕性能监控计数器（PMC）的概念构建，能够观察程序运行时发生的特定事件，例如缓存未命中和分支误预测。

4 性能分析中的术语和指标

与许多工程学科一样，性能分析大量使用特殊的术语和指标。对于初学者来说，查看由 Linux perf 或 Intel VTune Profiler 等分析工具生成的性能剖析可能非常困难。这些工具涉及许多复杂的术语和指标，但如果你想从事任何认真的性能工程工作，这些指标是“必须了解的”。

既然我提到了 Linux perf，让我简要介绍一下这个工具，因为我在本章和后续章节中有许多使用它的示例。Linux perf 是一个性能分析器，你可以用它来查找程序中的热点、收集各种低级 CPU 性能事件、分析调用栈等等。我将在本书中广泛使用 Linux perf，因为它是最流行的性能分析工具之一。我更喜欢展示 Linux perf 的另一个原因是它是开源软件，这使得热心的读者可以探索现代分析工具内部的工作机制。这对于学习本书中介绍的概念特别有用，因为基于 GUI 的工具（如 Intel® VTune™ Profiler）往往会隐藏所有复杂性。我们将在 Section 7 中更详细地介绍 Linux perf。

本章是性能分析中使用的基本术语和指标的简要介绍。我们将首先定义一些基本内容，如退休/执行指令、IPC/CPI、 μ ops、核心/参考时钟、缓存未命中和分支预测错误。然后，我们将了解如何度量系统的内存延迟和带宽，并介绍一些更高级的指标。最后，我们将对四个行业工作负载进行基准测试，并查看收集到的指标。

4.1 退休指令与执行指令

现代处理器通常执行的指令比程序流程要求的更多。这是因为某些指令是推测执行的，如 Section 3.3.3 中所讨论的。对于大多数指令，一旦结果可用且所有前序指令已经退休，CPU 就会提交结果。但对于推测执行的指令，CPU 保持其结果而不立即提交。当推测被证明是正确的时，CPU 解除这些指令的阻塞并正常继续。但当推测被证明是错误的时，CPU 丢弃推测指令所做的所有更改，并且不退休它们。因此，CPU 处理的指令可以被执行但不一定被退休。考虑到这一点，我们通常可以预期执行的指令数高于退休的指令数。

有一个例外。某些指令被识别为惯用法，在没有实际执行的情况下被解析。其中一些例子是 NOP、移动消除和清零，如 Section 3.8.2 中所讨论的。这些指令不需要执行单元，但仍然会被退休。因此，理论上可能存在退休指令数高于执行指令数的情况。

大多数现代处理器中有一个性能监控计数器（PMC），用于收集退休指令的数量。没有性能事件来收集执行指令，但有一种方法可以收集已执行和已退休的 μ ops，我们很快就会看到。使用 Linux perf 可以轻松获取退休指令数：

```
$ perf stat -e instructions -- ./a.exe
2173414 instructions # 0.80 insn per cycle
# 或者简单地运行：
$ perf stat -- ./a.exe
```

4.2 CPU 利用率

CPU 利用率是核心在一段时间内处于繁忙状态的时间百分比。从技术上讲，当 CPU 没有运行内核 idle 线程时，就被认为是正在被利用。

$$CPU\ Utilization = \frac{CPU_CLK_UNHALTED.REF_TSC}{TSC},$$

其中 `CPU_CLK_UNHALTED.REF_TSC` 计算核心未处于停机状态时的参考周期数。TSC 代表时间戳计数器（在 Section 2.5 中讨论），它一直在计时。

如果 CPU 利用率低，通常意味着应用程序性能较差，因为部分时间被 CPU 浪费了。然而，高 CPU 利用率并不总是良好性能的指标。它只是系统正在做一些工作的标志，但没有说明它在做什么：即使 CPU 在等待内存访问时停顿，它也可能被高度利用。在多线程上下文中，线程在等待资源继续时也可能自旋。稍后，在 Section 13.1 中，我们将讨论并行效率指标，特别是查看过滤自旋时间的“有效 CPU 利用率”。

Linux `perf` 自动计算系统中所有 CPU 的 CPU 利用率：

```
$ perf stat -- a.exe
0.634874 task-clock (msec) # 0.773 CPUs utilized
```

4.3 CPI 和 IPC

这两个基本指标代表：

- 每周期指令数 (IPC) - 平均每个周期退休多少条指令。

$$IPC = \frac{INST_RETIRED.ANY}{CPU_CLK_UNHALTED.THREAD},$$

其中 `INST_RETIRED.ANY` 计算退休指令数，`CPU_CLK_UNHALTED.THREAD` 计算线程未处于停机状态时的核心周期数。

- 每指令周期数 (CPI) - 平均退休一条指令需要多少周期。

$$CPI = \frac{1}{IPC}$$

使用哪一个取决于个人偏好。我更喜欢使用 IPC，因为它更容易比较。使用 IPC 时，我们希望每个周期有尽可能多的指令，因此 IPC 越高越好。使用 CPI 时则相反：我们希望每条指令的周期数尽可能少，因此 CPI 越低越好。使用“越高越好”指标的比较更简单，因为你不必每次都进行心理反转。在本书的其余部分，我们将主要使用 IPC，但同样，使用 CPI 也没有错。

IPC 和 CPU 时钟频率之间的关系非常有趣。从广义上讲，性能 = 工作 / 时间，其中我们可以将工作表示为指令数，时间表示为秒数。程序运行的秒数可以计算为 总周期数 / 频率：

$$Performance = \frac{instructions \times frequency}{cycles} = IPC \times frequency$$

如我们所见，性能与 IPC 和频率成正比。如果我们增加这两个指标中的任何一个，程序的性能都会增长。

从基准测试的角度来看，IPC 和频率是两个独立的指标。我见过一些工程师错误地将它们混淆，认为如果你增加频率，IPC 也会提高。但事实并非如此。如果你将处理器时钟设置为 1 GHz 而不是 5 GHz，对于许多应用程序，你仍然会得到相同的 IPC。^[1] 这可能听起来很令人困惑，特别是 IPC 与 CPU 时钟密切相关。然而，频率只告诉我们单个时钟周期有多快，而 IPC 计算每个周期完成多少工作。因此，从基准测试的角度来看，IPC 完全取决于处理器的设计，与频率无关。乱序核心通常比顺序核心具有更高的 IPC。当你增加 CPU 缓存大小或改进分支预测时，IPC 通常会上升。

现在，如果你问硬件架构师，他们肯定会告诉你 IPC 和频率之间存在依赖关系。从 CPU 设计的角度来看，你可以故意降低处理器时钟，这将使每个周期更长，并使每个周期能够挤入更多工作。最终，你会得到更高的 IPC 但更低的频率。硬件厂商以不同的方式处理这个性能方程。例如，Intel 和 AMD 芯片通常具有非常高的频率，最近的 Intel 13900KS 处理器无需超频即可提供 6 GHz 的 turbo 频率。另一方面，Apple M1/M2 芯片频率较低，但通过更高的 IPC 来补偿。较低的频率有助于降低功耗。而较高的 IPC 通常需要更复杂的设计、更多的晶体管 and 更大的芯片面积。我们不会在这里讨论所有的设计权衡，因为它们是另一本书的主题。

IPC 对于评估硬件和软件效率都很有用。硬件工程师使用此指标比较 CPU 代际和不同供应商的 CPU。由于 IPC 是 CPU 微架构性能的度量，工程师和媒体使用它来表示相对于前一代的收益。但是，为了进行公平的比较，你需要在相同的频率下运行两个系统。

IPC 也是评估软件的有用指标。它让你直观地了解应用程序中的指令在 CPU 流水线中进展的速度。在本章后面，你将看到几个具有不同 IPC 的生产应用程序。内存密集型应用程序通常具有较低的 IPC (0-1)，而计算密集型工作负载往往具有较高的 IPC (4-6)。

Linux perf 用户可以通过运行以下命令来度量其工作负载的 IPC：

```
“bash $ perf stat -e cycles,instructions - a.exe
```

4.4 微操作

具有 x86 架构的微处理器将复杂的 CISC 指令转换为简单的 RISC 微操作，简称为 μops 。像 `ADD rax, rbx` 这样的简单寄存器到寄存器加法指令只生成一个 μop ，而像 `ADD rax, [mem]` 这样更复杂的指令可能生成两个：一个用于从 `mem` 内存位置加载到临时（未命名）寄存器，另一个用于将其加到 `rax` 寄存器。指令 `ADD [mem], rax` 生成三个 μops ：一个用于从内存加载，一个用于加法，一个用于将结果存储回内存。

将指令分解为微操作的主要优点是 μops 可以被执行：

- **乱序**：考虑 `PUSH rbx` 指令，它将栈指针减 8 字节，然后将源操作数存储在栈顶。假设 `PUSH rbx` 在解码后被“破解”为两个依赖的微操作：

```
SUB rsp, 8
STORE [rsp], rbx
```

通常，函数序言通过使用多个 `PUSH` 指令来保存多个寄存器。在我们的例子中，下一个 `PUSH` 指令可以在前一个 `PUSH` 指令的 `SUB  $\mu\text{op}$` 完成后开始执行，而不必等待 `STORE  $\mu\text{op}$` ，后者现在可以异步执行。

- **并行**：考虑 `HADDPD xmm1, xmm2` 指令，它将 `xmm1` 和 `xmm2` 中的两个双精度浮点值相加（归约），并将两个结果存储在 `xmm1` 中，如下所示：

```
xmm1[63:0] = xmm2[127:64] + xmm2[63:0]
xmm1[127:64] = xmm1[127:64] + xmm1[63:0]
```

对该指令进行微编码的一种方式是:1) 归约 `xmm2` 并将结果存储在 `xmm_tmp1[63:0]` 中, 2) 归约 `xmm1` 并将结果存储在 `xmm_tmp2[63:0]` 中, 3) 将 `xmm_tmp1` 和 `xmm_tmp2` 合并到 `xmm1` 中。总共三个 μops 。注意步骤 1) 和 2) 是独立的, 因此可以并行完成。

尽管我们刚刚讨论了指令如何被分解为更小的部分, 但有时 μops 也可以融合在一起。现代 x86 CPU 中有两种融合类型:

- **微融合**: 融合来自同一机器指令的 μops 。微融合只能应用于两种组合: 内存写操作和读-修改操作。例如:

```
add    eax, [mem]
```

该指令中有两个 μops : 1) 读取内存位置 `mem`, 2) 将其加到 `eax`。通过微融合, 两个 μops 在解码步骤中融合为一个。

- **宏融合**: 融合来自不同机器指令的 μops 。在某些情况下, 解码器可以将算术或逻辑指令与后续的条件跳转指令融合为单个计算-分支 μop 。例如:

```
.loop:
    dec rdi
    jnz .loop
```

4.5 流水线槽

一些性能工具使用的另一个重要指标是流水线槽的概念。流水线槽表示处理一个 μop 所需的硬件资源。图 fig. 4.1 演示了一个每个周期有 4 个分配槽的 CPU 的执行流水线。这意味着核心可以为 4 个新的 μops 分配执行资源 (重命名的源和目标寄存器、执行端口、ROB 条目等)。这样的处理器通常称为 4 宽机器。在图中的六个连续周期中, 只有一半的可用槽被利用 (黄色高亮显示)。从微架构的角度来看, 执行此类代码的效率只有 50%。



Figure 4.1: 4 宽 CPU 的流水线图。

Intel 的 Skylake 和 AMD Zen3 核心具有 4 宽分配。Intel 的 Sunny Cove 微架构是 5 宽设计。截至 2023 年底, 最新的 Golden Cove 和 Zen4 架构都具有 6 宽分配。Apple M1 和 M2 设计是 8 宽, Apple M3 是 9- μop 执行带宽 (参见 [Apple, 2024, Table 4.10])。机器的宽度为 IPC 设定了上限。这意味着处理器可实现的最大 IPC 等于其宽度。⁴⁷ 例如, 当你的计算显示 Golden Cove 核心上超过 6 IPC 时, 你应该持怀疑态度。

⁴⁷ 虽然有一些例外。例如, 宏融合的比较-分支指令只需要一个流水线槽, 但被计为两条指令。在某些极端情况下, 这可能导致 IPC 大于机器宽度。

很少有应用程序能达到机器的最大 IPC。例如，Intel Golden Cove 核心理论上每个时钟可以执行四个整数加法/减法，加上一个加载，加上一个存储（总共六条指令），但应用程序不太可能具有适当的独立指令混合来利用所有这些潜在的并行性。

流水线槽利用率是 Top-down 微架构分析（参见 Section 6.1）的核心指标之一。例如，前端绑定和后端绑定指标表示为由于各种瓶颈而未利用的流水线槽的百分比。

4.6 核心周期与参考周期

大多数 CPU 使用时钟信号来调节其顺序操作。时钟信号由外部生成器产生，该生成器每秒提供一致数量的脉冲。时钟脉冲的频率决定了 CPU 执行指令的速率。因此，时钟越快，CPU 每秒执行的指令就越多。

$$Frequency = \frac{Clockticks}{Time}$$

大多数现代 CPU（包括 Intel 和 AMD CPU）没有固定的运行频率。相反，它们实现了动态频率缩放，在 Intel CPU 中称为 *Turbo Boost*，在 AMD 处理器中称为 *Turbo Core*。它使 CPU 能够动态增加和减少其频率。降低频率以性能为代价降低功耗，而增加频率提高性能但牺牲了节能。

核心时钟周期计数器以 CPU 核心实际运行的频率计算时钟周期。参考时钟事件按处理器以基本频率运行的方式计算周期。让我们看看在 Skylake i7-6000 处理器上运行单线程应用程序的实验，该处理器的基本频率为 3.4 GHz：

```
$ perf stat -e cycles,ref-cycles -- ./a.exe
43340884632 cycles          # 3.97 GHz
37028245322 ref-cycles     # 3.39 GHz
10,899462364 seconds time elapsed
```

`ref-cycles` 事件按没有频率缩放的方式计算周期。平台上的外部时钟频率为 100 MHz，如果我们按时钟倍频器缩放它，我们将获得处理器的基本频率。Skylake i7-6000 处理器的时钟倍频器等于 34：这意味着对于每个外部脉冲，当 CPU 以基本频率运行时（即 3.4 GHz），它执行 34 个内部周期。

`cycles` 事件计算实际 CPU 周期并考虑频率缩放。使用上面的公式，我们可以确认平均工作频率为 43340884632 周期 / 10.899 秒 = 3.97 GHz。当你比较一小段代码的两个版本的性能时，以时钟周期而不是纳秒来度量时间更好，因为你可以避免时钟频率上下波动的问题。

4.7 缓存未命中

如 Section 3.6 中所讨论的，在特定缓存级别未命中的任何内存请求必须由更高级别的缓存或 DRAM 提供服务。这意味着此类内存访问的延迟显著增加。内存子系统组件的典型延迟如表 {tbl. ??} 所示。当内存请求在最后一级缓存（LLC）中未命中并一路下降到主内存时，性能会受到严重影响。^[3]

内存层次结构组件延迟（	周期/时间）
L1 缓存 4	周期（~1 纳秒）
L2 缓存 10	~25 周期（5-10 纳秒）
L3 缓存 ~4	0 周期（20 纳秒）

内存层次结构组件延迟 (	周期/时间)
主内存 20	0+ 周期 (100 纳秒)

表: x86 平台上内存子系统的典型延迟。{#tbl:mem_latency}

指令和数据获取都可能在缓存中未命中。根据 Top-down 微架构分析 (参见 Section 6.1), 指令缓存 (I-cache) 未命中被描述为前端停顿, 而数据缓存 (D-cache) 未命中被描述为后端停顿。指令缓存未命中发生在 CPU 流水线中非常早的指令获取阶段。数据缓存未命中发生在指令执行阶段的较晚时候。

Linux perf 用户可以通过运行以下命令收集 L1 缓存未命中数:

```
$ perf stat -e
    mem_load_retired.fb_hit,mem_load_retired.l1_miss,
    mem_load_retired.l1_hit,mem_inst_retired.all_loads --
    a.exe
29580 mem_load_retired.fb_hit
19036 mem_load_retired.l1_miss
497204 mem_load_retired.l1_hit
546230 mem_inst_retired.all_loads
```

以上是 L1 数据缓存和填充缓冲区的所有加载的细分。加载可能命中已分配的填充缓冲区(fb_hit)、命中 L1 缓存 (l1_hit), 或者两者都未命中 (l1_miss), 因此 all_loads = fb_hit + l1_hit + l1_miss。[²] 我们可以看到只有 3.5% 的所有加载在 L1 缓存中未命中, 因此 L1 命中率为 96.5%。

我们可以通过运行以下命令进一步细分 L1 数据未命中并分析 L2 缓存行为:

```
“bash $ perf stat -e mem_load_retired.l1_miss,
```

4.8 分支预测错误

现代 CPU 试图预测条件分支指令的结果 (已采取或未采取)。例如, 当处理器看到这样的代码时:

```
dec eax
jz .zero
# eax 不为 0
...
zero:
# eax 为 0
```

在上面的示例中, jz 指令是一个条件分支。为了提高性能, 现代处理器每次看到分支指令时都会尝试猜测结果。这称为推测执行, 我们在 Section 3.3.3 中讨论过。处理器将推测, 例如, 分支不会被采取, 并执行对应于 eax 不为 0 情况的代码。但是, 如果猜测错误, 这称为分支预测错误, CPU 需要撤销它最近完成的所有推测工作。

分支预测错误通常涉及 10 到 25 个时钟周期的惩罚。首先, 所有基于错误预测获取和执行的指令需要从流水线中刷新。之后, 某些缓冲区可能需要清理以从错误推测开始的状态恢复。

Linux perf 用户可以通过运行以下命令检查分支预测错误数:

```
$ perf stat -e branches,branch-misses -- a.exe
358209 branches
14026 branch-misses # 3,92% of all branches
# 或者简单地:
$ perf stat -- a.exe
```

4.9 性能指标

能够收集各种性能事件对性能分析非常有帮助。但是，有一个注意事项。假设你运行了一个程序并收集了 MEM_LOAD_RETIREDD.L3_MISS 事件，该事件计算 LLC 未命中数，它显示的值为十亿。当然，这听起来很多，所以你决定调查这些缓存未命中的来源。错了！你确定这是一个问题吗？如果一个程序只进行二十亿次加载，那么是的，这是一个问题，因为一半的加载在 LLC 中未命中。相反，如果一个程序进行一万亿次加载，那么只有一千分之一的加载导致 L3 缓存未命中。

这就是为什么除了硬件性能事件之外，性能工程师还经常使用建立在原始事件之上的指标。表 {tbl. ??} 显示了 Intel 第 12 代 Golden Cove 架构的指标列表，以及描述和公式。该列表并不详尽，但它显示了最重要的指标。Intel CPU 的完整指标列表及其公式可以在 TMA_metrics.xlsx 中找到。^[1] Section 4.11 展示了如何在实践中使用性能指标。

指标名称	描述	公式
L1MPKI	L1 缓存真实未命中每千条退休需求加载的 I 未命中数。	$1000 * \frac{\text{MEM_LOAD_RETIREDD.L1_MISS_PS}}{\text{NST_RETIREDD.ANY}}$
L2MPKI	L2 缓存真实未命中每千条退休需求加载的 I 未命中数。	$1000 * \frac{\text{MEM_LOAD_RETIREDD.L2_MISS_PS}}{\text{NST_RETIREDD.ANY}}$
L3MPKI	L3 缓存真实未命中每千条退休需求加载的 I 未命中数。	$1000 * \frac{\text{MEM_LOAD_RETIREDD.L3_MISS_PS}}{\text{NST_RETIREDD.ANY}}$
分支预测错误率	所有分支中预测错误的 B 比率 B	$\frac{\text{R_MISP_RETIREDD.ALL_BRANCHES}}{\text{R_INST_RETIREDD.ALL_BRANCHES}}$
代码 STLB MPKI	STLB (二级 TLB) 代码推测未命中每千条 / 指令数 (完成页表遍历的任何页面大小的未命中数)	$1000 * \frac{\text{ITLB_MISSES.WALK_COMPLETED}}{\text{INST_RETIREDD.ANY}}$
加载 STLB MPKI	STLB 数据加载推测未命中每千条 / 指令数	$1000 * \frac{\text{DTLB_LD_MISSES.WALK_COMPLETED}}{\text{INST_RETIREDD.ANY}}$
存储 STLB MPKI	STLB 数据存储推测未命中每千条 / 指令数	$1000 * \frac{\text{DTLB_ST_MISSES.WALK_COMPLETED}}{\text{INST_RETIREDD.ANY}}$

表: Intel 第 12 代 Golden Cove 架构的性能指标。{#tbl:perf_metrics}

4.10 内存延迟和带宽

低效的内存访问通常是现代环境中的主要性能瓶颈。因此，处理器从内存子系统获取数据的速度是决定应用程序性能的关键因素。内存性能有两个方面：1) CPU 从内存获取单个字节的速度（延

迟)，2) 每秒可以获取多少字节（带宽）。这两个方面在各种场景中都很重要；我们稍后将看一些例子。在本节中，我们将专注于度量内存子系统组件的峰值性能。

在 x86 平台上可能有用的工具之一是 Intel 内存延迟检查器 (MLC)，^[1] 它在 Windows 和 Linux 上免费可用。MLC 可以使用不同的访问模式和负载来度量缓存和内存延迟和带宽。在基于 ARM 的系统上没有类似的工具，但是用户可以从源代码下载和构建内存延迟和带宽基准测试。此类项目的例子有 lmbench^[2]、bandwidth^[4] 和 Stream。^[3]

我们将只关注一部分指标，即空闲读取延迟和读取带宽。让我们从读取延迟开始。空闲意味着在我们进行度量时，系统是空闲的。这将给我们从内存系统组件获取数据所需的最短时间，但是当系统被其他“内存饥渴”应用程序加载时，由于在各个点可能有更多资源排队，访问延迟会增加。MLC 通过执行依赖加载（也称为指针追逐）来度量空闲延迟。一个度量线程分配一个非常大的缓冲区并初始化它，使缓冲区中的每个（64 字节）缓存行包含指向缓冲区中另一个但不相邻的缓存行的指针。通过适当调整缓冲区大小，我们可以确保几乎所有的加载都命中缓存的某个级别或在主内存中。

我的被测系统是一台 Intel Alder Lake 计算机，配备 Core i7-1260P CPU 和 16GB DDR4 @ 2400 MT/s 双通道内存。该处理器具有 4 个 P（性能）超线程核心和 8 个 E（高效）核心。每个 P 核心有 48 KB 的 L1 数据缓存和 1.25 MB 的 L2 缓存。每个 E 核心有 32 KB 的 L1 数据缓存，四个 E 核心形成一个集群，可以访问共享的 2 MB L2 缓存。系统中的所有核心都由 18 MB 的 L3 缓存支持。如果我们使用 10 MB 的缓冲区，我们可以几乎确定对该缓冲区的重复访问会在 L2 中未命中但在 L3 中命中。以下是示例 mlc 命令：

```
$ sudo ./mlc --idle_latency -c0 -L -b10m
Intel(R) Memory Latency Checker - v3.10
Command line parameters: --idle_latency -c0 -L -b10m
Using buffer size of 10.000MiB
Each iteration took 31.1 base frequency clocks (
    12.5    ns)
```

选项 `--idle_latency` 在不加载系统的情况下度量读取延迟。此外，MLC 有 `--loaded_latency` 选项来度量当其他线程产生内存流量时的延迟。选项 `-c0` 将度量线程固定到逻辑 CPU 0，该 CPU 位于 P 核心上。选项 `-L` 启用大页以限制我们度量中的 TLB 影响。选项 `-b10m` 告诉 MLC 使用 10MB 缓冲区，该缓冲区将适合我们系统上的 L3 缓存。

图 fig. 4.2 显示了 L1、L2 和 L3 缓存的读取延迟。图表上有四个不同的区域。左侧从 1 KB 到 48 KB 缓冲区大小的第一个区域对应于 L1 D-cache，它是每个物理核心私有的。我们可以观察到 E 核心的延迟为 0.9 纳秒，P 核心略高为 1.1 纳秒。此外，我们可以使用此图表来确认缓存大小。注意 E 核心延迟在缓冲区大小超过 32 KB 后开始攀升，而 P 核心延迟在 48 KB 之前保持恒定。这确认了 E 核心中的 L1 D-cache 大小为 32 KB，P 核心中为 48 KB。

第二个区域显示 L2 缓存延迟，E 核心的延迟几乎是 P 核心的两倍（5.9 纳秒 vs. 3.2 纳秒）。对于 P 核心，延迟在超过 1.25 MB 缓冲区大小后增加，这是预期的。我们预计 E 核心延迟在达到 2 MB 之前保持不变，但是根据我们的测量，它发生得更早。

从 2 MB 到 14 MB 的第三个区域对应于 L3 缓存延迟，两种核心类型大约为 12 纳秒。系统中所有核心共享的 L3 缓存总大小为 18 MB。有趣的是，我们从 15 MB 开始看到一些意外的动态，而不是 18 MB。这很可能与某些访问在 L3 中未命中并需要从主内存获取有关。

我没有显示图表中对应于内存延迟的部分，该部分在超过 18MB 边界后开始。延迟开始急剧攀升，

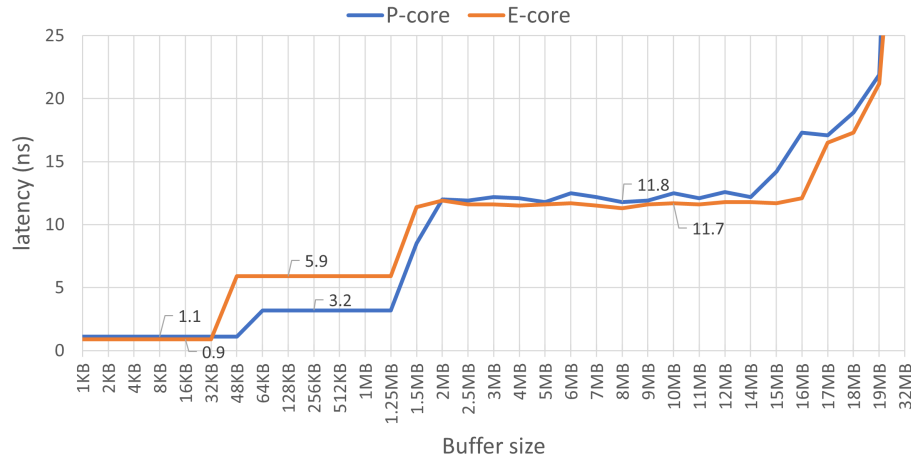


Figure 4.2: Intel Core i7-1260P 上的 L1/L2/L3 缓存读取延迟（越低越好），使用 MLC 工具测量，启用大页。

E 核心在 24 MB 处趋于平稳，P 核心在 64 MB 处趋于平稳。使用更大的缓冲区大小，例如 500 MB，E 核心访问延迟为 45 纳秒，P 核心为 90 纳秒。这度量的是内存延迟，因为几乎没有加载命中 L3 缓存。

使用类似的技术，我们可以度量内存层次结构各种组件的带宽。对于度量带宽，MLC 执行加载请求，其结果不被任何后续指令使用。这允许 MLC 生成最大可能的带宽。MLC 在每个配置的逻辑处理器上生成一个软件线程。每个线程访问的地址是独立的，线程之间没有数据共享。与延迟实验一样，线程使用的缓冲区大小决定了 MLC 是度量 L1/L2/L3 缓存带宽还是内存带宽。

```
$ sudo ./mlc --max_bandwidth -k0-15 -Y -L -u -b18m
Measuring Maximum Memory Bandwidths for the system
Bandwidths are in MB/sec (1 MB/sec = 1,000,000
Bytes/sec)
Using all the threads from each core if Hyper-threading
is enabled
Using traffic with the following read-write ratios
ALL Reads : 349670.42
```

4.11 案例研究：分析四个基准测试的性能指标

为了综合本章到目前为止讨论的所有内容，让我们看一些真实世界的例子。我们运行了来自不同领域的四个基准测试，并计算了它们的性能指标。首先，让我们介绍这些基准测试。

1. Blender 3.4 - 一个开源的 3D 创作和建模软件项目。此测试使用 BMW27 混合文件进行 Blender 的 Cycles 性能测试。使用所有硬件线程。URL: <https://download.blender.org/release>。命令行: `./blender -b bmw27_cpu.blend -noaudio --enable-autoexec -o output.test -x 1 -F JPEG -f 1`。
2. Stockfish 15 - 一个高级开源国际象棋引擎。此测试是 stockfish 内置基准测试。使用单个硬件线程。URL: <https://stockfishchess.org>。命令行: `./stockfish bench`

128 1 24 default depth。

3. Clang 15 自构建 - 此测试使用 Clang 15 从源代码构建 Clang 15 编译器。使用所有硬件线程。URL: <https://www.llvm.org>。命令行: `ninja -j16 clang`。
4. CloverLeaf 2018 - 一个拉格朗日-欧拉流体动力学基准测试。使用所有硬件线程。此测试使用 `clover_bm.in` 输入文件(问题 5)。URL: <http://uk-mac.github.io/CloverLeaf>。命令行: `./clover_leaf`。

对于这个练习,我在具有以下特征的机器上运行了所有四个基准测试:

- 第 12 代 Alder Lake Intel® Core™ i7-1260P CPU @ 2.10GHz (4.70GHz Turbo), 4P+8E 核心, 18MB L3 缓存
- 16 GB RAM, DDR4 @ 2400 MT/s
- 256GB NVMe PCIe M.2 SSD
- 64 位 Ubuntu 22.04.1 LTS (Jammy Jellyfish)
- Clang-15 C++ 编译器,使用以下选项: `-O3 -march=core-avx2`

为了收集性能指标,我使用了 Andi Kleen 的 `pmu-tools` 中的 `toplev.py` 脚本: [1]

```
$ ~/workspace/pmu-tools/toplev.py -m --global --no-desc
-v -- <app with args>
```

表 {tbl. ??} 提供了我们的四个基准测试的性能指标的并排比较。仅通过查看指标,我们就可以了解这些工作负载的性质。

指标名称核心	类型	Blen	der Stoc 自 构建	kfish Clan	g15- Clov	erLeaf
指令数 P 核	心	6.02	E+12 6.59	E+11 2.40	E+13 1.06	E+12
核心周期 P 核	心	4.31	E+12 3.65	E+11 3.78	E+13 5.25	E+12
IPC P 核	心	1.40	1.80	0.64	0.20	
CPI P 核	心	0.72	0.55	1.57	4.96	
指令数 E 核	心	4.97	E+12 0	1.43	E+13 1.11	E+12

表: 四个基准测试的性能指标比较。{#tbl:perf_metrics_case_study}

问题和练习

1. CPU 核心时钟和参考时钟之间有什么区别?
2. 退休指令和执行指令之间有什么区别?
3. 当你增加频率时,IPC 是上升、下降还是保持不变?
4. 查看表 {tbl. ??} 中的 DRAM BW Use 公式。你认为为什么有一个常数 64?
5. 使用 Intel MLC、Stream 或其他工具度量你用于开发/基准测试的机器上的缓存层次结构和内存的带宽和延迟。
6. 运行你日常使用的应用程序。收集性能指标。有什么让你惊讶的吗?

容量规划练习: 假设你是我们在案例研究中基准测试的四个应用程序的所有者。你公司的管理层要求你为每个应用程序构建一个小型计算集群,主要目标是最大性能(吞吐量)。给你的预算很紧,但

足以购买 1 台中端服务器系统 (Mac Studio、Supermicro/Dell/HPE 服务器机架等) 或 1 台高端台式机 (配备超频 CPU、液冷、顶级 GPU、快速 DRAM) 来运行每个工作负载, 总共 4 台机器。这些可以是所有四种不同的系统。另外, 你可以用这笔钱购买 3-4 台低端系统; 选择权在你。管理层希望将每个应用程序的成本控制在 10,000 美元以下, 但如果你能证明费用的合理性, 他们很灵活 (10-20%)。假设 Stockfish 仍然是单线程的。再次查看四个应用程序的性能特征, 并记下你将为每个工作负载购买哪些计算机部件 (CPU、内存、独立 GPU (如果需要))。你将优先考虑哪些参数? 你会在最昂贵的部分上花多少钱? 在哪里可以省钱? 尝试尽可能详细地描述, 并在网上搜索确切的组件及其价格。考虑系统的所有组件: 主板、磁盘驱动器、冷却解决方案、电源装置、机架/机箱/塔式等。你会运行哪些额外的性能实验来指导你的决定?

章节总结

- 在本章中, 我们介绍了性能分析中的基本指标, 如退休/执行指令、CPU 利用率、IPC/CPI、 μops 、流水线槽、核心/参考时钟、缓存未命中和分支预测错误。我们展示了如何使用 Linux perf 收集这些指标。
- 对于更高级的性能分析, 有许多衍生指标可以收集。例如, 每千条指令的缓存未命中数 (MPKI)、每函数调用指令数、分支、加载等 (I_p^*)、ILP、MLP 等。本章中的案例研究展示了如何通过分析这些指标获得可操作的见解。
- 仅通过查看汇总数据就得出结论要小心。不要陷入“Excel 性能工程”的陷阱, 即只收集性能指标而不查看代码。始终寻求第二个数据源 (例如, 稍后讨论的性能剖析) 来验证你的想法。
- 内存带宽和延迟是当今许多生产软件包性能的关键因素, 包括 AI、HPC、数据库和许多通用应用程序。内存带宽取决于 DRAM 速度 (以 MT/s 为单位) 和内存通道数。现代高端服务器平台有 8-12 个内存通道, 整个系统可达 500 GB/s, 单线程模式下可达 50 GB/s。内存延迟如今变化不大, 事实上, 随着新的 DDR4 和 DDR5 代际, 它略有恶化。大多数现代面向客户端的系统每次内存访问的延迟范围在 70-110 纳秒。服务器平台可能具有更高的内存延迟。

5 性能分析方法

当你正在进行高级优化时，例如将更好的算法集成到应用程序中，通常很容易判断性能是否提高，因为基准测试结果通常是显著的。大的加速，如 2 倍、3 倍等，从性能分析的角度来看相对明显。当你从程序中消除大量计算时，你期望看到运行时间的明显差异。

但是，也有这样的情况：你看到执行时间有很小的变化，比如 5%，但你不知道它来自哪里。仅靠计时或吞吐量测量无法解释性能上升或下降的原因。在这种情况下，我们需要更多关于程序如何执行的见解。这就是我们需要进行性能分析来理解我们观察到的减速或加速的底层性质的情况。

性能分析类似于侦探工作。要解决性能谜团，你需要收集所有可用的数据并尝试形成假设。一旦做出假设，你就设计一个实验来证明或反驳它。在你找到线索之前，它可能会来回几次。就像一个好侦探一样，你试图收集尽可能多的证据来确认或反驳你的假设。一旦你有了足够的线索，你就可以对你观察到的行为做出令人信服的解释。

当你刚开始处理性能问题时，你可能只有测量结果，例如代码更改前后的结果。根据这些测量，你得出结论程序变慢了 X%。如果你知道减速发生在某个提交之后，这可能已经给了你足够的信息来修复问题。但如果你没有好的参考点，那么减速的可能原因是无穷无尽的，你需要收集更多数据。收集此类数据最受欢迎的方法之一是分析应用程序并查看热点。本章介绍了这种方法以及其他几种已被证明在性能工程中有用的数据收集方法。

下一个问题是：“有哪些可用的性能数据以及如何收集它们？”堆栈的硬件和软件层都有设施来跟踪性能事件并在程序运行时记录它们。在这个上下文中，硬件是指执行程序的 CPU，软件是指操作系统、库、应用程序本身以及用于分析的其他工具。通常，软件堆栈提供高级指标，如时间、上下文切换次数和页面错误，而 CPU 监控缓存未命中、分支预测错误和其他与 CPU 相关的事件。根据你要解决的问题，某些指标比其他指标更有用。因此，这并不意味着硬件指标总是能给我们提供更精确的程序执行概述。它们只是不同。例如，某些指标（如上下文切换次数）无法由 CPU 提供。性能分析工具（如 Linux perf）可以同时使用来自操作系统和 CPU 的数据。

正如你可能猜测的那样，性能工程师可能使用数百种数据源。本章主要关于收集硬件级信息。我们将介绍一些最流行的性能分析技术：代码检测、跟踪、表征、采样和 Roofline 模型。我们还讨论不涉及运行实际应用程序的静态性能分析技术和编译器优化报告。

5.1 代码检测

可能发明的第一个性能分析方法是代码检测。这是一种将额外代码插入程序以收集特定运行时信息的技术。Listing ?? 显示了在函数开头插入 `printf` 语句以指示该函数是否被调用的最简单示例。之后，你运行程序并计算在输出中看到“foo is called”的次数。也许世界上的每个程序员在他们职业生涯的某个时刻都至少做过一次。

Listing: 代码检测

```
int foo(int x) {  
+ printf("foo is called\n");  
  // 函数体...  
}
```

行首的加号表示该行是添加的，不在原始代码中。通常，检测代码不应该被推送到代码库中；相反，它是用于收集所需数据，之后可以删除。

Listing ?? 展示了一个更有趣的代码检测示例。在这个虚构的代码示例中，函数 `findObject` 在地图上搜索具有某些属性 `p` 的对象的坐标。所有对象都保证最终会被定位。函数 `getNewCoords` 返回在作为参数提供的更大区域内的新坐标。函数 `findObj` 返回使用当前坐标 `c` 定位正确对象的置信度。如果是精确匹配，我们停止搜索循环并返回坐标。如果置信度高于 `threshold`，我们调用 `zoomIn` 来找到对象的更精确位置。否则，我们获取 `searchArea` 内的新坐标以在下次尝试搜索。

检测代码由两个类组成：`histogram` 和 `incrementor`。前者跟踪我们感兴趣的任何变量值及其出现频率，然后在程序完成后打印直方图。后者只是一个帮助类，用于将值推送到 `histogram` 对象中。它很简单，可以快速调整以适应你的特定需求。⁴⁸

Listing: 代码检测

```
+ struct histogram {
+   std::map<uint32_t, std::map<uint32_t, uint64_t>>
+       hist;
+   ~histogram() {
+       for (auto& tripCount : hist)
+           for (auto& zoomCount : tripCount.second)
+               std::cout << "[" << tripCount.first << "]"["
+                   << zoomCount.first << "]" : "
+                   << zoomCount.second << "\n";
+   }
+ };
+ histogram h;

+ struct incrementor {
+   uint32_t tripCount = 0;
+   uint32_t zoomCount = 0;
+   ~incrementor() {
+       h.hist[tripCount][zoomCount]++;
+   }
+ };

Coords findObject(const ObjParams& p, Coords
    searchArea) {
+ incrementor inc;
    Coords c = getNewCoords(searchArea);
    while (true) {
+     inc.tripCount++;
        float match = findObj(p, c);
        if (exactMatch(match))
            return c;
    }
```

⁴⁸ 我有一个稍微高级一些的版本，通常会复制粘贴到我正在处理的任何项目中，之后再删除。

```

    if (match > threshold) {
        searchArea = zoomIn(searchArea, c);
+       inc.zoomCount++;
    } else {
        c = getNewCoords(searchArea);
    }
}
return c;
}

```

在这个假设场景中，我们添加了检测代码来了解在找到对象之前 `zoomIn` 的频率。变量 `inc.tripCount` 计算循环在退出前运行的迭代次数，变量 `inc.zoomCount` 计算我们在同一次循环中减少搜索区域（调用 `zoomIn`）的次数。我们总是期望 `inc.zoomCount` 小于或等于 `inc.tripCount`。

`findObject` 函数使用各种输入被多次调用。以下是运行检测程序后我们可能观察到的输出：

```

// [tripCount][zoomCount]: 出现次数
[7][6]: 2
[7][5]: 6
[7][4]: 20
[7][3]: 156
[7][2]: 967
[7][1]: 3685
[7][0]: 251004
[6][5]: 2
[6][4]: 7
[6][3]: 39
[6][2]: 300
[6][1]: 1235
[6][0]: 91731
[5][4]: 9
[5][3]: 32
[5][2]: 160
[5][1]: 764
[5][0]: 34142
...

```

方括号中的第一个数字是循环的迭代次数，第二个是我们在同一次循环中调用 `zoomIn` 的次数。冒号后的数字是该特定组合出现的次数。例如，有两次我们观察到 7 次循环迭代和 6 次 `zoomIn`。251004 次循环运行了 7 次迭代且没有 `zoomIn`，依此类推。你也可以绘制数据以进行更好的可视化，或采用其他统计方法，但我们可以得出的主要结论是 `zoomIn` 并不频繁。

对 `findObject` 的总调用次数约为 40 万次；我们可以通过对直方图中所有桶求和来计算。如果我们对所有 `zoomCount` 非零的桶求和，得到约 1 万次；这是 `zoomIn` 函数被调用的次数。因此，每次 `zoomIn` 调用，我们大约调用 40 次 `findObject` 函数。

本书后续章节包含许多如何利用此类信息进行优化的示例。在我们的案例中，我们得出结论：`findObj` 经常找不到对象。这意味着循环的下次迭代将尝试使用新坐标但在同一搜索区域内查

找对象。了解这一点后，我们可以尝试一些优化：1) 并行运行多个搜索，如果任何一个成功则同步；2) 为当前搜索区域预计算某些内容，从而消除 `findObj` 内部的重复工作；3) 编写软件流水线，调用 `getNewCoords` 生成下一组所需坐标，并从内存中预取相应的映射位置。本书的第二部分更深入地探讨了其中一些技术。

代码检测在你需要关于程序执行的特定知识时提供了非常详细的信息。它允许我们跟踪程序中每个变量的任何信息。使用这种方法在优化大段代码时通常能获得最佳洞察，因为你可以使用自顶向下的方法（检测主函数然后深入到其调用者）来更好地理解应用程序的行为。代码检测使开发者能够观察应用程序的架构和流程。对于处理不熟悉代码库的人来说，这种技术特别有帮助。

代码检测技术在实时场景（如视频游戏和嵌入式开发）的性能分析中被大量使用。一些分析器将检测与其他技术（如跟踪或采样）相结合。我们将在 Section 7.7 中介绍一种这样的混合分析器 Tracy。

虽然代码检测在许多情况下很强大，但它不提供关于代码如何从操作系统或 CPU 角度执行的任何信息。例如，它不能告诉你进程被调度进出执行的频率（由操作系统知道）或发生了多少次分支误预测（由 CPU 知道）。检测代码是应用程序的一部分，与应用程序本身具有相同的权限。它在用户空间中运行，无法访问内核。

这种技术的一个更重要的缺点是，每次需要检测新的内容（比如另一个变量）时，都需要重新编译。这可能成为负担并增加分析时间。不幸的是，还有其他缺点。由于你通常关心应用程序中的热路径，你正在检测位于代码性能关键部分的内容。在热路径中注入检测代码很容易导致整体基准测试速度降低 2 倍。记住不要对检测过的程序进行基准测试。通过检测代码，你改变了程序的行为，因此你可能看不到之前观察到的相同效果。

以上所有因素都增加了实验之间的间隔时间并消耗了更多开发时间，这就是为什么工程师现在不太手动检测代码的原因。然而，自动化代码检测仍被编译器广泛使用。编译器能够自动检测整个程序（第三方库除外）以收集关于执行的有趣统计信息。自动化检测最广为人知的用例是代码覆盖率分析和配置文件引导优化（参见 Section 11.7）。

在讨论检测时，重要的是提到二进制检测技术。二进制检测背后的思想类似，但它是在已经构建的可执行文件上完成的，而不是在源代码上。有两种类型的二进制检测：静态（提前完成）和动态（检测代码在程序执行时按需插入）。动态二进制检测的主要优点是它不需要程序重新编译和重新链接。此外，使用动态检测，可以将检测量限制在仅感兴趣的代码区域，而不是检测整个程序。

二进制检测在性能分析和调试中非常有用。最流行的二进制检测工具之一是 Intel Pin⁴⁹ 工具。Pin 在发生有趣事件时拦截程序的执行，并从程序中的该点开始生成新的检测代码。这使得能够收集各种运行时信息。基于 Pin 构建的最流行工具之一是 Intel SDE (Software Development Emulator)⁵⁰。另一个知名的二进制检测工具叫做 DynamoRIO⁵¹。以下是使用二进制检测工具可以收集的一些内容：

- 指令计数和函数调用计数
- 指令组合分析
- 拦截函数调用和应用程序中任何指令的执行

⁴⁹ Pin - <https://software.intel.com/en-us/articles/pin-a-dynamic-binary-instrumentation-tool>

⁵⁰ Intel SDE - <https://www.intel.com/content/www/us/en/developer/articles/tool/software-development-emulator.html>

⁵¹ DynamoRIO - <https://github.com/DynamoRIO/dynamorio>。它支持 Linux 和 Windows 操作系统，并在 x86 和 ARM 硬件上运行。

- 内存强度和占用（参见 Section ??）

与代码检测一样，二进制检测只检测用户级代码，并且可能非常慢。

5.2 跟踪

跟踪在概念上与检测非常相似，但略有不同。代码检测假设用户可以完全访问其应用程序的源代码。另一方面，跟踪依赖于现有的检测。例如，`strace` 工具使我们能够跟踪系统调用，可以被视为 Linux 内核的检测。Intel 处理器跟踪（Intel PT，参见附录 C）使你能够记录处理器执行的指令，可以被视为 CPU 的检测。跟踪可以从预先适当检测且不受更改影响的组件中获得。跟踪通常用作黑盒方法，用户无法修改应用程序的代码，但希望了解程序正在做什么。

Listing ?? 提供了使用 Linux `strace` 工具跟踪系统调用的示例，显示了运行 `git status` 命令时的前几行输出。通过使用 `strace` 跟踪系统调用，可以知道每个系统调用的时间戳（最左列）、退出状态（= 号后）和每个系统调用的持续时间（以尖括号为单位）。

Listing: 使用 `strace` 跟踪系统调用。

```
$ strace -tt -T -- git status
17:46:16.798861 execve("/usr/bin/git", ["git",
    "status"], 0x7ffe705dcd78
    /* 75 vars */) = 0 <0.000300>
17:46:16.799493 brk(NULL) =
    0x55f81d929000 <0.000062>
17:46:16.799692 access("/etc/ld.so.nohwcap", F_OK) = -1
    ENOENT
    (No such file or directory) <0.000063>
17:46:16.799863 access("/etc/ld.so.preload", R_OK) = -1
    ENOENT
    (No such file or directory) <0.000074>
17:46:16.800032 openat(AT_FDCWD, "/etc/ld.so.cache",
    O_RDONLY|O_CLOEXEC) = 3
    <0.000072>
17:46:16.800255 fstat(3, {st_mode=S_IFREG|0644,
    st_size=144852, ...}) = 0
    <0.000058>
17:46:16.800408 mmap(NULL, 144852, PROT_READ,
    MAP_PRIVATE, 3, 0)
    = 0x7f6ea7e48000 <0.000066>
17:46:16.800619 close(3) = 0 <0.000123>
...
```

跟踪的开销取决于我们试图跟踪的具体内容。例如，如果我们跟踪一个很少进行系统调用的程序，在 `strace` 下运行它的开销将接近于零。另一方面，如果我们跟踪一个严重依赖系统调用的程序，开销可能非常大，例如 100 倍。⁵² 此外，跟踪可以生成大量数据，因为它不跳过任何样本。为了补偿这一点，跟踪工具提供过滤器，使你能够将数据收集限制在特定的时间段或特定的代码部分。

⁵² B. Gregg 关于 `strace` 的文章 - <http://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html>

与检测类似，跟踪可用于探索系统中的异常。例如，你可能想确定应用程序在 10 秒无响应期间发生了什么。正如你稍后将看到的，采样方法不是为此设计的，但使用跟踪，你可以看到导致程序无响应的原因。例如，使用 Intel PT，你可以重建程序的控制流，并准确知道执行了哪些指令。

跟踪对于调试也非常有用。其底层特性支持基于记录跟踪的“记录和重放”用例。其中一个工具是 Mozilla rr⁵³ 调试器，它执行进程的记录和重放，支持向后单步执行等等。大多数跟踪工具能够用时间戳修饰事件，这使我们能够找到与当时发生的外部事件的关联。也就是说，当我们观察到程序中的故障时，我们可以查看应用程序的跟踪，并将该故障与当时整个系统中发生的情况关联起来。

5.3 收集性能监控事件

性能监控计数器（PMCs）是低级性能分析的重要工具。它们可以提供关于程序执行的独特信息。PMCs 通常以两种模式使用：“计数”或“采样”。计数模式主要用于计算我们在 Section 4.9 中讨论的各种性能指标。采样模式用于查找热点，我们稍后将讨论。

计数背后的思想非常简单：我们想在程序运行时计算某些性能监控事件的总数。PMCs 在 Top-down 微架构分析（TMA）方法中被大量使用，我们将在 Section 6.1 中详细讨论。图 fig. 5.1 说明了从程序开始到结束计算性能事件的过程。

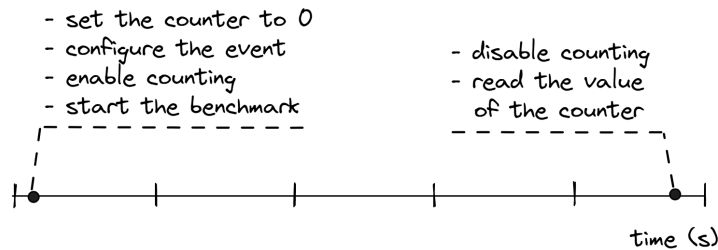


Figure 5.1: 计算性能事件。

图 fig. 5.1 中概述的步骤大致代表了典型分析工具计算性能事件的过程。perf stat 工具中实现了类似的过程，该工具可用于计算各种硬件事件，如指令数、周期数、缓存未命中数等。以下是 perf stat 输出的示例：

```
$ perf stat -- ./my_program.exe
10580290629 cycles          # 3,677 GHz
8067576938 instructions    # 0,76 insn per cycle
3005772086 branches        # 1044,472 M/sec
239298395 branch-misses    # 7,96% of all branches
```

这些数据可能变得非常有用。首先，它使我们能够快速发现一些异常，例如高分支预测错误率或低 IPC。此外，当你进行了代码更改并想验证更改是否提高了性能时，它可能派上用场。查看相关事件可能有助于你证明或拒绝代码更改。perf stat 工具可用作轻量级基准测试包装器。它可以作为性能调查的第一步。有时可以立即发现异常，这可以为你节省一些分析时间。

可用事件名称的完整列表可以通过 perf list 查看：

⁵³ Mozilla rr 调试器 - <https://rr-project.org/>。

```
$ perf list
cycles          [Hardware event]
ref-cycles      [Hardware event]
instructions    [Hardware event]
branches       [Hardware event]
branch-misses   [Hardware event]
...
cache:
  mem_load_retired.ll_hit
  mem_load_retired.ll_miss
  ...
```

现代 CPU 有数百个可观察的性能事件。很难记住所有这些事件及其含义。理解何时使用特定事件更加困难。这就是为什么通常我不建议手动收集特定事件，除非你真的知道自己在做什么。相反，我建议使用 Intel VTune Profiler 等工具，它们可以自动收集所需事件来计算各种指标。

性能事件并非在每个环境中都可用，因为访问 PMCs 需要 root 权限，而虚拟化环境中运行的应用程序通常没有这个权限。对于在公共云中执行的程序，如果虚拟机 (VM) 管理器没有向客户机正确暴露 PMU 编程接口，直接在客户机容器中运行基于 PMU 的分析器不会产生有用的输出。因此，基于 CPU 性能监控计数器的分析器在虚拟化和云环境中工作得不太好 [Du et al., 2010]，尽管情况正在改善。VMware® 是最早启用⁵⁴虚拟性能监控计数器 (vPMC) 的 VM 管理器之一。AWS EC2 云也为专用主机启用了⁵⁵ PMCs。

5.3.1 使用标记 API

在某些场景中，我们可能对分析特定代码区域的性能感兴趣，而不是整个应用程序。当你正在开发一段新代码并希望只关注该代码时，就会出现这种情况。自然地，你希望跟踪优化进度并捕获额外的性能数据来帮助你。大多数性能分析工具提供特定的标记 API，让你可以做到这一点。以下是两个例子：

- Intel VTune 有 `__itt_task_begin` / `__itt_task_end` 函数。
- AMD uProf 有 `amdProfileResume` / `amdProfilePause` 函数。

这种混合方法结合了检测和性能事件计数的优点。标记 API 允许我们将性能统计归因于代码区域（循环、函数）或功能部分（远程过程调用 (RPC)、输入事件等），而不是度量整个程序。你获得的数据质量很容易证明其价值。例如，在调查仅在特定类型 RPC 中发生的性能错误时，你可以仅为该类型 RPC 启用监控。

下面我们提供一个使用 `libpfm4` 的非常基本的示例，^[1] 这是用于收集性能监控事件的流行 Linux 库之一。它建立在 Linux `perf_events` 子系统之上，该子系统允许你直接访问性能事件计数器。`perf_events` 子系统相当底层，因此 `libpfm4` 包在这里很有用，因为它添加了一个发现工具来识别 CPU 上可用的事件，以及围绕原始 `perf_event_open` 系统调用的包装库。下面的代码列表显示了如何使用 `libpfm4` 来检测 C-Ray^[2] 基准测试的 `render` 函数。

```
“‘cpp + #include <perfmon/pfmlib.h> + #include <perfmon/pfmlib_perf_event.h>
```

⁵⁴ VMware vPMC - https://docs.vmware.com/en/VMware-vSphere/7.0/com.vmware.vsphere.vm_admin.doc/GUID-2E04FD07-8382-4D4E-B7B2-99F1F5AE7F37.html

⁵⁵ AWS EC2 PMCs - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/instance-performance-counters.html>

```

... /* 将 xsz/ysz 维度的一帧渲染到提供的帧缓冲区 / void render(int xsz,
int ysz, uint32_t fb, int samples) { ... + pfm_initialize(); + struct
perf_event_attr perf_attr; + memset(&perf_attr, 0, sizeof(perf_attr));
+ perf_attr.size = sizeof(struct perf_event_attr); + perf_attr.read_format =
PERF_FORMAT_TOTAL_TIME_ENABLED | PERF_FORMAT_TOTAL_TIME_RUNNING
| PERF_FORMAT_GROUP; +
+ pfm_perf_encode_arg_t arg; + memset(&arg, 0, sizeof(pfm_perf_encode_arg_t));
+ arg.size = sizeof(pfm_perf_encode_arg_t); + arg.attr = &perf_attr; +
+ pfm_get_os_event_encoding("instructions", PFM_PLM3, PFM_OS_PERF_EVENT_EXT,
&arg); + int leader_fd = perf_event_open(&perf_attr, 0, -1, -1, 0); +
+ pfm_get_os_event_encoding("cycles", PFM_PLM3, PFM_OS_PERF_EVENT_EXT,
&arg); + int event_fd = perf_event_open(&perf_attr, 0, -1, leader_fd, 0); +
+ pfm_get_os_event_encoding("branches", PFM_PLM3, PFM_OS_PERF_EVENT_EXT,
&arg); + event_fd = perf_event_open(&perf_attr, 0, -1, leader_fd, 0); +
+ pfm_get_os_event_encoding("branch-misses", PFM_PLM3, PFM_OS_PERF_EVENT_EXT,
&arg); + event_fd = perf_event_open(&perf_attr, 0, -1, leader_fd, 0); + +
struct read_format { uint64_t nr, time_enabled, time_running, values[4]; };

```

5.4 采样

采样是最常用的性能分析方法。人们通常将其与查找程序中的热点联系起来。更广泛地说，采样有助于找到代码中贡献最多特定性能事件的地方。如果我们想查找热点，问题可以重新表述为：“找到代码中消耗最多 CPU 周期的地方”。人们经常使用分析一词来描述技术上称为采样的内容。根据 Wikipedia，^[1] 分析是一个更广泛的术语，包括各种收集数据的技术，包括采样、代码检测、跟踪等。

可能会令人惊讶的是，可以想象的最简单的采样分析器是调试器。事实上，你可以按如下方式识别热点：a) 在调试器下运行程序；b) 每 10 秒暂停程序；c) 记录它停止的位置。如果你重复 b) 和 c) 多次，你可以从收集的样本构建直方图。你停止次数最多的代码行将是程序中最热的地方。当然，这不是查找热点的有效方式，我们不建议这样做。这只是为了说明概念。尽管如此，这是真实分析工具工作方式的简化描述。现代分析器每秒可以收集数千个样本，这给出了程序中最热位置的相当准确的估计。

与调试器示例一样，每次捕获新样本时，被分析程序的执行都会被中断。在中断时，分析器收集程序状态的快照，该快照构成一个样本。为每个样本收集的信息可能包括中断时执行的指令地址、寄存器状态、调用栈（参见 Section ??）等。收集的样本存储在转储文件中，该文件可用于显示程序中最耗时的部分、调用图等。

5.4.1 用户模式和基于硬件事件的采样

采样可以使用两种不同的模式执行：用户模式或基于硬件事件的采样（EBS）。用户模式采样是一种纯软件方法，将代理库嵌入到被分析的应用程序中。代理为应用程序中的每个线程设置一个操作系统定时器。定时器到期时，应用程序接收由代理处理的 SIGPROF 信号。EBS 使用硬件 PMC 来触发中断。特别是，使用 PMU 的计数器溢出功能，我们稍后将讨论。

用户模式采样只能用于识别热点，而 EBS 可用于涉及 PMC 的其他分析类型，例如在缓存未命中上采样、Top-down 微架构分析（参见 Section 6.1）等。

用户模式采样比 EBS 产生更高的运行时开销。当以 10ms 的间隔采样时，用户模式采样的平均开销约为 5%，而 EBS 的开销小于 1%。由于开销较少，你可以使用更高的采样率使用 EBS，这将提供更准确的数据。但是，用户模式采样生成的数据较少，处理时间也更短。

5.4.2 查找热点

在本节中，我们将讨论使用 PMC 和 EBS 的机制。图 fig. 5.2 说明了 PMU 的计数器溢出功能，该功能用于触发性能监控中断 (PMI)，也称为 SIGPROF。在基准测试开始时，我们配置要采样的事件。对周期进行采样是许多分析工具的默认设置，因为我们想知道程序在哪里花费了大部分时间。然而，这不是一条严格的规则；我们可以对我们想要的任何性能事件进行采样。例如，如果我们想知道程序经历最多 L3 缓存未命中的地方，我们将在相应的事件上采样，即 MEM_LOAD_RETIRED.L3_MISS。

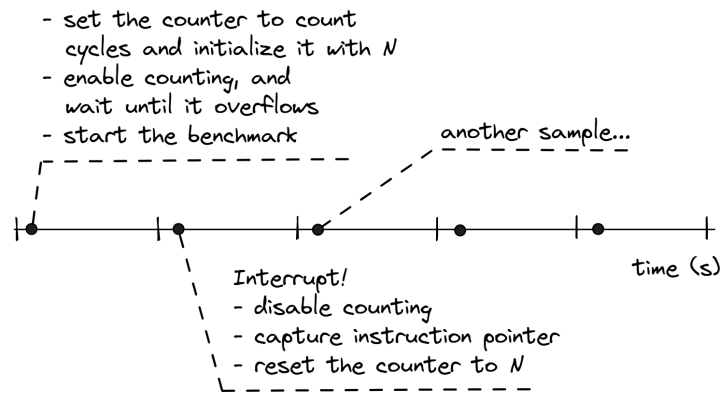


Figure 5.2: 使用性能计数器进行采样

初始化寄存器后，我们开始计数并让基准测试运行。由于我们配置了一个 PMC 来计算周期，它将在每个周期递增。最终，它将溢出。在寄存器溢出时，硬件将发出 PMI。分析工具配置为捕获 PMI，并有一个中断服务例程 (ISR) 来处理它们。我们在 ISR 内执行多个步骤：首先，我们禁用计数；之后，我们记录计数器溢出时 CPU 执行的指令；然后，我们将计数器重置为 N 并恢复基准测试。

现在，让我们回到值 N 。使用此值，我们可以控制我们希望多久获得一次新中断。假设我们想要更细的粒度，每 100 万个周期一个样本。为了实现这一点，我们可以将计数器设置为 (unsigned) -1,000,000，这样它将在每 100 万个周期后溢出。此值也称为 *sample after* 值。

我们重复此过程多次以构建足够的样本集合。如果我们稍后聚合这些样本，我们可以构建程序中最热位置的直方图，如下面 Linux perf record/report 输出中所示的那样。这给出了按降序排序的程序函数开销的细分 (热点)。在 Phoronix 测试套件^[8] 中采样 x264^[7] 基准测试的示例如下：

```
$ time -p perf record -F 1000 -- ./x264 -o /dev/null
--slow --threads 1
../Bosphorus_1920x1080_120fps_420_8bit_YUV.y4m
```

```
[ perf record: Captured and wrote 1.625 MB perf.data
(35035 samples) ]
real 36.20 sec
$ perf report -n --stdio
# Samples: 35K of event 'cpu_core/cycles/'
# Event count (approx.): 156756064947

# Overhead Samples Shared Object Symbol
# .....
.....
7.50%    2620      x264          [.]
x264_8_me_search_ref
7.38%    2577      x264          [.]
refine_subpel.lto_priv.0
```

5.5 Roofline 性能模型

Roofline 性能模型是一种面向吞吐量的性能模型，在 HPC 领域被广泛使用。它于 2009 年在加州大学伯克利分校开发 [Williams et al., 2009]。该模型中的”屋顶线”一词表达了应用程序的性能不能超过机器能力的事实。程序中的每个函数和每个循环都受到机器计算或内存带宽容量的限制。这个概念在图 fig. 5.3 中表示。应用程序的性能将始终受到某个”屋顶线”函数的限制。

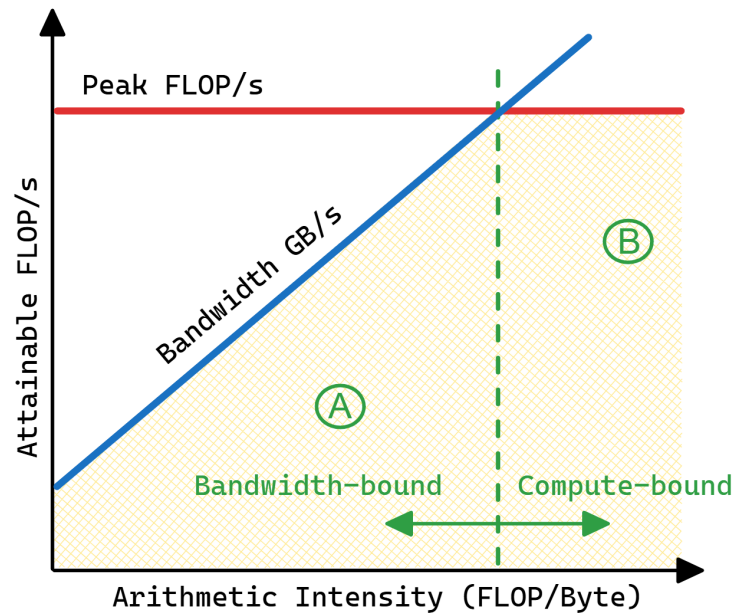


Figure 5.3: Roofline 性能模型。应用程序的最大性能受峰值 FLOPS（水平线）和平台带宽乘以算术强度（对角线）的最小值限制。

硬件有两个主要限制：它能多快进行计算（峰值计算性能，FLOPS）和它能多快移动数据（峰值内

存带宽, GB/s)。应用程序的最大性能受峰值 FLOPS (水平线) 和平台带宽乘以算术强度 (对角线) 的最小值限制。图 fig. 5.3 中的 Roofline 图表绘制了两个应用程序 A 和 B 相对于硬件限制的性能。应用程序 A 具有较低的算术强度, 其性能受内存带宽限制, 而应用程序 B 更计算密集, 不会像那样受到内存瓶颈的影响。类似地, A 和 B 可以表示程序中的两个不同的函数, 并具有不同的性能特征。Roofline 性能模型考虑了这一点, 可以在同一个图表上显示应用程序的多个函数和循环。但是, 请记住, Roofline 性能模型主要适用于具有少量计算密集型循环的 HPC 应用程序。我不建议将其用于通用应用程序, 如编译器、Web 浏览器或数据库。

算术强度是浮点操作 (FLOPs) [^7] 和字节之间的比率, 可以为程序中的每个循环计算。让我们计算 Listing ?? 中代码的算术强度。在最内层循环体中, 我们有一个浮点加法和一个乘法; 因此, 我们有 2 个 FLOP。另外, 我们有一个读操作和一个写操作; 因此, 我们传输 4 操作 * 4 字节 = 16 字节。该代码的算术强度为 $2 / 16 = 0.125$ 。算术强度是 Roofline 图表的 X 轴, 而 Y 轴度量给定程序的性能。

Listing: 朴素并行矩阵乘法。

```
1 void matmul(int N, float a[] [2048], float b[] [2048],
   float c[] [2048]) {
2     #pragma omp parallel for
3     for(int i = 0; i < N; i++) {
4         for(int j = 0; j < N; j++) {
5             for(int k = 0; k < N; k++) {
6                 c[i][j] = c[i][j] + a[i][k] * b[k][j];
7             }
8         }
9     }
10 }
```

加速应用程序性能的传统方法是充分利用机器的 SIMD 和多核能力。通常, 我们需要优化许多方面: 向量化、内存和线程。Roofline 方法可以帮助评估应用程序的这些特征。在 Roofline 图表上, 我们可以绘制标量单核、SIMD 单核和 SIMD 多核性能的理论最大值 (参见图 fig. 5.4)。这将使我们了解改进应用程序性能的范围。如果我们发现我们的应用程序是计算受限的 (即具有高算术强度) 并且低于峰值标量单核性能, 我们应该考虑强制向量化 (参见 Section 9.4) 并将工作分配给多个线程。相反, 如果应用程序具有低算术强度, 我们应该寻求改善内存访问的方法 (参见 Section 8)。使用 Roofline 模型优化性能的最终目标是将点在图表上向上移动。向量化和线程将点向上移动, 而通过增加算术强度来优化内存访问将点向右移动, 并且也可能提高性能。

理论最大值 (屋顶线) 通常在设备规格中给出, 可以轻松查找。此外, 可以根据你使用的机器特性计算理论最大值。通常, 一旦你知道机器的参数, 这并不难做到。对于 Intel Core i5-8259U 处理器, 使用 AVX2 和 2 个融合乘加 (FMA) 单元的最大 FLOPS (单精度浮点数) 可以计算为:

$$\begin{aligned} \text{Peak FLOPS} &= 8 \text{ (number of logical cores)} \times \frac{256 \text{ (AVX bit width)}}{32 \text{ bit (size of float)}} \times \\ &\quad 2 \text{ (FMA)} \times 3.8 \text{ GHz (Max Turbo Frequency)} \\ &= 486.4 \text{ GFLOPS} \end{aligned}$$

Intel NUC Kit NUC8i5BEH 的最大内存带宽 (我用于实验的设备) 可以如下计算。记住,

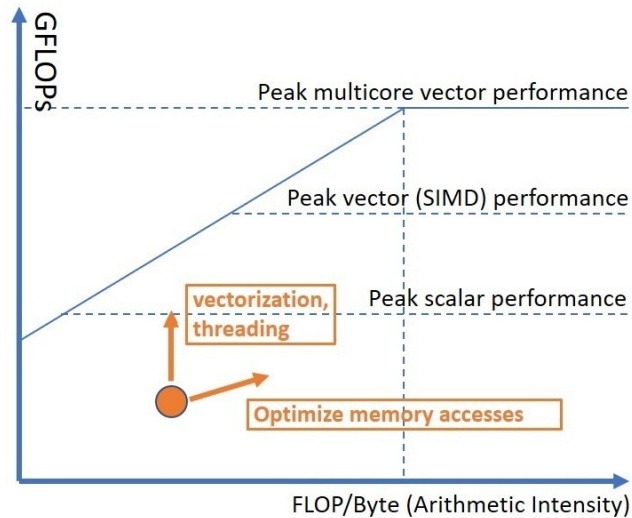


Figure 5.4: 程序的 Roofline 分析及其性能改进的潜在方法。

DDR 技术允许每次内存访问传输 64 位或 8 字节。

5.6 静态性能分析

如今，我们有广泛的静态代码分析工具。对于 C 和 C++ 语言，我们有众所周知的工具，如 Clang 静态分析器、Klocwork、Cppcheck 等。这些工具旨在检查代码的正确性和语义。同样，一些工具试图解决代码的性能方面。静态性能分析器不执行或分析程序。相反，它们模拟代码，就像它在真实硬件上执行一样。静态预测性能几乎是不可能的，因此这种类型的分析有许多限制。

首先，无法对 C/C++ 代码进行静态性能分析，因为我们不知道它将被编译成的机器代码。因此，静态性能分析在汇编代码上工作。

其次，静态分析工具模拟工作负载而不是执行它。这显然非常慢，因此无法静态分析整个程序。相反，工具获取一段汇编代码并尝试预测它在真实硬件上的行为。用户应该选择特定的汇编指令（通常是一个小循环）进行分析。因此，静态性能分析的范围非常窄。

静态性能分析器的输出相当底层，通常将执行分解为 CPU 周期。通常，开发人员使用它来对关键代码区域进行细粒度调优，其中每个 CPU 周期都很重要。

静态分析器与动态分析器

静态工具。它们不运行实际代码，而是尝试模拟执行，保留尽可能多的微架构细节。它们无法进行实际测量（执行时间、性能计数器），因为它们不运行代码。这里的好处是你不需要实际硬件，可以为不同的 CPU 代际模拟代码。另一个好处是你不需要担心结果的一致性：静态分析器总是给你确定性的输出，因为模拟（与在真实硬件上执行相比）没有任何偏差。静态工具的缺点是它们通常无法预测和模拟现代 CPU 中的所有内容：它们基于可能存在错误和限制的模型。静态性能分析器的例子有 UICA^[2] 和 llvm-mca。^[3]

动态工具。它们基于在真实硬件上运行代码并收集关于执行的各种信息。这是证明任何性能假设的唯一 100% 可靠的方法。作为缺点，通常你需要特权访问权限来收集低级性能数据，如 PMC。编写一个好的基准测试并度量你想度量的内容并不总是容易的。最后，你需要过滤噪声和各种副作用。动态微架构性能分析器的两个例子是 nanoBench^[5] 和 uarch-bench。^[4]

更完整的静态和动态微架构性能分析工具集合可在 [此处](#) 找到。^[7]

5.6.1 案例研究：使用 UICA 优化 FMA 吞吐量

开发人员经常问的问题之一是：“最新的处理器有 10 多个执行单元；我如何编写代码来使它们始终保持忙碌？”这确实是最难解决的问题之一。有时它需要仔细研究程序如何运行。UICA 模拟器就是这样一种显微镜，它帮助你深入了解你的代码如何在现代处理器中流动。

让我们看看 Listing ?? 中的代码。我故意尝试使示例尽可能简单。当然，现实世界的代码通常比这更复杂。该代码将数组 `a` 的每个元素缩放浮点值 `B`，并将乘积累加到 `sum` 中。在右边，我展示了 Clang-16 在使用 `-O3 -ffast-math -march=core-avx2` 编译时生成的循环的机器代码。

Listing: FMA 吞吐量

```
float foo(float * a, float B, int N){
    float sum = 0;
    ymm2, ymm1, [rdi + rsi]
    for (int i = 0; i < N; i++)
        ymm3, ymm1, [rdi + rsi + 32]
        sum += a[i] * B;
        ymm4, ymm1, [rdi + rsi + 64]
    return sum;
    ymm5, ymm1, [rdi + rsi + 96]
}

.loop:
    vfmadd231ps
    vfmadd231ps
    vfmadd231ps
    vfmadd231ps
    sub rsi, -128
    cmp rdx, rsi
    jne .loop
```

这是一个归约循环，即我们需要将所有乘积相加，最后返回一个浮点值。这段代码的编写方式存在 `sum` 上的循环携带依赖。在累加前一个乘积之前，你不能覆盖 `sum`。并行化这一点的聪明方法是使用多个累加器，最后将它们相加。因此，不是单个 `sum`，我们可以有 `sum1` 来累加偶数迭代的结果，`sum2` 来累加奇数迭代的结果。

这就是 Clang-16 所做的：它有 4 个向量 (`ymm2-ymm5`)，每个持有 8 个浮点累加器，加上它使用 FMA 将乘法和加法融合为一条指令。常数 `B` 被广播到 `ymm1` 寄存器。`-ffast-math` 选项允许编译器重新关联浮点操作；我们将在 Section 9.4 中讨论此选项如何帮助优化。^[9]

5.7 编译器优化报告

如今，软件开发在很大程度上依赖编译器进行性能优化。编译器在加速软件方面起着关键作用。大多数开发人员将优化代码的工作留给编译器，只在看到改进编译器无法完成的机会时才进行干预。可以说，这是一个很好的默认策略。但当你寻求最佳性能时，它就不那么有效了。如果编译器未能执行关键优化（如向量化循环）怎么办？你如何知道这一点？幸运的是，所有主要编译器都提供优化报告，我们现在将讨论这些报告。

假设你想知道一个关键循环是否被展开。如果它被展开了，展开因子是多少？有一种困难的方法可以知道：研究生成的汇编指令。不幸的是，并非所有人都习惯阅读汇编语言。如果函数很大，它调用其他函数或有许多也被向量化的循环，或者编译器创建了同一循环的多个版本，这可能尤其困难。大多数编译器，包括 GCC、Clang、Intel 编译器和 MSVC^[9] 都提供优化报告，以检查特定代码完成了哪些优化。

让我们看看 Listing ??，它显示了一个未被 clang 16.0 向量化的循环示例。

Listing: a.c

```
1 void foo(float* __restrict__ a,
2         float* __restrict__ b,
3         float* __restrict__ c,
4         unsigned N) {
5     for (unsigned i = 1; i < N; i++) {
6         a[i] = c[i-1]; // 从前一次迭代传递的值
7         c[i] = b[i];
8     }
9 }
```

要在 Clang 编译器中发出优化报告，你需要使用 `-Rpass*` 标志：

```
$ clang -O3 -Rpass-analysis=.* -Rpass=.*
-Rpass-missed=.* a.c -c
a.c:5:3: remark: loop not vectorized
[-Rpass-missed=loop-vectorize]
for (unsigned i = 1; i < N; i++) {
^
a.c:5:3: remark: unrolled loop by a factor of 8 with
run-time trip count [-Rpass=loop-unroll]
for (unsigned i = 1; i < N; i++) {
^
```

通过检查上面的优化报告，我们可以看到该循环没有被向量化，而是被展开了。对于开发人员来说，识别 Listing ?? 中第 6 行循环中的循环携带依赖并不总是容易的。`c[i-1]` 加载的值依赖于前一次迭代的存储（参见图 fig. ?? 中的操作 ② 和 ③）。可以通过手动展开循环的前几次迭代来揭示依赖关系：

“cpp

// 迭代 1 `a[1] = c[0];`

问题和练习

1. 在以下场景中，你会使用哪种性能分析方法？

- 场景 1：客户支持团队报告客户问题：升级到新版本应用程序后，某个操作的性能下降了 10%。
- 场景 2：客户支持团队报告客户问题：某些事务运行时间比其他事务长 2 倍，没有特定模式。
- 场景 3：你正在评估三种不同的压缩算法，你想知道每种算法存在哪些类型的性能瓶颈（内存延迟、计算、分支预测错误等）。

- 场景 4：有一个新的闪亮库声称比你当前集成到项目中的库更快；你决定比较它们的性能。
 - 场景 5：你被要求分析一些不熟悉的代码的性能，这些代码涉及一个热循环；你想知道循环正在执行多少次迭代。
2. 运行你日常使用的应用程序。使用我们在本章讨论的方法练习进行性能分析。收集各种 CPU 性能事件的原始计数，查找热点，收集 Roofline 数据，并为程序中的热函数生成和研究编译器优化报告。

章节总结

- 延迟和吞吐量通常是程序性能的最终指标。当寻求改进它们的方法时，我们需要获得有关应用程序如何执行的更详细信息。硬件和软件都提供可用于性能监控的数据。
- 代码检测使我们能够跟踪程序中的许多事情，但在开发和运行时都会造成相对较大的开销。虽然大多数开发人员不习惯手动检测他们的代码，但这种方法对于自动化过程仍然相关，例如配置文件引导优化（PGO）。
- 跟踪在概念上与检测相似，对于探索系统中的异常很有用。跟踪使我们能够捕获带有时间戳的整个事件序列。
- 性能监控计数器是低级性能分析的重要工具。它们通常以两种模式使用：“计数”或“采样”。计数模式主要用于计算各种性能指标。
- 采样跳过程序的大部分执行，只取一个样本，该样本应该代表整个间隔。尽管如此，采样通常产生足够精确的分布。采样最著名的用例是查找程序中的热点。采样是最流行的分析方法，因为它不需要重新编译程序，并且运行时开销非常小。
- 通常，计数和采样产生的运行时开销非常低（通常低于 2%）。当你开始在不同事件之间多路复用时，计数会变得更昂贵（5-15% 开销），而采样随着采样频率的增加而变得更昂贵 [Nowak & Bitzes, 2014]。
- Roofline 性能模型是一种面向吞吐量的性能模型，在高性能计算（HPC）领域被广泛使用。它将应用程序的性能可视化硬件限制。Roofline 模型有助于识别性能瓶颈，指导软件优化，并跟踪优化进度。
- 有一些工具试图静态分析代码的性能。这些工具模拟一段代码而不是执行它。这种方法有许多限制和约束，但你可以得到一个非常详细和底层的报告作为回报。
- 编译器优化报告有助于查找缺失的编译器优化。

6 用于性能分析的 CPU 特性

性能分析的最终目标是识别性能瓶颈并定位与之相关的代码部分。不幸的是，没有预定的步骤可以遵循，因此可以用许多不同的方法来处理。

通常，分析应用程序可以快速了解应用程序的热点。有时这就是帮助开发人员发现和修复性能问题所需要的。特别是高级性能问题通常可以通过分析来揭示。例如，考虑这样一种情况：你刚刚对应用程序中的函数 `foo` 进行了更改，突然看到明显的性能下降。因此，你决定分析应用程序。根据你对应用程序的心智模型，你期望 `foo` 是一个冷函数，它不会出现在热函数的前 10 名列表中。但当你打开性能剖析时，你看到它消耗的时间比以前多得多。你很快意识到你在代码中犯的错误并修复了它。如果性能工程中的所有问题都那么容易修复，这本书就不会存在。

当你开始从应用程序中榨取最后一丝性能的旅程时，仅仅查看热点列表是不够的。除非你有一个水晶球或整个 CPU 的精确模型在脑海中，否则你需要额外的支持来理解性能瓶颈是什么。

一些开发人员依赖他们的直觉并进行随机实验，尝试强制各种编译器优化，如循环展开、向量化、内联等等。确实，有时你可以很幸运，享受同事的一些赞美，甚至可能在团队中获得非官方的性能大师称号。但通常，你需要有非常好的直觉和运气。在本书中，我们不教你如何幸运。相反，我们展示在实践中被证明有效的方法。

现代 CPU 不断获得新功能，以不同的方式增强性能分析。使用这些功能大大简化了查找底层问题（如缓存未命中、分支预测错误等）。在本章中，我们将看看现代 CPU 上可用的一些硬件性能监控功能。来自不同供应商的处理器不一定具有相同的功能集。我们将探索 Intel、AMD 和 Arm 处理器中可用的性能监控功能。⁵⁶

- **Top-down 微架构分析 (TMA)** 方法，在 Section 6.1 中讨论。这是一种识别程序对 CPU 微架构低效使用的技术。它表征工作负载的瓶颈，并允许定位源代码中发生该问题的确切位置。它抽象了 CPU 微架构的复杂性，即使对于经验不足的开发人员也相对容易使用。
- **分支记录**，在 Section 6.2 中讨论。这是一种在执行程序的同时持续记录最近分支结果的机制。它用于收集调用栈、识别热分支、计算单个分支的预测错误率等。
- **基于硬件的采样**，在 Section 6.3.1 中讨论。这是一种增强采样的功能。其主要优点包括：降低采样开销和提供“精确事件”功能，能够精确定位导致特定性能事件的确切指令。
- **Intel 处理器跟踪 (PT)**，在附录 C 中讨论。这是一种以每条指令的时间戳记录和重建程序执行的设施。其主要用途是事后分析和根本原因性能故障。

Intel PT 功能在附录 C 中介绍。Intel PT 本应是性能分析的“终极方案”。凭借其低运行时开销，它是一个非常强大的分析功能。但事实证明它在性能工程师中并不十分流行。部分原因是工具中的支持还不成熟，部分原因是在许多情况下它是多余的，使用采样分析器更容易。此外，它会产生大量数据，对于长时间运行的工作负载来说不实用。尽管如此，它在某些行业（如高频交易 (HFT)）中很受欢迎。

上述硬件性能监控功能从 CPU 的角度提供了对程序效率的见解。在下一章中，我们将讨论分析工具如何使用这些功能提供许多不同类型的分析。

⁵⁶ RISC-V 生态系统还没有成熟的性能监控基础设施，因此我们不会在本书中介绍它。

6.1 Top-down 微架构分析

Top-down 微架构分析 (TMA) 方法是识别程序中 CPU 瓶颈的强大技术。该方法最好的部分是它不需要开发人员对系统中的微架构和 PMC 有深入的理解, 就能高效地找到 CPU 瓶颈。

在概念层面上, TMA 识别什么在阻止程序执行。图 fig. 6.1 说明了 TMA 的核心思想。以下是如何阅读此图的简短指南。正如我们在 Section 3 中所知, CPU 中有内部缓冲区, 用于跟踪正在执行的 μops 的信息。每当获取和解码一条新指令时, 就会在这些缓冲区中分配新条目。如果指令的 μop 在特定执行周期内未被分配, 可能有两个原因: 要么我们无法获取和解码它 (前端绑定), 要么后端工作过载, 无法为新的 μop 分配资源 (后端绑定)。如果 μop 被分配并调度执行但从未退休, 这意味着它来自预测错误的路径 (错误推测)。最后, 退休代表正常执行。这是我们希望所有 μops 所在的桶, 尽管有例外, 我们稍后会讨论。

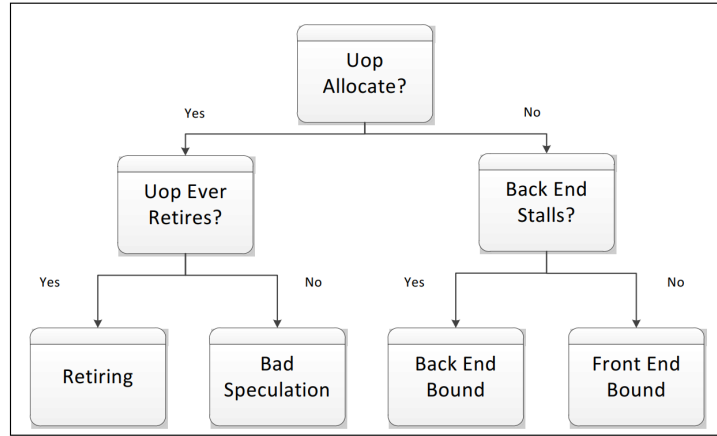


Figure 6.1: TMA 顶层分解背后的概念。© 来源: [Yasin, 2014]

这不是分析在实践中工作的方式, 因为分析每个微操作 (μop) 会非常慢。相反, TMA 通过监控一组特定的性能事件来观察程序的执行, 然后根据预定义的公式计算指标。使用这些指标, TMA 通过将程序分配到四个高级桶之一来表征程序。四个高级类别中的每一个都有几个嵌套级别, CPU 供应商可以选择不同的实现方式。每一代处理器可能有不同的公式来计算这些指标, 因此最好依靠工具进行分析, 而不是自己尝试计算。

在接下来的章节中, 我们将讨论 AMD、Arm 和 Intel 处理器中的 TMA 实现。

6.1.1 Intel 平台上的 TMA

TMA 方法由 Intel 于 2014 年首次提出, 从 Sandy Bridge 系列处理器开始支持。Intel 的实现支持每个高级桶的嵌套类别, 可以更好地理解程序中的 CPU 性能瓶颈 (参见图 fig. 6.2)。

工作流程旨在“深入”到 TMA 层次结构的较低级别, 直到我们获得性能瓶颈的非常具体的分类。首先, 我们收集四个主要桶的指标: 前端绑定、后端绑定、退休和错误推测。假设我们发现程序执行的很大一部分被内存访问阻塞 (这是后端绑定桶, 参见图 fig. 6.2)。下一步是再次运行工作负载, 仅收集内存绑定桶的特定指标。重复此过程, 直到我们确切知道根本原因, 例如 L3 绑定。

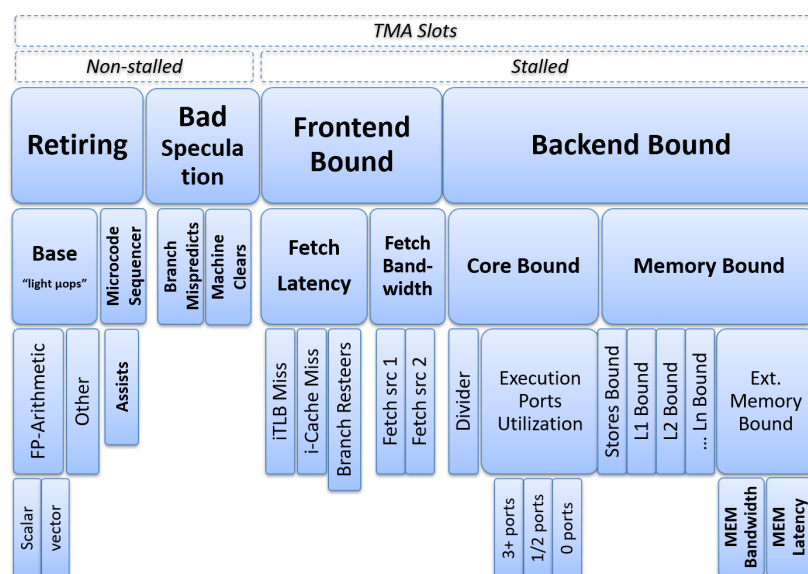


Figure 6.2: 性能瓶颈的 TMA 层次结构。© 图片由 Ahmad Yasin 提供。

多次运行工作负载是可以的，每次深入并专注于特定指标。但通常，运行一次工作负载并收集所有 TMA 级别所需的所有指标就足够了。分析工具通过在单次运行中在不同性能事件之间进行多路复用（参见 Section ??）来实现这一点。此外，在实际应用程序中，性能可能受多个因素限制。例如，它可能同时经历大量的分支预测错误（错误推测）和缓存未命中（后端绑定）。在这种情况下，TMA 将同时深入多个桶，并识别每种类型的瓶颈对程序性能的影响。Intel VTune Profiler、AMD uProf 和 Linux perf 等分析工具可以通过单次基准测试运行计算所有 TMA 指标。但是，这仅在工作负载稳定时才可接受。否则，你最好回到原始策略，进行多次运行并在每次运行中深入研究。

TMA 指标的前两个级别表示为程序执行期间可用的所有流水线槽（参见 Section 4.5）的百分比。这允许 TMA 在考虑处理器完整带宽的情况下，给出 CPU 微架构利用率的准确表示。到目前为止，一切应该很好地加起来达到 100%。但是，从第 3 级开始，桶可能以不同的计数域表示，例如时钟和停顿。因此，它们不一定与其他 TMA 桶直接可比。

一旦我们识别出性能瓶颈，我们需要知道它在代码中的确切位置。TMA 的第二步是将问题源定位到确切的代码行和相应的汇编指令。分析方法为每个类别的性能问题提供了你应该使用的性能事件。然后你可以对该事件进行采样，以找到源代码中导致第一阶段识别的性能瓶颈的行。如果这个过程对你来说听起来很复杂，请不要担心；一旦你通读案例研究，一切都会变得清楚。

案例研究：使用 TMA 减少缓存未命中数

作为本案例研究的示例，我采用了一个非常简单的基准测试，这样它易于理解和更改。它不能代表真实世界的应用程序，但它足以演示 TMA 的工作流程。我在本书的第二部分中有更多实际示例。

本书的大多数读者可能会将 TMA 应用于他们熟悉的应用程序。但即使你第一次看到该应用程序，TMA 也非常有效。因此，我不从展示基准测试的源代码开始。但这里有一个简短的描述：基准测

试在堆上分配一个 200 MB 的数组，然后进入 1 亿次迭代的循环。在循环的每次迭代中，它生成一个随机索引到已分配的数组中，执行一些虚拟工作，然后从该索引读取值。

我在配备 Intel Core i5-8259U CPU（基于 Skylake）和 16GB DRAM（DDR4 2400 MT/s）的机器上运行了实验，运行 64 位 Ubuntu 20.04（内核版本 5.13.0-27）。

第 1 步：识别瓶颈

作为第一步，我们运行微基准测试并收集一组有限的事件，这些事件将帮助我们计算第 1 级指标。在这里，我们尝试通过将高级性能瓶颈归因于四个 L1 桶来识别应用程序的高级性能瓶颈：前端绑定、后端绑定、退休和错误推测。可以使用 Linux perf 工具收集第 1 级指标。perf stat 命令有一个专用的 --topdown 选项。在更新的版本中，它将默认输出这些指标。以下是我们基准测试的细分。本节中所有命令的输出都经过修剪以节省空间。

```
$ perf stat -- ./benchmark.exe
...
TopdownL1 (cpu_core) # 53.4 % tma_backend_bound
                        <==
                        # 0.2 % tma_bad_speculation
                        # 13.8 % tma_frontend_bound
                        # 32.5 % tma_retiring
...
```

通过查看输出，我们可以判断应用程序的性能受 CPU 后端限制。让我们深入一层。为了获得 TMA 指标第 2 级、第 3 级和更高级别，我将使用 toplev 工具，它是 pmu-tools^[7]的一部分，由 Andi Kleen 编写。它在 Python 中实现，并在底层使用 Linux perf。必须启用特定的 Linux 内核设置才能使用 toplev；有关更多详细信息，请检查文档。

```
$ ~/pmu-tools/toplev.py --core S0-C0 -l2 -v --no-desc
taskset -c 0 ./benchmark.exe
```

6.1.2 AMD 平台上的 TMA

从 Zen4 开始，AMD 处理器支持第 1 级和第 2 级 TMA 分析。根据 AMD 文档，它被称为“流水线利用率”分析，但思想保持不变。L1 和 L2 桶也与 Intel 的非常相似。从内核 6.2 开始，Linux 用户可以使用 perf 工具收集流水线利用率数据。

接下来，我们将研究 Crypto++^[1] 的 SHA-256（安全哈希算法 256）实现，这是比特币挖矿中的基本加密算法。Crypto++ 是一个开源的 C++ 加密算法类库，包含许多算法的实现，而不仅仅是 SHA-256。然而，对于我们的示例，我通过注释掉 bench1.cpp 中 BenchmarkUnkeyedAlgorithms 函数中的相应行来禁用其他算法的基准测试。

我在配备 Ubuntu 22.04、Linux 内核 6.5 的 AMD Ryzen 9 7950X 机器上运行了测试。我使用 GCC 12.3 C++ 编译器编译了 Crypto++ 版本 8.9。我使用了默认的 -O3 优化选项，但由于代码是用 x86 内置函数编写的（参见 Section 9.5）并利用了 SHA x86 ISA 扩展，因此它对性能影响不大。

以下是我用于获取 L1 和 L2 流水线利用率指标的命令。输出经过修剪，一些统计数据被删除以消除不必要的干扰。

```
$ perf stat -M PipelineL1,PipelineL2 -- ./cryptest.exe
b1 10
0.0 % bad_speculation_mispredicts      (20.08%)
0.0 % bad_speculation_pipeline_restarts (20.08%)
0.0 % bad_speculation                  (20.08%)
6.1 % frontend_bound                   (20.00%)
6.1 % frontend_bound_bandwidth          (20.00%)
0.1 % frontend_bound_latency            (20.00%)
65.9 % backend_bound_cpu                (20.00%)
1.7 % backend_bound_memory              (20.00%)
67.5 % backend_bound                   (20.00%)
26.3 % retiring                         (20.08%)
20.2 % retiring_fastpath                (19.99%)
6.1 % retiring_microcode                (19.99%)
```

在输出中，方括号中的数字表示运行时持续时间的百分比，当监控该指标时。如我们所见，由于多路复用，每个指标只被监控了 20% 的时间。在我们的案例中，由于 SHA256 具有一致的行为，这可能不是问题，但情况并非总是如此。为了最小化多路复用的影响，你可以在单次运行中收集有限的指标集，例如 `perf stat -M frontend_bound,backend_bound`。

上面显示的流水线利用率指标的描述可以在 [AMD, 2024, Chapter 2.8 Pipeline Utilization] 中找到。通过查看指标，我们可以看到 SHA256 中没有发生分支预测错误 (`bad_speculation` 为 0%)。只有 26.3% 的可用分派槽被使用 (`retiring`)，这意味着剩余的 73.7% 由于前端和后端停顿而被浪费。

高级加密指令并不简单，因此它们在内部被分解为更小的部分 (μops)。一旦处理器遇到这样的指令，它就从微代码中检索其 μops 。微操作从微代码序列器获取的带宽低于常规指令解码器，使其成为潜在的性能瓶颈来源。Crypto++ SHA256 实现大量使用 SHA256MSG2、SHA256RND2 等指令，根据 uops.info [2] 网站，这些指令由多个 μops 组成。

`retiring_microcode` 指标表明 6.1% 的分派槽被最终退休的微代码操作使用。与同级指标 `retiring_fastpath` 相比，我们可以说大约每 4 条指令中就有一条是微代码操作。如果我们现在查看 `frontend_bound_bandwidth` 指标，我们将看到 6.1% 的分派槽由于 CPU 前端的带宽瓶颈而未被使用。这表明 6.1% 的分派槽被浪费了，因为微代码序列器在后端可以消耗时没有提供 μops 。在这个例子中，`retiring_microcode` 和 `frontend_bound_bandwidth` 指标紧密相关，然而，它们相等的事实仅仅是巧合。

大多数周期在 CPU 后端停顿 (`backend_bound`)，但只有 1.7% 的周期在等待内存访问时停顿 (`backend_bound_memory`)。所以，我们知道基准测试主要受机器计算能力的限制。正如你将在本书第 2 部分中所知，这可能与数据流依赖或某些加密操作的执行吞吐量有关。它们比传统的 ADD、SUB、CMP 和其他指令更少见，因此通常只能在单个执行单元上执行。大量此类操作可能饱和该特定单元的执行吞吐量。进一步的分析应涉及仔细查看源代码和生成的汇编、检查执行端口利用率、查找数据依赖等。

总之，Crypto++ 在 AMD Ryzen 9 7950X 上的 SHA-256 实现仅使用了 26.3% 的可用分派槽；6.1% 的分派槽由于微代码序列器带宽而被浪费，65.9% 由于缺乏机器计算资源而停顿。该算法确实遇到了一些硬件限制，因此不清楚其性能是否可以改进。

在 Windows 方面，在撰写本文时，TMA 方法仅在 AMD 服务器平台（代号 Genoa）上支

持，而不在客户端系统（代号 Raphael）上支持。TMA 支持在 AMD uProf 4.1 版本中添加，但仅在 AMD uProf 安装中的命令行工具 AMDuProfPcm 中。你可以查阅 [AMD, 2024, Chapter 2.8 Pipeline Utilization] 了解更多关于如何运行分析的详细信息。AMD uProf 的图形版本还没有 TMA 分析功能。

6.1.3 Arm 平台上的 TMA

Arm CPU 架构师也为他们的处理器开发了 TMA 性能分析方法，我们接下来将讨论。Arm 在其文档中称其为“Topdown”[Arm, 2023]，因此我们将使用他们的命名。在撰写本文时（2023 年底），Topdown 仅在 Arm 设计的核心上支持，例如 Neoverse N1 和 Neoverse V1，[5] 以及它们的衍生产品，例如 Ampere Altra 和 AWS Graviton3。如果你需要刷新对 Arm 芯片系列的记忆，请参阅本书末尾的主要 CPU 微架构列表。Apple 设计的处理器尚不支持 Arm Topdown 性能分析方法。

Arm Neoverse V1 是 Neoverse 系列中第一个支持完整第 1 级 Topdown 指标集的 CPU：Bad Speculation、Frontend Bound、Backend Bound 和 Retiring。在 V1 核心之前，Neoverse N1 仅支持两个 L1 类别：Frontend Stalled Cycles 和 Backend Stalled Cycles。[6]

为了演示 Arm 在基于 V1 的处理器上的 Topdown 分析，我启动了一个由 AWS Graviton3 提供支持的 AWS EC2 m7g.metal 实例。请注意，由于虚拟化，Topdown 可能无法在其他非 metal 实例类型上工作。我使用了由 AWS 管理的 64 位 ARM Ubuntu 22.04 LTS 和 Linux kernel 6.2。提供的 m7g.metal 实例具有 64 个 vCPU 和 256 GB RAM。

我将 Topdown 方法应用于 AI Benchmark Alpha，[1] 这是一个用于评估各种硬件平台（包括 CPU、GPU 和 TPU）AI 性能的开源 Python 库。该基准测试依赖于 TensorFlow 机器学习库来测量关键深度学习模型的推理和训练速度。AI Benchmark Alpha 总共包含 42 个测试，包括分类、图像分割、文本翻译等。

Arm 工程师开发了 topdown-tool[2]，我们将在下面使用。该工具在 ARM 上的 Linux 和 Windows 上都能工作。在 Linux 上，它利用标准的 perf 工具，而在 Windows 上，它使用 WindowsPerf[3]，这是一个 Windows on ARM 性能分析工具。与 Intel 的 TMA 类似，Arm 方法采用“深入”概念，即你首先确定高级性能瓶颈，然后深入进行更细致的根本原因分析。以下是我们使用的命令：

```
$ topdown-tool --all-cpus -m Topdown_L1 -- python -c
    "from ai_benchmark import AIBenchmark; results =
      AIBenchmark(use_CPU=True).run()"
Stage 1 (Topdown metrics)
=====
[Topdown Level 1]
Frontend Bound... 16.48% slots
Backend Bound.... 54.92% slots
Retiring..... 27.99% slots
Bad Speculation.. 0.59% slots
```

其中 --all-cpus 选项启用所有 CPU 的系统级收集，-m Topdown_L1 收集 Topdown 第 1 级指标。-- 之后的所有内容是要运行的 AI Benchmark Alpha 套件的命令行。

从上面的输出中，我们得出结论基准测试没有遭受分支预测错误。此外，如果没有对所涉及工作负载的更深入理解，很难说 16.5% 的 Frontend Bound 是否值得研究，因此我们将注意力转向 Backend Bound 指标，这是停顿周期的主要来源。基于 Neoverse V1 的芯片没有二级分解，相反，该方法建议通过收集一组相应的指标来进一步探索有问题的类别。以下是我们如何深入更详细的 Backend Bound 分析：

```
$ topdown-tool --all-cpus -n BackendBound -- python -c
    "from ai_benchmark import AIBenchmark; results =
      AIBenchmark(use_CPU=True).run()"
Stage 1 (Topdown metrics)
=====
[Topdown Level 1]
Backend Bound..... 54.70% slots

Stage 2 (uarch metrics)
=====
[Data TLB Effectiveness]
DTLB MPKI..... 0.413 misses per
1,000 instructions
L1 Data TLB MPKI..... 3.779 misses per
1,000 instructions
L2 Unified TLB MPKI..... 0.407 misses per
1,000 instructions
```

6.1.4 TMA 总结

TMA 非常适合识别 CPU 性能瓶颈。理想情况下，我们希望看到 Retiring 指标达到 100%。虽然有例外。Retiring 指标为 100% 意味着 CPU 完全饱和，它以全速处理指令。但这并不能说明这些指令的质量。程序可能在紧密循环中自旋等待锁；这会显示高 Retiring 指标，但它不做任何有用的工作。

另一个你可能会看到高 Retiring 指标但整体性能缓慢的例子是当程序有一个未量化的热点时。你通过让处理器运行简单的非量化操作给了它一个“轻松”的时间，但这是使用可用 CPU 资源的最佳方式吗？当然不是。如果 CPU 执行你的代码没有问题，这并不意味着性能不能提高。注意这种情况，记住 TMA 识别 CPU 性能瓶颈，但不会将它们与程序的性能相关联。一旦你进行了必要的实验，你就会发现这一点。

虽然在玩具程序上可以实现接近 100% 的 Retiring，但实际应用程序远未达到这一点。图 fig. 6.3 显示了 Google 数据中心工作负载的顶级 TMA 指标，以及在 Intel Ivy Bridge 服务器处理器上运行的几个 SPEC CPU2006⁵⁷ 基准测试。我们可以看到大多数数据中心工作负载在 Retiring 桶中占很小的比例。这意味着大多数数据中心工作负载在各种瓶颈上停顿。BackendBound 是性能问题的主要来源。FrontendBound 类别对数据中心工作负载来说比在 SPEC2006 中是一个更大的问题，因为这些应用程序通常具有较大的代码库且局部性差。最后，一些工作负载比其他工作负载遭受更多的分支预测错误，例如 search2 和 445.gobmk。

请记住，对于其他 CPU 代际，数字可能会发生变化，因为架构师不断尝试改进 CPU 设计。对于其他指令集架构 (ISA) 和编译器版本，数字也可能发生变化。

⁵⁷SPEC CPU 2006 - <http://spec.org/cpu2006/>.

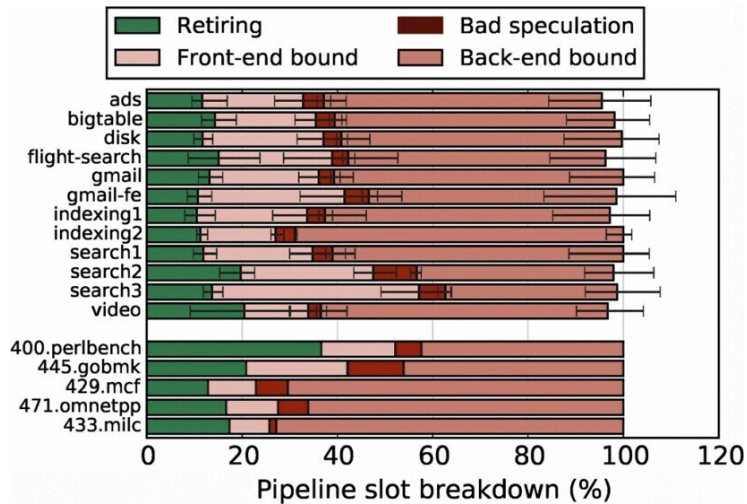


Figure 6.3: Google 数据中心工作负载与几个 SPEC CPU2006 基准测试的 TMA 分解, © 来源: [Kanev et al., 2015]

在我们继续之前的一些最后想法.....正如我们在本章开头所提到的, 不建议在具有主要性能缺陷的代码上使用 TMA, 因为它可能会将你引向错误的方向, 你将不是修复真正的高级性能问题, 而是调整糟糕的代码, 这只是浪费时间。同样, 确保环境不会妨碍分析。例如, 如果你丢弃文件系统缓存并在 TMA 下运行基准测试, 它可能会显示你的应用程序受内存限制, 而当文件系统缓存预热时, 这可能实际上是错误的。

TMA 提供的工作负载特征分析可以将潜在优化的范围扩展到源代码之外。例如, 如果应用程序受内存带宽限制, 并且在软件层面加速它的所有可能方法都已用尽, 那么通过用更快的内存芯片升级内存子系统可能提高性能。这展示了使用 TMA 诊断性能瓶颈如何支持你在新硬件上花钱的决定。

6.2 分支记录机制

现代高性能 CPU 提供分支记录机制, 使处理器能够持续记录一组先前执行的分支。但在深入细节之前, 你可能会问: 为什么我们对分支如此感兴趣? 因为这是我们确定程序控制流的方式。我们基本上忽略基本块 (参见 Section 11.2) 中的其他指令, 因为分支总是基本块中的最后一条指令。由于基本块中的所有指令都保证执行一次, 我们只能关注将“代表”整个基本块的分支。因此, 如果我们可以跟踪每个分支的结果, 就可以重建程序的整个逐行执行路径。这就是 Intel 处理器跟踪 (PT) 功能能够做到的, 在附录 C 中讨论。我们将在本节讨论的分支记录机制基于采样, 而不是跟踪, 因此具有不同的用例和能力。

Intel、AMD 和 Arm 设计的处理器都宣布了它们的分支记录扩展。确切的实现可能不同, 但思想是相同的: 硬件在执行程序的同时记录每个分支的“from”和“to”地址以及一些附加数据。如果我们收集足够长的源-目标对历史, 我们将能够展开程序的控制流, 就像调用栈一样, 但深度有限。此类扩展旨在对运行中的程序造成最小的减速 (通常在 1% 以内)。

有了分支记录机制, 我们可以在分支 (或周期, 这不重要) 上采样, 但在每个样本期间, 查看先前执行的 N 个分支。这为我们提供了热代码路径中控制流的合理覆盖, 但不会因为太多信息而不堪重

负，因为只检查了少量的总分支。重要的是要记住这仍然是采样，因此并非每个执行的分支都可以被检查。CPU 通常运行得太快，无法做到这一点。

重要的是要记住，只有被采取的分支才会被记录。Listing ?? 显示了分支结果如何被跟踪的示例。此代码表示一个具有三条可能改变程序执行路径的指令的循环，即循环回边 JNE (1)、条件分支 JNS (2)、函数 CALL (3) 和从此函数返回的地址 (4)。

Listing: 记录分支的示例。

```

----> 4eda10: mov    edi,DWORD PTR [rbx]
|      4eda12: test   edi,edi
| --- 4eda14: jns     4eda1e             <== (2)
| |    4eda16: mov    eax,edi
| |    4eda18: shl    eax,0x7
| |    4eda1b: lea    edi,[rax+rdi*8]
| > 4eda1e: call   4edb26             <== (3)
|      4eda23: add    rbx,0x4            <== (4)
|      4eda27: mov    DWORD PTR [rbx-0x4],eax
|      4eda2a: cmp    rbx,rbp
----- 4eda2d: jne     4eda10             <== (1)

```

Listing ?? 显示了使用分支记录机制可以记录的一个可能的分支历史。它显示了在执行 CALL 指令时最后 7 个分支结果（未显示更多）。因为在循环的最新迭代中，JNS 分支（4eda14 → 4eda1e）未被采取，所以它没有被记录，因此不出现在历史中。

Listing: 可能的分支历史。

	Source Address	Destination Address	
	...	...	
(1)	4eda2d	4eda10	<== 下一次迭代
(2)	4eda14	4eda1e	<== jns 被采取
(3)	4eda1e	4edb26	<== 调用函数
(4)	4b01cd 时间	4eda23	<== 从函数返回
(1)	4eda2d	4eda10	<== 下一次迭代
(3)	4eda1e	4edb26	<== 最新分支
	V		

6.3 基于硬件的采样功能

主要 CPU 供应商提供一组额外功能来增强采样。由于 CPU 供应商以不同的方式处理性能监控，这些能力不仅在称呼上有所不同，而且在你可以用它们做什么方面也有所不同。在 Intel 处理器中，它被称为处理器基于事件的采样 (PEBS)，首先在 NetBurst 微架构中引入。AMD 处理器上的类似功能称为基于指令的采样 (IBS)，从 AMD Opteron Family (10h 代) 核心开始可用。接下来，我们将更详细地讨论这些功能，包括它们的相似之处和不同之处。

6.3.1 Intel 平台上的 PEBS

与 Last Branch Record 功能类似, PEBS 在分析程序时用于在每个收集的样本中捕获附加数据。当性能计数器配置为 PEBS 时, 处理器保存一组附加数据, 该数据具有定义的格式, 称为 PEBS 记录。Intel Skylake CPU 的 PEBS 记录格式如图 fig. 6.4 所示。它包含通用寄存器的状态 (EAX、EBX、ESP 等)、EventingIP、Data Linear Address 和 Latency value, 以及其他一些字段。PEBS 记录的内容布局因微架构而异, 参见 [Intel, 2023, Volume 3B, Chapter 20 Performance Monitoring]。

Byte Offset	Field	Byte Offset	Field
00H	R/EFLAGS	68H	R11
08H	R/EIP	70H	R12
10H	R/EAX	78H	R13
18H	R/EBX	80H	R14
20H	R/ECX	88H	R15
28H	R/EDX	90H	Applicable Counter
30H	R/ESI	98H	Data Linear Address
38H	R/EDI	A0H	Data Source Encoding
40H	R/EBP	A8H	Latency value (core cycles)
48H	R/ESP	B0H	EventingIP
50H	R8	B8H	TX Abort Information (Section 18.3.6.5.1)
58H	R9	C0H	TSC
60H	R10		

Figure 6.4: 第 6 代、第 7 代和第 8 代 Intel Core 处理器系列的 PEBS 记录格式。© 来源: [Intel, 2023, Volume 3B, Chapter 20]。

从 Skylake 开始, PEBS 记录已得到增强, 可以收集 XMM 寄存器和 Last Branch Record (LBR) 记录。格式已重构, 字段分为 Basic 组、Memory 组、GPR 组、XMM 组和 LBR 组。性能分析工具可以选择感兴趣的数据组, 从而减少记录开销。默认情况下, PEBS 记录只包含 Basic 组。

使用 PEBS 的一个显著好处是与常规基于中断的采样相比, 采样开销更低。回想一下, 当计数器溢出时, CPU 生成中断以收集一个样本。频繁生成中断并让分析工具本身在中断服务例程内捕获程序状态是非常昂贵的, 因为它涉及操作系统交互。

另一方面, PEBS 维护一个缓冲区来临时存储多个 PEBS 记录。假设我们正在使用 PEBS 采样加载指令。当性能计数器配置为 PEBS 时, 计数器中的溢出条件不会触发中断, 而是会激活 PEBS 机制。该机制然后将捕获下一个加载, 捕获一个新记录, 并将其存储在专用的 PEBS 缓冲区区域中。该机制还负责清除计数器溢出状态并用初始值重新加载计数器。只有当专用缓冲区满时, 处理器才会发出中断, 缓冲区才会刷新到内存。此机制通过触发更少的中断来降低采样开销。

Linux 用户可以通过执行 `dmesg` 来检查是否启用了 PEBS:

```
$ dmesg | grep PEBS
[ 0.113779] Performance Events: XSAVE Architectural
LBR, PEBS fmt4+-baseline,
AnyThread deprecated, Alderlake Hybrid events, 32-deep
LBR, full-width counters, Intel PMU driver.
```

对于 LBR, Linux perf 在每个收集的样本时转储 LBR 堆栈的全部内容。因此, 可以分析 Linux perf 收集的原始 LBR 转储。但是, 对于 PEBS, Linux perf 不会像对 LBR 那样导出原始输出。相反, 它处理 PEBS 记录并仅根据特定需求提取数据的子集。因此, 无法使用 Linux perf 访问原始 PEBS 记录的集合。但是, Linux perf 提供了一些从原始样本处理的 PEBS 数据, 可以通过 `perf report -D` 访问。要转储原始 PEBS 记录, 你可以使用 `pebs-grabber`[1]。

6.3.2 AMD 平台上的 IBS

基于指令的采样 (IBS) 是 AMD64 处理器的一项功能, 可用于收集与指令获取和指令执行相关的特定指标。AMD 处理器的流水线由两个独立的阶段组成: 获取 AMD64 指令字节的前端阶段和执行 ops 的后端阶段。由于阶段在逻辑上是分离的, 因此有两种独立的采样机制: IBS Fetch 和 IBS Execute。

- IBS Fetch 监控流水线的前端, 并提供有关 ITLB (命中或未命中)、I-cache (命中或未命中)、获取地址、获取延迟等信息。
- IBS Execute 监控流水线的后端, 并通过跟踪单个 op 的执行来提供有关指令执行行为的信息。例如, 分支 (已采取或未采取、已预测或未预测) 和加载/存储 (在 D-cache 和 DTLB 命中或未命中、线性地址、加载延迟)。

PMC 和 AMD 处理器中的 IBS 有几个重要区别。PMC 计数器是可编程的, 而 IBS 充当固定计数器。IBS 计数器只能启用或禁用用于监控, 不能编程为任何选择性事件。IBS Fetch 和 Execute 计数器可以独立启用/禁用。使用 PMC, 用户必须提前决定要监控哪些事件。使用 IBS, 为每条采样指令收集丰富的数据集, 然后由用户分析他们感兴趣的数据部分。IBS 选择并标记要监控的指令, 然后在执行期间捕获由该指令引起的微架构事件。Intel PEBS 和 AMD IBS 的更详细比较可以在 [Sasongko et al., 2023] 中找到。

由于 IBS 集成到处理器流水线中并充当固定事件计数器, 因此样本收集开销最小。分析器需要处理 IBS 生成的数据, 根据采样间隔、配置的线程数、是否配置 Fetch/Execute 等, 这些数据可能非常大。在 Linux 内核版本 6.1 之前, IBS 总是为所有核心收集样本。此限制导致大量数据收集和处理的开销。从内核 6.2 开始, Linux perf 仅支持为配置的核心进行 IBS 样本收集。

Linux perf 和 AMD uProf 分析器支持 IBS。以下是收集 IBS Execute 和 Fetch 样本的示例命令:

问题和练习

1. 命名 TMA 性能方法中的四个第 1 级类别。
2. 基于硬件事件的采样有什么好处?
3. 什么是性能事件滑移?
4. 研究你用于开发/基准测试的机器中 CPU 上可用的性能分析功能。

章节总结

- 现代处理器提供增强性能分析的功能。使用这些功能大大简化了查找底层优化的机会。
- Top-down 微架构分析 (TMA) 方法是识别程序对 CPU 微架构低效使用的技术, 即使对于经验不足的开发人员也易于使用。TMA 是一个迭代过程, 包括多个步骤, 包括表征工

作负载和定位源代码中发生瓶颈的确切位置。我们建议 TMA 应该是每次底层调优工作的起点之一。

- 分支记录机制，如 Intel 的 LBR、AMD 的 LBR 和 ARM 的 BRBE，在执行程序的同时持续记录最近的分支结果，造成最小的减速。这些设施的主要用途之一是收集调用栈。此外，它们有助于识别热分支和预测错误率，并能够精确计时机器代码。
- 现代处理器通常提供基于硬件的采样功能用于高级分析。这些功能通过将多个样本存储在专用缓冲区中而不使用软件中断来降低采样开销。它们还引入了“精确事件”，能够精确定位导致特定性能事件的确切指令。此外，还有其他一些不太重要的用例。此类基于硬件的采样功能的示例实现包括 Intel 的 PEBS、AMD 的 IBS 和 ARM 的 SPE。
- Intel 处理器跟踪 (PT) 是一项 CPU 功能，通过以高度压缩的二进制格式编码数据包来记录程序执行，可用于重建每条指令带有时间戳的执行流程。PT 具有广泛的覆盖范围和相对较小的开销。其主要用途是事后分析和查找性能故障的根本原因。Intel PT 功能在附录 C 中介绍。基于 ARM 架构的处理器也具有称为 Arm CoreSight 的跟踪功能，⁵⁸ 但它主要用于调试而不是性能分析。

⁵⁸ Arm CoreSight - <https://developer.arm.com/ip-products/system-ip/coresight-debug-and-trace>

7 性能分析工具概述

在上一章中，我们探讨了现代处理器中为辅助性能分析而实现的功能。但是，如果你要直接开始使用这些功能，它很快就会变得非常微妙，因为需要大量低级编程才能利用它们。幸运的是，性能分析工具负责有效使用这些硬件性能监控功能所需的所有复杂性。这使得分析过程顺利进行，但对这些工具如何获取和解释数据有直觉仍然很重要。这就是为什么我们现在在讨论了 CPU 性能监控功能之后才讨论分析工具。

本章概述了主要平台上可用的最流行的性能分析工具。其中一些工具是跨平台的，但大多数不是，因此了解哪些工具对你可用很重要。分析工具通常由硬件供应商自己开发和维护，因为它们是知道如何正确使用其处理器上可用的性能监控功能的人。因此，高级性能工程工作的工具选择取决于你使用的操作系统和 CPU。

阅读本章后，花时间练习使用你最终可能使用的工具。熟悉这些工具的接口和 workflows。分析你日常使用的应用程序。即使你没有发现任何可操作的见解，当实际需求出现时，你也会有更好的准备。

7.1 Intel VTune Profiler

VTune Profiler (以前称为 VTune Amplifier) 是一个用于 x86 机器的性能分析工具，具有丰富的图形界面。它可以在 Linux 或 Windows 操作系统上运行。我们跳过讨论 VTune 对 MacOS 的支持，因为它在 Apple 的芯片 (如 M1 和 M2) 上不起作用，而且基于 Intel 的 Macbook 正在迅速过时。

VTune 可以在 Intel 和 AMD 系统上使用。但是，高级基于硬件的采样需要 Intel 制造的 CPU。例如，你将无法在 AMD 系统上使用 Intel VTune 收集硬件性能计数器。

截至 2023 年初，VTune 作为独立工具或作为 Intel oneAPI Base Toolkit 的一部分免费提供。^[1]

如何配置

在 Linux 上，VTune 可以使用两种数据收集器：Linux perf 和 VTune 自己的驱动程序 SEP。第一种类型用于用户模式采样，但如果你想执行高级分析，你需要构建并安装 SEP 驱动程序，这并不太难。

```
# 转到 vtune 安装目录中的 sepdk 文件夹
$ cd ~/intel/oneapi/vtune/latest/sepdk/src
# 构建驱动程序
$ ./build-driver
# 添加 vtune 组并将你的用户添加到该组
# 创建新的 shell，或重启系统
$ sudo groupadd vtune
$ sudo usermod -a -G vtune `whoami`
# 安装 sep 驱动程序
$ sudo ./insmod-sep -r -g vtune
```

完成上述步骤后，你应该能够使用高级分析类型，如微架构探索和内存访问。

Windows 在安装 VTune 后不需要任何额外配置。收集硬件性能事件需要管理员权限。

你能用它做什么：

- 查找热点：函数、循环、语句。
- 监控各种 CPU 特定的性能事件，例如分支预测错误和 L3 缓存未命中。
- 定位发生这些事件的代码行。
- 使用 TMA 方法表征 CPU 性能瓶颈。
- 过滤特定函数、进程、时间段或逻辑核心的数据。
- 观察工作负载随时间的行为（包括 CPU 频率、内存带宽利用率等）。

VTune 可以提供关于正在运行的进程的非常丰富的信息。如果你想要提高应用程序的整体性能，它是正确的工具。VTune 总是提供一段时间内的聚合数据，因此它可以用于查找“平均情况”的优化机会。

7.2 AMD uProf

uProf 分析器是 AMD 开发的用于监控在 AMD 处理器上运行的应用程序性能的工具。虽然 uProf 也可以在 Intel 处理器上使用，但你只能使用与 CPU 无关的功能。该分析器可免费下载，可用于 Windows、Linux 和 FreeBSD。AMD uProf 可用于在多个虚拟机（VM）上进行分析，包括 Microsoft Hyper-V、KVM、VMware ESXi 和 Citrix Xen，但并非所有功能在所有 VM 上都可用。此外，uProf 支持分析用各种语言编写的程序，包括 C、C++、Java、.NET/CLR。

如何配置

在 Linux 上，uProf 使用 Linux perf 进行数据收集。在 Windows 上，uProf 使用自己的采样驱动程序，在安装 uProf 时安装，不需要额外配置。AMD uProf 支持命令行界面（CLI）和图形界面。CLI 界面需要两个单独的步骤——收集和报告，类似于 Linux perf。

你能用它做什么：

- 查找热点：函数、语句、指令。
- 监控各种硬件性能事件并定位发生这些事件的代码行。
- 过滤特定函数或线程的数据。
- 观察工作负载随时间的行为：在时间线图表中查看各种性能事件。
- 分析热调用路径：调用图、火焰图和自底向上图表。

此外，uProf 可以在 Linux 上监控各种操作系统事件：线程状态、线程同步、系统调用、页面错误等。你可以用它来分析 OpenMP 应用程序以检测线程不平衡，并分析 MPI⁵⁹ 应用程序以检测 MPI 集群节点之间的负载不平衡。有关 uProf 各种功能的更多详细信息，请参阅用户指南⁶⁰。

你不能用它做什么：

由于工具的采样性质，它最终会错过持续时间非常短的事件。报告的样本是统计估计的数字，大多数时候足以分析性能，但不是事件的确切计数。

⁵⁹ MPI - 消息传递接口，分布式内存系统上并行编程的标准。

⁶⁰ AMD uProf 用户指南 - <https://www.amd.com/en/developer/uprof.html#documentation>

示例

为了演示 AMD uProf 工具的外观和感觉，我们在 AMD Ryzen 9 7950X 上运行了 Scimark2⁶¹ 基准测试中的密集 LU 矩阵分解组件，运行 Windows 11，配备 64 GB RAM。

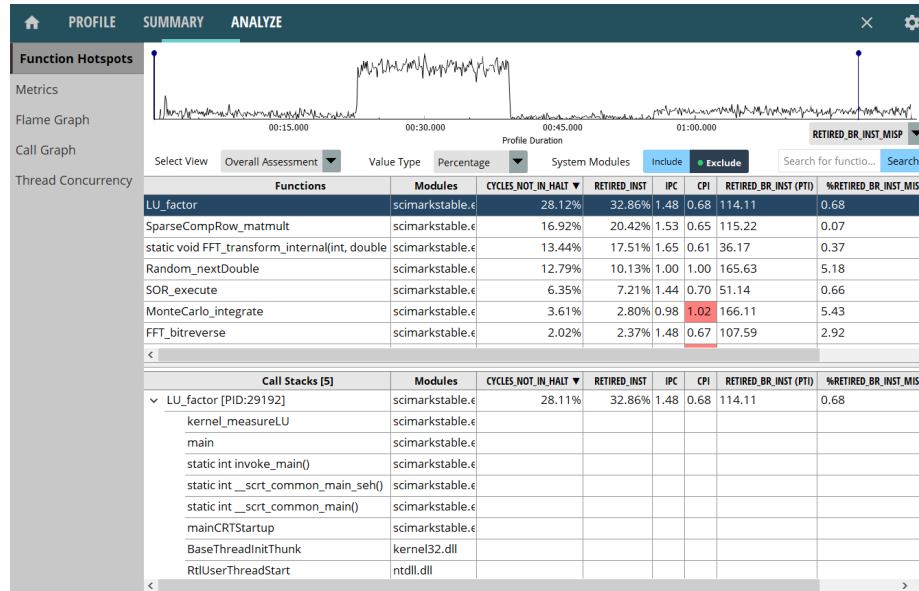


Figure 7.1: uProf 的函数热点视图。

图 fig. 7.1 显示了函数热点分析（在图像左侧的菜单列表中选择）。在图像顶部，你可以看到一个事件时间线，显示在应用程序执行的各个时间观察到的事件数量。在右侧，你可以选择要绘制的指标；我们选择了 RETIRED_BR_INST_MISP。注意在 20s 到 40s 的时间范围内分支预测错误的尖峰。你可以选择此区域以密切分析那里发生了什么。一旦你这样做，它将更新底部面板以仅显示该时间间隔的统计信息。

在时间线图下方，你可以看到函数表中所选函数的自底向上调用栈视图。如我们所见，所选的 LU_factor 函数是从 kernel_measureLU 调用的，而后者又是从 main 调用的。在 Scimark2 基准测试中，这是 LU_factor 的唯一调用栈，即使它显示 Call Stacks [5]。这是可以忽略的收集伪像。但在其他应用程序中，热函数可以从许多不同的地方调用，因此你也需要检查其他调用栈。

如果你双击任何函数，uProf 将打开该函数的源代码/汇编视图。为简洁起见，我们不显示此视图。在左侧面板中，还有其他可用的视图，如指标、火焰图、调用图视图和线程并发。它们对分析也很有用，但我们决定跳过它们。读者可以自己尝试并查看这些视图。

7.3 Apple Xcode Instruments

在 MacOS 上进行类似性能分析的最方便方法是使用 Xcode Instruments。这是一个随 Xcode 免费提供的应用程序性能分析器和可视化工具。Instruments 分析器建立在从 Solaris

⁶¹ Scimark2 - <https://math.nist.gov/scimark2/index.html>

移植到 macOS 的 DTrace 跟踪框架之上。它有许多工具来检查应用程序的性能，并使我们能够执行其他分析器（如 Intel VTune）可以执行的大多数基本操作。获取分析器的最简单方法是从 Apple App Store 安装 Xcode。该工具不需要配置；安装后即可使用。

在 Instruments 中，你使用称为 instruments 的专用工具来跟踪应用程序、进程和设备随时间的不同方面。Instruments 具有强大的可视化机制。它在分析时收集数据，并实时向你展示结果。你可以收集不同类型的数据并并排查看它们，这使你能够看到执行中的模式，关系统事件并发现非常细微的性能问题。

在本章中，我们将只展示最与本书相关的“CPU Counters”instrument。Instruments 还可以可视化 GPU、网络和磁盘活动，跟踪内存分配和释放，捕获用户事件（如鼠标点击），提供功耗效率见解等。你可以在 Instruments 文档中阅读更多关于这些用例的信息。^[1]

你能用它做什么：

- 访问 Apple 处理器上的硬件性能计数器。
- 查找程序中的热点及其调用栈。
- 将生成的 ARM 汇编代码与源代码并排检查。
- 过滤时间线上选定间隔的数据。

你不能用它做什么：

与其他基于采样的分析器类似，Xcode Instruments 具有与 VTune 和 uProf 相同的盲点。

示例：分析 Clang 编译

在这个示例中，我将展示如何在配备 M1 处理器、macOS 13.5.1 Ventura 和 16 GB RAM 的 Apple Mac mini 上收集硬件性能计数器。我从 LLVM 代码库中取了最大的文件之一，并使用 Clang C++ 编译器的 15.0 版本分析其编译。

以下是我使用的命令行：

```
$ clang++ -O3 -DNDEBUG -arch arm64 <other options ...>
-c llvm/lib/Transforms/Vectorize/LoopVectorize.cpp
```

图 fig. 7.2 显示了 Xcode Instruments 的主时间线视图。此截图是在编译完成后拍摄的。我们稍后会回到它，但首先，让我们展示如何启动分析会话。

首先，打开 *Instruments* 并选择 *CPU Counters* 分析类型。你需要做的第一步是配置收集。点击并按住红色目标图标（参见图 fig. 7.2 中的 ①），然后从菜单中选择 *Recording Options...*。它将显示图 fig. 7.3 中所示的对话框窗口。这是你可以添加硬件性能监控事件进行收集的地方。Apple 在其手册 [Apple, 2024, Section 6.2 Performance Monitoring Events] 中记录了其硬件性能监控事件。

第二步是设置分析目标。为此，点击并按住应用程序的名称（在图 fig. 7.2 中标记为 ②）并选择你感兴趣的程序。设置参数和环境变量（如果需要）。现在，你可以开始收集了；按红色目标图标 ①。

Instruments 显示时间线并不断更新关于正在运行的应用程序的统计信息。程序完成后，Instruments 将显示如图 fig. 7.2 所示的结果。编译花费了 7.3 秒，我们可以看到事件数量随时间的变

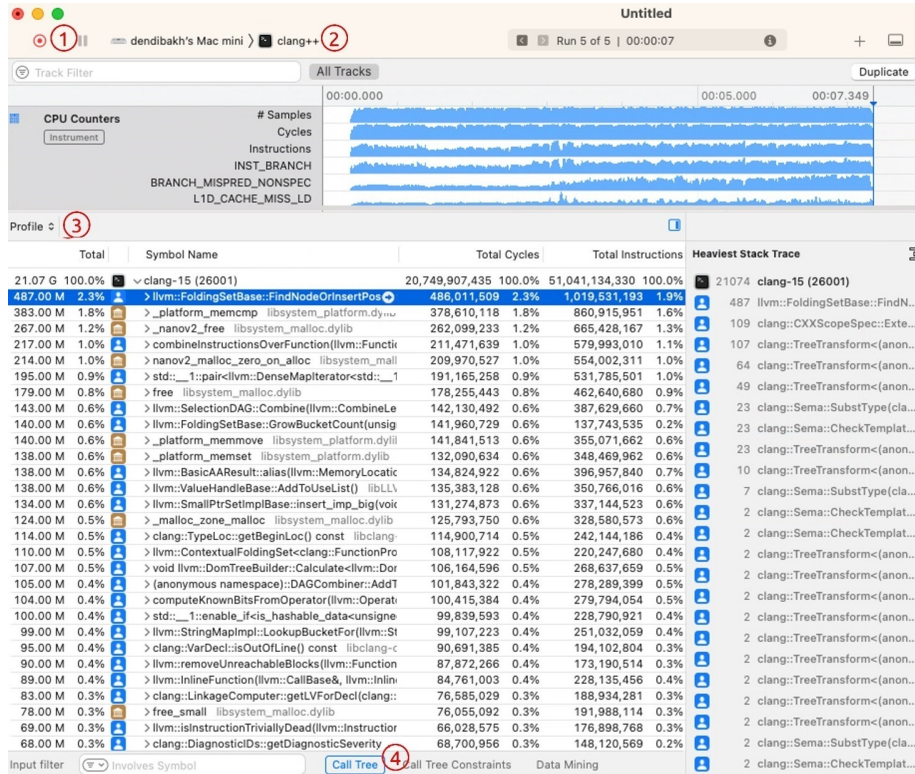


Figure 7.2: Xcode Instruments: 时间线和统计面板。

化。例如，执行的分支指令和预测错误的数量在运行时接近尾声时增加。你可以在时间线上放大到该间隔以检查涉及的函数。

7.4 Linux Perf

Linux Perf 可能是世界上使用最多的性能分析器，因为它在大多数 Linux 发行版上可用，这使其可被广泛的用户访问。Perf 在许多流行的 Linux 发行版中被原生支持，包括 Ubuntu、Red Hat 和 Debian。它包含在内核中，因此你可以在任何运行 Linux 的系统上获得操作系统级统计信息（页面错误、CPU 迁移等）。截至 2024 年中，该分析器支持 x86、ARM、PowerPC64、UltraSPARC 和其他几种 CPU 类型。⁶² 在这些平台上，perf 提供对硬件性能监控功能的访问，例如性能计数器。有关 Linux perf 的更多信息，请参阅其 wiki 页面⁶³。

如何配置

安装 Linux perf 非常简单，可以用一条命令完成：

⁶² RISCv 尚未作为官方内核的一部分支持，尽管供应商存在自定义工具。

⁶³ Linux perf wiki - https://perf.wiki.kernel.org/index.php/Main_Page。

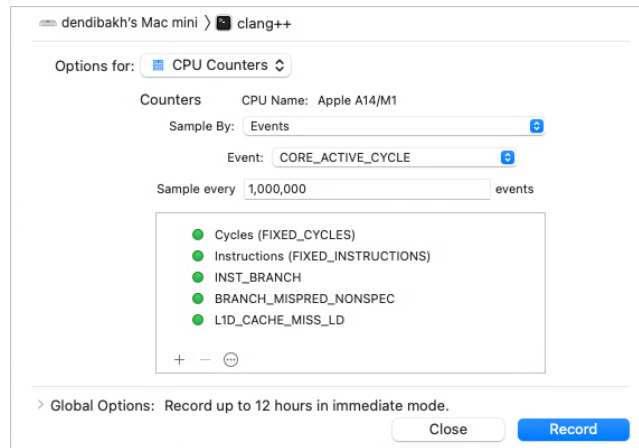


Figure 7.3: Xcode Instruments: CPU Counters 选项。

```
$ sudo apt-get install linux-tools-common
    linux-tools-generic linux-tools-`uname -r`
```

另外，除非安全是问题，否则考虑更改以下默认值：

```
# 允许非特权用户进行内核分析和访问 CPU 事件
$ echo 0 | sudo tee /proc/sys/kernel/perf_event_paranoid
$ echo kernel.perf_event_paranoid=0 | sudo tee -a
  /etc/sysctl.d/local.conf
# 启用非特权用户的内核模块符号解析
$ echo 0 | sudo tee /proc/sys/kernel/kptr_restrict
$ echo kernel.kptr_restrict=0 | sudo tee -a
  /etc/sysctl.d/local.conf
```

你能用它做什么：

通常，Linux perf 可以做其他分析器可以做的大多数事情。硬件供应商优先在 Linux perf 中启用其功能，以便在新 CPU 上市时，perf 已经支持它。大多数人使用两个主要命令。第一个 perf stat 报告指定性能事件的计数。第二个 perf record 在采样模式下分析应用程序或系统，通常后跟 perf report 从采样数据生成报告。

perf record 命令的输出是样本的原始转储。许多建立在 Linux perf 之上的工具解析原始转储文件并提供新的分析类型。以下是最值得注意的：

- 火焰图，在 Section 7.5 中讨论。
- KDAB Hotspot,⁶⁴ 一个使用与 Intel VTune 非常相似的界面可视化 Linux perf 数据的工具。如果你使用过 Intel VTune，KDAB Hotspot 会看起来非常熟悉。
- Netflix Flamescope。^[4] 此工具显示应用程序运行时采样事件的热图。你可以观察工作负载行为中的不同阶段和模式。Netflix 工程师使用此工具发现了一些非常细微的性能错误。此外，你可以在热图上选择时间范围并为该时间范围生成火焰图。

⁶⁴ KDAB Hotspot - <https://github.com/KDAB/hotspot>。

你不能用它做什么：

Linux perf 是一个命令行工具，缺少图形用户界面（GUI），这使得过滤数据、观察工作负载行为随时间的变化、放大运行时的一部分等变得困难。通过 `perf report` 命令提供了有限的控制台输出，这对于快速分析来说虽然不如其他 GUI 分析器方便，但已经足够了。幸运的是，正如我们刚才提到的，有一些 GUI 工具可以后处理和可视化 Linux perf 的原始输出。

7.5 火焰图

火焰图是可视化分析数据和程序中最频繁代码路径的流行方式。它使我们能够看到哪些函数调用占用了执行时间的最大部分。图 fig. 7.4 显示了 x264 视频编码基准测试的火焰图示例，由 Brendan Gregg 开发的开源 scripts⁶⁵ 生成。如今，几乎所有的分析器都可以自动生成火焰图，只要在分析会话期间收集了调用栈。

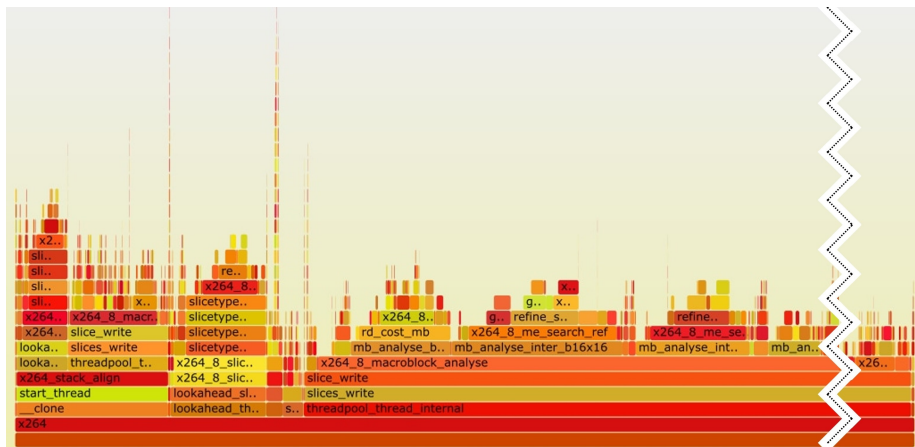


Figure 7.4: x264 基准测试的火焰图。

在火焰图上，每个矩形（水平条）代表一个函数调用，矩形的宽度表示函数本身及其被调用者所花费的相对执行时间。函数调用从底部到顶部发生，因此我们可以看到程序中最热的路径是 `x264` → `threadpool_thread_internal` → ... → `x264_8_macroblock_analyse`。函数 `threadpool_thread_internal` 及其被调用者占程序花费时间的 74%。但自身时间，即函数本身花费的时间相当小。类似地，我们可以对 `x264_8_macroblock_analyse` 进行同样的分析，它占运行时的 66%。这种可视化让你对最时间花费在哪里有很好的直觉。

火焰图是交互式的。你可以点击图像上的任何条，它将放大到该特定代码路径。你可以持续放大，直到找到一个不符合你期望的地方或到达叶子/尾部函数——现在你有了可以在分析中使用的可操作信息。另一种策略是找出程序中最热的函数（从这个火焰图中不立即清楚），然后自底向上通过火焰图，试图理解这个最热的函数是从哪里被调用的。

一些工具更喜欢使用冰柱图，它是火焰图的倒置版本（参见 Section 7.9 中的示例）。

⁶⁵ Brendan Gregg 的火焰图 - <https://github.com/brendangregg/FlameGraph>

7.6 Windows 事件跟踪

Microsoft 开发了一个名为 Windows 事件跟踪 (ETW) 的系统级跟踪设施。它最初旨在帮助设备驱动程序开发人员,但后来也用于分析通用应用程序。ETW 在所有受支持的 Windows 平台 (x86 和 ARM) 上可用,具有相应的平台相关安装包。ETW 在用户和内核代码中记录结构化事件,具有完整的调用栈跟踪支持,使你能够观察运行系统中的软件动态并解决许多具有挑战性的性能问题。

如何配置

从 Windows 10 开始,使用 WPR.exe 无需额外下载即可录制 ETW 数据。但要启用系统级分析,你必须是管理员并启用 SeSystemProfilePrivilege。Windows 性能记录器工具支持一组内置录制配置文件,适用于常见的性能问题。你可以通过编写具有 .wprp 扩展名的自定义性能记录器配置文件 XML 文件来定制你的录制需求。

如果你想不仅录制而且查看录制的 ETW 数据,你需要安装 Windows 性能工具包 (WPT)。你可以从 Windows SDK⁶⁶ 或 ADK⁶⁷ 下载页面下载它。Windows SDK 很大;你不一定需要它的所有部分。在我们的案例中,我们只启用了 Windows 性能工具包的复选框。你被允许将 WPT 作为你自己应用程序的一部分重新分发。

你能用它做什么:

- 使用可配置的 CPU 采样率 (从 125 微秒到 10 秒) 识别热点。默认值为 1 毫秒,成本约为 5-10% 的运行开销。
- 确定什么阻塞了某个线程以及多长时间 (例如,延迟的事件信号、不必要的线程休眠等)。
- 检查磁盘提供读/写请求的速度,并发现什么启动了该工作。
- 检查文件访问性能和模式 (包括导致无磁盘 IO 的缓存读/写)。
- 跟踪 TCP/IP 堆栈以查看数据包如何在网络接口和计算机之间流动。

上面列出的所有项目都在系统范围内为所有进程录制,具有可配置的调用栈跟踪 (内核和用户模式调用栈被组合)。也可以添加你自己的 ETW 提供程序,将系统级跟踪与你的应用程序行为关联起来。你可以通过检测代码来扩展收集的数据量。例如,你可以在函数中注入进入/离开 ETW 跟踪钩子到你的源代码中,以测量某个函数被执行的频率。

你不能用它做什么:

ETW 跟踪不适用于检查 CPU 微架构瓶颈。为此,请使用供应商特定的工具,如 Intel VTune、AMD uProf、Apple Instruments 等。

ETW 跟踪捕获系统级别所有进程的动态,但它可能会生成大量数据。例如,捕获线程上下文切换数据以观察各种等待和延迟,每分钟很容易生成 1-2 GB。这就是为什么在不覆盖先前存储的跟踪的情况下记录高容量事件数小时是不切实际的。

如果你想了解更多关于 ETW 的信息,附录 D 中有更详细的讨论。我们探讨了录制和分析 ETW 的工具,并介绍了调试程序启动缓慢的案例研究。

⁶⁶ Windows SDK 下载 - <https://developer.microsoft.com/en-us/windows/downloads/sdk-archive/>

⁶⁷ Windows ADK 下载 - <https://learn.microsoft.com/en-us/windows-hardware/get-started/adk-install#other-adk-downloads>

7.7 专用和混合分析器

到目前为止探索的大多数工具都属于采样分析器类别。当你想要识别代码中的热点时，这些工具很棒，但在某些情况下，它们可能无法提供所需的分析粒度。根据分析器采样频率和程序行为，大多数函数可能足够快，以至于它们不会出现在分析器中。在某些场景中，你可能想要手动定义程序的哪些部分需要持续测量。例如，视频游戏平均每秒渲染 60 帧 (FPS)；某些显示器允许高达 144 FPS。在 60 FPS 时，每帧只有 16 毫秒的时间来完成工作，然后才能进入下一帧。开发人员特别注意超过此阈值的帧，因为这会导致游戏中出现可见的卡顿，并可能破坏玩家体验。这种情况很难用采样分析器捕获。

开发人员创建了特定环境中提供有用功能的分析器，通常带有一个你可以用来手动检测代码的标记 API。这使你能够观察特定函数或代码块（以后称为区域）的性能。继续游戏行业，这个领域有几个工具：一些直接集成到游戏引擎中，如 Unreal，而其他工具作为外部库和工具提供，可以集成到你的项目中。一些最常用的分析器是 Tracy、RAD Telemetry、Remotery 和 Optick（仅限 Windows）。接下来，我们展示 Tracy，^[1] 因为这似乎是最受欢迎的项目之一；但是，这些概念也适用于其他分析器。

Tracy 能做什么：

- 调试程序中的性能异常，例如慢帧。
- 将慢事件与系统中的其他事件关联。
- 查找慢事件之间的共同特征。
- 检查源代码和汇编。
- 在代码更改后进行“之前/后”比较。

Tracy 不能做什么：

- 检查 CPU 微架构问题，例如收集各种性能计数器。

案例研究：使用 Tracy 分析慢帧

在这个示例中，我们将使用 ToyPathTracer^[2] 程序，这是一个简单的路径跟踪器，它是一种简化的光线跟踪技术，每像素向场景发射数千条光线以渲染逼真的图像。为了处理一帧，实现将每像素的处理分配给一个单独的线程。

为了模拟 Tracy 可以帮助诊断问题根本原因的典型场景，我们手动修改了代码，使某些帧比其他帧消耗更多时间。Listing ?? 显示了代码大纲以及添加的 Tracy 检测。注意，我们随机选择帧来减慢速度。我们还包含了 Tracy 的头文件，并在我们要跟踪的函数中添加了 ZoneScoped 和 FrameMark 宏。FrameMark 宏可以插入以在分析器中识别各个帧。每帧的持续时间将在时间线上可见，这非常有用。

Listing: Tracy 检测

```
{#lst:TracyInstrumentation .cpp} #include "tracy/Tracy.hpp"

void DoExtraWork() { ZoneScoped; // 模拟有用的工作 }

void TraceRowJob() { ZoneScoped; if (frameCount == randomlySelected)
DoExtraWork(); // ... }
```

7.8 内存分析

到目前为止，在本章中，我们讨论了识别程序花费大部分时间的位置的工具。在本节中，我们将专注于程序与内存的交互。这通常称为内存分析。特别是，我们将学习如何收集内存使用情况、分析堆分配和度量内存占用。内存分析帮助你了解应用程序如何随时间使用内存，并帮助你构建程序与内存交互的准确心智模型。以下是它可以回答的一些问题：

- 程序的总虚拟内存消耗是多少，它如何随时间变化？
- 程序在何处以及何时进行堆分配？
- 分配内存最多的代码位置在哪里？
- 程序每秒访问多少内存？
- 程序的总内存占用是多少？

7.8.1 内存使用

内存使用通常由虚拟内存大小（VSZ）和常驻集大小（RSS）描述。VSZ 包括进程可以访问的所有内存，例如堆栈、堆、用于编码可执行文件指令的内存以及来自链接共享库的指令，包括换出到磁盘/SSD 的内存。另一方面，RSS 度量分配给进程的内存量驻留在 RAM 中。因此，RSS 不包括换出的内存或该进程从未触及的内存。RSS 也不包括未加载到内存中的共享库的内存。使用 mmap 映射到内存的文件也对 VSZ 和 RSS 使用有贡献。

考虑一个示例。进程 A 有 200K 的堆栈和堆分配，其中 100K 驻留在主内存中；其余被换出或未使用。它有一个 500K 的二进制文件，其中只有 400K 被触及。进程 A 链接到 2500K 的共享库，只加载了 1000K 到主内存中。

VSZ: $200K + 500K + 2500K = 3200K$

RSS: $100K + 400K + 1000K = 1500K$

开发人员可以使用标准 top 工具在 Linux 上观察 RSS 和 VSZ，但是，这两个指标可能变化得非常快。幸运的是，一些工具可以记录和可视化随时间变化的内存使用情况。图 fig. 7.5 显示了 PSPNet 图像分割算法的内存使用情况，它是 AI Benchmark Alpha 的一部分。^[5] 此图表是基于名为 memory_profiler^[6] 的工具的输出创建的，这是一个建立在跨平台 psutil^[7] 包之上的 Python 库。

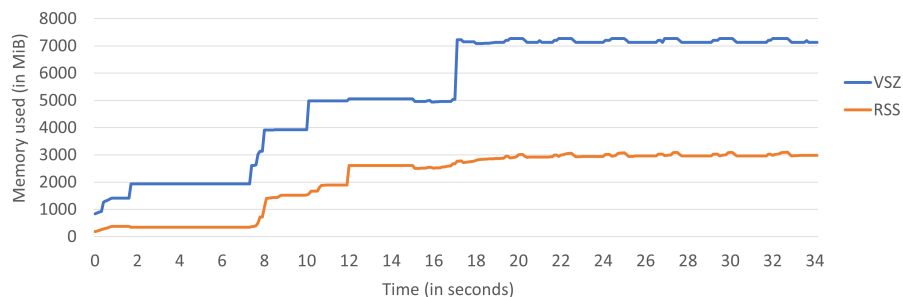


Figure 7.5: AI_bench PSPNet 图像分割的 RSS 和 VSZ 内存利用率。

除了标准的 RSS 和 VSZ 指标外，人们还开发了一些更复杂的指标。由于 RSS 包括进程独有的内存和与其他进程共享的内存，因此不清楚进程自己拥有多少内存。USS（唯一集大小）是进程独有的内存，如果进程现在被终止，这些内存将被释放。PSS（比例集大小）表示唯一内存加上

共享内存量，在共享它的进程之间均匀分配。例如，如果一个进程有 10 MB 全部给自己 (USS) 和 10 MB 与另一个进程共享，它的 PSS 将是 15 MB。psutil 库支持度量这些指标（仅限 Linux），可以由 memory_profiler 可视化。

在 Windows 上，类似的概念由已提交内存大小和工作集大小定义。它们不是 VSZ 和 RSS 的直接等效项，但可用于有效估计 Windows 应用程序的内存使用情况。RAMMap[⁸] 工具提供了有关系统和单个进程内存使用的丰富信息。

当开发人员谈论内存消耗时，他们隐含地指的是堆使用。堆实际上是大多数应用程序中最大的内存消耗者，因为它容纳所有动态分配的对象。但堆不是唯一的内存消耗者。为了完整性，让我们提及其他方面：

- 堆栈：应用程序中堆栈帧使用的内存。应用程序内的每个线程都有自己的堆栈内存空间。通常，堆栈大小只有几 MB，如果超过限制，应用程序将崩溃。例如，Linux 上堆栈内存的默认大小通常为 8MB，尽管它可能因发行版和内核设置而异。macOS 上的默认堆栈大小也是 8MB，但在 Windows 上只有 1 MB。总堆栈内存消耗与系统中运行的线程数成正比。
- 代码：用于存储应用程序及其库的代码（指令）的内存。在大多数情况下，它对内存消耗的贡献不大，但也有例外。例如，Clang 17 C++ 编译器有 33 MB 的代码段，而最新的 Windows Chrome 浏览器在其 219MB 的 chrome.dll 中有 187MB 专用于代码。但是，并非所有代码部分在程序运行时都会被频繁使用。我们将在 Section 11.9 中展示如何度量代码占用。

由于堆通常是内存资源的最大消耗者，因此开发人员在分析应用程序的内存使用情况时关注内存的这一部分是有意义的。在下一节中，我们将研究流行的实际应用程序中的堆消耗和内存分配。

7.8.2 案例研究：分析 Stockfish 的堆分配

在这个案例研究中，我使用 heaptrack[²]，这是 KDE 开发的开源 Linux 堆内存分析器。Ubuntu 用户可以非常容易地使用 `apt install heaptrack heaptrack-gui` 安装它。Heaptrack 可以找到代码中最大和最频繁分配发生的位置以及其他许多事情。在 Windows 上，你可以使用 Mttuner[³]，它具有与 Heaptrack 类似的功能。

7.9 持续分析

在 Section 5 中，我们介绍了可用于进行性能分析的各种方法，包括但不限于检测、跟踪和采样。在这三种方法中，采样施加相对较小的运行时开销，并且需要最少的前期工作量，同时仍然提供对应用程序热点的有价值的见解。但是，这种见解仅限于收集样本的特定时间点。如果我们能给这种采样添加一个时间维度呢？与其知道 FunctionA 在某个特定时间点消耗 30% 的 CPU 周期，不如跟踪 FunctionA 的 CPU 使用率在几天、几周或几个月内的变化？或者检测其调用栈在相同时间跨度内的变化，所有这些都在生产环境中？持续分析的出现将这些目标变为现实。

持续分析 (CP) 是一种系统范围的、基于采样的分析器，它始终开启，尽管采样率较低以最小化运行时影响。从所有进程持续收集数据有助于分析为什么代码在不同时间的执行方式不同，并且还有助于在事件发生后调试事件。CP 工具提供了有关哪些代码使用最多资源的有价值见解，这有助于工程师减少生产环境中的资源使用，从而节省资金。与典型的分析器（如 Linux perf 或 Intel VTune）不同，CP 可以从任何给定日期和时间精确定位从应用程序堆栈到内核堆栈的性能问题，并支持任意两个日期/时间之间的调用栈比较以突出性能差异。

念，该论文倡导在生产环境中始终开启分析的价值。然而，它花了近十年时间才在行业中获得关注：

1. 2019 年 3 月，Google Cloud 发布了其持续分析器。
2. 2020 年 7 月，AWS 发布了 CodeGuru Profiler。
3. 2020 年 8 月，Datadog 发布了其持续分析器。
4. 2020 年 12 月，New Relic 收购了 Pixie 持续分析器。
5. 2021 年 1 月，Pyroscope 发布了其开源持续分析器。
6. 2021 年 10 月，Elastic 收购了 Optimyze 及其持续分析器 (Prodfiler)；Polar Signals 发布了其 Parca 持续分析器。它于 2024 年 4 月开源。
7. 2021 年 12 月，Splunk 发布了其 AlwaysOn Profiler。
8. 2022 年 3 月，Intel 收购了 Granulate 及其持续分析器 (gProfiler)。它于 2024 年 3 月开源。

这个领域的新进入者继续在开源和商业领域涌现。其中一些产品比其他产品需要更多的引导。例如，一些需要源代码或配置文件更改才能开始分析。其他产品需要不同的代理来处理不同的语言运行时（例如，Ruby、Python、Golang、C/C++/Rust）。其中最好的围绕 eBPF 调制了秘制酱料，因此除了简单地安装运行时代理之外，不需要其他任何东西。

它们在支持的语言运行时数量、获取调试符号以获得可读调用栈所需的工作量，以及除 CPU 之外可以分析的系统资源类型（例如，内存、I/O 或锁定）方面也存在差异。虽然持续分析器在上述方面有所不同，但它们都共享为各种语言运行时提供低开销、基于采样的分析以及用于基于 Web 的搜索和查询功能的远程调用栈存储的共同功能。

持续分析走向何方？Optimyze 的联合创始人 Thomas Dullien（开发了创新的持续分析器 Prodfiler）在 QCon London 2023 上发表了主题演讲，他表达了希望有一个集群范围的工具能够回答“为什么这个请求慢？”或“为什么这个请求昂贵？”的问题。在多线程应用程序中，某个特定函数可能在分析中显示为最高的 CPU 和内存消耗者，但其职责可能完全不在应用程序的关键路径之外，例如，一个维护线程。同时，另一个 CPU 执行时间微不足道以至于在分析中几乎没有注册的函数，可能对整体应用程序延迟和/或吞吐量产生巨大影响。典型的分析器无法解决这个缺点。由于 CP 工具基本上是始终运行的分析器，它们继承了同样的盲点。

值得庆幸的是，新一代 CP 工具已经出现，它们采用 AI 和受大型语言模型启发的架构来处理分析样本、分析函数之间的关系，并最终以高精度定位直接影响整体吞吐量和延迟的函数和库。Raven.io 是目前提供这种功能的公司之一。随着这个领域的竞争加剧，创新能力将继续增长，使 CP 工具变得像典型分析器一样强大和稳健。

问题和练习

1. 回想一下我们在本章中使用的所有工具。在以下场景中，你会使用哪些？
 - 场景 1：客户支持团队报告客户问题：升级到新版本应用程序后，某个操作的性能下降了 10%。
 - 场景 2：客户支持团队报告客户问题：某些事务完成时间比平时长 2 倍，没有特定模式。
 - 场景 3：你正在评估三种不同的压缩算法，你想知道每种算法的性能瓶颈是什么。
 - 场景 4：有一个新的闪亮库声称比你当前集成到项目中的库更快；你决定比较它们的性能。
2. 运行你日常使用的应用程序。根据你想要做的改进，哪个工具最适合分析你的应用程序？练习使用这个工具。

3. 假设你在单台机器上运行同一程序的多个副本。分析其中一个副本是否足够，还是需要分析整个系统？

章节总结

- 我们快速概述了三大主要平台 (Linux、Windows 和 MacOS) 上可用的最流行工具。根据 CPU 供应商的不同，分析工具的选择会有所不同。对于具有 Intel 处理器的系统，我们建议使用 VTune；对于具有 AMD 处理器的系统，使用 uProf；在 Apple 平台上，使用 Xcode Instruments。
- Linux perf 可能是 Linux 上最常用的分析工具。它支持所有主要 CPU 供应商的处理器。它没有图形界面。但是，有一些工具可以可视化 perf 的分析数据。
- 我们还讨论了 Windows 事件跟踪 (ETW)，它旨在观察运行系统中的软件动态。Linux 有一个类似的工具叫 KUtrace，⁷⁰ 它在 [Sites, 2022] 一书中有介绍。
- 有一些混合分析器结合了代码检测、采样和跟踪等技术。这吸取了这些方法的优点，允许用户获得关于特定代码片段的非常详细的信息。在本章中，我们研究了 Tracy，它在游戏开发者中相当流行。
- 内存分析器提供有关内存使用、堆分配、内存占用和其他指标的信息。内存分析帮助你了解应用程序如何随时间使用内存。
- 持续分析工具已经成为生产环境中监控性能的重要组成部分。它们收集系统范围的性能指标和调用栈，持续数天、数周甚至数月。这些工具使发现性能变化开始的时间点并确定问题的根本原因变得更容易。

⁷⁰ KUtrace - <https://github.com/dicksites/KUtrace>

第二部分：源代码调优

在本书的第一部分中，我们讨论了如何找到代码中的性能瓶颈。在第二部分中，我们将讨论如何通过使用底层源代码优化技术（也称为调优）来修复这些瓶颈。

现代 CPU 是一个非常复杂的设备，几乎不可能预测某些代码片段的运行速度。软件和硬件性能取决于许多因素，活动部件的数量太大，人类思维无法容纳。幸运的是，得益于我们在本书第一部分中讨论的所有性能监控功能，从 CPU 的角度观察代码如何运行是可能的。我们将广泛依赖本书前面学到的方法和工具来指导我们的性能工程过程。

在非常高的层次上，软件优化可以分为五类。

- **算法优化。**分析程序中使用的算法和数据结构，看看是否能找到更好的。例如：使用快速排序而不是冒泡排序。
- **并行化计算。**如果算法高度可并行化，使程序多线程化，或考虑在 GPU 上运行它。目标是同时做多件事。并发已经在硬件和软件堆栈的所有层中使用。例如：将工作分配给多个线程；在数据中心的许多服务器之间平衡负载；使用异步 IO 避免在等待 IO 操作时阻塞；保持多个并发网络连接以重叠请求延迟。
- **消除冗余工作。**不做不需要或已经做过的工作。例如：利用使用更多 RAM 来减少必须使用的 CPU 和 IO（缓存、查找表、压缩）；预计算编译时已知的值；将循环不变计算移到循环外；通过引用传递 C++ 对象以消除按值传递导致的过多复制。
- **批处理。**聚合多个类似操作并一次性完成，从而减少重复执行操作的开销。例如：发送大的 TCP 数据包而不是许多小的；分配一大块内存而不是为数百个小对象分配空间。
- **排序。**重新排列算法中操作的顺序。例如：更改数据布局以启用顺序内存访问；根据类型对 C++ 多态对象数组进行排序以允许更好的虚函数调用预测；将热函数分组并将其放置在二进制文件中更近的位置。

本书中讨论的许多优化属于多个类别。例如，我们可以说向量化是并行化和批处理的组合；而循环阻塞（分块）是批处理和消除冗余工作的体现。

为了完整起见，让我们也列出其他可能明显但仍然相当合理的加速方式：

- **用另一种语言重写代码：**如果程序是用解释型语言（Python、JavaScript 等）编写的，将其性能关键部分用开销较低的语言重写，例如 C++、Rust、Go 等。
- **调整编译器选项：**检查你是否至少使用了这三个编译器标志：-O3（启用与机器无关的优化）、-march（启用针对特定 CPU 架构的优化）、-flto（启用过程间优化）。但不要止步于此；还有许多其他影响性能的选项。我们将在后面的章节中研究其中一些。你可以考虑为应用程序挖掘最佳选项集；可以使用自动化此过程商业产品。
- **优化第三方软件包：**绝大多数软件项目都利用了专有和开源代码层。这包括操作系统、库和框架。你可以通过替换、修改或重新配置其中一个部分来寻求改进。
- **购买更快的硬件：**显然，这是一个带有相关成本的商业决策，但有时当其他选项已经用尽时，它是提高性能的唯一方法。当你识别出应用程序中的性能瓶颈并清楚地与高层管理人员沟通时，更容易证明购买的合理性。例如，一旦你发现内存带宽限制了多线程程序的性能，你可能会建议购买具有更多内存通道和 DIMM 插槽的服务器主板和处理器。

算法优化

标准算法和数据结构并不总是对性能关键型工作负载有效。例如，传统上，链表的每个新节点都是动态分配的。除了可能调用许多昂贵的内存分配之外，这还可能导致列表的所有元素分散在内存中。遍历这样的数据结构对缓存不友好。即使算法复杂度仍然是 $O(N)$ ，但实际上时间会比普通数组差得多。一些数据结构，如二叉树，具有自然的类似链表的表示，因此用指针追逐方式实现它们可能是诱人的。然而，存在更高效的“扁平”版本的数据结构，例如 `boost::flat_map` 和 `boost::flat_set`。

在为手头的问题选择算法时，你可能会快速选择最流行的选项并继续前进.....即使它可能不是你特定情况下的最佳选择。假设你需要在排序数组中找到一个元素。大多数开发人员考虑的第一个选项是二分查找，对吧？它非常著名，并且在算法复杂度 $O(\log N)$ 方面是最优的。如果我告诉你数组保存 32 位整数值，并且数组大小通常很小（少于 20 个元素），你会改变决定吗？最终，测量应该指导你的决定，但二分查找遭受分支预测错误，因为每个元素值的测试都有 50% 的概率为真。这就是为什么在小数组上，线性扫描通常更快，尽管它的算法复杂度更差。^[1]

数据驱动开发

数据驱动开发（DDD）的主要思想是研究程序如何访问数据：它在内存中如何布局以及它如何在程序中被转换；然后相应地修改程序（更改数据布局、更改访问模式）。这种方法的一个经典示例是结构数组（AOS）到数组结构（SOA）的转换，如 Listing ?? 所示。

Listing: SOA 到 AOS 转换。

```
{#lst:AOSToSOA .cpp}
```

8 优化内存访问

现代计算机仍然基于经典的冯·诺依曼架构构建，该架构将 CPU、内存和输入/输出单元解耦。如今，内存操作（加载和存储）占性能瓶颈和功耗的最大部分。我们从这个类别开始也就不足为奇了。

内存层次结构性能至关重要的说法可以通过图 fig. 8.1 来说明。它显示了内存和处理器之间性能差距的增长。垂直轴是对数刻度，显示了 CPU-DRAM 性能差距的增长。内存基线是 1980 年 64 KB DRAM 芯片的内存访问延迟。典型的 DRAM 性能改进为每年 7%，而 CPU 享受每年 20-50% 的改进。根据这张图，处理器性能已经趋于平稳，但即便如此，性能差距仍然存在。[Hennessy & Patterson, 2017]

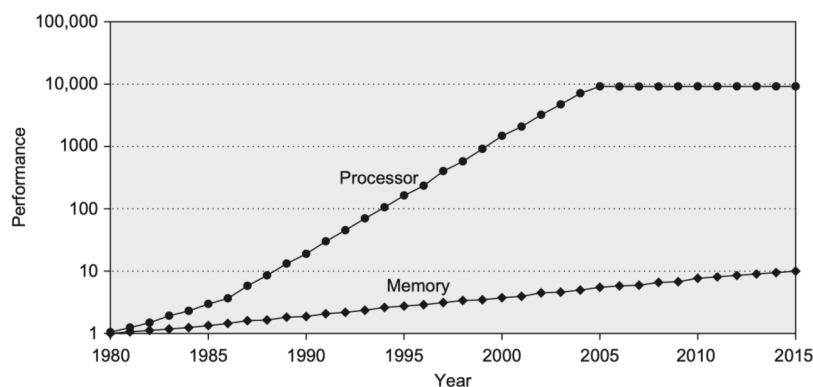


Figure 8.1: 内存和处理器之间的性能差距。© 来源: [Hennessy & Patterson, 2017]。

变量可以在几个时钟周期内从最小的 L1 缓存中获取，但如果它不在 CPU 缓存中，从 DRAM 获取变量可能需要超过三百个时钟周期。从 CPU 的角度来看，最后一级缓存未命中感觉像是非常长的时间，特别是如果处理器在此期间没有做任何有用的工作。当系统被以非常高的速率访问内存的线程高度加载，并且没有可用的内存带宽来及时满足所有加载和存储时，执行线程也可能被饿死。

当应用程序执行大量内存访问并花费大量时间等待它们完成时，这样的应用程序被描述为受内存限制。这意味着为了进一步提高其性能，我们可能需要改进访问内存的方式、减少此类访问的数量，或升级内存子系统本身。

在 TMA 方法中，Memory Bound 指标估计 CPU 流水线因加载或存储指令需求而可能停顿的槽位比例。解决此类性能问题的第一步是定位导致高 Memory Bound 指标的内存访问（参见 Section 6.1.1）。一旦识别出有问题的内存访问问题，可以应用几种优化策略。在本章中，我们将讨论改善内存访问模式的技术。

8.1 缓存友好数据结构

编写缓存友好的算法和数据结构是高性能应用程序的关键要素之一。缓存友好代码的关键支柱是我们在 Section 3.6 中介绍的时间和空间局部性原则。这里的目标是具有可预测的内存访问模式并高效地存储数据。

缓存行是可以在缓存和主内存之间传输的最小数据单元。在设计缓存友好代码时，不仅要考虑单个变量及其在内存中的位置，还要考虑缓存行，这很有帮助。

接下来，我们将讨论使数据结构更缓存友好的几种技术。

8.1.1 顺序访问数据

利用缓存空间局部性的最佳方式是进行顺序内存访问。通过这样做，我们使硬件预取机制（参见 Section ??）能够识别内存访问模式并提前获取下一块数据。Listing ?? 中显示了行主序与列主序遍历的示例。注意，代码中只有一个微小的变化（交换了 col 和 row 下标），但它对性能有很大的影响。

左边的代码不是缓存友好的，因为它在内循环的每次迭代中跳过 NCOLS 个元素。这导致缓存的使用非常低效：我们没有在缓存行被驱逐之前充分利用整个预取的缓存行。相反，右边的代码按照矩阵在内存中布局的顺序访问元素。这保证了缓存行在被驱逐之前会被完全使用。行主序遍历利用空间局部性，是缓存友好的。图 fig. 8.2 说明了两种遍历模式之间的区别。

Listing: 缓存友好的内存访问。

```
// 列主序顺序                                // 行主序顺序
for (row = 0; row < NROWS; row++)              for
    (row = 0; row < NROWS; row++)
    for (col = 0; col < NCOLS; col++)
        for (col = 0; col < NCOLS; col++)
            matrix[col][row] = row + col;      =>
            matrix[row][col] = row + col;
```

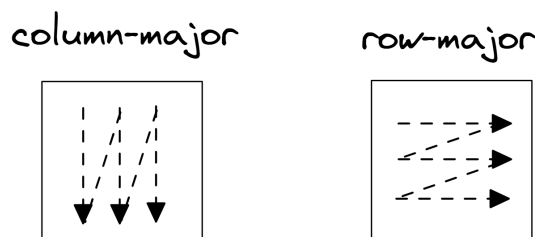


Figure 8.2: 列主序与行主序遍历。

上面展示的例子是经典的，但通常，现实世界的应用程序比这复杂得多。有时你需要额外努力来编写缓存友好的代码。如果数据在内存中的布局不是算法最优的，可能需要先重新排列数据。

考虑在大型排序数组中实现二分查找的标准方法，在每次迭代中，你访问中间元素，将其与你正在搜索的值进行比较，然后向左或向右。该算法不利用空间局部性，因为它测试位于不同位置的元素，这些元素彼此相距很远，并且不共享相同的缓存行。解决此问题的最著名方法是使用 Eytzinger 布局 [Khuong & Morin, 2015] 存储数组元素。其思想是维护一个隐式二叉树搜索树，使用类似 BFS 的布局（通常在二叉堆中看到）打包到数组中。如果代码在数组中执行大量二分查找，将其转换为 Eytzinger 布局可能是有益的。

8.1.2 使用适当的容器。

几乎任何语言中都有各种各样的即用型容器。但了解它们的底层存储和性能影响很重要。记住数据将如何被访问和操作。你应该考虑的不仅是数据结构操作的时间和空间复杂性，还有与之相关的硬件影响。

默认情况下，远离依赖指针的数据结构，例如链表或树。遍历元素时，它们需要额外的内存访问来跟随指针。如果最大元素数量相对较小且在编译时已知，C++ `std::array` 可能比 `std::vector` 更好。如果你需要关联容器但不需要按排序顺序存储元素，`std::unordered_map` 应该比 `std::map` 更快。选择适当 C++ 容器的逐步指南可以在 [Fog, 2023, Section 9.7 Data structures, and container classes] 中找到。

有时，存储指向包含对象的指针而不是对象本身更有效。考虑需要在数组中存储许多对象的情况，同时每个对象的大小很大。此外，对象经常被洗牌、删除和插入。在数组中存储对象将要求每次对象顺序更改时移动大块内存，这是昂贵的。在这种情况下，最好在数组中存储指向对象的指针。这样，只移动指针，这便宜得多。然而，这种方法有其缺点。它需要指针的额外内存，并引入了额外的间接级别。

8.1.3 数据打包

数据缓存的利用率也可以通过使数据更紧凑来提高。打包数据有许多方法。经典的例子之一是使用位字段。Listing ?? 中显示了数据打包可能有益的代码示例。如果我们知道 `a`、`b` 和 `c` 表示需要一定位数来编码的枚举值，我们可以减少结构体 `S` 的存储。

8.2 动态内存分配

开发人员应该意识到动态内存分配有许多相关成本。在堆栈上分配对象是即时完成的，无论对象大小如何：你只需要移动堆栈指针。但是，动态内存分配是一个更复杂的操作。它涉及调用标准库函数，如 `malloc`，这可能会将分配委托给操作系统。避免不必要的动态内存分配是避免这些成本的第一步。这个过程简单目标是临时分配，即直接跟随其释放的分配。在 Section 7.8.2 中，我们展示了如何使用 `heaptrack` 来查找动态内存分配的来源。

你可以通过分配一个大块来摊薄许多小分配的成本。这是 `arena` 分配器⁷¹ 和内存池背后的核心思想。它为手动内存管理提供了更多灵活性。你可以获取操作系统分配的内存区域，并在该区域之上设计自己的分配策略。一个简单的策略可能是将该区域分成两部分：一部分用于热数据，一部分用于冷数据。并提供两种分配方法，它们将使用各自的 `arena`。将热数据放在一起为更好的缓存利用创造了机会。它也可能改善 TLB 利用率，因为热数据将更紧凑，占用更少的内存页。

动态内存分配的另一个成本出现在应用程序使用多个线程时。当两个线程同时尝试分配内存时，操作系统必须同步它们。在高度并发的应用程序中，线程可能花费大量时间等待公共锁来分配内存。内存释放也是如此。同样，自定义分配器可以帮助避免这个问题，例如，为每个线程使用单独的 `arena`。

有许多标准动态内存分配例程 (`malloc` 和 `free`) 的直接替代品，它们更快、更具可扩展性，并且更好地解决碎片问题。一些最流行的内存分配库是 `jemalloc`⁷² 和 `tcmalloc`⁷³。一些项目采用 `jemalloc` 和 `tcmalloc` 作为其默认内存分配器，并且它们看到了显著的性能改进。

⁷¹ 基于区域的内存管理 - https://en.wikipedia.org/wiki/Region-based_memory_management

⁷² `jemalloc` - <http://jemalloc.net/>。

⁷³ `tcmalloc` - <https://github.com/google/tcmalloc>

最后，动态内存分配的一些成本是隐藏的⁷⁴，不容易度量。在所有主要操作系统中，`malloc` 返回的指针只是一个承诺——操作系统承诺当页面被触及时它将提供所需的内存，但实际的物理页面在虚拟地址被访问之前不会被分配。这称为请求调页，它对每个新分配的页面产生一个小页面错误的成本。我们将在 Section 12.3.1 中讨论如何减轻这种成本。此外，出于安全原因，所有现代操作系统在将页面交给下一个进程之前都会擦除其内容（写零）。操作系统维护一个已清零页面的池，以便为分配做好准备。但是当这个池用完可用的已清零页面时，操作系统必须按需清零页面。这个过程不是超级昂贵，但也不是免费的，可能会增加内存分配调用的延迟。

8.3 解决内存带宽限制

如 Section ?? 中所讨论的，处理器通过内存总线从内存获取数据。使用最新的 DDR5 内存技术，最大理论内存带宽为每通道 51.2 GB/s。现代系统具有多个内存通道；例如，典型的笔记本电脑通常有两个内存通道，而服务器系统可以有 4 到 12 个通道。看起来即使是笔记本电脑每秒也可以来回移动大量数据，但实际上，内存带宽在许多应用程序中成为一个限制。

我们应该记住，内存通道在系统中的所有核心之间共享。一旦许多核心参与内存密集型活动，流经内存总线的流量可能会变得拥塞。这可能导致内存请求返回的等待时间增加。现代系统设计为适应多个内存密集型线程同时工作，因此通常需要多个线程来饱和内存带宽。新兴的 AI 工作负载以其“内存饥渴”和高度并行化而闻名，因此内存带宽是它们的首要瓶颈。

解决内存带宽限制的第一步是确定最大理论和预期的内存带宽。理论最大内存带宽可以从内存技术规格中计算，正如我们在 Section 5.5 中所显示的那样。预期的内存带宽可以使用 Intel Memory Latency Checker 或 `lmbench` 等工具来度量，我们在 Section 4.10 中讨论过。Intel VTune 可以在分析应用程序启动之前自动度量内存带宽。

第二步是在应用程序运行时度量内存带宽利用率。如果内存流量接近最大度量带宽，那么你的应用程序的性能可能受内存带宽限制。绘制内存带宽利用率随时间变化的图表是个好主意，以查看是否存在内存强度飙升或下降的不同阶段。如果你在分析配置中勾选“Evaluate max DRAM bandwidth”复选框，Intel VTune 可以提供这样的图表。

如果你已确定你的应用程序受内存带宽限制，第一个建议是看看是否可以降低应用程序的内存强度。这并不总是可能的，但你可以考虑禁用应用程序的一些内存密集型功能，实时重新计算数据而不是缓存结果，或者压缩数据。在 AI 领域，大多数大型语言模型 (LLM) 以 fp32 精度提供，这意味着每个参数占用 4 字节。通过量化技术可以实现最大的性能提升，将参数精度降低到 fp16 或 int8。这将分别减少 2 倍或 4 倍的内存流量。有时，使用 4 位甚至 5 位量化来减少内存流量并在推理性能和质量之间取得适当的平衡。

值得指出的是，对于饱和可用内存带宽的工作负载，代码优化不像对计算受限的工作负载那样发挥重要作用。对于计算受限的应用程序，代码优化（如向量化）通常会转化为显著的性能提升。但是，对于受内存限制的工作负载，向量化可能没有类似的效果，因为处理器无法向前推进，简单地说是因为它缺乏要处理的数据。我们无法使内存总线运行得更快，这就是为什么内存带宽通常是一个难以克服的硬限制。

最后，如果所有选项都用尽了，内存带宽仍然是一个限制，改善这种情况的唯一方法是购买更好的硬件。你可以投资购买具有更多内存通道或更快传输速度的 DRAM 模块的服务器。这可能是昂贵的，但仍然是加速应用程序的可行选择。

⁷⁴ Bruce Dawson: 内存分配的隐藏成本 - <https://randomascii.wordpress.com/2014/12/10/hidden-costs-of-memory-allocation/>。

8.4 减少 DTLB 未命中

如 Section 3.7.1 中所讨论的, TLB 是一个快速但有限的每核心缓存, 用于虚拟到物理地址的内存地址转换。没有它, 应用程序的每次内存访问都将需要耗时的内核页表遍历来计算每个引用虚拟地址的正确物理地址。在具有 5 级页表的系统中, 它将需要访问至少 5 个不同的内存位置来获取地址转换。在第 Section 11.8 节中, 我们将讨论如何将大页用于代码。这里我们将看到如何将它们用于数据。

任何对大内存区域进行随机访问的算法都可能遭受 DTLB 未命中。此类应用程序的例子包括大数组中的二分查找、访问大型哈希表和遍历图。使用大页有潜力加速此类应用程序。

在 x86 平台上, 默认页面大小为 4KB。考虑一个频繁引用 20MB 内存空间的应用程序。使用 4KB 页面, 操作系统需要分配许多小页面。此外, 该进程将触及许多 4KB 大小的页面, 每个页面将争夺有限数量的 TLB 条目。相比之下, 使用 2MB 大页, 20MB 的内存可以用仅仅 10 个页面映射, 而使用 4KB 页面, 你需要 5120 个页面。这意味着使用大页时需要更少的 TLB 条目, 这反过来减少了 TLB 未命中的数量。由于 2MB 条目的数量少得多, 减少不会按 512 的比例。例如, 在 Intel 的 Skylake 核心系列中, L1 DTLB 有 64 个 4KB 页面的条目, 只有 32 个 2MB 页面的条目。除了 2MB 大页之外, AMD 和 Intel 的 x86 芯片还支持 1GB 巨页用于数据, 但不用于指令。使用 1GB 页面而不是 2MB 页面进一步减少了 TLB 压力。

利用大页通常会导致更少的页面遍历, 并且由于表本身更紧凑, 在 TLB 未命中事件中遍历内核页表的惩罚会降低。利用大页的性能提升有时可以高达 30%, 具体取决于应用程序经历的 TLB 压力。期望 2 倍加速要求太高, 因为 TLB 未命中很少是主要瓶颈。论文 [Luo et al., 2015] 介绍了在 SPEC2006 基准测试套件上使用大页的评估。结果可以总结如下。在套件中的 29 个基准测试中, 15 个的加速在 1% 以内, 可以作为噪声丢弃。六个基准测试的加速在 1%-4% 范围内。四个基准测试的加速在 4% 到 8% 范围内。两个基准测试的加速为 10%, 另外两个基准测试分别获得了 22% 和 27% 的加速。

许多实际应用程序已经利用了大页, 例如 KVM、MySQL、PostgreSQL、Java 的 JVM 等。通常, 这些软件包提供启用该功能的选项。每当你使用类似的应用程序时, 请检查其文档以查看是否可以启用大页。

Windows 和 Linux 都允许应用程序建立大页内存区域。如何在 Windows 和 Linux 上启用大页的说明可以在附录 B 中找到。在 Linux 上, 有两种方式在应用程序中使用大页: 显式大页和透明大页。Windows 的支持不如 Linux 丰富, 将在后面讨论。

8.4.1 显式大页

显式大页 (EHP) 作为系统内存的一部分可用, 并作为大页文件系统 `hugetlbfs` 暴露。EHP 应在系统启动时或在应用程序启动之前保留。有关如何操作的说明, 请参见附录 B。在启动时保留 EHP 增加了成功分配的可能性, 因为内存尚未被严重碎片化。显式预分配的页面驻留在物理内存的保留块中, 在内存压力下不能被换出。此外, 此内存空间不能用于其他目的, 因此用户应小心, 只保留他们需要的页面数量。

在 Linux 应用程序中使用 EHP 的最简单方法是使用 `MAP_HUGETLB` 调用 `mmap`, 如 Listing ?? 所示。在此代码中, 指针 `ptr` 将指向为 EHP 显式保留的 2MB 内存区域。注意, 如果 EHP 没有提前保留, 分配可能会失败。在用户代码中使用 EHP 的其他不太流行的方式在附录 B 中提供。此外, 开发人员可以编写自己的基于 `arena` 的分配器来使用 EHP。

Listing: 从未分配的大页映射内存区域。

```

void ptr = mmap(nullptr, size, PROT_READ | PROT_WRITE,
                MAP_PRIVATE | MAP_ANONYMOUS |
                MAP_HUGETLB, -1, 0);
if (ptr == MAP_FAILED)
    throw std::bad_alloc{};
...
munmap(ptr, size);

```

过去，有一个选项可以使用 `libhugetlbfs` 库，它覆盖了现有动态链接可执行文件中使用的 `malloc` 调用，以在 EHP 中分配内存。不幸的是，这个项目不再维护了。它不需要用户修改代码或重新链接二进制文件。他们只需在命令行前加上 `LD_PRELOAD=libhugetlbfs.so HUGETLB_MORECORE=yes <your app command line>` 即可使用它。但幸运的是，其他库支持使用 `malloc` 的大页（不是 EHP），我们接下来将讨论。

8.4.2 透明大页

Linux 还提供透明大页支持（THP），它有两种操作模式：系统范围和每进程。当 THP 在系统范围启用时，内核自动管理大页，对应用程序透明。当需要大块内存且可以分配时，操作系统内核会尝试将大页分配给任何进程，因此大页不需要手动保留。如果 THP 按进程启用，内核仅将大页分配给归因于 `madvise` 系统调用的单个进程的内存区域。你可以通过以下方式检查 THP 是否在系统中启用：

```

$ cat /sys/kernel/mm/transparent_hugepage/enabled
always [madvise] never

```

8.5 显式内存预取

到现在，你应该知道未从缓存解析的内存访问通常非常昂贵。现代 CPU 通过预测程序将来会访问哪些内存位置并提前预取它们来努力降低缓存未命中的惩罚。如果请求的内存位置在程序请求时不在缓存中，那么我们将遭受缓存未命中惩罚，因为我们必须去 DRAM 获取数据。但是，如果 CPU 能够及时将该内存位置带入缓存，或者如果请求被预测且数据正在传输中，那么缓存未命中的惩罚将低得多。

现代 CPU 有两种自动机制来解决这个问题：硬件预取和 OOO 执行。硬件预取器通过在重复内存访问模式上发起预取请求来帮助隐藏内存访问延迟。OOO 引擎向前查看 N 条指令并提前发出加载，以实现未来将需要此数据的指令的平滑执行。

当数据访问模式太复杂而无法预测时，硬件预取器会失败。软件开发人员对此无能为力，因为我们无法控制这个单元的行为。另一方面，OOO 引擎不像硬件预取那样尝试预测将来需要的内存位置。因此，它的成功衡量标准是它通过提前调度加载能够隐藏多少延迟。

考虑 Listing ?? 中的一小段代码，其中 `arr` 是一个包含一百万个整数的数组。索引 `idx` 被赋值为一个随机值，立即用于访问 `arr` 中的一个位置，这几乎肯定会因为它是随机的而在缓存中未命中。硬件预取器无法预测它，因为每次加载都到达内存中一个全新的位置。从知道内存位置的地址（从函数 `random_distribution` 返回）到请求该内存位置的值（调用 `doSomeExtensiveComputation`）的时间间隔称为预取窗口。在这个示例中，OOO 引擎没有机会提前发出加载，因为预取窗口非常小。这导致内存访问 `arr[idx]` 的延迟在执行循环时处于关键路径上，如图 fig. 8.3 所示。程序等待值返回（图中带阴影的填充矩形），而没有向前推进。

你可能会想：“但循环的下一迭代应该开始并行推测执行”。这是正确的，事实上，它在图 fig. 8.3 中有所反映。doSomeExtensiveComputation 函数需要大量工作，当执行接近第一次迭代结束时，CPU 推测地开始执行下一迭代的指令。这在迭代之间创建了正的执行重叠。事实上，我们提出了一个乐观场景，其中处理器能够生成下一个随机数并与循环的上一次迭代并行发出加载。但是，CPU 无法完全隐藏加载的延迟，因为它无法向前查看当前迭代那么远来提前发出加载。也许未来的处理器将具有更强大的 OOO 引擎，但目前，存在需要程序员干预的情况。

Listing: 随机数是后续加载的索引。

```
for (int i = 0; i < N; ++i) {
    size_t idx = random_distribution(generator);
    int x = arr[idx]; // 缓存未命中
    doSomeExtensiveComputation(x);
}
```

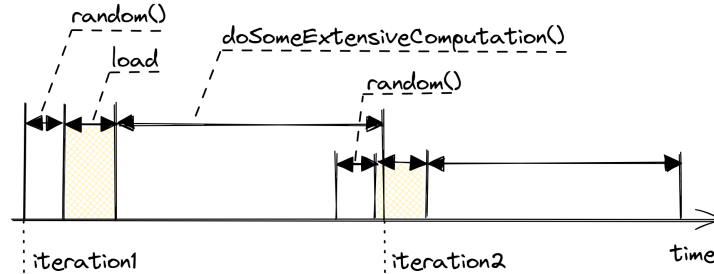


Figure 8.3: 显示加载延迟处于关键路径上的执行时间线。

幸运的是，这不是死胡同，因为有一种方法可以通过将加载与 doSomeExtensiveComputation 的执行完全重叠来加速此代码，这将隐藏缓存未命中的延迟。我们可以通过称为软件流水线和显式内存预取的技术来实现这一点。Listing ?? 中展示了此思想的实现。我们流水线化随机数的生成，并在与 doSomeExtensiveComputation 并行时开始为下一次迭代预取内存位置。

Listing: 利用显式软件内存预取提示。

```
size_t idx = random_distribution(generator);
for (int i = 0; i < N; ++i) {
    int x = arr[idx];
    idx = random_distribution(generator);
    // 为下一次迭代预取元素
    __builtin_prefetch(&arr[idx]);
    doSomeExtensiveComputation(x);
}
```

这种转换的图形说明如图 fig. ?? 所示。我们利用软件流水线为下一次迭代生成随机数。换句话说，在迭代 M 上，我们生成一个将在迭代 M+1 上使用的随机数。这使我们能够提前发出内存请求，因为我们已经知道数组中的下一个索引。这种转换使我们的预取窗口大得多，并完全隐藏了缓存未命中的延迟。在迭代 M+1 上，实际加载有很大的机会命中缓存，因为它在迭代 M 上被预取了。

问题和练习

1. 解决 `perf-ninja::data_packing` 实验室作业，在该作业中你需要使数据结构更紧凑。
2. 使用我们在 Section 8.4 中讨论的方法解决 `perf-ninja::huge_pages_1` 实验室作业。观察性能、`/proc/meminfo` 中的大页分配以及度量 DTLB 加载和未命中的 CPU 性能计数器的任何变化。
3. 通过为未来的循环迭代实现显式内存预取来解决 `perf-ninja::swmem_prefetch_1` 实验室作业。
4. 一般性地描述一段代码要缓存友好需要什么。
5. 运行你日常使用的应用程序。度量其内存使用情况，并使用我们在 Section 7.8 中讨论的内存分析器分析堆分配。使用 Linux perf、Intel VTune 或其他分析器识别热内存访问。是否有方法可以改进这些访问？

章节总结

- 大多数实际应用程序都经历与内存相关的性能瓶颈。新兴的应用程序领域，如机器学习和大数据，在内存带宽和延迟方面要求特别高。
- 内存子系统的性能增长不如 CPU 性能快。然而，内存访问是许多应用程序中性能问题的常见来源。加速此类程序需要修改它们访问内存的方式。
- 在 Section 8 中，我们讨论了开发缓存友好数据结构的常用方法，探索了数据重组技术，学习了如何利用大内存页来提高 DTLB 性能，以及如何使用显式内存预取来减少缓存未命中的数量。

9 优化计算

在上一章中，我们讨论了如何为高效的内存访问扫清道路。完成后，是时候看看 CPU 如何处理它从内存中获取的数据了。现代应用程序需要大量的 CPU 计算，特别是涉及复杂图形、人工智能、加密货币挖矿和大数据处理的应用程序。在本章中，我们将专注于可以减少 CPU 需要完成的工作量并提高程序整体性能的计算。

当应用 TMA 方法时，低效计算通常反映在 Core Bound 类别中，以及在一定程度上反映在 Retiring 类别中。Core Bound 类别表示 CPU 乱序执行引擎中未由内存问题引起的所有停顿。有两个主要类别：

- 软件指令之间的数据依赖限制了性能。例如，一长串依赖操作可能导致低指令级并行 (ILP) 并浪费许多执行槽。下一节将更详细地讨论数据依赖链。
- 硬件计算资源短缺。这表明某些执行单元过载（也称为执行端口争用）。当工作负载频繁执行许多相同类型的指令时，就会发生这种情况。例如，AI 算法通常执行大量乘法。科学应用程序可能运行许多除法和平方根运算。但是，任何给定 CPU 核心中乘法器和除法器的数量是有限的。因此，当端口争用发生时，指令排队等待轮到它们执行。这种类型的性能瓶颈对于特定的 CPU 微架构非常具体，通常没有治愈方法。

在本章中，我们将研究著名的技术，如函数内联、向量化和循环优化。这些代码转换旨在减少执行指令的总数或用更有效的指令替换它们。

9.1 数据依赖

当一个程序语句引用前一个语句的输出时，我们说这两个语句之间存在数据依赖。有时人们也使用术语 `__ 依赖链 __` 或数据流依赖。我们最熟悉的例子是遍历链表（参见图 fig. 9.1）。要访问节点 `N+1`，我们应该首先解引用指针 `N->next`。对于右边的循环，这是一个循环数据依赖，意味着它跨越循环的多次迭代。遍历链表是一个非常长的依赖链。



Figure 9.1: 遍历链表时的数据依赖。

传统程序是假设顺序执行模型编写的。在这个模型中，指令一个接一个地执行，原子地且按照程序指定的顺序。然而，正如我们已经知道的，这不是现代 CPU 的构建方式。它们被设计为乱序、并行地执行指令，并以最大化可用执行单元利用率的方式执行。

当出现长数据依赖时，处理器被迫顺序执行代码，只利用其全部能力的一部分。长依赖链阻碍并行性，这破坏了现代超标量 CPU 的主要优势。例如，指针追逐不会从 OOO 执行中受益，因此将以顺序 CPU 的速度运行。正如我们将在本节中看到的，依赖链是性能瓶颈的主要来源。

你不能消除数据依赖；它们是程序的基本属性。任何程序都需要输入来计算某些东西。事实上，人们已经开发了技术来发现语句之间的数据依赖并构建数据流图。这称为依赖分析，更适合编译器开

发人员，而不是性能工程师。我们对为整个程序构建数据流图不感兴趣。相反，我们想要在热代码片段（如循环或函数）中找到关键的依赖链。

你可能想知道：“如果你不能摆脱依赖链，你能做什么？”好吧，有时这将是性能的限制因素，不幸的是，你将不得不接受它。但有些情况下，你可以打破不必要的数据依赖链或重叠它们的执行。Listing ?? 中展示了一个这样的例子。与其他一些情况类似，我们在左边展示源代码，在右边展示相应的 ARM 汇编。此外，此代码示例包含在 Performance Ninja 在线课程的 dep_chains_2 实验室作业中，因此你可以自己尝试。[^2]

这个小程序模拟随机粒子运动。我们有 1000 个粒子在 2D 表面上移动，没有约束，这意味着它们可以离起始位置任意远。每个粒子由其在 2D 表面上的 x 和 y 坐标以及速度定义。初始 x 和 y 坐标在 [-1000,1000] 范围内，速度在 [0,1] 范围内，不会改变。程序为每个粒子模拟 1000 个移动步骤。对于每个步骤，我们使用随机数生成器 (RNG) 生成一个角度，该角度设置粒子的移动方向。然后我们相应地调整粒子的坐标。

鉴于手头的任务，你决定编写自己的 RNG、正弦和余弦函数，以牺牲一些精度并使其尽可能快。毕竟，这是随机移动，所以这是一个很好的权衡。你选择了一个中等质量的 XorShift RNG，因为它只有 3 次移位和 3 次 XOR。还有什么比这更简单的？此外，你搜索了网络并找到了使用多项式进行正弦和余弦近似的算法，这些算法足够精确且非常快。

Listing: 2D 表面上的随机粒子运动

```
{#lst:DepChain .cpp .numberLines} struct Particle { float x; float y; float
velocity; }; class XorShift32 { uint32_t val; public: XorShift32 (uint32_t
seed) : val(seed) { uint32_t gen() { val ^= (val << 13); val ^= (val >> 17);
val ^= (val << 5); return val; .loop: } eor w0, w0, w0, lsl #13 }; eor w0,
w0, w0, lsr #17 eor w0, w0, w0, lsl #5 static float sine(float x) { ucvtf s1, w0
const float B = 4 / PI_F; fmov s2, w9 const float C = -4 / ( PI_F * PI_F);
fmul s2, s1, s2
```

9.2 内联函数

如果你是那些经常查看汇编代码的开发人员之一，你可能见过 CALL、PUSH、POP 和 RET 指令。在 x86 ISA 中，CALL 和 RET 指令用于调用和从函数返回。PUSH 和 POP 指令用于在堆栈上保存寄存器值并恢复它。

函数调用的细微差别由调用约定描述：参数如何传递以及以什么顺序传递，结果如何返回，被调用函数必须保留哪些寄存器，以及调用者和被调用者之间的工作如何分配。基于调用约定，当调用者进行函数调用时，它期望某些寄存器在被调用者返回后保持相同的值。因此，如果被调用者需要更改应保留的寄存器之一，它需要在返回给调用者之前保存 (PUSH) 和恢复 (POP) 它们。一系列 PUSH 指令称为序言，一系列 POP 指令称为尾声。

当函数很小时，调用函数的开销（序言和尾声）可能非常显著。通过将函数体内联到调用它的位置，可以消除此开销。函数内联是将对函数 foo 的调用替换为使用调用的实际参数专门化的 foo 代码的过程。内联是最重要的编译器优化之一。不仅因为它消除了调用函数的开销，还因为它启用了其他优化。当编译器内联函数时，编译器分析的范围会扩展到更大的代码块，从而实现这一点。然而，也有缺点：内联可能会增加代码大小和编译时间。[^20]

许多编译器中函数内联的主要机制依赖于成本模型。例如，在 LLVM 编译器中，函数内联基于计算每个函数调用点的成本。调用点是代码中调用函数的位置。内联函数调用的成本基于该函数中的

指令数量和类型。如果成本低于阈值（通常是固定数字），则会发生内联；但在某些情况下可以改变。^[21]除了通用成本模型之外，许多启发式方法可以在某些情况下覆盖成本模型决策。例如：

- 微小函数（包装器）几乎总是被内联。
- 具有单个调用点的函数是内联的首选候选者。
- 大函数通常不被内联，因为它们会膨胀调用者的代码。

此外，内联存在问题的情况：

- 递归函数不能内联到自身中，除非它是尾递归函数（参见下一节）。此外，如果递归深度通常很小，可以部分内联递归函数，即将递归函数的主体内联到自身几次，然后像以前一样保留递归调用。如果递归调用深度通常很小，这可以消除函数调用的开销。
- 通过指针引用的函数可以在直接调用的位置内联，但函数必须保留在二进制文件中，即它不能被完全内联和消除。具有外部链接的函数也是如此。

正如我之前写的，编译器在决定内联函数时倾向于使用成本模型方法，这在实践中通常效果良好。通常，依靠编译器做出所有内联决策并在需要时进行调整是一个好策略。成本模型无法考虑每种可能的情况，这留下了改进的空间。有时编译器需要开发人员的特殊提示。在程序中找到潜在内联候选者的一种方法是查看分析数据，特别是函数的序言和尾声有多热。Listing ?? 是一个具有热序言和尾声的函数配置文件示例，占函数时间的 ~50%：

Listing: 函数 foo 的配置文件，具有热序言和尾声

```
{#lst:FuncInlining .cpp} 开销 | 函数 foo 的 (%) | 源代码和反汇编
-----
3.77 : 418be0: push r15 # 序言 4.62 : 418be2: mov r15d,0x64
2.14 : 418be8: push r14 1.34 : 418bea: mov r14,rsi 3.43 : 418bed: push r13 3.08
: 418bef: mov r13,rdi 1.24 : 418bf2: push r12 1.14 : 418bf4: mov r12,rcx 3.08
: 418bf7: push rbp 3.43 : 418bf8: mov rbp,rdx 1.94 : 418bfb: push rbx 0.50 :
418bfc: sub rsp,0x8 ...
```

9.3 循环优化

循环是几乎所有高性能程序的核心。由于循环代表执行大量次数的代码，它们占执行时间的大部分。在这个关键代码片段中的微小变化可能对程序的性能产生很大影响。这就是为什么仔细分析程序中热循环的性能并了解可能的改进方法如此重要。

在本节中，我们将研究解决上述类型瓶颈的最著名的循环优化。我们首先讨论低级优化，在 Section 9.3.1 中，这些优化只在单个循环内移动代码。接下来，在 Section ?? 中，我们将研究重构循环的高级优化，这通常会影响多个循环。注意，我在这里介绍的内容不是所有已知循环转换的完整列表。有关下面讨论的每个转换的更详细信息，读者可以参考 [Cooper & Torczon, 2012] 和 [Allen & Kennedy, 2001]。

9.3.1 低级优化。

让我们从转换单个循环内代码的简单循环优化开始：循环不变代码移动、循环展开、循环强度折减和循环开关。通常，编译器擅长进行此类转换；但是，仍然存在编译器可能需要开发人员支持的情况。我们将在后续章节中讨论这一点。

循环不变代码移动 (LICM)：值在循环的迭代中从不改变的表达式称为循环不变。由于其值在循环迭代中不会改变，我们可以将循环不变表达式移到循环外。我们通过将结果存储在临时变量中并

在循环内使用它来做到这一点（参见 Listing ??）。[17] 如今，所有优秀的编译器在大多数情况下都能成功执行 LICM。

Listing: 循环不变代码移动

```
for (int i = 0; i < N; ++i)          for (int i = 0;
    i < N; ++i) {                    i < N; ++i) {
    for (int j = 0; j < N; ++j)      =>    auto temp =
        c[i];                        for (int j =
        a[j] = b[j] * c[i];          0; j < N; ++j)
        0; j < N; ++j)              a[j] = b[j]
                                    * temp;
                                    }
    }
```

循环展开：__ 归纳变量 __ 是循环中的变量，其值是循环迭代次数的函数。例如， $v = f(i)$ ，其中 i 是迭代次数。我们不是在每次迭代时修改归纳变量，而是展开循环并为归纳变量的每次增量执行多次迭代（参见 Listing ??）。

Listing: 循环展开

```
for (int i = 0; i < N; ++i)          int i = 0;
    i+=2) {                          for (; i+1 < N;
    a[i] = b[i] * c[i];              a[i] = b[i]
        * c[i];                      a[i+1] =
                                     b[i+1] * c[i+1];
    b[i+1] * c[i+1];                }
                                     // 余数 (当 N
                                     为奇数时)
    ++i)                            for (; i < N;
                                     a[i] = b[i] *
                                     c[i];
```

循环展开的主要好处是每次迭代执行更多计算。在每次迭代结束时，索引值必须递增并测试，然后如果它有更多迭代要处理，我们就返回循环顶部。这项工作通常称为“循环开销”或“循环税”，可以通过循环展开来减少。例如，通过将 Listing ?? 中的循环展开 2 倍，我们将执行的比较和分支指令的数量减少 2 倍。

我不推荐手动展开循环，除非你需要打破循环携带的依赖，如 Listing ?? 所示。首先，因为编译器非常擅长自动完成此操作，通常可以选择最优的展开因子。第二个原因是处理器具有“嵌入式展开器”，这要归功于其乱序推测执行引擎（参见 Section 3）。当处理器等待第一次迭代的长延迟指令完成（例如加载、除法、微编码指令、长依赖链）时，它将推测地开始执行第二次迭代的指令，并且只等待循环携带的依赖。这跨越了多个迭代，有效地在指令重排序缓冲区（ROB）中展开了循环。第三个原因是展开过多可能由于代码膨胀而导致负面后果。

9.4 向量化

在现代处理器上，使用 SIMD 指令可以比常规未向量化（标量）代码获得巨大的加速。在进行性能分析时，软件工程师的首要任务之一是确保代码的热部分被向量化。本节指导工程师发现向量化机会。有关现代 CPU 的 SIMD 功能的回顾，读者可以查看 Section 3.4。

向量化通常在没有任何用户干预的情况下自动发生；这称为编译器自动向量化。在这种情况下，编译器自动识别从源代码生成 SIMD 机器代码的机会。

自动向量化非常方便，因为现代编译器可以自动为各种程序生成快速的 SIMD 代码。然而，在某些情况下，没有软件工程师的干预，自动向量化不会成功。现代编译器具有扩展功能，允许高级用户控制自动向量化过程，并确保代码的某些部分被高效地向量化。我们将提供几个使用编译器自动向量化提示的示例。

在本节中，我们将讨论如何利用编译器自动向量化，特别是内循环向量化，因为它是自动向量化最常见的类型。另外两种类型（外循环向量化和超级字级并行向量化）不在本书中讨论。

9.4.1 编译器自动向量化

多个障碍可能阻止自动向量化，其中一些是编程语言语义固有的。例如，编译器必须假设循环索引可能溢出，这可能阻止某些循环转换。另一个例子是 C 语言的假设：程序中的指针可能指向重叠的内存区域，这会使程序分析变得非常困难。

另一个主要障碍是处理器本身的设计。在某些情况下，处理器没有针对某些操作的有效向量指令。例如，谓词（位掩码控制）加载和存储操作在大多数处理器上不可用。尽管存在所有这些挑战，你可以解决其中的许多问题并启用自动向量化。在本节后面，我们提供关于如何与编译器协作并确保热代码被编译器向量化的指导。

向量化器通常分三个阶段构建：合法性检查、盈利性检查和转换本身：

- **合法性检查：**在此阶段，编译器检查是否可以合法地将循环（或另一种类型的代码区域）转换为使用向量。合法性阶段收集需要发生的一系列要求，以使循环的向量化合法。循环向量化器检查循环的迭代是否连续，这意味着循环线性推进。向量化器还确保循环中的所有内存和算术操作都可以展宽为连续操作。循环的控制流在所有 lane 上是统一的，内存访问模式是统一的。编译器必须检查或确保生成的代码不会触及它不应该触及的内存，并且操作顺序将被保留。编译器需要分析指针的可能范围，如果它缺少某些信息，它必须假设转换是非法的。
- **盈利性检查：**接下来，向量化器检查转换是否盈利。它比较不同的向量化宽度，并找出哪个执行最快。向量化器使用成本模型来预测不同操作的成本，例如标量加法或向量加载。它需要考虑将数据混洗到寄存器中添加的指令，预测寄存器压力，并估计确保允许向量化的前提条件的循环保护的成本。检查盈利性的算法很简单：1) 将代码中所有操作的成本相加，2) 比较每个版本代码的成本，3) 将成本除以预期执行次数。例如，如果标量代码成本为 8 个周期，而向量化代码成本为 12 个周期但一次执行 4 次循环迭代，那么循环的向量化版本可能更快。
- **转换：**最后，在向量化器确定转换是合法且盈利的之后，它转换代码。此过程还包括插入启用向量化的保护。例如，大多数循环使用未知的迭代次数，因此编译器必须生成循环的标量版本（余数），以及循环的向量化版本，以处理最后几次迭代。编译器还必须检查指针是否不重叠等。所有这些转换都使用在合法性检查阶段收集的信息完成。

9.4.2 发现向量化机会。

发现改进向量化的机会应该从分析程序中的热循环开始，并检查编译器执行了哪些优化。检查编译器向量化报告（参见 Section 5.7）是了解这一点的最简单方法。现代编译器可以报告某个循环是否被向量化，并提供其他详细信息，例如向量化因子（VF）。当编译器无法向量化循环时，它也能够说明失败的原因。

使用编译器优化报告的另一种方法是检查汇编输出。最好分析来自分析工具的输出，该工具显示给定循环的源代码和生成的汇编指令之间的对应关系。这样，你只关注重要的代码，即热代码。然而，理解汇编语言比像 C++ 这样的高级语言要困难得多。可能需要一些时间来弄清楚编译器生成的指令的语义。然而，这项技能非常有价值，通常提供有价值的见解。

有经验的开发人员可以通过查看指令助记符和这些指令使用的寄存器名称来快速判断代码是否已被向量化。例如，在 x86 ISA 中，向量指令对打包数据进行操作（因此名称中有 P）并使用 XMM、YMM 或 ZMM 寄存器，例如，VMULPS XMM1, XMM2, XMM3 将 XMM2 和 XMM3 中的四个单精度浮点数相乘，并将结果保存在 XMM1 中。但要小心，人们通常从看到 XMM 寄存器被使用就得出结论它是向量代码——不一定。例如，VMULSS XMM1, XMM2, XMM3 指令将只乘以一个单精度浮点值，而不是四个。

潜在向量化机会的另一个指标是高 Retiring 指标（高于 80%）。在 Section 6.1 中，我们说过 Retiring 指标是性能良好代码的良好指标。其背后的原理是执行没有停顿，CPU 以高速率退休指令。然而，有时它可能隐藏真正的性能问题，即低效计算。也许工作负载执行大量可以被向量指令替换的简单指令。在这种情况下，高 Retiring 指标不会转化为高性能。

开发人员在尝试加速可向量化代码时经常遇到几种常见情况。下面我们介绍四种典型场景，并就每种情况给出一般性指导。

9.4.2.1 向量化是非法的。 在某些情况下，遍历数组元素的代码根本不可向量化。优化报告非常有效地解释了出了什么问题以及为什么编译器无法向量化代码。Listing ?? 显示了阻止向量化的循环内依赖的示例。[31]

9.5 编译器内置函数

有些类型的应用程序具有值得大力调优的热点。但是，编译器并不总是能够在这些热点中生成我们想要的代码。有时人类专家可以想出比编译器生成的代码性能更好的代码。它通常涉及一些棘手或专门的算法，这些算法对于编译器来说可能非常困难甚至不可能找出。在这些情况下，可能无法使用 C 和 C++ 语言的标准构造让编译器生成所需的汇编代码。

当绝对有必要生成特定汇编指令时，你不应该依赖编译器自动向量化。在这种情况下，代码可以使用编译器内置函数编写。在大多数情况下，编译器内置函数提供与汇编指令的 1 到 1 映射。Listing ?? 中的示例展示了如何使用编译器内置函数编写数组元素的水平求和（参见 Listing ??）的 C++ 版本。

Listing: 使用编译器内置函数对数组元素求和。

```
1 #include <immintrin.h>
2
3 float calcSum(float* a, unsigned N) {
4     __m128 sum = _mm_setzero_ps();    // 用零初始化求和
5     unsigned i = 0;
```

```

6  for (; i + 3 < N; i += 4) {
7      __m128 vec = _mm_loadu_ps(a + i); // 从数组加载 4
        个浮点数
8      sum = _mm_add_ps(sum, vec);      // 将 vec 累加到
        sum
9  }
10
11  // 128 位向量的水平求和
12  __m128 shuf = _mm_movehdup_ps(sum); // 将元素 3,1
        广播到 2,0
13  sum = _mm_add_ps(sum, shuf);        // 部分和 [0+1]
        和 [2+3]
14  shuf = _mm_movehl_ps(shuf, sum);    // 高半部分 ->
        低半部分
15  sum = _mm_add_ss(sum, shuf);        // 结果在低元素中
16  float result = _mm_cvtss_f32(sum); //
        空操作（编译器消除它）
17
18  // 处理剩余元素
19  for (; i < N; i++)
20      result += a[i];
21  return result;
22 }

```

当为 SSE 目标编译 Listing ?? 时，编译器将生成与 Listing ?? 基本相同的汇编代码。我展示这个例子只是为了说明目的。显然，如果编译器可以生成相同的机器代码，就没有必要使用内置函数。你应该仅在编译器无法生成所需代码时使用内置函数。

当你利用编译器自动向量化时，它将插入所有必要的运行时检查。例如，它将确保有足够的元素来馈送向量执行单元（参见 Listing ??，第 6 行）。此外，编译器将生成循环的标量版本来处理余数（第 19 行）。当你使用内置函数时，你必须自己处理安全方面。

内置函数比内联汇编更好，因为编译器执行类型检查，负责寄存器分配，并进行进一步的优化，例如窥孔转换和指令调度。然而，它们通常仍然冗长且难以阅读。

当你使用不可移植的平台特定内置函数编写代码时，你还应该为其他架构提供后备选项。Intel 平台上所有可用内置函数的列表可以在这个参考^[11]中找到。对于 ARM，你可以在 Arm 的网站上找到这样的列表。^[14]

问题和练习

1. 使用我们在本章中讨论的技术解决以下实验室作业：

- perf-ninja::function_inlining 1
- perf-ninja::vectorization 1 和 2
- perf-ninja::dep_chains 1 和 2
- perf-ninja::compiler_intrinsics 1 和 2
- perf-ninja::loop_interchange 1 和 2

- `perf-ninja::loop_tiling_1`
2. 描述你将采取的步骤来找出应用程序是否正在利用 SIMD 代码的所有机会。
 3. 在真实代码上练习手动进行循环优化。确保所有测试仍然通过。
 4. 假设你正在处理一个具有非常低 IpCall（每调用指令数）指标的应用程序。你将尝试应用/强制哪些优化？
 5. 运行你日常使用的应用程序。找到最热的循环。它是否被向量化？是否可以强制编译器自动向量化？循环是否受依赖链或执行吞吐量限制？

章节总结

- 低效计算在实际应用程序的瓶颈中占很大比例。现代编译器非常擅长通过执行许多不同的代码转换来消除不必要的计算开销。尽管如此，我们仍然有很大的机会做得比编译器所能提供的更好。
- 在 Section 9 中，我们展示了如何通过强制某些代码优化来搜索程序中的性能空间。我们讨论了诸如函数内联、循环优化和向量化等流行的转换。

10 优化分支预测

到目前为止，我们一直在讨论优化内存访问和计算。然而，我们还没有讨论另一类重要的性能瓶颈。它与推测执行有关，推测执行是所有现代高性能 CPU 核心中都存在的特性。为了刷新你的记忆，请参阅 Section 3.3.3，我们在其中讨论了如何使用推测执行来提高性能。在本章中，我们将探索减少分支预测错误数量的技术。

通常，现代处理器非常擅长预测分支结果。它们不仅遵循静态预测规则，还检测动态模式。通常，分支预测器保存先前结果的历史记录，并尝试猜测下一个结果是什么。但是，当模式变得难以遵循 CPU 分支预测器时，它可能会损害性能。

当分支预测错误定期发生时，它可能会带来显著的惩罚。当发生此类事件时，CPU 需要清除所有先前完成的推测工作，这些工作后来被证明是错误的。它还需要刷新流水线并开始用正确路径的指令填充它。通常，现代 CPU 因分支预测错误而经历 10 到 25 个周期的惩罚。确切的周期数取决于微架构设计，即流水线的深度和用于从预测错误中恢复的机制。

分支预测错误最频繁的原因可能是因为它具有复杂的模式（例如，表现出伪随机行为），这对处理器来说是不可预测的。为了完整起见，让我们介绍分支预测错误背后的其他不太频繁的原因。分支预测器使用缓存和历史寄存器，因此容易出现与缓存相关的问题，即：

- **冷未命中**：当没有动态历史记录可用且使用静态预测时，分支的第一次动态发生时可能会发生预测错误。
- **容量未命中**：由于程序中分支数量非常多或动态模式过长而导致动态历史丢失而产生的预测错误。
- **冲突未命中**：分支使用其虚拟和/或物理地址的组合映射到缓存桶（关联集）。如果太多活动分支映射到同一集合，则可能发生历史丢失。冲突未命中的另一个实例是别名，当两个独立分支映射到同一缓存条目并可能相互干扰从而降低预测历史时。

程序将始终经历非零数量的分支预测错误。你可以通过查看 TMA Bad Speculation 指标来了解程序因分支预测错误而遭受多少损失。对于通用应用程序，Bad Speculation 指标在 5-10% 范围内是正常的。我的建议是，一旦该指标超过 10%，就要密切关注。

过去，开发人员有一种选项可以通过分支指令前缀的形式向 x86 处理器提供预测提示（0x2E：Branch Not Taken、0x3E：Branch Taken）。这可能会在较旧的微架构（如 Pentium 4）上提高性能。但是，现代 x86 处理器过去忽略这些提示，直到 Intel 的 RedwoodCove 开始再次使用它。它的分支预测器仍然擅长寻找动态模式，但现在它将使用编码的预测提示来处理以前从未见过的分支（即当没有关于分支的存储信息时）。[Intel, 2023, Section 2.1.1.1 Branch Hint]

有间接的方法可以通过减少动态分支指令数量来降低分支预测错误率。这种方法有帮助，因为它减轻了分支预测器结构的压力。当程序执行较少的分支指令时，它可能会间接改善以前遭受容量和冲突未命中的分支的预测。编译器转换（如循环展开和向量化）有助于减少动态分支计数，尽管它们并不专门针对改善任何给定条件语句的预测率。配置文件引导优化（PGO）和链接后优化器（例如 BOLT）也通过改善直通率（拉直代码）来有效减少分支预测错误。我们将在下一章讨论这些技术。

75

消除分支预测错误的唯一直接方法是消除分支指令本身。在接下来的章节中，我们将研究改善分支预测的直接和间接方法。特别是，我们将探索以下技术：用查找表替换分支、算术、选择和 SIMD 指令。

10.1 用查找替换分支

避免频繁预测错误分支的一种方法是使用查找表。Listing ?? 中显示了这种转换可能有益的代码示例。与往常一样，原始版本在左边，改进版本在右边。函数 `mapToBucket` 将 `[0-50)` 范围内的值映射到相应的五个桶，并为超出此范围的值返回 `-1`。对于均匀分布的 `v` 值，`v` 落入任何桶的概率相等。在原始版本的生成汇编中，我们可能会看到许多分支，这些分支可能具有高预测错误率。希望可以使用单个数组查找来重写函数 `mapToBucket`，如右边所示。

Listing: 用查找表替换分支。

<pre>int8_t mapToBucket(unsigned v) { buckets[50] = { if (v < 10) return 0; 0, 0, 0, 0, 0, else if (v < 20) return 1; 1, 1, 1, 1, 1, else if (v < 30) return 2; 2, 2, 2, 2, 2, else if (v < 40) return 3; 3, 3, 3, 3, 3, else if (v < 50) return 4; 4, 4, 4, 4, 4 }; return -1; }</pre>	=>	<pre>int8_t 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, int8_t mapToBucket(unsigned v) { if (v < (sizeof(buckets) / sizeof(int8_t))) return buckets[v]; return -1; }</pre>
--	----	---

对于右边改进版本的 `mapToBucket`，编译器可能会生成单个分支指令来保护对 `buckets` 数组的越界访问。通过此函数的典型热路径将执行未采取的分支和一个加载指令。该分支将被 CPU 分支预测器很好地预测，因为我们期望大多数输入值落入 `buckets` 数组覆盖的范围内。查找也将很快，因为 `buckets` 数组很小，并且可能在 L1 D-cache 中。

如果我们需要映射更大的值范围，例如 `[0-1M)`，分配一个非常大的数组是不切实际的。在这种情况下，我们可以使用区间映射数据结构，使用更少的内存但对数查找复杂度来实现该目标。读者可

⁷⁵ 有一个传统观点认为永远不会采取的分支对分支预测是透明的，不会影响性能，因此从预测的角度来看，删除它们没有太大意义。然而，与这种观点相反，BOLT 优化器的作者进行的一项实验表明，在大型代码占用应用程序（如 Clang C++ 编译器）中，用等大小的空操作替换永远不会采取的分支，在现代 Intel CPU 上可带来约 5% 的加速。因此，尝试消除所有分支仍然是值得的。

以在 Boost⁷⁶ 和 LLVM⁷⁷ 中找到区间映射容器的现有实现。

10.2 用算术替换分支

在某些场景中，可以用算术替换分支。Listing ?? 中的代码也可以使用简单的算术公式重写，如 Listing ?? 所示。注意，对于此代码，Clang-17 编译器将昂贵的除法替换为更便宜的乘法和右移操作。

Listing: 用算术替换分支。

```
int8_t mapToBucket(unsigned v) {
    al, -1
    constexpr unsigned BucketRangeMax = 50;
    edi, 49
    if (v < BucketRangeMax)
        .exit
        return v / 10;
    eax, dil
    return -1;
    eax, eax, 205
}
    eax, 11
    .exit:
    ret
```

截至 2024 年，编译器通常无法自己找到这些捷径，因此由程序员手动完成。如果你能找到一种方法用算术替换经常预测错误的分支，你很可能会看到性能改进。

10.3 用选择替换分支

某些分支可以通过执行分支的两个部分然后选择正确的结果来有效消除。Listing ?? 中显示了这种转换可能有利的代码示例。如果 TMA 建议 if (cond) 分支具有非常多的预测错误，你可以尝试通过右边所示的转换来消除分支。

Listing: 用选择替换分支。

```
int a;
x = computeX();
if (cond) { /* 经常预测错误 */ => int y =
    computeY();
    a = computeX();
    a = cond ? x : y;
} else {
    foo(a);
    a = computeY();
}
```

⁷⁶ C++ Boost interval_map - https://www.boost.org/doc/libs/1_65_0/libs/icl/doc/html/boost/icl/interval_map.html

⁷⁷ LLVM 的 IntervalMap - https://llvm.org/doxygen/IntervalMap_8h_source.html

```
foo(a);
```

对于右边的代码，编译器可以替换来自三元运算符的分支，并生成 CMOV x86 指令。CMOVcc 指令检查 EFLAGS 寄存器中一个或多个状态标志（CF、OF、PF、SF 和 ZF）的状态，并在标志处于指定状态或条件时执行移动操作。类似的转换可以使用 FCMOVcc 和 VMAXSS/VMINSS 指令对浮点数完成。在 ARM ISA 中，有 CSEL（条件选择）指令，但也有 CSINC（选择并递增）、CSNEG（选择并取反）和其他一些条件指令。

Listing: 用选择替换分支 - x86 汇编代码。

# 原始版本	# 无分支版本
400504: test ebx,ebx	400537: mov eax,0x0
400506: je 400514	40053c: call <computeX>
# 计算 x; a = x	
400508: mov eax,0x0	400541: mov ebp,eax
# ebp = x	
40050d: call <computeX>	=> 400543: mov eax,0x0
400512: jmp 40051e	400548: call <computeY>
# 计算 y; a = y	
400514: mov eax,0x0	40054d: test ebx,ebx
# 测试 cond	
400519: call <computeY>	40054f: cmovne eax,ebp
# 如果需要，用 x 覆盖 a	
40051e: mov edi,eax	400552: mov edi,eax
400521: call <foo>	400554: call <foo>

Listing ?? 显示了原始版本和无分支版本的汇编列表。与原始版本相比，无分支版本没有跳转指令。但是，无分支版本独立计算 x 和 y，然后选择一个值并丢弃另一个。虽然这种转换消除了分支预测错误的惩罚，但它比原始代码做了更多的工作。

我们已经知道原始版本左边的分支很难预测。这就是我们首先尝试无分支版本的动机。在这个例子中，此更改的性能增益取决于 computeX 和 computeY 函数的特性。如果函数很小 [1] 并且编译器可以内联它们，那么选择可能会带来显著的性能优势。如果函数很大 [2]，承担分支预测错误的成本可能比执行 computeX 和 computeY 函数更便宜。最终，性能测量总是决定哪个版本更好。

再看看 Listing ??。在左边，处理器可以预测，例如，je 400514 分支将被采取，推测地调用 computeY，并开始从函数 foo 运行代码。记住，分支预测发生在我们知道分支实际结果之前的许多周期。到我们开始解析分支时，我们可能已经完成了 foo 函数的一半，尽管它仍然是推测性的。如果我们正确，我们节省了很多周期。如果我们错误，我们必须承担惩罚并从正确路径重新开始。在后一种情况下，我们不会从已经完成 foo 的一部分这一事实中获得任何收益，它都必须被丢弃。如果预测错误发生得太频繁，恢复惩罚就会超过推测执行的收益。

使用条件选择，情况就不同了。没有分支，所以处理器不必推测。它可以并行执行 computeX 和 computeY 函数。但是，在计算 CMOVNE 指令的结果之前，它无法开始从 foo 运行代码，因为 foo 将其用作参数（数据依赖）。当你使用条件选择指令时，你将控制流依赖转换为数据流依赖。

10.4 多个测试单个分支

我们在本章中讨论的最后一种技术旨在通过组合多个测试来最小化动态分支指令的数量。这里的主要思想是避免对大数组的每个元素执行分支。相反，目标是同时执行多个测试，这主要涉及使用 SIMD 指令。多个测试的结果是一个向量掩码，可以转换为字节掩码，通常可以用单个分支指令处理。这使我们能够消除许多分支指令，你很快就会看到。你可能会在各种算法（如 JSON/HTML 解析、媒体编解码器等）的 SIMD 实现中遇到这种技术。

Listing ?? 显示了一个通过逐个测试字符来查找输入字符串中最长行的函数。我们遍历输入字符串并搜索换行符 (eol) (\n, ASCII 中的 0x0A)。对于每个找到的 eol 字符，我们检查当前行是否是最长的，并将当前行的长度重置为零。此代码将为每个字符执行一条分支指令。^[1]

Listing: 查找最长行 (逐个字符)。

```
uint32_t longestLine(const std::string &str) {
    uint32_t maxLen = 0;
    uint32_t curLen = 0;
    for (auto s : str) {
        if (s == '\n') {
            maxLen = std::max(curLen, maxLen);
            curLen = 0;
        } else {
            curLen++;
        }
    }
    // 如果最后没有换行符
    maxLen = std::max(curLen, maxLen);
    return maxLen;
}
```

考虑 Listing ?? 中显示的替代实现，它一次测试八个字符。你通常会看到使用编译器内置函数 (参见 Section 9.5) 实现的这个想法，但是，为了清晰起见，我决定展示标准 C++ 代码。这个确切的案例出现在 Performance Ninja 的一个实验室作业中，^[2] 所以你可以自己尝试编写 SIMD 代码。请记住，我展示的代码是不完整的，因为它缺少一些角落情况；我提供它只是为了说明这个想法。

Listing: 查找最长行 (一次 8 个字符)。

```
{#lst:LongestLineSIMD .cpp .numberLines} uint32_t longestLine(const
std::string &str) { uint32_t maxLen = 0; const uint64_t eol = 0x0a0a0a0a0a0a0a0a;
auto buf = str.data(); uint32_t lineBeginPos = 0; for (uint32_t pos = 0; pos
+ 7 < str.size(); pos += 8) { // 加载输入字符串的 8 字节块。uint64_t vect =
((const uint64_t*)(buf + pos)); // 检查此块中的所有字符。
```

问题和练习

1. 重新审视 Listing ?? 中右边显示的代码示例。假设我们开始频繁获得 [0-50) 范围之外的数字。这将为保护对 buckets 数组越界访问的分支引入许多新的预测错误。你将如何更改代码来消除这些新引入的预测错误？
2. 使用我们在本章中讨论的技术解决以下实验室作业：

- `perf-ninja::branches_to_cmov_1`
- `perf-ninja::lookup_tables_1`
- `perf-ninja::virtual_call_mispredict`
- `perf-ninja::conditional_store_1`

3. 运行你日常使用的应用程序。收集 TMA 分解并检查 `BadSpeculation` 指标。查看被归因于最多分支预测错误的代码。是否有使用我们在本章中讨论的技术来避免分支的方法？

编码练习：编写一个微基准测试，该测试将经历 50% 的预测错误率或尽可能接近。你的目标是编写一半分支指令被预测错误的代码。这不像你想的那么简单。一些提示和想法：

- 分支预测错误率衡量为 `BR_MISP_RETIRED.ALL_BRANCHES / BR_INST_RETIRED.ALL_BRANCHES`。
- 如果你用 C++ 编码，你可以使用 1) 类似于 `perf-ninja` 的 Google benchmark 库，2) 编写一个普通的控制台程序并使用 Linux `perf` 收集 CPU 计数器，或者 3) 将 `libpfm` 库集成到微基准测试中（参见 Section 5.3.1）。
- 没有必要发明一些复杂的算法。一个简单的方法是生成 `[0;100)` 范围内的伪随机数，并检查它是否小于 50。随机数可以提前预生成。
- 请记住，现代 CPU 可以记住长（但仍然有限）的分支结果序列。

章节总结

- 现代处理器非常擅长预测分支结果。因此，我建议仅当 TMA 指向高 `Bad Speculation` 指标时才关注分支预测错误。
- 当分支结果模式变得难以遵循 CPU 分支预测器时，应用程序的性能可能会受到影响。在这种情况下，算法的无分支版本可能具有更好的性能。在本章中，我展示了如何用查找表、算术和选择替换分支。
- 无分支算法并非普遍有益。始终进行测量以找出在你的特定情况下什么更有效。
- 有间接的方法可以通过减少程序中动态分支指令的数量来降低分支预测错误率。这种方法有帮助，因为它减轻了分支预测器结构的压力。此类技术的示例包括循环展开/向量化、用位操作替换分支以及使用 SIMD 指令。

11 机器代码布局优化

CPU 前端 (FE) 负责获取和解码指令并将它们传递给乱序后端 (BE)。随着新处理器获得更多的执行”马力”，CPU FE 需要足够强大以保持机器平衡。如果 FE 无法跟上提供指令，BE 将得不到充分利用，整体性能将受到影响。这就是为什么 FE 被设计为始终远在实际执行之前运行，以消除可能发生的任何小问题，并始终有指令准备执行。例如，2016 年发布的 Intel Skylake 每个周期可以获取最多 16 条指令。

大多数时候，CPU FE 中的低效可以描述为后端等待执行指令但前端无法提供它们的情况。结果，CPU 周期被浪费而没有做任何实际有用的工作。回想一下，现代 CPU 每个周期可以处理多条指令，现在范围从 4 到 9 宽。并非所有可用槽都被填充的情况经常发生。这代表了许多领域应用程序的低效来源，例如数据库、编译器、Web 浏览器等。

TMA 方法在 Frontend Bound 指标中捕获 FE 性能问题。它表示 CPU FE 无法向 BE 提供指令的周期百分比，而 BE 本可以接受这些指令。大多数实际应用程序经历非零的”Frontend Bound”指标，意味着运行时间的一定百分比将因次优的指令获取和解码而损失。低于 10% 是常态。如果你看到”Frontend Bound”指标超过 20%，就值得花时间处理它。

FE 无法向执行单元提供指令的原因可能有很多。大多数时候，这是由于次优的代码布局，导致 I-cache 和 ITLB 利用率低下。具有大代码库（例如数百万行代码）的应用程序特别容易受到 FE 性能问题的影响。在本章中，我们将研究一些改进机器代码布局的典型优化。

11.1 机器代码布局

当编译器将源代码转换为机器代码时，它生成一个线性字节序列。Listing ?? 显示了一个小 C++ 代码片段的二进制布局示例。一旦编译器完成生成汇编指令，它需要编码它们并在内存中按顺序排列它们。

Listing: 机器代码布局示例

C++ 代码	汇编列表	反汇编机器代码
.....		
if (a <= b)	; a 在 edi 中	401125
cmp esi, edi		
bar();	; b 在 esi 中	401128
jb 401131		
else	cmp esi, edi	40112a
call bar		
baz();	jb .label1	40112f
jmp 401136		
	call bar()	401131
call baz		
	jmp .label2	401136
...		
	.label1:	

```

        call baz()
.label2:
    ...

```

代码在目标文件中的放置方式称为机器代码布局。请注意，对于相同的程序，可以以多种不同的方式布局代码。对于 Listing ?? 中的代码，编译器可能决定以某种方式反转分支，使对 baz 的调用首先出现。此外，函数 bar 和 baz 的主体可以按两种不同的顺序放置：我们可以先在可执行映像中放置 bar，然后是 baz，或者反转顺序。这会影响指令在内存中放置的偏移量，这反过来可能影响生成程序的性能，正如你稍后将看到的。在本章的以下部分中，我们将研究一些机器代码布局的典型优化。

11.2 基本块

基本块是具有单个入口和单个出口的指令序列。图 fig. 11.1 显示了一个基本块的简单示例，其中 MOV 指令是入口，JA 是出口指令。如果控制流可以从 BB1 到 BB2，则基本块 BB1 是基本块 BB2 的前驱。类似地，如果控制流可以从 BB2 到 BB3，则基本块 BB3 是基本块 BB2 的后继。虽然一个基本块可以有一个或多个前驱和后继，但中间的指令不能进入或退出基本块。

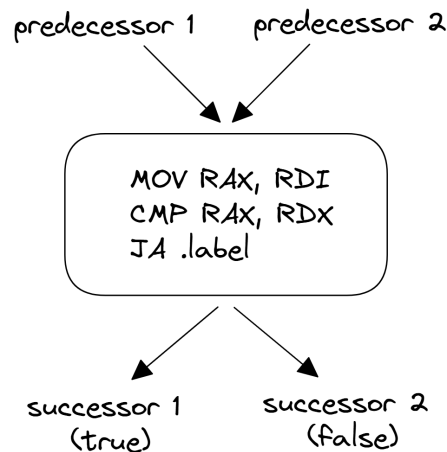


Figure 11.1: 汇编指令的基本块。

可以保证基本块中的每条指令将只执行一次。这是许多编译器转换所利用的重要属性。例如，它大大减少了控制流图分析和转换的问题，因为对于某些类别的问题，我们可以将基本块中的所有指令视为一个实体。

11.3 基本块放置

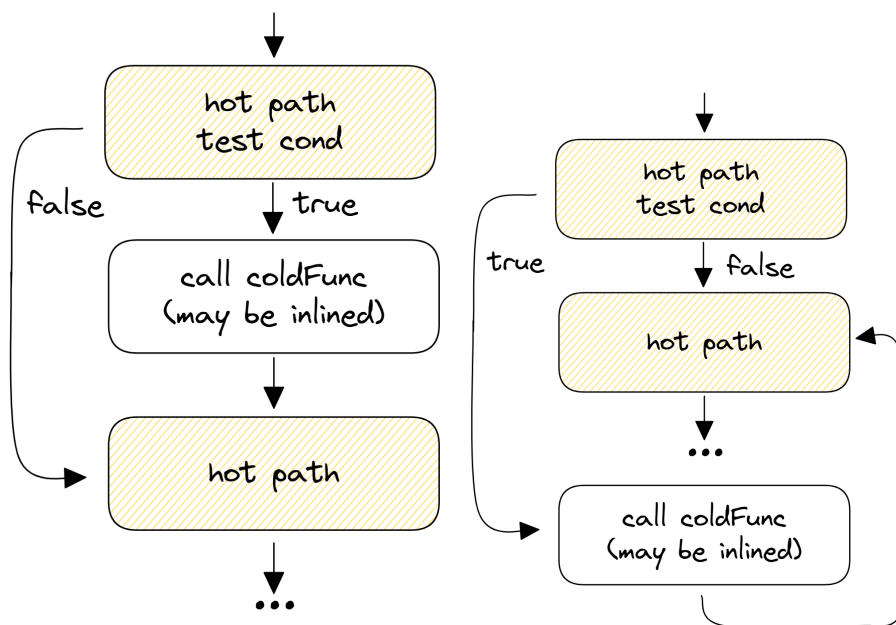
假设我们有一个热路径，其中有一些错误处理代码 (coldFunc)：

```

// 热路径
if (cond)
    coldFunc();
// 热路径继续

```

图 fig. 11.3 显示了此代码片段的两种可能的物理布局。图 fig. ?? 是大多数编译器在没有提供提示时默认发出的布局。如果我们反转条件 `cond` 并将热代码作为直通放置，则可以实现图 fig. ?? 中显示的布局。



上述代码片段的两种机器代码布局版本。

哪种布局更好？好吧，这取决于 `cond` 通常为真还是假。如果 `cond` 通常为真，那么我们最好选择默认布局，否则我们将执行两次跳转而不是一次。此外，如果 `coldFunc` 是一个相对较小的函数，我们希望将其内联。但是，在这个特定示例中，我们知道 `coldFunc` 是一个错误处理函数，可能不经常执行。通过选择布局 fig. ??，我们保持热代码之间的直通，并将采取的分枝转换为未采取的分支。

图 fig. ?? 中显示的布局性能更好有几个原因。首先，图 fig. ?? 中的布局更好地利用了指令和 μop 缓存 (DSB, 参见 Section 3.8.1)。所有热代码连续，没有缓存行碎片：L1 I-cache 中的所有缓存行都由热代码使用。 μop 缓存也是如此，因为它也基于底层代码布局进行缓存。其次，采取的分支对获取单元来说也更昂贵。CPU 的前端获取连续对齐的字节块，通常为 16、32 或 64 字节，具体取决于架构。对于每个采取的分支，跳转指令之后和分支目标之前的获取块中的字节未被使用。这降低了最大有效获取吞吐量。最后，在某些架构上，未采取的分支从根本上比采取的分支更便宜。例如，Intel Skylake CPU 每个周期可以执行两个未采取的分支，但每两个周期只能执行一个采取的分支。[²]

要建议编译器生成改进版本的机器代码布局，你可以使用 `[[likely]]` 和 `[[unlikely]]` 属性提供提示，这些属性自 C++20 以来可用。使用此提示的代码如下所示：

```
// 热路径
if (cond) [[unlikely]]
    coldFunc();
```

// 热路径继续

在上面的代码中, `[[unlikely]]` 提示将指示编译器 `cond` 不太可能为真, 因此编译器应该相应地调整代码布局。在 C++20 之前, 开发人员可以使用 `__builtin_expect[3]` 构造。他们通常创建 `LIKELY` 包装器提示以使代码更可读。例如:

```
#define LIKELY(EXPR)    __builtin_expect((bool)(EXPR),
                        true)
#define UNLIKELY(EXPR) __builtin_expect((bool)(EXPR),
                        false)
// 热路径
if (UNLIKELY(cond)) // NOT
    coldFunc();
```

11.4 基本块对齐

有时性能可能会根据指令在内存中放置的偏移量而发生显著变化。考虑 Listing ?? 中显示的一个简单函数, 以及使用 `-O3 -march=core-avx2 -fno-unroll-loops` (禁用循环展开以说明该思想) 编译时相应的机器代码。

Listing: 基本块对齐

<code>void benchmark_func(int* a) {</code>	00000000004046a0
<code><_Z14benchmark_funcPi>:</code>	
<code>for (int i = 0; i < 32; ++i)</code>	4046a0: mov
<code>rax, 0xffffffffffff80</code>	
<code>a[i] += 1;</code>	4046a7: vpcmpeqd
<code>ymm0, ymm0, ymm0</code>	
<code>}</code>	4046ab: nop DWORD
<code>[rax+rax+0x0]</code>	
	4046b0: vmovdqu
<code>ymm1, [rdi+rax+0x80] # 循环开始</code>	
	4046b9: vpsubd
<code>ymm1, ymm1, ymm0</code>	
	4046bd: vmovdqu
<code>[rdi+rax+0x80], ymm1</code>	
	4046c6: add rax, 0x20
	4046ca: jne 4046b0
<code># 循环结束</code>	
	4046cc: vzeroupper
	4046cf: ret

代码本身是合理的, 但其布局并不完美 (参见图 fig. 11.2)。对应于循环的指令以黄色高亮显示。粗框表示缓存行边界。缓存行长 64 字节。

Listing ?? 中循环的两种不同代码布局。

注意循环跨越多个缓存行: 它开始于缓存行 `0x80-0xBF`, 结束于缓存行 `0xC0-0xFF`。为了获取循环中执行的指令, 处理器需要读取两个缓存行。这种情况有时会导致 CPU 前端出现性能问题, 特别是对于像 Listing ?? 中所示的小循环。

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x80																
0x90																
0xA0	mov	...	...	...	...	...	...	vpcmp	...	...	...	nop	...	...	...	...
0xB0	vmov	...	...	...	...	...	...	...	...	vpsub	...	...	...	vmov	...	...
0xC0	...	...	...	...	...	...	add	...	...	...	jne	...	vzero	...	...	ret
0xD0																
0xE0																
0xF0																

} cache line 0x80-0xBF
 } cache line 0xC0-0xFF

Figure 11.2: 默认布局

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x80																
0x90																
0xA0	mov	...	...	...	...	...	...	vpcmp	...	...	...	nop	...	...	...	...
0xB0	nop	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
0xC0	vmov	...	...	...	...	...	...	...	...	vpsub	...	...	...	vmov	...	...
0xD0	...	...	...	...	...	...	add	...	...	...	jne	...	nop	...	...	...
0xE0	vzero	...	...	ret												
0xF0																

} cache line 0x80-0xBF
 } cache line 0xC0-0xFF

Figure 11.3: 改进布局

为了解决这个问题，我们可以使用单个 NOP 指令将循环指令向前移动 16 字节，使整个循环驻留在一个缓存行中。图 fig. 11.3 显示了使用 NOP 指令（以蓝色高亮显示）执行此操作的效果。

有趣的是，即使你只在微基准测试中运行这个热循环，性能影响也是可见的。这有点令人费解，因为代码量很小，在任何现代 CPU 上都不应该饱和 L1 I-cache 大小。图 fig. 11.3 中布局性能更好的原因并不简单，将涉及相当多的微架构细节，我们不在本书中讨论。感兴趣的读者可以在 Easyperf 博客的相关文章中找到更多信息。^[1]

默认情况下，LLVM 编译器识别循环并在 16B 边界处对齐它们，正如我们在图 fig. 11.2 中看到的。要达到我们示例中所需代码放置（如图 fig. 11.3 所示），你可以使用 `-mllvm -align-all-blocks=5` 选项，该选项将在对象文件中将每个基本块对齐到 32 字节边界。但是，我不建议使用此选项和类似选项，因为它们会影响翻译单元中所有函数的代码布局。还有其他侵入性较小的选项。

11.5 函数分割

函数分割背后的思想是将热代码与冷代码分离。这种转换通常也称为函数轮廓化。这种优化对于具有复杂控制流图和热路径中大块冷代码的相对较大的函数是有益的。Listing ?? 中显示了这种转换可能有益的代码示例。为了从热路径中移除冷基本块，我们剪切并将它们粘贴到一个新函数中，并创建对它的调用。

Listing: 函数分割：冷代码轮廓化到新函数。

```

void foo(bool cond1,
         bool cond2) {
    cond2) {
        // 热路径
        if (cond1) {
            /* 冷代码 (1) */
        }
        // 热路径
        if (cond2) {
            /* 冷代码 (2) */
        }
    }
}

__attribute__((noinline))

__attribute__((noinline))

void foo(bool cond1,
         bool
         // 热路径
         if (cond1) {
             cold1();
         }
         // 热路径
         if (cond2) {
             cold2();
         }
     }
}

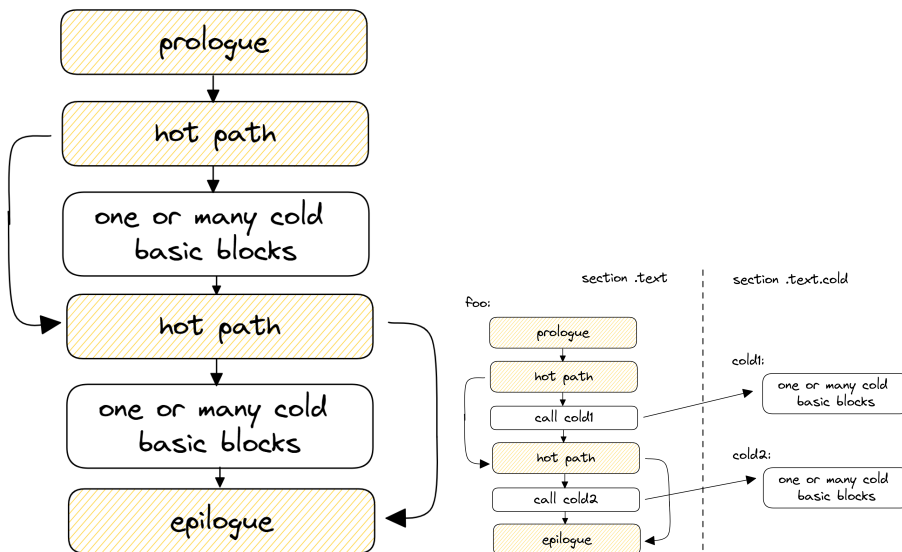
void cold1()
{ /* 冷代码 (1) */ }

void cold2()
{ /* 冷代码 (2) */ }

```

注意，我们通过使用 `noinline` 属性禁用了冷函数的内联。因为如果没有它，编译器可能会决定内联它，这将有效地撤销我们的转换。或者，我们可以在 `cond1` 和 `cond2` 分支上应用 `[[unlikely]]` 宏（参见 Section 11.3），以向编译器传达不希望内联 `cold1` 和 `cold2` 函数。

foo:



将冷代码分割到单独的函数中。

图 fig. 11.5 给出了此转换的图形表示。在改进的布局中，我们在热路径中只保留了一个 CALL 指令，下一个热指令很可能与前一个位于同一缓存行中。这改善了 CPU 前端数据结构（如 I-cache 和 μ op-cache）的利用率。

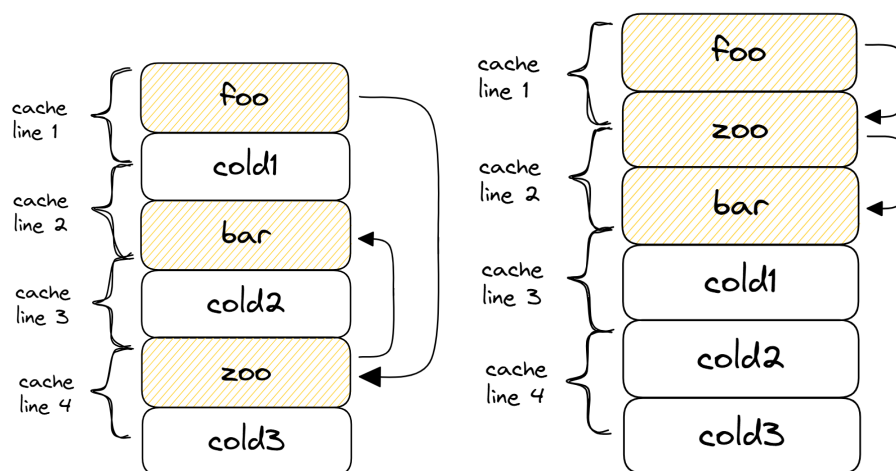
轮廓化函数应该创建在 .text 段之外，例如在 .text.cold 中。如果函数从未被调用，这可以改善内存占用，因为它不会在运行时加载到内存中。

11.6 函数重排序

遵循前面章节中描述的原则，可以将热函数分组在一起，以进一步改善 CPU 前端缓存的利用率。当热函数分组时，它们开始共享缓存行，这减少了代码占用，即 CPU 需要获取的缓存行总数。

图 fig. 11.6 给出了重排序热函数 foo、bar 和 zoo 的图形表示。图像上的箭头显示最频繁的调用模式，即 foo 调用 zoo，而 zoo 又调用 bar。在默认布局中（参见图 fig. ??），热函数彼此不相邻，一些冷函数放置在它们之间。因此，两个函数调用的序列（foo $\rightarrow$ zoo $\rightarrow$ bar）需要四次缓存行读取。⁷⁸

我们可以重新排列函数的顺序，使热函数彼此靠近（参见图 fig. ??）。在改进版本中，foo、bar 和 zoo 函数的代码适合三个缓存行。另外，请注意函数 zoo 现在根据函数调用的顺序放置在 foo 和 bar 之间。当从 foo 调用 zoo 时，zoo 的开头已经在 I-cache 中了。



重排序热函数。

与以前的优化类似，函数重排序改善了 I-cache 和 μ op-cache 的利用率。当有许多小热函数时，此优化效果最好。

链接器负责在最终二进制输出中布局程序的所有函数。虽然开发人员可以尝试自己重新排序程序中的函数，但不能保证期望的物理布局。几十年来，人们一直在使用链接器脚本来实现这一目标。如果你使用 GNU 链接器，这仍然是可行的方法。Gold 链接器 (ld.gold) 有一个更简单的方法来解决这个问题。要使用 Gold 链接器获得二进制文件中函数的期望排序，你可以首先使用 `-ffunction-sections` 标志编译代码，这会将每个函数放入一个单独的段中。然后使用

⁷⁸ 此外，位于共享库中的函数不参与机器代码的精心布局。

`--section-ordering-file=order.txt` 选项提供一个包含排序函数名列表的文件，该列表反映了期望的最终布局。LLD 链接器中也存在相同的功能，它是 LLVM 编译器基础设施的一部分，可通过 `--symbol-ordering-file` 选项访问。

Meta 的工程师在 2017 年引入了一种解决分组热函数问题的有趣方法。他们实现了一个名为 HFSort⁷⁹ 的工具，该工具根据分析数据自动生成段排序文件 [Ottoni & Maher, 2017]。使用此工具，他们观察到大型分布式云应用程序（如 Facebook、Baidu 和 Wikipedia）的 2% 性能加速。HFSort 已集成到 Meta 的 HHVM、LLVM BOLT 和 LLD 链接器中⁸⁰。此后，该算法首先被 HFSort+ 取代，最近又被 Cache-Directed Sort (CDSort⁸¹) 取代，为具有大代码占用的工作负载带来了更多改进。

11.7 配置文件引导优化

编译程序并生成最优汇编完全依赖于启发式方法。代码转换算法有许多边缘情况，旨在特定情况下实现最佳性能。对于编译器做出的许多决策，它都尝试根据某些典型情况猜测最佳选择。例如，在决定是否应内联特定函数时，编译器可以考虑该函数将被调用的次数。问题是编译器事先不知道这一点。它首先需要运行程序来了解。没有任何运行时信息，编译器将不得不猜测。

这就是配置文件信息变得有用的时候。有了配置文件信息，编译器可以做出更好的优化决策。大多数编译器中有一组转换可以根据提供给它们的配置文件数据调整其算法。这组转换称为配置文件引导优化 (PGO)。当配置文件数据可用时，编译器可以使用它来指导优化。否则，它将退回到使用其标准算法和启发式方法。有时在文献中，你可以找到反馈引导优化 (FDO) 一词，它与 PGO 指的是同一件事。

图 fig. 11.4 显示了使用 PGO 的传统工作流程，也称为检测 PGO。首先，你编译程序并告诉编译器自动检测代码。这将向函数中插入一些簿记代码以收集运行时统计信息。第二步是使用代表应用程序典型工作负载的输入数据运行检测后的二进制文件。这将生成配置文件数据，一个包含运行时统计信息的新文件。它是一个原始转储文件，包含有关函数调用次数、循环迭代次数和其他基本块命中次数的信息。此工作流程中的最后一步是使用配置文件数据重新编译程序以生成优化的可执行文件。

开发人员可以通过使用 `-fprofile-instr-generate` 选项构建程序来在 LLVM 编译器中启用 PGO 检测（步骤 1）。这将指示编译器添加检测代码以在运行时收集配置文件信息。此类信息通常包括我们进入程序中每个基本块的次数。

之后，LLVM 编译器可以使用 `-fprofile-instr-use` 选项使用配置文件数据重新编译程序并输出 PGO 调优的二进制文件。在 Clang 中使用 PGO 的指南在文档中描述。⁸² GCC 编译器使用不同的选项集：`-fprofile-generate` 和 `-fprofile-use`，如文档中所述。⁸³

PGO 帮助编译器改善函数内联、代码放置、寄存器分配和其他代码转换。PGO 主要用于具有大代码库的项目，例如 Linux 内核、编译器、数据库、Web 浏览器、视频游戏、生产力工具等。对

⁷⁹ HFSort - <https://github.com/facebook/hhvm/tree/master/hphp/tools/hfsort>

⁸⁰ LLD 中的 HFSort - <https://github.com/llvm-project/lld/blob/master/ELF/CallGraphSort.cpp>

⁸¹ LLVM 中的 Cache-Directed Sort - <https://github.com/llvm-project/blob/main/llvm/lib/Transforms/Utils/CodeLayout.cpp>

⁸² Clang 中的 PGO - <https://clang.llvm.org/docs/UsersManual.html#profiling-with-instrumentation>

⁸³ GCC 中的 PGO - <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html#Optimize-Options>

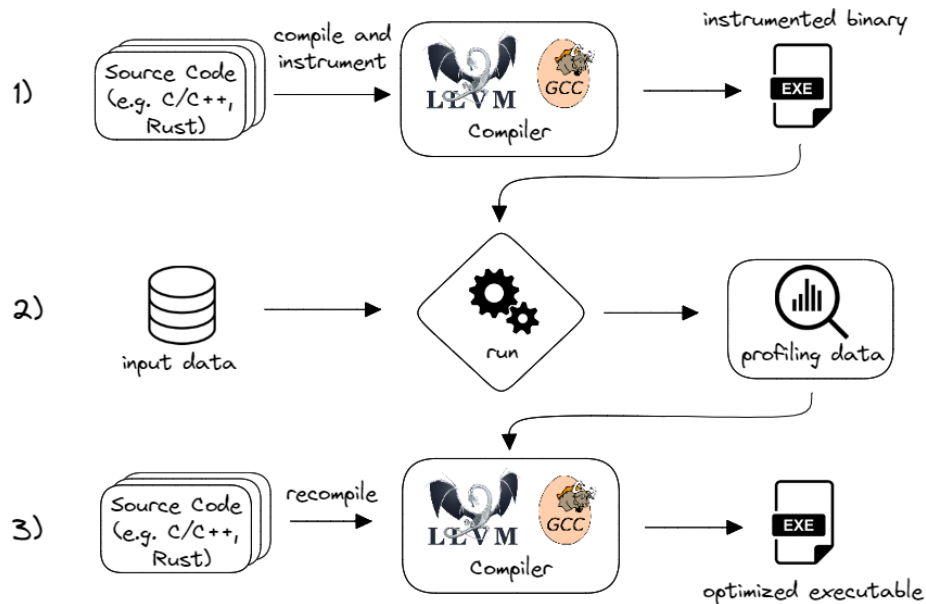


Figure 11.4: 检测 PGO 工作流。

于具有数百万行代码的应用程序，它是改善机器代码布局的唯一实用方法。

并非所有工作负载都能从 PGO 中受益。具有严重前端瓶颈的工作负载可能通过使用 PGO 看到高达 30% 的加速。但是，计算受限的工作负载（如科学计算）可能看不到任何好处。

虽然一些软件项目将检测 PGO 作为其构建过程的一部分采用，但采用率仍然非常低。有几个原因。主要原因是检测可执行文件的巨大运行时开销。运行检测后的二进制文件并收集配置文件数据通常会带来 5-10 倍的减速，这使得构建步骤更长，并阻止直接从生产系统（无论是客户端设备还是云中）收集配置文件。不幸的是，你不能收集一次配置文件数据并将其用于所有未来的构建。随着应用程序源代码的发展，配置文件数据变得过时（不同步）并需要重新收集。

PGO 流程中的另一个注意事项是，编译器应该只使用代表性的场景来训练你的应用程序的使用方式。否则，你可能会降低程序的性能。编译器”盲目地”使用你提供的配置文件数据。它假设程序将始终以相同的方式运行，无论输入数据是什么。PGO 的用户应小心选择用于收集配置文件数据（步骤 2）的输入数据，因为在改进应用程序的一个用例的同时，其他用例可能会被优化。幸运的是，它不必是完全单一的工作负载，因为来自不同工作负载的配置文件数据可以合并以代表应用程序的一组用例。

2016 年，Google 率先提出了基于采样的 PGO 的替代解决方案。[Chen et al., 2016] 不是检测代码，而是可以从标准分析工具（如 Linux perf）的输出中获取配置文件数据。Google 开发了一个名为 AutoFDO⁸⁴ 的开源工具，将 Linux perf 生成的采样数据转换为 GCC 和 LLVM 等编译器可以理解的格式。

这种方法比检测 PGO 有几个优势。首先，它从 PGO 构建工作流中消除了一个步骤，即步骤 1，

⁸⁴ AutoFDO - <https://github.com/google/autofdo>

因为不需要构建检测后的二进制文件。其次，配置文件数据收集在已优化的二进制文件上运行，因此运行时开销要低得多。这使得在生产环境中更长时间地收集配置文件数据成为可能。由于此方法基于硬件收集，它还启用了使用检测 PGO 不可能实现的新优化。一个例子是分支到 `cmov` 转换，这是一种将条件跳转替换为条件移动以避免分支预测错误成本的转换（参见 Section 10.3）。为了有效地执行此转换，编译器需要知道原始分支被预测错误的频率。此信息在现代 CPU（Intel Skylake+）上通过基于采样的 PGO 可用。

2018 年中期，Meta 开源了其二进制优化工具 BOLT⁸⁵，它适用于已编译的二进制文件。它首先反汇编代码，然后使用采样分析器（如 Linux `perf`）收集的配置文件信息进行各种布局转换，然后重新链接二进制文件。[Panchenko et al., 2018] 截至今天，BOLT 有超过 15 个优化传递，包括基本块重排序、函数分割和重排序等。与传统 PGO 类似，BOLT 优化的主要候选者是遭受指令缓存和 ITLB 未命中的程序。自 2022 年 1 月以来，BOLT 已成为 LLVM 项目的一部分，并作为独立工具可用。

BOLT 引入几年后，Google 开源了其二进制重链接工具 Propeller。它服务于类似的目的，但它不是反汇编原始二进制文件，而是依赖于链接器输入，因此可以分布在多台机器上以获得更好的扩展性和更少的内存消耗。Post-link 优化器（如 BOLT 和 Propeller）可以与传统 PGO（和链接时优化）结合使用，通常提供额外的 5-10% 的性能加速。此类技术开启了基于硬件预测的新型二进制重写优化。

11.8 减少 ITLB 未命中

调优前端效率的另一个重要领域是内存地址的虚拟到物理地址转换。这些转换主要由 TLB（参见 Section 3.7.1）提供服务，TLB 在专用条目中缓存最近使用的内存页转换。当 TLB 无法服务转换请求时，会进行耗时的内核页表遍历，以计算每个引用虚拟地址的正确物理地址。每当你从 TMA 摘要中看到高百分比的 ITLB 开销时，本节中的建议可能会派上用场。

通常，相对较小的应用程序不容易受到 ITLB 未命中的影响。例如，Golden Cove 微架构可以在其 ITLB 中覆盖多达 1MB 的内存空间。如果你的应用程序的机器代码适合 1MB，你应该不会受到 ITLB 未命中的影响。当应用程序的频繁执行部分分散在内存中时，问题开始出现。当许多函数开始频繁相互调用时，它们开始争夺 ITLB 中的条目。其中一个例子是 Clang 编译器，在撰写本文时，其代码段约为 60MB。在配备主流 Intel Coffee Lake 处理器的笔记本电脑上运行的 ITLB 开销约为 7%，这意味着 7% 的周期用于处理 ITLB 未命中：执行需要的页面遍历和填充 TLB 条目。

另一组经常受益于使用大页的大型内存应用程序包括关系数据库（例如 MySQL、PostgreSQL、Oracle）、托管运行时（例如 JavaScript V8、Java JVM）、云服务（例如 Web 搜索）、Web 工具（例如 `node.js`）。

减少 ITLB 压力的总体思想是将应用程序的性能关键代码部分映射到 2MB（大）页上。通常，为了简单起见，应用程序的整个代码段会被重新映射。实现此转换的关键要求是代码段在 2MB 边界上对齐。在 Linux 上，这可以通过两种不同的方式实现：使用额外的链接器选项重新链接二进制文件，或在运行时重新映射代码段。两种选项都在 Easyperf⁸⁶ 博客中进行了展示。据我所知，在 Windows 上不可能，所以我将只展示如何在 Linux 上执行此操作。

⁸⁵ BOLT - <https://code.fb.com/data-infrastructure/accelerate-large-scale-applications-with-bolt/>

⁸⁶ “使用大页处理代码的性能优势” - <https://easyperf.net/blog/2022/09/01/Utilizing-Huge-Pages-For-Code>。

第一个选项可以通过使用以下选项链接二进制文件来实现：`-Wl,-zcommon-page-size=2097152` `-Wl,-zmax-page-size=2097152`。这些选项指示链接器将代码段放置在 2MB 边界处，以便在启动时由加载器放置在 2MB 页上。这种放置的缺点是链接器将被迫插入多达 2MB 的填充（浪费）字节，使二进制文件更加膨胀。在 Clang 编译器的示例中，它将二进制文件的大小从 111 MB 增加到 114 MB。重新链接二进制文件后，我们在 ELF 二进制文件头中设置一个特殊位，该位决定文本段是否应默认使用大页支持。最简单的方法是使用 `libhugetlbfs`⁸⁷ 包中的 `hugeedit` 或 `hugectl` 工具。例如：

```
# 在 ELF 二进制文件头中永久设置特殊位。
$ hugeedit --text /path/to/clang++
# 代码段将默认使用大页加载。
$ /path/to/clang++ a.cpp
```

```
# 在运行时覆盖默认行为。
$ hugectl --text /path/to/clang++ a.cpp
```

第二个选项是在运行时重新映射代码段。此选项不需要代码段与 2MB 边界对齐，因此无需重新编译应用程序即可工作。当你无法访问源代码时，这特别有用。此方法背后的思想是在程序启动时分配大页并将代码段转移到那里。此方法的参考实现在 `iodlr`⁸⁸ 库中实现。一个选项是从你的 `main` 函数调用该功能。另一个更简单的选项是构建动态库并在命令行中预加载它：

```
$ LD_PRELOAD=/usr/lib64/liblppreload.so clang++ a.cpp
```

虽然第一种方法仅适用于显式大页，但第二种使用 `iodlr` 的方法同时适用于显式和透明大页。有关如何在 Windows 和 Linux 上启用大页的说明，请参见附录 B。

将代码段映射到大页可以将 ITLB 未命中减少多达 50% [Suresh Srinivas, 2019]，从而为某些应用程序带来高达 10% 的加速。然而，与许多其他功能一样，大页并不适用于所有应用程序。可执行文件只有几 KB 大小的小程序最好使用常规 4KB 页面而不是 2MB 大页；这样可以更有效地使用内存。

除了使用大页之外，标准的 I-cache 性能优化技术也可用于改善 ITLB 性能。即，重排序函数以使热函数更好地共位，通过链接时优化（LTO/IPO）减小热区域的大小，使用配置文件引导优化（PGO）和 BOLT，以及较少激进的内联。

BOLT 提供 `-hugify` 选项，根据配置文件数据自动使用大页处理热代码。使用此选项时，`llvm-bolt` 将注入代码以在运行时将热代码放在 2MB 页上。该实现利用了 Linux 透明大页（THP）。这种方法的好处是只有一小部分代码被映射到大页，并且所需的大页数量被最小化，因此页面碎片减少了。

11.9 案例研究：度量代码占用

正如我在本章中多次提到的，代码布局优化对具有大量代码的应用程序影响最大。澄清程序中热代码大小不确定性的最佳方法是度量其代码占用，它定义为程序执行期间触及的具有机器指令的字节/缓存行/页数。

大的代码占用本身不一定对性能产生负面影响。代码占用不是决定性指标，它不能立即告诉你是否有问题。然而，它已被证明在性能分析中作为额外数据点很有用。结合 TMA 的 `Frontend_Bound`、

⁸⁷ `libhugetlbfs` - <https://github.com/libhugetlbfs/libhugetlbfs/blob/master/HOWTO>。

⁸⁸ `iodlr` 库，仅限 Linux - <https://github.com/intel/iodlr>。

L1 指令缓存未命中率和其他指标，它可以加强为优化应用程序的机器代码布局投入时间的论点。

目前，可用于可靠度量代码占用的工具非常少。在本案例研究中，我将演示 `perf-tools`，^[1] 这是一个建立在 Linux `perf` 之上的开源分析工具集合。为了估计 ^[2] 代码占用，`perf-tools` 利用 Intel 的 LBR（参见 Section 6.2），因此它目前不适用于基于 AMD 或 ARM 的系统。以下是收集代码占用数据的示例命令：

```
$ perf-tools/do.py profile --profile-mask 100 -a <your
benchmark>
```

`--profile-mask 100` 启动 LBR 采样，`-a` 允许你指定要运行的程序。此命令将收集代码占用以及各种其他数据。我不显示该工具的输出，好奇的读者可以研究文档并尝试该工具。

我取了一组四个基准测试：Clang C++ 编译、Blender 光线追踪、Cloverleaf 流体动力学和 Stockfish 国际象棋引擎；这些工作负载你应该已经从 Section 4.11 中熟悉了，我们在其中分析了它们的性能特征。我在 Intel 的 Alder Lake 处理器上运行了它们。^[5]

在我们开始查看结果之前，让我们花一些时间在术语上。程序代码的不同部分可能以不同的频率执行，因此某些部分会比其他部分更热。`perf-tools` 包不区分这一点，并使用术语“非冷代码”来指代至少执行过一次的代码。这称为双向拆分，因为它将代码分为冷和非冷部分。其他工具（例如 Meta 的 HHVM）使用三向拆分，并区分热、暖和冷代码，具有可调整的阈值。在本节中，我们使用术语“热代码”来指代非冷代码。

四个基准测试的每个结果如表 `tbl. ??` 所示。二进制和 `.text` 大小是使用标准 Linux `readelf` 工具获得的，而其他指标是使用 `perf-tools` 收集的。`non-cold code footprint [KB]` 指标是程序至少触及一次的具有机器指令的千字节数。指标 `non-cold code [4KB-pages]` 告诉我们程序至少触及一次的具有机器指令的非冷 4KB 页面数。它们帮助我们理解这些非冷内存位置的密集程度或稀疏程度。最后，我们还展示了前端绑定百分比，这是从关于 TMA 的 Section 6.1 中你应该已经熟悉的指标。

指标	Clang17 编译	Blender C	loverLeaf S	tockfish
二进制大小 [KB]	113844	223914	672	39583
<code>.text</code> 大小 [KB]	67309	133009	598	238
非冷代码占用 [KB] 5042	313	104	99	
非冷代码 [4KB 页面]	6614	546	104	61
前端绑定 [%]	52.3 2	9.4	5.3	25.8

表：案例研究中使用的基准测试的代码占用。{#tbl:code_footprint}

让我们首先看看二进制和 `.text` 大小。CloverLeaf 与 Clang17 和 Blender 相比是一个很小的应用程序；Stockfish 嵌入了神经网络文件，占二进制文件的大部分，但其代码段相对较小；Clang17 和 Blender 具有巨大的代码库。`.text size` 指标是我们应用程序的上限，即我们假设 ^[3] 代码占用不应超过 `.text` 大小。

通过分析代码占用数据可以进行一些有趣的观察。首先，即使 Blender `.text` 段非常大，Blender 代码中不到 1% 是非冷的：133 MB 中的 313 KB。因此，仅仅因为二进制文件很大，并不意味着应用程序遭受 CPU 前端瓶颈。重要的是热代码的数量。对于其他基准测试，这个比率更高：

Clang17 7.5%，CloverLeaf 17.4%，Stockfish 41.6%。从绝对数字来看，Clang17 编译触及的机器指令字节数比其他三个应用程序的总和多一个数量级。

问题和练习

1. 解决 `perf-ninja::pgo` 和 `perf-ninja::lto` 实验室作业。
2. 尝试为代码段使用大页。取一个大型应用程序（访问源代码是加分项但不是必需的），二进制文件大小超过 100MB。尝试使用 Section 11.8 中描述的方法之一将其代码段重新映射到大页上。观察性能、`/proc/meminfo` 中的大页分配以及度量 ITLB 加载和未命中的 CPU 性能计数器的任何变化。
3. 运行你日常使用的应用程序。应用 PGO、llvm-bolt 或 Propeller 并检查结果。比较“之前”和“之后”的配置文件以了解加速来自哪里。

章节总结

CPU 前端优化的总结如表 {tbl. ??} 所示。

转换如何转	换？为什么有帮助	？最适合	由谁完成
基本块放置	保持直通未热代码分	采取的任支更便宜；有更好的缓存利用率	何代码，特别是编译很多分支的代码
基本块对齐	使用 NOP 移动更热代码利	好的缓存热用率	循环编译
函数分割	分割冷代码块更并将它们放置利在单独的函数中	好的缓存具用率函	有复杂 CFG 的编译数，当热部分之间有大块冷代码时
函数重排序	将热函数分组更在一起利	好的缓存许用率	多小热函数链接

表：CPU 前端优化总结。{#tbl:CPU_FE_OPT}

- 代码布局改进通常被低估和忽视。CPU 前端性能问题（如 I-cache 和 ITLB 未命中）占浪费周期的很大一部分，特别是对于具有大代码库的应用程序。但即使是中小型应用程序也可以从优化机器代码布局中受益。
- 如果你能为应用程序提供一组典型的用例，使用 LTO、PGO、BOLT 和类似工具来改善代码布局通常是最佳选择。对于大型应用程序，这是唯一实用的选择。

12 其他调优领域

在本章中，我们将研究一些与前三章涵盖的任何类别没有特别关系的优化主题，但仍然重要到足以在本书中找到它们的位置。

我们将首先讨论如何在代码中引入 CPU 特定的优化。然后，我们将介绍一些可能对应用程序性能产生可衡量影响的边缘情况。接下来，我们继续探索低延迟调优技术，这些技术对于需要快速响应时间的应用程序至关重要。最后，我们将提供有关调整系统设置的建议。

12.1 CPU 特定优化

为特定 CPU 微架构优化软件涉及调整代码以利用该微架构的优势并减轻其劣势。当你知道应用程序的确切目标 CPU 时，这样做更容易。但是，大多数应用程序在各种 CPU 上运行。优化具有非常高速度要求的跨平台应用程序的性能可能具有挑战性，因为来自不同供应商的平台具有不同的设计和实现。尽管如此，可以编写在不同供应商的 CPU 上性能合理的代码，同时为特定微架构提供精细调整的版本。

x86（被视为 CISC）和 RISC ISA（如 ARM 和 RISC-V）之间的主要差异总结如下：

- x86 指令是可变长度的，而 ARM 和 RISC-V 指令是固定长度的。这使得解码 x86 指令更加复杂。
- x86 ISA 具有许多寻址模式，而 ARM 和 RISC-V 具有很少的寻址模式。ARM 和 RISC-V 指令中的操作数是寄存器或立即值，而 x86 指令输入也可以来自内存。这增加了 x86 指令的数量，但也允许更强大的单条指令。例如，ARM 需要先加载内存位置，然后执行操作；x86 可以在一条指令中完成两者。

除此之外，在为特定微架构进行优化时，你还应该考虑一些其他差异。截至 2024 年，最新的 x86-64 ISA 有 16 个架构通用寄存器，而最新的 ARMv8 和 RV64 要求 CPU 提供 32 个通用寄存器。额外的架构寄存器减少了寄存器溢出，从而减少了加载/存储的数量。Intel 宣布了一个名为 APX^[1] 的新扩展，它将寄存器数量增加到 32 个。

x86 和 ARM 之间的内存页大小也存在差异。x86 平台的默认页大小为 4 KB，而大多数 ARM 系统（例如 Apple MacBooks）使用 16 KB 页大小，尽管两个平台都支持更大的页大小（参见 Section 3.7.2 和 Section 8.4）。当这些差异成为瓶颈时，它们都会影响应用程序的性能。

虽然 ISA 差异可能对特定应用程序的性能产生切实影响，但大量研究表明，平均而言，两个最流行的 ISA（即 x86 和 ARM）之间的差异不会产生可衡量的性能影响。在整本书中，我小心地避免了任何产品的广告（例如 Intel vs. AMD vs. Apple）和任何宗教 ISA 辩论（x86 vs. ARM vs. RISC-V）。^[5] 以下是一些我认为可以结束辩论的参考：

- 性能或能耗差异不是由 ISA 差异产生的，而是由微架构实现产生的。[Blem et al., 2013]
- ISA 对执行指令的数量和类型没有很大影响。[Weaver & McIntosh-Smith, 2023] [Blem et al., 2013]
- CISC 代码不比 RISC 代码更密集。[Geelnard, 2022]
- ISA 开销可以通过微架构实现有效缓解。例如， μop 缓存最小化解码开销；指令缓存最小化代码密度影响。[Blem et al., 2013] [ChipsAndCheese, 2024]

尽管如此，这并没有消除架构特定优化的价值。在本节中，我们将讨论如何为特定平台进行优化。我们将介绍 ISA 扩展、CPU 分发技术，并讨论如何推理指令延迟和吞吐量。

12.1.1 ISA 扩展

ISA 演进一直在持续。它专注于加速专门的工作负载，例如加密、AI、多媒体等。利用 ISA 扩展通常会带来显著的性能改进。开发人员不断找到在通用应用程序中利用这些扩展的聪明方法。因此，即使你在这些高度专业化的领域之外，你仍然可能从使用 ISA 扩展中受益。

不可能学习所有特定指令。但我建议你熟悉目标平台上可用的主要 ISA 扩展。例如，如果你正在开发使用 fp16（16 位半精度浮点）数据类型的 AI 应用程序，并且你的目标是现代 ARM 处理器之一，请确保你的程序的机器代码包含相应的 fp16 ISA 扩展。如果你正在开发加密/解密软件，请检查它是否利用了目标 ISA 的加密扩展。等等。

以下是一些著名的 x86 ISA 扩展列表：

- SSE/AVX/AVX2：提供用于浮点和整数操作的 SIMD 指令。
- AVX512：使用 512 位寄存器和许多新指令扩展 AVX2。
- AVX512_FP16/AVX512_BF16：添加对 16 位半精度和 Bfloat16 浮点值的支持。
- AES/SHA：提供用于 AES 加密、解密和 SHA 哈希的指令。
- BMI/BMI2：提供用于位操作的指令。
- AVX_VNNI/AVX512_VNNI：用于加速深度学习工作负载的向量神经网络指令。
- AMX：用于加速矩阵乘法的高级矩阵扩展。

以下是一些著名的 ARM ISA 扩展列表：

12.2 微架构特定性能问题

在本节中，我们将讨论影响大多数现代处理器的一些常见微架构特定问题。我称它们为微架构特定的，因为它们是由特定微架构功能的实现方式引起的。这些问题非常具体，不会经常作为主要性能瓶颈出现。通常，它们被其他更重要的性能问题稀释。因此，这些微架构特定的性能问题被认为是边缘情况，不如我们在本书中已经讨论的其他问题知名。尽管如此，它们可能导致非常不受欢迎的性能惩罚。请注意，特定问题的影响在一种平台上可能比另一种更明显。此外，请记住，下面涵盖的微架构特定问题列表并非详尽无遗。

12.2.1 内存顺序违规

我在 Section 3.8.3 中介绍了内存排序的概念。内存重排序是现代 CPU 的一个关键方面，因为它使它们能够并行和乱序执行内存请求。加载/存储重排序中的关键元素是内存消歧，它预测让加载提前于较早的存储是否安全。由于内存消歧是推测性的，如果处理不当，它可能导致性能问题。

考虑 Listing ?? 左边的示例。此代码片段计算 8 位灰度图像的直方图，即图像中某种颜色出现的次数。除了无数其他地方之外，此代码可以在 Otsu 阈值算法 [1] 中找到，该算法用于将灰度图像转换为二值图像。由于输入图像是 8 位灰度，因此只有 256 种不同的颜色。

对于图像上的每个像素，你需要读取该像素颜色的当前直方图计数，递增它，并将其存回。这是通过内存的经典读取-修改-写入依赖。假设我们在图像中有以下连续像素：pixels = [0xFF, 0xFF, 0x00, 0xFF, ...] 等等。pixels[1] 的直方图计数的加载值来自上一次迭代

的结果 (pixels[0])。pixels[2] 的直方图计数来自内存；它是独立的，可以被重排序。但同样，pixels[3] 的直方图计数依赖于处理 pixels[1] 的结果，依此类推。迭代 0、1 和 3 是依赖的，不能被重排序。

Listing: 内存顺序违规示例。

```
std::array<uint32_t, 256> hist;
    std::array<uint32_t, 256> hist1;
hist.fill(0);
    std::array<uint32_t, 256> hist2;
int N = width * height;
for (int i = 0; i < N; ++i)    =>    hist1.fill(0);
    hist[image[i]]++;           hist2.fill(0);
    * height;                  int N = width
                                int i = 0;
                                for (; i + 1
                                < N; i += 2) {
                                hist1[image[i+0]]++;
                                hist2[image[i+1]]++;
                                }
                                // 余数
                                for (; i < N;
                                ++i)
                                hist1[image[i]]++;
                                //
                                合并部分直方图
                                for (int i =
                                0; i < hist1.size(); ++i)
                                    hist1[i] +=
                                hist2[i];
```

回想 Section 3.8.3，处理器不一定知道潜在的存储到加载转发，因此它必须做出预测。如果它正确预测了颜色 0xFF 两次更新之间的内存顺序违规，那么这些访问将被序列化。性能不会很好，但这是我们可以用初始代码期望的最好结果。相反，如果处理器预测没有内存顺序违规，它将推测地让两个更新并行运行。稍后它将识别错误，刷新流水线，并重新执行两个更新中较新的一个。这对性能非常有害。

性能将很大程度上取决于输入图像的颜色模式。具有长序列相同颜色像素的图像将比颜色不经常重复的图像性能更差。只要两个相同颜色像素之间的距离足够长，初始版本的性能就会很好。在此上下文中，“足够长”一词由乱序指令窗口的大小决定。如果在同一颜色的重复读取-修改-写入发生在几个循环迭代内，它们可能会触发排序违规，但如果它们相隔一百多个循环迭代发生则不会。

Listing ?? 右边显示了内存顺序违规问题的治愈方法。如你所见，我复制了直方图，现在像素的处理在两个部分直方图之间交替。最后，我们组合两个部分直方图以获得最终结果。这个具有两个部分直方图的新版本仍然容易出现潜在的问题模式，例如 0xFF 0x00 0xFF 0x00 0xFF ...。

但是，通过此更改，原始最坏情况（例如 0xFF 0xFF 0xFF ...）将比以前快两倍。根据输入图像的颜色模式，创建四个或八个部分直方图可能是有益的。这段确切的代码出现在 Performance Ninja 课程的 `mem_order_violation_1[2]` 实验室作业中，所以请随意实验。

在少量输入图像上，我在各种平台上观察到 10% 到 50% 的加速。值得一提的是，右边的版本消耗了 1 KB 的额外内存，在这种情况下可能不是很大，但需要注意。

12.3 低延迟调优技术

到目前为止，我们已经讨论了旨在提高应用程序整体性能的各种软件优化。在本节中，我们将讨论在低延迟系统中使用的其他调优技术，例如实时处理和高频交易（HFT）。在这种环境中，主要优化目标是使程序的特定部分尽可能快地运行。当你在 HFT 行业工作时，每一微秒和纳秒都很重要，因为它直接影响利润。通常，低延迟部分实现实时或 HFT 系统的关键循环，例如移动机械臂或向交易所发送订单。优化关键路径的延迟有时是以牺牲程序的其他部分为代价完成的。一些技术甚至牺牲了系统的整体吞吐量。

当开发人员为延迟进行优化时，他们会避免在热路径上需要支付的任何不必要的成本。这通常涉及系统调用、内存分配、I/O 以及其他具有非确定性延迟的任何内容。为了达到尽可能低的延迟，热路径需要所有资源立即就绪和可用。

一个相对简单的技术是预计算你在热路径上执行的一些操作。这会带来使用更多内存的成本，这些内存将对系统中的其他进程不可用，但它可能会在关键路径上为你节省一些宝贵的周期。然而，请记住，有时计算比从内存中获取结果更快。

由于这是一本关于底层 CPU 性能的书，我们将跳过讨论类似于我们刚才提到的高级技术。相反，我们将讨论如何避免页面错误、缓存未命中、TLB 射击和关键路径上的核心节流。

12.3.1 避免小页面错误

虽然该术语包含“小”这个词，但小页面错误对运行时延迟的影响一点也不小。回想一下，当用户代码分配内存时，操作系统只承诺提供一个页面，但它不会立即通过给我们一个清零的物理页面来执行承诺。相反，它将等待用户代码第一次访问它，然后操作系统才履行其职责。对新分配页面的第一次写入会触发小页面错误，这是由操作系统处理的硬件中断。小错误的延迟影响从不到一微秒到几微秒不等，特别是如果你使用具有 5 级页表而不是 4 级页表的 Linux 内核。

如何检测应用程序中的运行时小页面错误？一个简单的方法是使用 `top` 工具（添加 `-H` 选项以获取线程级视图）。将 `vMn` 字段添加到显示列的默认选择中，以查看每次显示刷新间隔内发生的小页面错误数量。Listing ?? 显示了在编译大型 C++ 项目时 `top` 命令的转储，其中包含前 10 个进程。额外的 `vMn` 列显示在最后 3 秒内发生的小页面错误数量。

Listing: 编译大型 C++ 项目时带有额外 `vMn` 字段的 Linux `top` 命令转储。

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU
%MEM	TIME+	COMMAND	vMn					
341763	dendiba+	20	0	303332	165396	83200	R	99.3
1.0	0:05.09	c++	13k					
341705	dendiba+	20	0	285768	153872	87808	R	99.0
1.0	0:07.18	c++	5k					
341719	dendiba+	20	0	313476	176236	83328	R	94.7
1.1	0:06.49	c++	8k					

```

341709 dendiba+ 20 0 301088 162800 82944 R 93.4
1.0 0:06.46 c++ 2k
341779 dendiba+ 20 0 286468 152376 87424 R 92.4
1.0 0:03.08 c++ 26k
341769 dendiba+ 20 0 293260 155068 83072 R 91.7
1.0 0:03.90 c++ 22k
341749 dendiba+ 20 0 360664 214328 75904 R 88.1
1.3 0:05.14 c++ 18k
341765 dendiba+ 20 0 351036 205268 76288 R 87.1
1.3 0:04.75 c++ 18k
341771 dendiba+ 20 0 341148 194668 75776 R 86.4
1.2 0:03.43 c++ 20k
341776 dendiba+ 20 0 286496 147460 82432 R 76.2
0.9 0:02.64 c++ 25k

```

检测运行时小页面错误的另一种方法是使用 `perf stat -e page-faults` 附加到正在运行的进程。

在 HFT 世界中，任何超过 0 的都是问题。但对于其他业务领域中的低延迟应用程序，每秒 100-1000 次错误的恒定发生应该促使进一步调查。调查运行时小页面错误的根本原因可以简单到启动 `perf record -e page-faults`，然后 `perf report` 来定位有问题的源代码行。

为了避免运行时的页面错误惩罚，你应该在启动时为应用程序预取所有内存。一个简单的示例可能如下所示：

```
“‘cpp char *mem = malloc(size);
```

12.4 系统调优

在完成所有调优应用程序以利用 CPU 微架构的所有复杂功能的艰苦工作之后，我们最不希望的是系统固件、操作系统或内核破坏我们所有的努力。如果一个高度调优的应用程序不时被停止整个系统的系统中断打断，那么它将意义不大。这样的中断可能一次运行数十到数百毫秒。

开发人员通常对应用程序执行的环境几乎没有控制权。当我们发布产品时，调整客户可能拥有的每个设置是不现实的。通常，足够大的组织有单独的运维团队（Ops），他们处理此类问题。我们有责任为他们提供建议，以设置系统从我们的应用程序中获得最佳性能。

现代系统中有很多需要调整的东西，避免基于系统的干扰并非易事。x86 服务器部署的性能调优手册的一个例子是 Red Hat 指南⁸⁹。在那里，你将找到消除或显着最小化来自系统 BIOS、Linux 内核和设备驱动程序等来源的缓存干扰中断的提示，以及许多其他应用程序干扰源。这些指南应作为所有新服务器构建的基准映像，然后再将任何应用程序部署到生产环境中。

大多数开箱即用的平台都配置为在尽可能节省功耗的同时实现最佳吞吐量。但有些行业有实时需求，它们更关心低延迟而不是其他一切。这种行业的一个例子是在汽车装配线中运行的机器人。这些机器人执行的动作由外部事件触发，并且通常有预定的时间预算来完成，因为下一个中断很快就会到来（通常称为“控制循环”）。满足此类平台的实时目标可能需要牺牲机器的整体吞吐量或允许其消

⁸⁹ Red Hat 低延迟调优指南 - <https://access.redhat.com/sites/default/files/attachments/2015-01-perf-brief-low-latency-tuning-rhel7-v2.1.pdf>

耗更多能量。该领域的一种流行技术是禁用处理器睡眠状态⁹⁰，以保持其准备好立即响应。另一个有趣的技术称为缓存锁定，⁹¹ 其中 CPU 缓存的一部分被保留用于特定的数据集。它有助于简化应用程序内的内存延迟。

超频 CPU 是提高性能的更极端尝试。超频是将 CPU 以高于其设计频率运行的过程。这是一种有风险的操作，因为它可能使保修无效并可能损坏 CPU。超频不适合生产环境，通常由愿意为性能承担风险的爱好者执行。要超频 CPU，你需要拥有合适的部件，主要是支持超频的主板、具有解锁时钟频率的 CPU，以及可以处理增加的热输出的冷却系统。2024 年初，超频专家在广泛可用的 CPU 上突破了 9 GHz 大关。⁹²

了解应用程序中的性能瓶颈有助于调整正确的设置。可扩展性研究可以帮助你确定你的应用程序对各种系统设置的敏感性。例如，你可能会发现你的应用程序不随核心数量扩展（参见 Section 13.2.1），或者它受内存延迟限制。使用此信息，你可以就调整系统设置或为计算系统购买新硬件组件做出明智的决定。在下一个案例研究中，我们将展示如何确定应用程序是否对最后一级缓存（LLC）的大小敏感。

12.5 案例研究：对最后一级缓存大小的敏感性

在这个案例研究中，我们多次运行相同的应用程序集，LLC 大小各不相同。现代服务器处理器允许用户控制 LLC 空间对处理器线程的分配。这样，用户可以限制每个线程仅使用其分配的共享资源量。此类设施通常称为服务质量（QoS）扩展。它们可用于优先考虑性能关键型应用程序，并减少与同一系统中其他线程的干扰。除了 LLC 分配之外，QoS 扩展还支持限制内存读取带宽。

我们的分析将帮助我们识别当 LLC 大小减小时性能显著下降的应用程序。我们说这些应用程序对 LLC 大小敏感。此外，我们还识别了不敏感的应用程序，即 LLC 大小对性能没有影响。此结果可用于正确确定处理器 LLC 的大小，特别是考虑到市场上可用的广泛范围。例如，我们可以确定应用程序受益于更大的 LLC。那么，对新硬件的投资可能是合理的。相反，如果应用程序的性能不会因拥有大 LLC 而改善，那么我们可能可以购买更便宜的处理器。

对于本案例研究，我们使用 AMD Milan 处理器，但其他服务器处理器（如 Intel Xeon [Herdrich et al., 2016] 和 Arm ThunderX [Wang et al., 2017]）也包括硬件支持，允许用户控制处理器线程的 LLC 空间和内存读取带宽的分配。根据我们的测试，本节中描述的方法在 AMD Zen4 桌面处理器（如 7950X 和 7950X3D）上同样有效。

目标机器：AMD EPYC 7313P

我们使用了一个配备 16 核 AMD EPYC 7313P 处理器的服务器系统，代号 Milan，AMD 于 2021 年推出。该系统的主要特性如表 tbl. ?? 所示。

特性值	
处理器	AMD EPYC 7313P

⁹⁰ 电源管理状态：P 状态、C 状态 - <https://software.intel.com/content/www/us/en/develop/articles/power-management-states-p-states-c-states-and-package-c-states.html>

⁹¹ 缓存锁定。缓存锁定技术调查 [Mittal, 2016]。伪锁定缓存部分的示例，然后在 Linux 文件系统中作为字符设备暴露并可用于 mmap: <https://events19.linuxfoundation.org/wp-content/uploads/2017/11/Introducing-Cache-Pseudo-Locking-to-Reduce-Memory-Access-Latency-Reinette-Chatre-Intel.pdf>

⁹² CPU 超频记录 - <https://press.asus.com/news/press-releases/rog-maximus-z790-apex-enclave-sets-3-overclocking-world-records/>

特性值

核心 x 线程 16 \$	×\$ 2
配置 4 CC	X × 4 核心/CCX
频率 3.0/	3.7 GHz, 基础/最大
L1 缓存 (I, D) 8 路	, 32 KiB (每核心), 64 字节行
L2 缓存 8 路	, 512 KiB (每核心), 64 字节行
LLC 16 路	, 32 MB, 非包含 (每 CCX), 64 字节行
主内存 512	GiB DDR4, 8 通道, 标称峰值带宽: 204.8 GB/s
TurboBoost 已禁	用
超线程已禁	用 (1 线程/核心)
操作系统 Ubun	tu 22.04, 内核 5.15.0-76

问题和练习

1. 解决 Performance Ninja 在线课程中的以下实验室作业：
 - perf-ninja::mem_order_violation_1
 - perf-ninja::mem_alignment_1
 - perf-ninja::io_opt1
2. 研究你使用的处理器支持的 ISA 扩展。检查你正在开发的应用程序是否使用这些扩展。如果没有，它能否从中受益？
3. 运行你日常使用的应用程序。查找热点。检查它们是否遭受我们在本章中讨论的任何微架构特定问题。
4. 描述如何避免应用程序关键路径上的页面错误。

章节总结

- 来自不同供应商的处理器并不相同。它们在支持的指令集架构 (ISA) 和微架构实现方面存在差异。达到峰值性能通常需要利用最新的 ISA 扩展并针对特定 CPU 微架构调整应用程序。
- CPU 分发是一种技术，使你能够引入平台特定的优化。使用它，你可以为特定微架构提供快速路径，同时为其他平台保留通用实现。
- 我们探讨了由应用程序与 CPU 微架构的交互引起的几个性能边缘情况。这些包括内存顺序违规、未对齐的内存访问、缓存别名和非规格化浮点数。
- 我们还讨论了几个低延迟调优技术，这些技术对于需要快速响应时间的应用程序至关重要。我们展示了如何避免关键路径上的页面错误、缓存未命中、TLB 射击和核心节流。
- 系统调优是拼图的最后一块。一些旋钮和设置可能会影响应用程序的性能。确保系统固件、操作系统或内核不会破坏所有调优工作至关重要。

13 优化多线程应用程序

现代 CPU 每年都在获得越来越多的核心。截至 2024 年，你可以购买一个拥有超过 200 个核心的服务器处理器！即使具有 16 个执行线程的笔记本电脑在现在也是相当常见的配置。由于每个 CPU 中有如此多的处理能力，有效利用所有硬件线程变得更具挑战性。为软件做好准备以随着 CPU 核心数量的增长而良好扩展，对于应用程序未来的成功非常重要。

服务器和客户端产品利用并行性的方式存在差异。大多数服务器平台设计用于处理来自大量客户的请求。这些请求通常是相互独立的，因此服务器可以并行处理它们。如果系统上有足够的负载，应用程序本身可以是单线程的，平台利用率仍然会很高。但是，当你将服务器平台用于 HPC 或 AI 计算时；那么你需要你拥有的所有计算能力。另一方面，客户端平台（如笔记本电脑和台式机）具有为单个用户服务的所有资源。在这种情况下，应用程序必须利用所有可用的核心来提供最佳用户体验。在本章中，我们将专注于可以扩展到大量核心的应用程序。

从软件的角度来看，实现并行性有两种主要方式：多进程和多线程。在多进程应用程序中，多个独立进程并发运行。每个进程有自己的内存空间，并通过进程间通信机制（如管道、套接字或共享内存）与其他进程通信。在多线程应用程序中，单个进程包含多个线程，这些线程共享进程的相同内存空间和资源。同一进程内的线程可以更容易地通信和共享数据，因为它们可以直接访问相同的内存空间。但是，线程之间的同步通常更复杂，并且容易出现竞态条件和死锁等问题。在本章中，我们将主要关注多线程应用程序，但是一些技术也可以应用于多进程应用程序。我们将在本章中展示两种类型应用程序的示例。

当谈论面向吞吐量的应用程序时，我们可以区分以下两种类型的应用程序：

- **大规模并行应用程序。**此类应用程序通常随着核心数量的增加而良好扩展。它们被设计为处理大量独立任务。大规模并行程序通常使用分治技术将工作分成较小的任务（也称为工作线程）并并行处理它们。此类应用程序的示例包括科学计算、视频渲染、数据分析、AI 等。此类应用程序的主要障碍是共享资源（如内存带宽）的饱和，这可以有效地阻塞进程中的所有工作线程。
- **需要同步的应用程序。**此类应用程序的工作线程共享资源以完成其任务。工作线程相互依赖，这会产生一些线程被阻塞的时期。此类应用程序的示例包括数据库、Web 服务器和其他服务器应用程序。此类应用程序的主要挑战是尽量减少所需的同步并避免共享资源上的争用。

在本章中，我们将探讨如何分析两种类型应用程序的性能。由于这是一本关于底层性能的书，我们不会讨论算法级别的优化，如无锁数据结构，这些在其他书中有很好的介绍。

13.1 并行效率指标

让我们首先介绍一些对分析多线程应用程序性能很重要的指标。在处理多线程应用程序时，工程师在分析基本指标时应小心，例如 CPU 利用率。其中一个线程可能显示高 CPU 利用率，但结果可能是该线程只是在等待锁时在忙等待循环中自旋。这就是为什么在评估应用程序的并行效率时，建议使用有效 CPU 利用率，它仅基于有效时间。

有效 CPU 利用率

此指标表示应用程序利用可用 CPU 的效率。它显示系统上所有逻辑 CPU 的平均 CPU 利用率百分比。它仅基于有效时间，不包括并行运行时系统⁹³和自旋时间引入的开销。有效 CPU 利用率为 100% 意味着你的应用程序在其运行的整个时间内使所有逻辑 CPU 核心保持忙碌。

对于指定的时间间隔 T，有效 CPU 利用率可以计算为

⁹³ 像 pthread、OpenMP 和 Intel TBB 这样的线程库会因创建和管理线程而产生额外开销。

$$\text{有效 CPU 利用率} = \frac{\sum_{i=1}^{\text{线程数}} \text{有效 CPU 时间 (T,i)}}{T \times \text{线程数}}$$

$$\text{有效 CPU 时间} = \text{CPU 时间} - (\text{开销时间} + \text{自旋时间})$$

度量开销和自旋时间可能具有挑战性，我建议使用像 Intel VTune Profiler 这样的性能分析工具，它可以提供这些指标。

线程数

大多数并行应用程序具有可配置的线程数，这允许它们在具有不同核心数的平台上高效运行。使用比系统上可用的更少的线程数运行应用程序会使其资源利用不足。另一方面，运行过多的线程可能导致过度订阅；一些线程将等待轮到它们运行。

除了实际的工作线程之外，多线程应用程序通常还有其他维护线程：主线程、输入/输出线程等。如果这些线程消耗大量时间，它们将从工作线程中夺取执行时间，因为它们也需要 CPU 核心来运行。这就是为什么知道总线程数并正确配置工作线程数很重要的原因。

为了避免线程创建和销毁的惩罚，工程师通常分配一个线程池^[14]，其中多个线程等待由监督程序分配任务进行并发执行。这对于执行短期任务特别有益。

等待时间

等待时间发生在软件线程因阻塞或导致上下文切换的 API 而等待时。等待时间是按线程计算的；因此，总等待时间可以超过应用程序经过的时间。

线程可能由于同步或抢占而被操作系统调度器切换出执行。因此，等待时间可以进一步分为同步等待时间和抢占等待时间。大量的同步等待时间可能表明应用程序具有高度争用的同步对象。我们将在以下部分中探讨如何找到它们。显著的抢占等待时间可能表示线程过度订阅问题，要么是因为应用程序线程数量过多，要么是因为与系统上的操作系统线程或其他应用程序冲突。在这种情况下，开发人员应考虑减少线程总数或增加每个工作线程的任务粒度。

自旋时间

自旋时间是 CPU 忙碌时的等待时间。当同步 API 导致 CPU 在软件线程等待时进行轮询时，通常会发生这种情况。实际上，内核同步原语的实现会在锁上自旋一段时间，而不是立即让出给另一个线程。然而，过多的自旋时间可能反映出 productive work 的机会丧失。

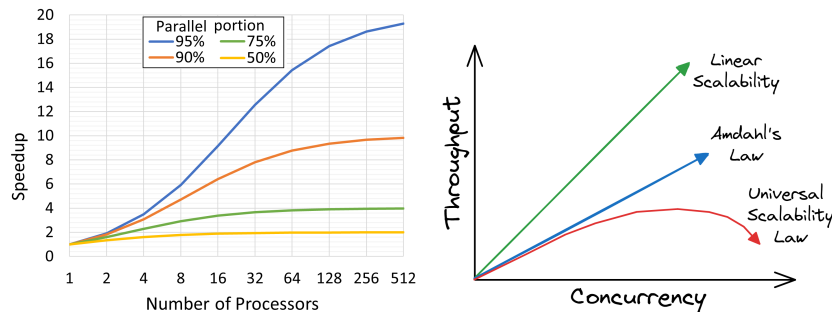
其他并行效率指标的列表可以在 Intel VTune 页面上找到。^[15]

13.2 多线程程序中的性能扩展

在处理单线程应用程序时，优化程序的一部分通常会产生积极的性能结果。但是，对于多线程应用程序，情况并非总是如此。可能存在一个应用程序，其中线程 A 执行长时间运行的操作，而线程 B 提前完成其任务并只是等待线程 A 完成。无论我们如何改进线程 B，应用程序延迟都不会降低，因为它将受更长时间运行的线程 A 的限制。

这种效果被广泛称为阿姆达尔定律，⁹⁴ 它认为并行程序的加速受其串行部分的限制。图 fig. ?? 说明了程序执行延迟作为执行它的处理器数量函数的理论加速。对于 75% 并行的程序，加速因子收敛到 4。

⁹⁴ 阿姆达尔定律 - https://en.wikipedia.org/wiki/Amdahl's_law。



阿姆达尔定律和通用可扩展性定律。

实际上,进一步向系统添加计算节点可能会产生倒退加速。我们将在下一节中看到它的例子。这种效应被 Neil Gunther 解释为通用可扩展性定律⁹⁵ (USL),它是阿姆达尔定律的扩展。USL 将计算节点(线程)之间的通信描述为限制性能的另一个因素。随着系统的扩展,开销开始抵消收益。超过一个临界点后,系统的能力开始下降(参见图 fig. ??)。USL 被广泛用于建模系统的容量和可扩展性。

USL 描述的减速由几个因素驱动。首先,随着计算节点数量的增加,它们开始争夺资源(争用)。这导致额外的时间花在同步这些访问上。另一个问题发生在许多工作者共享的资源上。我们需要在许多工作者之间保持共享资源的一致状态(一致性)。例如,当多个工作者频繁更改全局可见对象时,这些更改需要广播给所有使用该对象的节点。突然之间,由于维护一致性的额外需求,通常的操作开始花费更多时间。优化多线程应用程序不仅涉及本书到目前为止描述的所有技术,还涉及检测和缓解上述争用和一致性的影响。

13.2.1 线程数扩展案例研究

线程数扩展可能是你可以对多线程应用程序进行的最有价值的分析。它显示了应用程序可以利用现代多核系统的程度。正如你将看到的,你可以在这个过程中学到大量信息。不再做更多介绍,让我们开始吧。在这个案例研究中,我们将分析以下基准测试的线程数扩展,其中一些你应该已经从前几章中熟悉了:

1. Blender 3.4 - 一个开源 3D 创作和建模软件项目。此测试使用 BMW27 混合文件进行 Blender 的 Cycles 性能测试。URL: <https://download.blender.org/release>。命令行: `./blender -b bmw27_cpu.blend -noaudio --enable-autoexec -o output.test -x 1 -F JPEG -f 1 -t N`, 其中 N 是线程数。
2. Clang 17 构建 - 此测试使用 Clang 15 从源代码构建 Clang 17 编译器。URL: <https://www.llvm.org>。命令行: `ninja -jN clang`, 其中 N 是线程数。
3. Zstandard v1.5.5, 一种快速无损压缩算法。URL: <https://github.com/facebook/zstd>。用于压缩的数据集: <http://wanos.co/assets/silesia.tar>。命令行: `./zstd -TN -3 -f -- silesia.tar`, 其中 N 是压缩工作线程数。
4. CloverLeaf 2018 - 一个拉格朗日-欧拉流体动力学基准测试。使用所有硬件线程。此测试使用输入文件 `clover_bm.in` (问题 5)。URL: <http://uk-mac.github.io/CloverLeaf>。命令行: `export OMP_NUM_THREADS=N; ./clover_leaf`, 其中 N 是线程数。
5. CPython 3.12, Python 编程语言的参考实现。URL: <https://github.com/python/cpython>。我运行了一个用 Python 编写的简单多线程二分搜索脚本,它在 1,000,000 个元素的有序列表(干草堆)中搜索 10,000 个随机数字(针)。命令行: `./python3 binary_search.py N`, 其中 N 是线程数。针在各个线程之间平均分配。

基准测试在以下配置的机器上执行:

⁹⁵ USL 定律 - http://www.perfdynamics.com/Manifesto/USLscalability.html#tth_sEc1。

- 第 12 代 Alder Lake Intel® Core™ i7-1260P CPU @ 2.10GHz (4.70GHz Turbo), 4P+8E 核心, 18MB L3 缓存。
- 16 GB RAM, DDR4 @ 2400 MT/s。
- Clang 15 编译器, 使用以下选项: `-O3 -march=core-avx2`。
- 256GB NVMe PCIe M.2 SSD。
- 64 位 Ubuntu 22.04.1 LTS (Jammy Jellyfish, Linux 内核 6.5)。

这显然不是顶级硬件配置, 而是主流计算机, 不一定设计用于处理媒体、开发者或 HPC 工作负载。然而, 它是我们的案例研究的绝佳平台, 因为它演示了线程数扩展的各种效果。由于资源有限, 即使使用少量线程, 应用程序也开始遇到性能障碍。请记住, 在更好的硬件上, 扩展结果会有所不同。

我的处理器有四个 P 核心和八个 E 核心。P 核心启用 SMT, 这意味着此平台上的总线线程数为十六。默认情况下, Linux 调度器将首先尝试使用空闲的物理 P 核心。前四个线程将利用四个空闲 P 核心上的四个线程。当它们被完全利用时, 它将开始在 E 核心上调度线程。因此, 接下来的八个线程将被调度在八个 E 核心上。最后, 剩余的四个线程将被调度在 P 核心的 4 个兄弟 SMT 线程上。

我使用上述方案在固定线程亲和性的情况下运行基准测试, Zstd 和 CPython 除外。不运行亲和性可以更好地代表真实场景, 但线程亲和性使线程数扩展分析更清晰。由于性能数字非常相似, 在本案例研究中, 我展示了使用线程亲和性时的结果。

基准测试执行固定数量的工作。无论线程数如何, 退休指令数几乎相同。在所有基准测试中, 算法的最大部分使用分治范式实现, 其中工作被分成相等的部分, 每个部分可以独立处理。理论上, 这允许应用程序随着核心数量的增加而良好扩展。然而, 在实践中, 扩展通常远非最优。

图 fig. 13.1 显示了所选基准测试的线程数可扩展性。x 轴表示线程数, y 轴显示相对于单线程执行的加速比。加速比计算为单线程执行的执行时间除以多线程执行的执行时间。加速比越高, 应用程序随线程数的扩展越好。

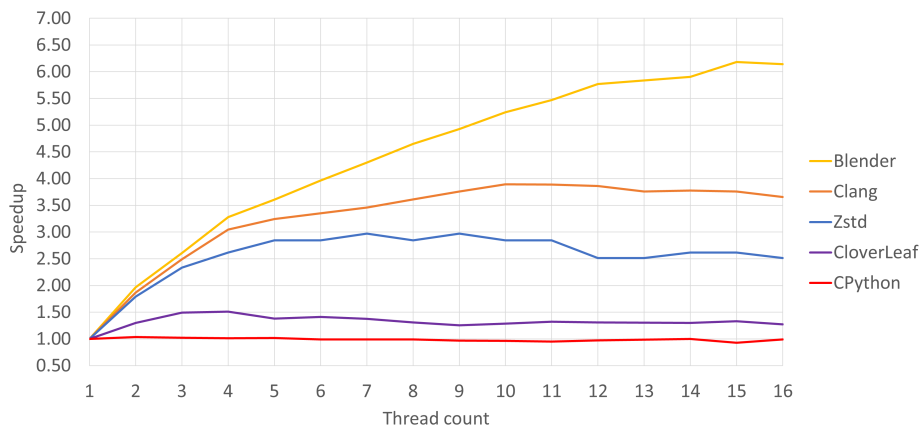


Figure 13.1: 五个选定基准测试的线程数可扩展性图表。

如你所见, 大多数都远低于线性扩展, 这相当令人失望。在本案例研究中扩展性最好的基准测试 Blender 在使用 16 个线程时仅实现 6 倍加速。例如, CPython 根本不受线程数扩展。Clang 和 Zstd 的性能在超过 10 个线程时会下降。为了理解为什么会发生这种情况, 让我们深入了解每个基准测试的细节。

Blender

Blender 是我们套件中唯一一个持续扩展到系统中所有 16 个线程的基准测试。原因是工作负载高度可并行化。渲染过程被分成小块，每个块可以独立渲染。然而，即使具有这种高水平的并行性，扩展也只有 6.1 倍加速 / 16 个线程 = 38%。这种次优扩展的原因是什么？

根据 Section 4.11，我们知道 Blender 的性能受浮点计算限制。它还具有相对较高比例的 SIMD 指令。P 核心在处理此类指令方面比 E 核心好得多。这就是为什么我们看到在 4 个线程之后，随着 E 核心开始被使用，加速曲线的斜率下降。性能扩展以相同的速度持续到 12 个线程，然后开始再次下降。这是使用 SMT 兄弟线程的效果。两个活动的兄弟 SMT 线程争夺有限数量的 FP/SIMD 执行单元。要度量 SMT 扩展，我们需要将两个 SMT 线程 (2T1C - 两个线程一个核心) 的性能除以单个 P 核心 (1T1C) 的性能。^[5] 对于 Blender，SMT 扩展约为 1.3 倍。

还有另一个也适用于 Blender 的扩展降级方面，我们将在讨论 Clang 的线程数扩展时讨论。

13.3 任务调度

随着混合处理器的出现，任务调度变得非常具有挑战性。例如，最新的 Intel Meteor Lake 芯片有三种类型的核心；都具有不同的性能特征。正如你将在本节中看到的，通过次优地调度任务，很容易降低多线程应用程序的性能。实现通用任务调度策略很棘手，因为它在很大程度上取决于运行任务的性质。以下是一些示例：

- 计算密集型线程工作负载（例如数据压缩）必须仅在 P 核心上服务。
- 后台任务（例如视频通话）可以在 E 核心上运行以节省功耗。
- 对于需要高响应性的突发应用程序（例如生产力软件），系统应仅使用 P 核心。
- 具有持续性能需求的多线程程序（例如视频渲染）应同时利用 P 和 E 核心。

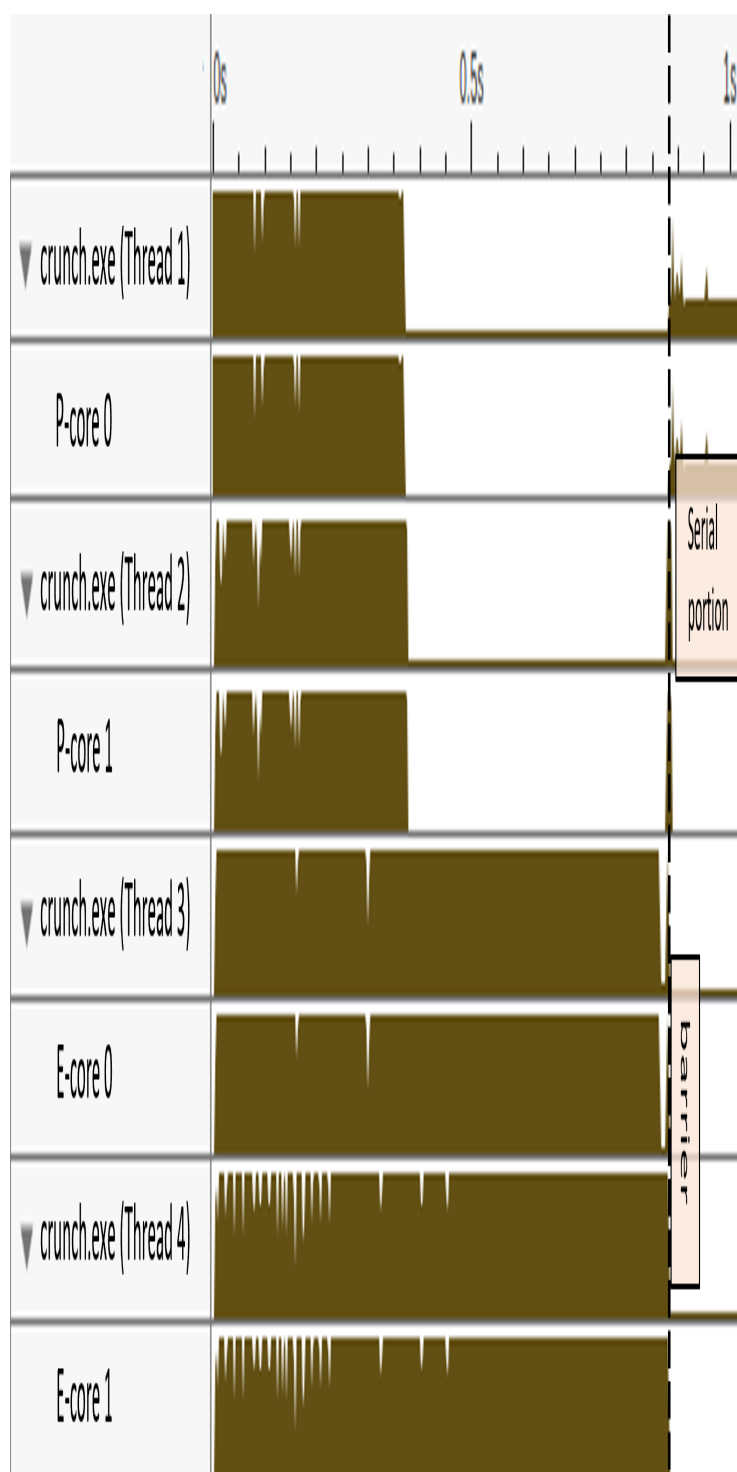
在大多数情况下，现代操作系统中的任务调度器会处理这些以及许多其他边缘情况。例如，Intel 的 Thread Director 帮助实时监控和分析性能数据，以将正确的应用程序线程无缝放置在正确的核心上。我在这里的总体建议是让操作系统完成其工作，不要过度限制它。操作系统知道如何调度任务以最小化争用、最大化缓存中的数据重用，并最终最大化性能。如果你正在开发旨在不同硬件配置上运行的跨平台软件，这将发挥重要作用。

下面我展示了非对称系统中任务调度的一些典型陷阱。我使用了与前一个案例研究中相同的系统：第 12 代 Alder Lake Intel® Core™ i7-1260P CPU，它有四个 P 核心和八个 E 核心。为简单起见，我只启用了两个 P 核心和两个 E 核心；其余核心被暂时禁用。我还禁用了两个活动 P 核心上的 SMT 兄弟线程。我编写了一个简单的 OpenMP 应用程序，其中每个工作线程在大型数组的每个 32 位整数元素上执行几个位操作。工作线程完成处理后，它遇到一个屏障并被迫等待其他线程完成其部分。之后，主数组清理并重复处理。该程序使用 GCC 13.2 和 `-O3 -march=core-avx2` 编译，这启用了向量化。

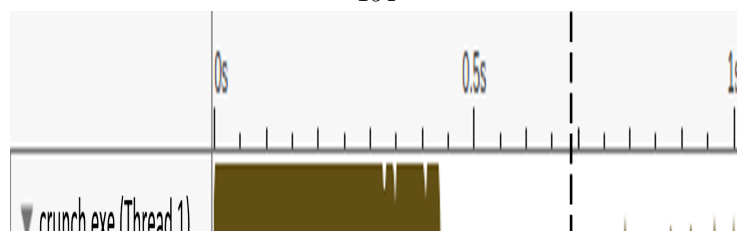
图 fig. 13.2 显示了三种策略，它们突出了我在实践中经常看到的常见问题。这些屏幕截图是使用 Intel VTune 捕获的。时间线上的条形图表示 CPU 时间，即线程正在运行的时间段。对于每个软件线程，有一个或两个对应的 CPU 核心。使用此视图，我们可以看到每个线程在任何给定时刻在哪个核心上运行。

我们的第一个示例使用静态分区，它将大型数组的处理分成四个相等的块（因为我启用了四个核心）。对于每个块，OpenMP 运行时生成一个新线程。此外，我使用了 `OMP_PROC_BIND=true`，它指示 OpenMP 运行时将生成的线程固定到 CPU 核心。图 fig. 13.2a 演示了效果：P 核心在处理 SIMD 指令方面比 E 核心好得多，它们完成工作快两倍（参见线程 1 和线程 2）。但是，线程亲和性不允许线程 3 和线程 4 迁移到等待屏障的 P 核心。这导致高延迟，其速度受 E 核心限制。

我的建议是避免将线程固定到核心。在工作不平衡的情况下，固定可能会限制工作窃取，将长执行尾部留给 E 核心。在 macOS 上，无法将线程固定到核心，因为操作系统不提供相应的 API。



(a) 静态分区, 将线程固定到核心: `#pragma omp for schedule(static)` 配合 `OMP_PROC_BIND=true`。



13.4 缓存一致性

多处理器系统包含确保每个核心在共享内存使用期间数据一致性的方法，每个核心包含自己的独立缓存实体。如果没有这样的协议，如果 CPU A 和 B 都将内存位置 L 读入它们各自的缓存，然后 CPU B 随后修改其缓存值 L，那么 CPU A 将具有相同内存位置 L 的不一致值。缓存一致性协议确保对缓存条目的任何更新都会在相同位置的任何其他缓存条目中被忠实地更新或无效化。

13.4.1 缓存一致性协议

最著名的缓存一致性协议之一是 MESI (Modified Exclusive Shared Invalid)，它用于支持现代 CPU 中使用的写回缓存。它的首字母缩写表示缓存行可以被标记的四种状态（参见图 fig. 13.3）：

- **已修改**：缓存行仅存在于当前缓存中，并且已从 RAM 中的值修改
- **独占**：缓存行仅存在于当前缓存中，并且与 RAM 中的值匹配
- **共享**：缓存行存在于此处和其他缓存行中，并且与 RAM 中的值匹配
- **无效**：缓存行未使用（即不包含任何 RAM 位置）

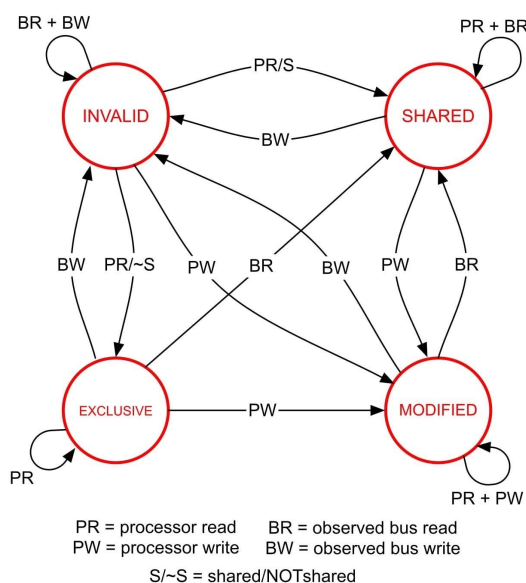


Figure 13.3: MESI 状态图。© 来源：华盛顿大学 *via courses.cs.washington.edu*.

从内存获取时，每个缓存行都有一个状态编码到其标签中。然后缓存行状态从一个状态转换到另一个状态。[²⁵] 实际上，CPU 供应商通常实现稍作改进的 MESI 变体。例如，Intel 使用 MESIF，[²⁶] 它添加了转发 (F) 状态，而 AMD 使用 MOESI，[²⁷] 它添加了拥有 (O) 状态。然而，这些协议仍然保持基本 MESI 协议的本质。

缺乏缓存一致性可能导致顺序不一致的程序。这个问题可以通过让 __snoop__ 缓存监视所有内存事务并相互协作以保持内存一致性来缓解。不幸的是，这需要付出代价，因为一个核心所做的修改会使另一个核心缓存中的相应缓存行无效。这会导致内存停顿并浪费系统带宽。与只能为应用程序性能设定上限的序列化和锁定问题相比，一致性问题可能导致 USL 在 Section 13.2 中归因的倒退效应。两种广泛知晓的一致性问题是真共享和假共享，我们接下来将探讨。

13.4.2 真共享

当两个不同的核心访问同一个变量时，就会发生真共享（参见 Listing ??）。

Listing: 真共享示例。

```
unsigned int sum; // 在所有线程之间共享
{ // 由线程 A 执行的代码      { // 由线程 B 执行的代码
    for (int i = 0; i < N; i++)    for (int i = 0; i < N;
        i++)                      i++)
        sum += a[i];              sum += b[i];
    }                              }
```

首先，除了真共享之外，我们还有一个更大的问题。我们实际上有一个数据竞争，这有时可能很难检测。注意，我们没有适当的同步机制，这可能导致不可预测或不正确的程序行为，因为对共享数据的操作可能相互干扰。幸运的是，有一些工具可以帮助识别此类问题。来自 Clang 的 Thread sanitizer^[30] 和 helgrind^[31] 就是此类工具。为了防止 Listing ?? 中的数据竞争，你应该将 sum 变量声明为 `std::atomic<unsigned int> sum`。

使用 C++ 原子类型可以帮助解决真共享发生时的数据竞争。但是，它有效地序列化了对原子变量的访问，这可能会损害性能。解决我们真共享问题的更好方法是使用线程本地存储（TLS）。TLS 是给定多线程进程中每个线程可以分配内存来存储线程特定数据的方法。通过这样做，线程修改它们的本地副本，而不是争夺全局可用的内存位置。Listing ?? 中的示例可以通过使用 TLS 类说明符声明 sum 来修复：`thread_local unsigned int sum`（自 C++11 起）。然后主线程应该合并每个工作线程的所有本地副本的结果。

13.4.3 假共享

如果不小心，你可能会尝试如 Listing ?? 所示解决真共享问题。此解决方案引入了另一个问题：假共享。当两个不同的核心修改恰好位于同一缓存行上的不同变量时，就会发生这种情况。在 Listing ?? 中显示的代码示例中，即使线程 A 和 B 更新结构体 S 的不同字段，它们也很可能位于同一缓存行上，这将触发假共享问题。图 fig. ?? 说明了这个问题。

13.5 高级分析工具

已经开发了许多工具来解决传统分析器无法提供足够可见性的特定用例。在本节中，我们将介绍 Coz 和 eBPF 工具。我们鼓励你对这些和其他工具进行进一步研究。

13.5.1 Coz

在 Section 13.2 中，我们定义了识别影响多线程程序整体性能的代码部分的挑战。由于各种原因，优化多线程程序的一部分可能并不总是产生可见的结果。传统的基于采样的分析器仅显示大部分时间花费的代码位置。然而，这不一定对应于程序员应集中优化工作的地方。

Coz⁹⁶ 是一个解决此问题的分析器。它使用一种称为因果分析的新技术，通过在应用程序运行时通过虚拟加速代码段来预测某些优化的总体效果来进行实验。它通过插入暂停来减慢所有其他同时运行的代码来实现这些“虚拟加速”。此外，Coz 量化了优化的潜在影响。[Curtsinger & Berger, 2018]

图 fig. 13.4 显示了将 Coz 分析器应用于 C-Ray⁹⁷ 基准测试的示例。根据图表，如果我们将 `c-ray-mt.c` 中第 540 行的性能提高 20%，Coz 预计 C-Ray 基准测试的整体应用程序性能将相应增加约 17%。一旦

⁹⁶ COZ 源代码 - <https://github.com/plasma-umass/coz>。

⁹⁷ C-Ray 基准测试 - <https://github.com/jtsiomb/c-ray>。

我们在该行上达到约 45% 的改进，根据 Coz 的估计，对应用程序的影响将趋于平稳。有关此示例的更多详细信息，请参见 Easyperf 博客上的文章⁹⁸。

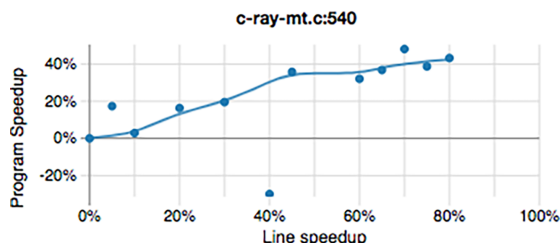


Figure 13.4: C-Ray 基准测试的 Coz 配置文件。

13.5.2 eBPF 和 GAPP

Linux 支持各种线程同步原语：互斥锁、信号量、条件变量等。内核通过 `futex` 系统调用支持这些线程原语。因此，通过在内核中跟踪 `futex` 系统调用的执行，同时从相关线程收集有用的元数据，可以更容易地识别争用瓶颈。Linux 提供了使这成为可能的内核跟踪和分析工具，其中最强大的是 Extended Berkeley Packet Filter⁹⁹ (eBPF)。

eBPF 基于在内核中运行的沙箱虚拟机，允许在内核内安全高效地执行用户定义的程序。用户定义的程序可以用 C 编写，并由 BCC 编译器¹⁰⁰ 编译为 BPF 字节码，准备加载到内核虚拟机中。这些 BPF 程序可以被编写为在某些内核事件执行时启动，并通过各种方式将原始或处理后的数据传回用户空间。

开源社区提供了许多通用的 eBPF 程序。其中一个工具是通用自动并行分析器 (GAPP)，¹⁰¹ 它有助于跟踪多线程争用问题。GAPP 使用 eBPF 通过识别序列化瓶颈的关键性来跟踪多线程应用程序的争用开销，并收集被阻塞线程和导致阻塞的线程的堆栈跟踪。GAPP 最好的地方是它不需要代码更改、昂贵的检测或重新编译。GAPP 分析器的创建者能够确认已知瓶颈，并在 Parsec 3.0 Benchmark Suite¹⁰² 和一些大型开源项目中发现新的、以前未报告的瓶颈。[Nair & Field, 2020]

作为结束语，我想再次强调优化多线程应用程序的重要性。从我们在本书中讨论的所有内容来看，本章的建议可能会带来最显著的性能改进。在多线程应用程序中，魔鬼在于细节。微妙的同步问题或数据共享中的小低效可能导致显著的性能降级。展望未来，多核处理器和并行工作负载的趋势只会加速。多线程优化的复杂性会增长，但掌握它的机会也会增长。

问题和练习

1. 解决 `perf-ninja::false_sharing` 实验室作业。
2. 多线程与多进程应用程序有什么好处？
3. 阿姆达尔定律和通用可扩展性定律的本质是什么？
4. 运行你日常使用的应用程序。它是多线程的吗？如果不是，选择一个流行的多线程基准测试。运行线程数扩展研究并分析结果。生成一个可视化工作线程在时间线上的图表。你能识别任何调度问题吗？识别热锁和导致这些锁的代码路径。你能改进锁定吗？

⁹⁸ 博客文章“COZ vs 采样分析器” - <https://easyperf.net/blog/2020/02/26/coz-vs-sampling-profilers>。

⁹⁹ eBPF 文档 - <https://prototype-kernel.readthedocs.io/en/latest/bpf/>

¹⁰⁰ BCC 编译器 - <https://github.com/iovisor/bcc>

¹⁰¹ GAPP - <https://github.com/RN-dev-repo/GAPP/>

¹⁰² Parsec 3.0 基准测试套件 - <https://parsec.cs.princeton.edu/index.htm>

章节总结

- 不利用现代多核 CPU 的应用程序正在落后于其竞争对手。为软件做好准备以随着 CPU 核心数量的增长而良好扩展，对于应用程序未来的成功非常重要。
- 在处理单线程应用程序时，优化程序的一部分通常会产生积极的性能结果。但是，对于多线程应用程序，情况并非总是如此。这种效果被广泛称为阿姆达尔定律，它认为并行程序的加速受其串行部分的限制。
- 线程通信可能导致倒退加速，正如通用可扩展性定律所解释的那样。这为调优多线程程序带来了额外的挑战。
- 正如我们在线程数案例研究中所看到的，频率节流、内存带宽饱和和其他问题可能导致性能扩展不佳。
- 混合处理器上的任务调度具有挑战性。注意次优的任务调度，并在不需要时不要限制操作系统调度器。
- 优化多线程应用程序的性能还涉及检测和缓解缓存一致性的影响，如真共享和假共享。
- 在过去的几年中，出现了一些新工具，涵盖了传统分析器无法覆盖的多线程应用程序性能分析空白。我们介绍了 Coz 和 GAPP，它们具有独特的功能集。

后记

感谢你阅读整本书。我希望你喜欢它并觉得它有用。如果这本书能帮助你解决现实世界的问题，我会更加高兴。在这种情况下，我会认为这是成功的证明，我的努力没有白费。在你继续你的事业之前，让我简要强调本书的要点并给你最终建议：

- 现代软件极其低效。有重大的优化机会可以减少碳排放并提供更好的用户体验。人们讨厌使用慢速软件，尤其是当他们的生产力因此下降时。不是所有快速的软件都是世界级的，但所有世界级的软件都是快速的。性能是 __ 杀手级 __ 特性。
- 单线程 CPU 性能的增长速度不如几十年前那样快。当硬件每一代不再提供显著的性能提升时，开发者应该开始优化其软件的代码。
- 多年来，性能工程一直是一个小众领域。但现在随着软件供应商意识到其优化不佳的软件对利润的影响，它已成为主流。性能调优比过去 40 年更重要。它将是未来性能提升的关键驱动力之一。
- 低级性能调优的重要性不应被低估，即使只是 1% 的改进。这些小改进的累积效应才是造成差异的原因。
- Donald Knuth 有一句名言：“过早优化是万恶之源”。[Knuth, 1974] 相反的情况也经常是真的。延迟的性能工程工作可能为时已晚，造成与过早优化同样多的恶果。在设计未来产品时不要忽视性能方面。通过将自动化性能基准测试集成到你的 CI/CD 管道中来挽救你的项目。尽早测量，经常测量。
- 要达到峰值性能，需要了解 CPU 微架构。然而，你的心智模型永远不可能像 CPU 的实际微架构设计那样准确。因此，当在代码中进行特定更改时，不要仅仅依赖你的直觉。预测特定代码片段的性能几乎是不可能的。总是测量！
- 度量性能时，了解你观察到的性能结果的底层技术原因。总是深入一层度量，并收集尽可能多的指标来支持你的结论。
- 性能工程很困难，因为没有预定的步骤可以遵循，没有算法。工程师需要从不同角度解决问题。了解可用的性能分析方法和工具（硬件和软件）。我强烈建议采用自顶向下微架构分析（TMA）方法。它将帮助你在工作引向正确的方向。
- 当你识别出应用程序的性能限制因素时，你已经完成了一半以上。根据我的经验，修复通常比找到问题的根本原因更容易。在第 2 部分中，我们涵盖了一些针对每种 CPU 性能瓶颈的基本优化：如何优化内存访问和计算、如何消除分支预测错误、如何改善机器代码布局以及其他一些。使用第 2 部分的章节作为参考，了解当你的应用程序遇到这些问题时有哪些选项可用。
- 来自不同供应商的处理器并不相同。它们在支持的指令集架构（ISA）和微架构实现方面存在差异。在给定平台上达到峰值性能需要利用最新的 ISA 扩展、避免常见的微架构特定问题，并根据特定 CPU 微架构的优势调整代码。
- 多线程程序为性能调优增加了一个复杂性维度。它们引入了新的瓶颈类型，并需要额外的工具和方法来分析和优化。检查应用程序如何随线程数扩展是识别多线程程序中瓶颈的有效方法。

我希望你现在对低级性能优化有了更好的理解。当然，这本书没有涵盖你在日常工作中可能遇到的每一个可能场景。我的目标是给你一个起点，并向你展示在现代 CPU 上处理性能分析和调优的潜在选项和策略。我希望你体验发现应用程序中性能瓶颈的乐趣和修复它们的满足感。

快乐的性能调优！

我将在我的博客上发布本书的勘误表和其他信息，网址为：<https://easypref.net/blog/2024/11/11/Book-Updates-Errata>。

如果你还没有解决 perf-ninja 练习，我鼓励你花时间去完成。它们将帮助你巩固知识，并为你应对现实世界的性能工程挑战做好准备。

P.S. 如果你喜欢阅读这本书，请确保将其传给你的朋友和同事。我将感激你在社交媒体平台上认可这本书来

帮助传播这本书。

致谢

我怀着深切的感激之情写下这一节，感谢下面提到的所有人。我很谦卑地向你们传达这些人的知识。

Mark E. Dawson, Jr. 撰写了 Section 12.3 “低延迟调优技术”和 Section 7.9 “持续分析”。他还为本书的第一版做出了很大贡献。Mark 是高频交易行业的公认专家，目前在 WH Trading 工作。Mark 还有一个博客 (<https://www.jabperf.com>)，他在那里写关于低延迟和其他性能优化的内容。



来自萨拉戈萨大学的 Agustín Navarro-Torres、Jesús Alastruey-Benedé、Pablo Ibáñez-Marín、Víctor Viñals-Yúfera 撰写了 Section 12.5 “案例研究的敏感性”。他们是计算机科学领域的研究人员，发表了多篇与性能工程相关的论文。



Universidad
Zaragoza

Jan Wassenberg 撰写了 Section ?? “内置函数的包装器库”，并对 Section 9.4 “向量化”和 Section 3.4 “SIMD 多处理器”提出了许多改进建议。Jan 是 Google DeepMind 的软件工程师，负责领导 Gemma.cpp 的开发。^[1]他还发表了多篇研究论文。他的个人网页是 <https://wassenberg.dreamhosters.com>。



Swarup Sahoo 帮助我撰写了关于 Section 6 中 AMD PMU 功能的内容，并撰写了 Section 7.1 关于 AMD uProf 的内容。Swarup 是 AMD 的高级开发人员，负责 HPC (OpenMP、MPI) 应用程序的 uProf 性能分析工具。Swarup 的 LinkedIn 页面可以在 <https://www.linkedin.com/in/swarupsahoo> 找到。



支持本书

如果你喜欢这本书并愿意支持它，有以下几种方式。

Paypal (one-time)



Buy Me a Coffee (one-time)



GitHub Sponsors
(one-time or monthly)



Patreon (monthly)



术语表

\begin{multicols}{2}

AOS 结构数组

BB 基本块

BIOS 基本输入输出系统

CI/CD 持续集成/持续开发

CFG 控制流图

CP 持续分析

CPI 每指令周期数

CPU 中央处理器

DDD 数据驱动开发

DSB 解码流缓冲区

DRAM 动态随机存取存储器

DTLB 数据转换后备缓冲区

EBS 基于事件的采样

EHP 显式大页

FLOPS 每秒浮点运算次数

FMA 融合乘加

FPGA 现场可编程门阵列

主要 CPU 微架构列表

在下面的表格中，我们展示了 Intel、AMD 和基于 ARM 的供应商的最新 ISA 和微架构。当然，并非所有设计都在此列出。我们只包含本书中引用的那些，或者它们代表了平台演进中的重大转变。

名称	三字母缩写发缩写	布年份支持的客	ISA 户端/服务器芯片
Nehalem	NHM	2008	SSE4.2
Sandy Bridge	SNB	2011	AVX
Haswell	HSW	2013	AVX2
Skylake	SKL	2015	AVX2 / AVX512
Sunny Cove	SNC	2019	AVX512
Golden Cove	GLC	2021	AVX2 / AVX512
Redwood Cove	RWC	2023	AVX2 / AVX512
Lion Cove	LNC	2024	AVX2

表：最近的 Intel Core 微架构列表。{#tbl:IntelUarchs}

名称发布年份支持的 ISA

Steamroller 2014 AVX
Excavator 2015 AVX2
Zen 2017 AVX2

附录 A. 减少测量噪声

以下是一些可能导致性能测量中非确定性增加的功能示例，以及一些减少噪声的技术。我在 Section 2.1 中介绍了该主题。

本节主要特定于 Linux 操作系统。鼓励读者在网上搜索有关如何配置其他操作系统的说明。

动态频率缩放

动态频率缩放^[11] (DFS) 是一种通过在系统运行要求高的任务时自动提高 CPU 运行频率来提高系统性能的技术。作为 DFS 实现的一个示例，Intel CPU 具有称为 Turbo Boost 的功能，AMD CPU 使用 Turbo Core 功能。

以下是 Turbo Boost 对在 Intel® Core™ i5-8259U 上运行的单线程工作负载的影响示例：

```
# TurboBoost 已启用
$ cat /sys/devices/system/cpu/intel_pstate/no_turbo
0
$ perf stat -e task-clock,cycles -- ./a.exe
    11984.691958 task-clock (msec) #    1.000 CPUs utilized
    32,427,294,227 cycles           #    2.706 GHz
    11.989164338 seconds time elapsed

# TurboBoost 已禁用
$ echo 1 | sudo tee
/sys/devices/system/cpu/intel_pstate/no_turbo
1
$ perf stat -e task-clock,cycles -- ./a.exe
    13055.200832 task-clock (msec) #    0.993 CPUs utilized
    29,946,969,255 cycles           #    2.294 GHz
    13.142983989 seconds time elapsed
```

启用 Turbo Boost 时平均频率更高 (2.7 GHz 对 2.3 GHz)。

DFS 可以在 BIOS 中永久禁用。要在 Linux 系统上以编程方式禁用 DFS 功能，你需要 root 访问权限。以下是如何实现这一点：

```
# Intel
echo 1 > /sys/devices/system/cpu/intel_pstate/no_turbo
```

附录 B. 启用大页

Windows

要在 Windows 上使用大页, 你需要启用 `SeLockMemoryPrivilege` 安全策略。这可以通过 Windows API 以编程方式完成, 或者通过安全策略 GUI 完成。

1. 单击开始 → 搜索“secpol.msc”, 然后启动它。
2. 在左侧选择“本地策略” → “用户权限分配”, 然后双击“锁定内存中的页”。

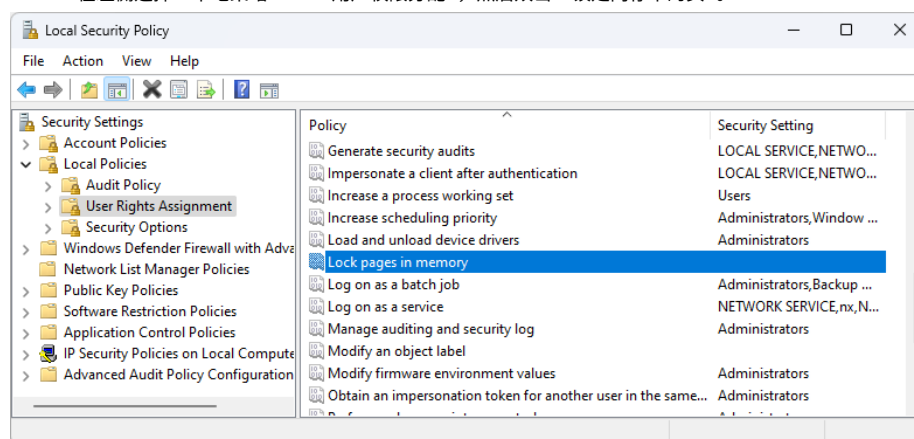


Figure 13.5: Windows 安全: 锁定内存中的页

3. 添加你的用户并重新启动机器。
4. 使用 RAMMap 工具检查运行时是否使用了大页。

在代码中使用大页:

```
void* p = VirtualAlloc(NULL, size, MEM_RESERVE |
                        MEM_COMMIT |
                        MEM_LARGE_PAGES,
                        PAGE_READWRITE);
...
VirtualFree(ptr, 0, MEM_RELEASE);
```

Linux

在 Linux 操作系统上, 有两种方式在应用程序中使用大页: 显式大页和透明大页。

显式大页

显式大页可以在系统启动时或在应用程序启动之前保留。要永久更改以强制 Linux 内核在启动时分配 128 个大页, 请运行以下命令:

```
$ echo "vm.nr_hugepages = 128" >> /etc/sysctl.conf
```

附录 C. Intel 处理器跟踪

Intel 处理器跟踪 (PT) 是一项 CPU 功能，它通过以高度压缩的二进制格式编码数据包来记录程序执行，可用于重建每条指令带有时间戳的执行流程。PT 具有广泛的覆盖范围和相对较小的开销，^[1] 通常低于 5%。其主要用途是事后分析和根本原因性能故障。

工作流程

与采样技术类似，PT 不需要对源代码进行任何修改。收集跟踪所需的所有操作只是在支持 PT 的工具下运行程序。一旦 PT 启用并启动基准测试，分析工具就开始将 PT 数据包写入 DRAM。

与 LBR (最后分支记录) 类似，Intel PT 通过记录分支来工作。在运行时，每当 CPU 遇到任何分支指令时，PT 将记录该分支的结果。对于简单的条件跳转指令，CPU 将使用 1 位记录它是被采取 (T) 还是未采取 (NT)。对于间接调用，PT 将记录目标地址。请注意，无条件分支被忽略，因为我们静态地知道它们的目标。

图 fig. 13.6 显示了一个小指令序列的编码示例。像 PUSH、MOV、ADD 和 CMP 这样的指令被忽略，因为它们不改变控制流。但是，JE 指令可能跳转到 .label，因此需要记录其结果。后来有一个间接调用，其目标地址被保存。

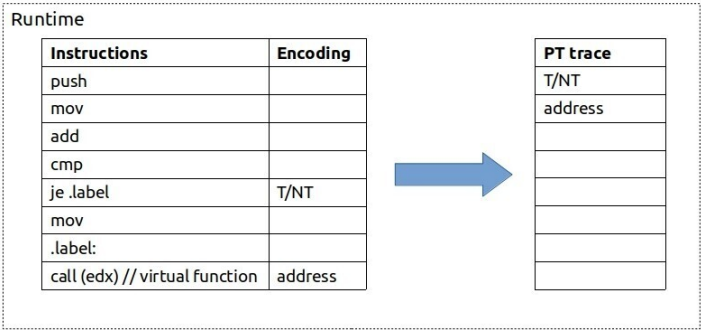


Figure 13.6: Intel 处理器跟踪编码

在分析时，我们将应用程序二进制文件和收集的 PT 跟踪结合在一起。软件解码器需要应用程序二进制文件来重建程序的执行流程。它从入口点开始，然后使用收集的跟踪作为查找参考来确定控制流。

图 fig. 13.7 显示了解码 Intel 处理器跟踪的示例。假设 PUSH 指令是应用程序二进制文件的入口点。然后，PUSH、MOV、ADD 和 CMP 被原样重建，而不查看编码的跟踪。稍后，软件解码器遇到 JE 指令，这是一个条件分支，我们需要查找其结果。根据图 fig. 13.7 中的跟踪，JE 被采取 (T)，因此我们跳过下一个 MOV 指令并转到 CALL 指令。同样，CALL (edx) 是改变控制流的指令，因此我们在编码的跟踪中查找目标地址，即 0x407e1d8。

以黄色突出显示的指令是在我们的程序运行时执行的。请注意，这是程序执行的精确重建；我们没有跳过任何指令。稍后，我们可以使用调试信息将汇编指令映射回源代码，并拥有逐行执行的源代码日志。

时间数据包

使用 Intel PT，不仅可以跟踪执行流，还可以跟踪时间信息。除了保存跳转目标之外，PT 还可以发出时间数据包。图 fig. 13.8 提供了如何使用时间数据包为指令恢复时间戳的可视化。与前面的示例一样，我们首先

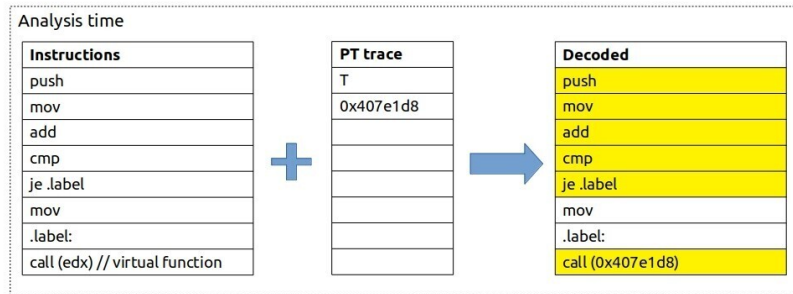


Figure 13.7: Intel 处理器跟踪解码

看到 JNZ 未被采取 (NT)，因此我们更新它以及上面所有时间戳为 0ns 的指令。然后我们看到 2ns 的时间更新和 JE 被采取，因此我们更新它以及 JE 上方 (和 JNZ 下方) 的所有指令，时间戳为 2ns。之后，有一个间接调用 (CALL (edx))，但没有附加时间数据包，因此我们不更新时间戳。然后我们看到 100ns 过去，JB 未被采取，因此我们更新其上方的所有指令，时间戳为 102ns。

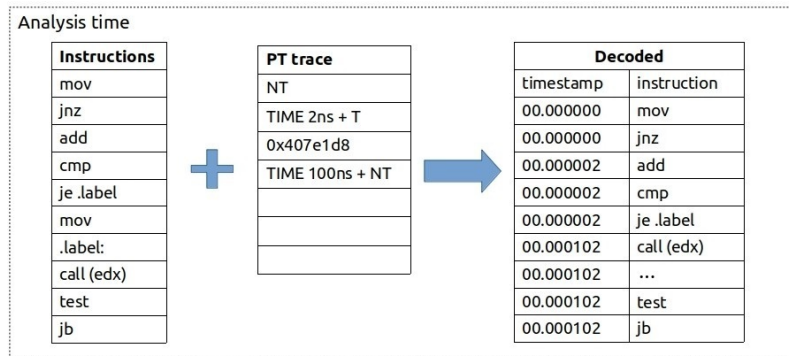


Figure 13.8: Intel 处理器跟踪计时

在图 fig. 13.8 中显示的示例中，指令数据 (控制流) 完全准确，但时间信息不够准确。显然，CALL (edx)、TEST 和 JB 指令不是同时发生的，但我们没有它们更准确的时间信息。有了时间戳，我们可以将程序的时间间隔与系统中的另一个事件对齐，并且很容易与墙钟时间进行比较。某些实现中的跟踪计时可以通过周期精确模式进一步改进，其中硬件在正常数据包之间保持周期计数记录 (更多详细信息请参见 [Intel, 2023, Volume 3C, Chapter 36])。

13.6 收集和解码跟踪

Intel PT 跟踪可以使用 Linux perf 工具轻松收集：

```
$ perf record -e intel_pt/cyc=1/u -- ./a.out
```

附录 D. Windows 事件跟踪分析

我们在 Section 7.6 中介绍了 ETW。在本节中，我们将继续之前的内容，探索用于记录和分析 ETW 的工具。为了展示这些工具的外观，我们介绍了一个调试程序启动缓慢的案例研究。

记录 ETW 跟踪的工具

以下是你可用于捕获 ETW 跟踪的工具列表：

- **WPR.exe**: 命令行记录工具，Windows 10 和 Windows Performance Toolkit 的一部分。
- **WPRUI.exe**: 用于记录 ETW 数据的简单 UI，Windows Performance Toolkit 的一部分。
- **xperf**: wpr 的命令行前身，Windows Performance Toolkit 的一部分。
- **PerfView**: [^3] 以 .NET 应用程序为主要焦点的图形记录和分析工具。这是 Microsoft 开发的开源应用程序。
- **Performance HUD**: [^7] 一个鲜为人知但非常强大的 GUI 工具，用于跟踪 UI 延迟以及通过所有未平衡资源分配的实时 ETW 记录的用户/句柄泄漏，以及泄漏/阻塞调用栈跟踪的实时显示。
- **ETWController**: [^4] 一个记录工具，能够记录键盘输入和屏幕截图以及 ETW 数据。这个由 Alois Kraus 开发的开源应用程序还支持在两台机器上同时进行分布式分析。
- **UIForETW**: [^6] 这个由 Bruce Dawson 开发的开源应用程序是 xperf 的包装器，具有用于记录 Google Chrome 问题的特殊选项。它还可以记录键盘和鼠标输入。

查看和分析 ETW 跟踪的工具

- **Windows Performance Analyzer (WPA)**: 查看 ETW 数据的最强大的 UI。WPA 可以可视化和叠加磁盘、CPU、GPU、网络、内存、进程等多种数据源，以全面了解系统的行为和正在执行的操作。虽然 UI 非常强大，但对于初学者来说也可能相当复杂。WPA 支持插件来处理来自其他来源的数据，而不仅仅是 ETW 跟踪。可以导入由 Linux perf、LTNG、Perfetto 等工具生成的 Linux/Android[^8] 分析数据，以及多种日志文件格式：dmesg、Cloud-Init、WaLinuxAgent 和 AndroidLogcat。
- **ETWAnalyzer**: [^5] 读取 ETW 数据并生成聚合摘要 JSON 文件，可以在命令行中查询、过滤和排序，或导出到 CSV 文件。
- **PerfView**: 主要用于排查 .NET 应用程序。为垃圾回收和 JIT 编译触发的 ETW 事件被解析并易于作为报告或 CSV 数据访问。

案例研究 - 程序启动缓慢

现在我们将看一个使用 ETWController 捕获 ETW 跟踪和 WPA 进行可视化的示例。

问题陈述: 在 Windows 资源管理器中双击下载的可执行文件时，启动有明显的延迟。似乎有什么东西延迟了进程启动。可能的原因是什么？磁盘慢？

设置

- 下载 ETWController 以记录 ETW 数据和屏幕截图。
- 下载最新的 Windows 11 Performance Toolkit[^1]，以便能够使用 WPA 查看数据。确保较新的 Win 11 WPR.exe 在你的路径中排在第一位，方法是在系统环境对话框中将 WPT 的安装文件夹移到 C:\Windows\system32 之前。它应该是这样的：

```
C> where wpr
C:\Program Files (x86)\Windows Kits\10\Windows Performance
  Toolkit\WPR.exe
C:\Windows\System32\WPR.exe
```

References

- [Akinshin, 2019] Akinshin, A. (2019). *Pro .NET Benchmarking* (1 ed.). Apress. <https://doi.org/10.1007/978-1-4842-4941-3>
- [Allen & Kennedy, 2001] Allen, R. & Kennedy, K. (2001). *Optimizing Compilers for Modern Architectures: A Dependence-based Approach* (1 ed.). Morgan-Kaufmann.
- [AMD, 2023] AMD (2023). *AMD64 Architecture Programmer's Manual*. Advanced Micro Devices, Inc. <https://www.amd.com/content/dam/amd/en/documents/processor-tech-docs/programmer-references/24593.pdf>
- [AMD, 2024] AMD (2024). *AMD uProf User Guide, Revision 4.2*. Advanced Micro Devices, Inc. <https://www.amd.com/content/dam/amd/en/documents/developer/version-4-2-documents/uprof/uprof-user-guide-v4.2.pdf>
- [Apple, 2024] Apple (2024). *Apple Silicon CPU Optimization Guide: 3.0*. Apple® Inc. <https://developer.apple.com/documentation/apple-silicon/cpu-optimization-guide>
- [Arm, 2022] Arm (2022). *Arm Neoverse™ V1 PMU Guide, Revision: r1p2*. Arm Limited. <https://developer.arm.com/documentation/PJDOC-1063724031-605393/2-0/?lang=en>
- [Arm, 2023] Arm (2023). *Arm Neoverse V1 Core: Performance Analysis Methodology*. Arm Limited. <https://armkeil.blob.core.windows.net/developer/Files/pdf/white-paper/neoverse-v1-core-performance-analysis.pdf>
- [Blem et al., 2013] Blem, E., Menon, J., & Sankaralingam, K. (2013). Power struggles: Revisiting the risc vs. cisc debate on contemporary arm and x86 architectures. *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, 1–12. <https://doi.org/10.1109/HPCA.2013.6522302>
- [Chen et al., 2016] Chen, D., Li, D. X., & Moseley, T. (2016). Autofdo: Automatic feedback-directed optimization for warehouse-scale applications. *CGO 2016 Proceedings of the 2016 International Symposium on Code Generation and Optimization*, 12–23. <https://ieeexplore.ieee.org/document/7559528>
- [ChipsAndCheese, 2024] ChipsAndCheese (2024). *Why x86 doesn't need to die*. <https://chipsandcheese.com/2024/03/27/why-x86-doesnt-need-to-die/>
- [Cooper & Torczon, 2012] Cooper, K. & Torczon, L. (2012). *Engineering a Compiler*. Morgan Kaufmann. Morgan Kaufmann. <https://books.google.co.in/books?id=CGTOIAEACAAJ>
- [Curtsinger & Berger, 2013] Curtsinger, C. & Berger, E. D. (2013). Stabilizer: Statistically sound performance evaluation. *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '13*, 219–228. <https://doi.org/10.1145/2451116.2451141>
- [Curtsinger & Berger, 2018] Curtsinger, C. & Berger, E. D. (2018). Coz: Finding code that counts with causal profiling. *Commun. ACM*, 61(6), 91–99. <https://doi.org/10.1145/3205911>
- [domo.com, 2017] domo.com (2017). *Data Never Sleeps 5.0*. Domo, Inc. https://www.domo.com/learn/data-never-sleeps-5?aid=ogsm072517_1&sf100871281=1

-
- [Du et al., 2010] Du, J., Sehwat, N., & Zwaenepoel, W. (2010). Performance profiling in a virtualized environment. *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing*, HotCloud’10, 2. https://www.usenix.org/legacy/event/hotcloud10/tech/full_papers/Du.pdf
- [Fog, 2023] Fog, A. (2023). *Optimizing software in c++: An optimization guide for windows, linux and mac platforms*. https://www.agner.org/optimize/optimizing_cpp.pdf
- [Geelnard, 2022] Geelnard, M. (2022). *Debunking cisc vs risc code density*. <https://www.bitsnbites.eu/cisc-vs-risc-code-density/>
- [Hennessy, 2018] Hennessy, J. L. (2018). *The future of computing*. <https://youtu.be/Azt8Nc-mtKM?t=329>
- [Hennessy & Patterson, 2017] Hennessy, J. L. & Patterson, D. A. (2017). *Computer Architecture, Sixth Edition: A Quantitative Approach* (6th ed.). Morgan Kaufmann Publishers Inc.
- [Herdrich et al., 2016] Herdrich, A. et al. (2016). Cache QoS: From concept to reality in the Intel® Xeon® processor E5-2600 v3 product family. *HPCA*, 657–668. <https://ieeexplore.ieee.org/document/7446102>
- [Intel, 2023] Intel (2023). *Intel® 64 and IA-32 Architectures Optimization Reference Manual*. Intel® Corporation. <https://software.intel.com/content/www/us/en/develop/download/intel-64-and-ia-32-architectures-optimization-reference-manual.html>
- [Jin et al., 2012] Jin, G., Song, L., Shi, X., Scherpelz, J., & Lu, S. (2012). Understanding and detecting real-world performance bugs. *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’12, 77–88. <https://doi.org/10.1145/2254064.2254075>
- [Kanev et al., 2015] Kanev, S., Darago, J. P., Hazelwood, K., Ranganathan, P., Moseley, T., Wei, G.-Y., & Brooks, D. (2015). Profiling a warehouse-scale computer. *SIGARCH Comput. Archit. News*, 43(3S), 158–169. <https://doi.org/10.1145/2872887.2750392>
- [Khuong & Morin, 2015] Khuong, P.-V. & Morin, P. (2015). *Array layouts for comparison-based searching*. <https://arxiv.org/ftp/arxiv/papers/1509/1509.05053.pdf>
- [Knuth, 1974] Knuth, D. E. (1974). Structured programming with go to statements. *ACM Comput. Surv.*, 6, 261–301. <https://api.semanticscholar.org/CorpusID:207630080>
- [Leiserson et al., 2020] Leiserson, C. E., Thompson, N. C., Emer, J. S., Kuszmaul, B. C., Lamson, B. W., Sanchez, D., & Schardl, T. B. (2020). There’s plenty of room at the top: What will drive computer performance after moore’s law? *Science*, 368(6495). <https://doi.org/10.1126/science.aam9744>
- [Luo et al., 2015] Luo, T., Wang, X., Hu, J., Luo, Y., & Wang, Z. (2015). Improving tlb performance by increasing hugepage ratio. *2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*, 1139–1142. <https://doi.org/10.1109/CCGrid.2015.36>
- [Matteson & James, 2014] Matteson, D. S. & James, N. A. (2014). A nonparametric approach for multiple change point analysis of multivariate data. *Journal of the American Statistical Association*, 109(505), 334–345. <https://doi.org/10.1080/01621459.2013.849605>

- [Mittal, 2016] Mittal, S. (2016). A survey of techniques for cache locking. *ACM Transactions on Design Automation of Electronic Systems*, 21. <https://doi.org/10.1145/2858792>
- [Mytkowicz et al., 2009] Mytkowicz, T., Diwan, A., Hauswirth, M., & Sweeney, P. F. (2009). Producing wrong data without doing anything obviously wrong! *SIGPLAN Not.*, 44(3), 265–276. <https://doi.org/10.1145/1508284.1508275>
- [Nair & Field, 2020] Nair, R. & Field, T. (2020). Gapp: A fast profiler for detecting serialization bottlenecks in parallel linux applications. *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. <https://doi.org/10.1145/3358960.3379136>
- [Nowak & Bitzes, 2014] Nowak, A. & Bitzes, G. (2014). The overhead of profiling using pmu hardware counters. <https://zenodo.org/record/10800/files/TheOverheadOfProfilingUsingPMUhardwareCounters.pdf>
- [Ottoni & Maher, 2017] Ottoni, G. & Maher, B. (2017). Optimizing function placement for large-scale data-center applications. *Proceedings of the 2017 International Symposium on Code Generation and Optimization, CGO '17*, 233–244. <https://ieeexplore.ieee.org/document/7863743>
- [Ousterhout, 2018] Ousterhout, J. (2018). Always measure one level deeper. *Commun. ACM*, 61(7), 74–83. <https://doi.org/10.1145/3213770>
- [Panchenko et al., 2018] Panchenko, M., Auler, R., Nell, B., & Ottoni, G. (2018). BOLT: A practical binary optimizer for data centers and beyond. *CoRR*, abs/1807.06735. <http://arxiv.org/abs/1807.06735>
- [Paoloni, 2010] Paoloni, G. (2010). *How to Benchmark Code Execution Times on Intel® IA-32 and IA-64 Instruction Set Architectures*. Intel® Corporation. <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/ia-32-ia-64-benchmark-code-execution-paper.pdf>
- [Ren et al., 2010] Ren, G., Tune, E., Moseley, T., Shi, Y., Rus, S., & Hundt, R. (2010). Google-wide profiling: A continuous profiling infrastructure for data centers. *IEEE Micro*, 65–79. <http://www.computer.org/portal/web/cSDL/doi/10.1109/MM.2010.68>
- [Sasongko et al., 2023] Sasongko, M. A., Chabbi, M., Kelly, P. H. J., & Unat, D. (2023). Precise event sampling on amd versus intel: Quantitative and qualitative comparison. *IEEE Transactions on Parallel and Distributed Systems*, 34(5), 1594–1608. <https://doi.org/10.1109/TPDS.2023.3257105>
- [Shen & Lipasti, 2013] Shen, J. & Lipasti, M. (2013). *Modern Processor Design: Fundamentals of Superscalar Processors*. Waveland Press, Inc.
- [Sites, 2022] Sites, R. (2022). *Understanding Software Dynamics*. Addison-Wesley professional computing series. Addison-Wesley. <https://books.google.com/books?id=TklozgEACAAJ>
- [statista.com, 2024] statista.com (2024). *Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2020, with forecasts from 2021 to 2025*. Statista, Inc. <https://www.statista.com/statistics/871513/worldwide-data-created/>
- [Suresh Srinivas, 2019] Suresh Srinivas, e. a. (2019). *Runtime performance optimization blueprint: Intel® architecture optimization with large code pages*. <https://www.intel.com/content/www/us/en/develop/articles/runtime-performance-optimization-blueprint-intel-architecture-optimization-with-large-code.html>

- [Wang et al., 2017] Wang, X. et al. (2017). SWAP: effective fine-grain management of shared last-level caches with minimum hardware support. *HPCA*, 121–132. <https://ieeexplore.ieee.org/document/7920819>
- [Weaver & McIntosh-Smith, 2023] Weaver, D. & McIntosh-Smith, S. (2023). An empirical comparison of the risc-v and aarch64 instruction sets. *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, SC-W '23, 1557–1565. <https://doi.org/10.1145/3624062.3624233>
- [Williams et al., 2009] Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4), 65–76. <https://doi.org/10.1145/1498765.1498785>
- [Yasin, 2014] Yasin, A. (2014). A top-down method for performance analysis and counters architecture. 35–44. <https://doi.org/10.1109/ISPASS.2014.6844459>